3D Computer Vision

CS 532 study textbook

Oct 22, 2026

CONTENTS

Before you begin 3

Notation and coordinate conventions 3
- Frames, axes, and transformations . . . 3
- Equality and homogeneous scale . . . 4
- Numerical checks . . . 5

Prerequisite refresher 6
- A quick readiness check . . . 6
- Matrix and vector tools used in Lecture 1 . . . 6
- Rank, nullspaces, and SVD . . . 7

I Lectures 1–6 8

1 Lecture 1: Foundations and 3D geometry 9
- 1.1 Why reconstruct a world from photographs? . . . 10
- 1.2 Points, lines, and the meaning of infinity . . . 14
- 1.3 Move points without losing the geometry . . . 18
- 1.4 Points, planes, and lines in 3D . . . 22
- 1.5 Rotations: several descriptions of the same motion . . . 28
- 1.6 Reference frames and two worked quiz problems . . . 33
- 1.7 Image formation: where does a 3D point appear? . . . 37
- 1.8 Foundation quiz practice . . . 44
- 1.9 Original quiz questions and worked solutions . . . 46
- 1.10 Review: choose the right geometric operation . . . 56
- 1.11 TLDR . . . 59

2 Lecture 2: Projective geometry and image warping 65
- 2.1 Reading route . . . 65
- 2.2 Quaternions: convert without changing the rotation . . . 66
- 2.3 Homographies: straighten a photographed plane . . . 68
- 2.4 Image warping: where do the new pixels come from? . . . 72
- 2.5 Estimate a reliable mapping . . . 74
- 2.6 Distortion and two-view geometry . . . 79
- 2.7 Original quiz questions and worked solutions . . . 83
- 2.8 Review and formula checklist . . . 87
- 2.9 TLDR . . . 88

3 Lecture 3: Put camera geometry to work **94**
3.1 Reading route . . . 94
3.2 From field of view to a camera matrix . . . 95
3.3 Find depth when the point lies on a known plane . . . 96
3.4 Recover a genuinely projective homography . . . 98
3.5 Probability and the pentagon puzzle . . . 100
3.6 Original quiz questions and worked solutions . . . 103
3.7 Review and source checklist . . . 117
3.8 TLDR . . . 120

4 Lecture 4: Epipolar geometry and stereo depth **126**
4.1 Find the worked math quickly . . . 126
4.2 Reading route . . . 127
4.3 Epipolar geometry: visual guide and worked quiz examples . . . 127
4.4 Estimate F from noisy matches . . . 132
4.5 Recover camera pose with calibration . . . 135
4.6 Stereo depth, sensitivity and worked error examples . . . 137
4.7 Find matches across an image . . . 143
4.8 Original quiz questions and worked solutions . . . 148
4.9 Review and source checklist . . . 164
4.10 TLDR . . . 166

5 Lecture 5: Find points you can track **172**
5.1 Reading route . . . 172
5.2 Find a small set of recognizable points . . . 172
5.3 Why corners are easier to locate . . . 174
5.4 Track a feature by solving for its motion . . . 180
5.5 Describe a patch, then judge its matches . . . 185
5.6 Original quiz questions and worked solutions . . . 190
5.7 Formula and source checklist . . . 196
5.8 TLDR . . . 197

6 Lecture 6: Reconstruct a scene—and improve it **202**
6.1 What can the photographs actually determine? . . . 202
6.2 Factorization: split one matrix into cameras and shape . . . 205
6.3 Triangulation: find the 3D point behind two pixels . . . 208
6.4 Grow a reconstruction one view at a time . . . 210
6.5 Bundle adjustment: move the model until the pixels agree . . . 213
6.6 Original quiz questions and worked solutions . . . 218
6.7 Formula and source checklist . . . 228
6.8 TLDR . . . 230

Study and reference **235**

Formula reference **235**
Lecture 1 . . . 235
Lecture 2 . . . 235
Lecture 3 . . . 235
Lecture 4 . . . 235

Lecture 5 . . . 236
Lecture 6 . . . 236

Quiz practice index **237**
Lecture 1 . . . 237
Lecture 2 . . . 237
Lecture 3 . . . 238
Lecture 4 . . . 239
Lecture 5 . . . 240
Lecture 6 . . . 241
Outside the core scope . . . 242

Glossary **243**

Sources and figure credits **249**
Reading list named in the lecture . . . 249
Lecture 2 . . . 249
Lecture 3 . . . 250
Lecture 4 . . . 250
Complete quiz workbooks . . . 251
Lecture 5–6 illustrations and method references . . . 251

Errata and assumptions **252**
Lecture 1: verified qualifications and convention translations . . . 252
Lecture 2: verified corrections and assumptions . . . 253
Lecture 3 qualifications . . . 253
Lecture 4 verified corrections and conventions . . . 254
Lecture 5–6 slide qualifications . . . 254

Index **255**

From images to geometry

How can photographs reveal the shape of a scene and the motion of a camera? This book develops the geometric ideas behind that question through the first six CS 532 lectures.

Read in lecture order, beginning with the *notation guide* and *prerequisite refresher*. Worked examples connect the lecture slides to the course quizzes. The *quiz index* links every included source to its full worked solutions.

> **Note**
>
> All six lectures contain explanations, credited figures, formula/application boxes, complete quiz workbooks, and final TLDR quizzes. Each lecture has one interactive demonstration with a full static counterpart for print.

How to study a section

1. Start with the visual question and explain the intuition in your own words.
2. Read the formula's assumptions and symbol definitions before following the derivation.
3. Work through the application procedure, then attempt the example without its solution.
4. Use the chapter checklist and formula reference to identify what still needs practice.

The website and printable edition are built from the same Markdown pages. Essential explanations and solutions belong in both editions.

Before you begin

NOTATION AND COORDINATE CONVENTIONS

These conventions apply throughout the book. A source using another convention will be translated explicitly.

Symbol	Meaning	Shape / units
$\mathbf{X}$	Homogeneous 3D point	4×1; scale arbitrary
$\mathbf{x}$	Homogeneous image point	3×1; scale arbitrary
(u, v)	Inhomogeneous pixel coordinates	pixels
$\mathbf{X}_c$	Euclidean camera point $(X_c, Y_c, Z_c)^T$	3×1; one consistent length unit
$\mathbf{K}$	Camera intrinsic matrix	3×3; focal entries in pixels
$\mathbf{R}, \mathbf{t}$	World-to-camera rotation and translation	3×3, 3×1
$\mathbf{C}$	Camera center in world coordinates	3×1; $\mathbf{C} = -\mathbf{R}^T\mathbf{t}$
$\mathbf{P}$	Homogeneous camera projection matrix	3×4
$\mathbf{H}, \mathbf{F}, \mathbf{E}$	Homography, fundamental matrix, essential matrix	each 3×3, with different constraints
ℓ, π	Homogeneous line / plane coefficients	3×1 / 4×1

Frames, axes, and transformations

Vectors are columns. Image u increases rightward and v downward. Use right-handed camera axes with positive Z_c forward. Specify any different image-plane convention in the figure and derivation.

Write $\mathbf{T}_{B \leftarrow A}$ for a transform that maps coordinates from frame A into frame B. In a product, the rightmost transform acts first. State whether a rotation acts on points or changes the coordinate frame. Use scalar-first unit quaternions (q_w, q_x, q_y, q_z) and define the rotation convention before applying them.

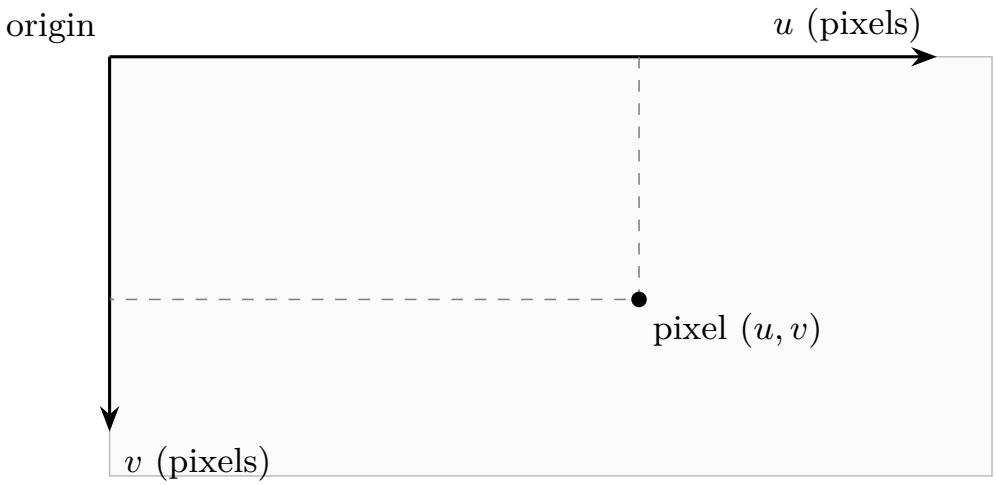


Image coordinates increase rightward and downward.
Distances are measured in pixels.

Fig. 1: The pixel-axis convention used throughout the book. The marked pixel is illustrative, not a calibrated measurement. Original diagram; editable TikZ source is retained with the figure assets.

Equality and homogeneous scale

Homogeneous coordinates represent a point by a nonzero vector up to nonzero scale; see *homogeneous coordinates*. Use $=$ for equality of vectors and $\sim$ for equality up to scale. A zero vector is never a projective point.

Formula — homogeneous scale equivalence

$$\mathbf{x} \sim \lambda \mathbf{x}, \qquad \lambda \neq 0. \tag{1}$$

Meaning: scaling all coordinates by the same nonzero number preserves the represented point. Here $\mathbf{x} \in \mathbb{R}^3 \setminus \{\mathbf{0}\}$ and λ is a scalar. This is an exact projective equivalence, not a numerical approximation. When the third coordinate w is nonzero, the corresponding Euclidean point is $(x_1/w, x_2/w)$. When $w = 0$, the point is ideal and this division is invalid.

Application — compare two representations

1. Check that neither vector is zero.
2. Determine whether one nonzero scalar multiplies every component of the first vector to produce the second.
3. For finite points, divide by the last coordinate and compare the resulting Euclidean coordinates.

For example, $(2, 4, 2)^T$ and $(1, 2, 1)^T$ represent the same finite point $(1, 2)$, although the two vectors are not equal. This is an original notation example; lecture-specific derivations and slide references will be added with the chapters.

Numerical checks

Keep pixels distinct from physical lengths. Convert centimeters and meters before substitution. Report exact values before rounded approximations. Distinguish a formula's mathematical conditions from numerical conditioning and state a justified tolerance when checking floating-point results.

You should be comfortable with algebra, trigonometry, basic probability, matrix operations, and introductory calculus. You do not need previous experience with 3D computer vision. Each lecture will refresh the specific tools it uses.

A quick readiness check

- **Vectors:** interpret a dot product, a cross product, and vector length geometrically.
- **Matrices:** check multiplication dimensions, distinguish an inverse from a transpose, and solve a small linear system.
- **Calculus:** interpret a derivative as local sensitivity and a first-order Taylor expansion as a local approximation.
- **Linear algebra:** distinguish eigenvectors from singular vectors and understand a nullspace as the set of solutions to a homogeneous linear system.
- **Probability:** calculate the complement of repeated independent failures.

Attempt the check below. Further numerical refreshers will be added when the corresponding lectures are written.

Check — can these matrices be multiplied?

A matrix A has shape 3×4 and a vector X has shape 4×1. What is the shape of AX? Can XA be formed?

Solution

The inner dimensions of AX agree, so its shape is 3×1. The product XA cannot be formed: its inner dimensions would be 1 and 3. This is the dimension check used later in camera projection.

Matrix and vector tools used in Lecture 1

A **basis** is a set of independent directions that can build every vector in the space. In 2D, $(1, 0)$ and $(0, 1)$ are the standard basis. The columns of a transformation matrix tell you where these basis vectors go. This is the derivation behind the rotation matrix in *Move points without losing the geometry*.

A **dot product** multiplies matching entries and adds them. For perpendicular vectors it is zero. In geometry it can test whether a point satisfies line or plane coefficients; the roles of the input vectors matter. A **cross product** of two 3-vectors is perpendicular to both, with magnitude equal to the area of their spanned parallelogram. Parallel inputs give zero. See (1.2) for its components and a geometric application.

The **transpose** swaps rows and columns; it does not generally invert a matrix. An **inverse** undoes the transformation and exists only for an invertible square matrix. Rotation matrices are special: their inverse equals their

transpose. A **determinant** is a signed area/volume scale for a square linear map. Zero means collapse of at least one independent direction; a proper rotation has determinant $+1$.

Rank, nullspaces, and SVD

Rank counts independent output directions. If a matrix sends a nonzero vector to zero, that vector belongs to its **nullspace**. A 3×4 matrix of rank three has a one-dimensional nullspace: every solution is a scalar multiple of one nonzero vector. This is precisely the scale ambiguity of a homogeneous plane or point.

A **singular value decomposition (SVD)** writes a real matrix as $A = U\Sigma V^T$, where U, V have orthonormal columns in their full forms and Σ contains nonnegative singular values. Geometrically, the map changes orthonormal coordinates, scales directions, then changes coordinates again. A right singular vector belonging to zero singular value is mapped to zero.

For the 3×4 incidence systems in *Points, planes, and lines in 3D*, a full SVD supplies a 4×4 V: its last column spans the nullspace when rank is three. A reduced decomposition may omit that column. This implementation detail matters even when the geometric derivation is correct.

An **eigenvector** satisfies $Av = \lambda v$: its direction is preserved, though its scale may change. A rotation axis has eigenvalue one. Eigenvectors and singular vectors solve different questions and should not be interchanged.

Refresher — a null vector is not the zero answer

For $A = \begin{pmatrix} 1 & 0 & -2 \\ 0 & 1 & -3 \end{pmatrix}$, verify a nonzero null vector and explain what "up to scale" means.

Solution

$A(2, 3, 1)^T = (0, 0)^T$. Every nonzero multiple of $(2, 3, 1)^T$ is another valid null vector. The zero vector is also an algebraic solution, but it cannot represent a projective point. The matrix has rank two and one free scale direction.

Part I

Lectures 1–6

CHAPTER

ONE

LECTURE 1: FOUNDATIONS AND 3D GEOMETRY

Original quiz questions + fully worked solutions — complete problem statements, numbered red steps with a plain-English reason, and blue mathematics showing intermediate operations.

Can photographs taken from different viewpoints reveal a shared 3D world? Lecture 1 introduces that problem and builds the geometric language used to solve it.

This chapter follows the actual supplied deck, including its early introduction to camera projection. Later lectures revisit these foundations and develop their estimation algorithms. Read the sections in order; use the review checklist to find a formula or the final TLDR for a quick recap and quiz.

What you will learn

- Explain why a pixel alone does not determine scene depth.
- Join points, intersect lines, and recognize ideal points in homogeneous coordinates.
- Distinguish point transformations from line/plane transformations.
- Construct planes and spatial lines, and recognize degenerate inputs.
- Apply and compare rotation matrices, ordered angles, axis–angle, and quaternions.
- Compose rigid frame transforms with explicit directions.
- Project a world point into a camera and reason about camera motion.

Reading route

1. *Why reconstruct a world from photographs?*
2. *Points, lines, and infinity*
3. *2D transformations and invariants*
4. *Planes and lines in 3D*
5. *Rotation representations*
6. *Frames and integrated quiz problems*
7. *Camera projection and the hands-on exercise*
8. *Foundation quiz practice*
9. *Review and source-coverage checklist*
10. *TLDR: key takeaways and a short quiz*

How the quizzes fit

Fall 2025 Quiz 1, questions 3–4, develops relative camera pose and a camera/object plane. Fall 2017 Quiz 9 supplies the tetrahedron normals/rotation problem and a supplementary depth-backprojection bridge. The supplied lecture mapping classifies both quiz sets as supplementary; the core geometric ideas are still prerequisites for the higher-priority camera and two-view topics.

Spring 2020 Quiz 1 is also solved here: its original Lecture 2 assignment is preserved in the quiz index, but its foundation topics are taught only in this chapter.

Source: `wk1_2_cs532s26_IntroToGeometry.pdf`, all 114 PDF pages audited. Repeated visual builds are consolidated. Course logistics are excluded from the mathematics; research applications are summarized rather than treated as extra algorithms. Geometry slide sections credit M. Pollefeys and D. Cappelleri; image-formation sections credit John Oliensis. All newly added numerical values and diagrams are labeled as original or adapted.

1.1 Why reconstruct a world from photographs?

Imagine collecting photographs of the same building from people who never met. They used different cameras, stood in different places, and photographed it at different times. Could those disconnected pictures become one 3D model?

That is the opening challenge of Lecture 1: **"the Internet as a camera."** Here the phrase describes a collection of viewpoints, not one physical camera or a claim that arbitrary pictures can always be reconstructed. The reconstruction must discover which images overlap, which pixels refer to the same scene features, and where the cameras were.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 2–61. Printed slide labels match PDF pages through page 24; from PDF page 25 onward labels are offset by one. The application descriptions below summarize the supplied lecture, not the current state of a research project.

1.1.1 Work backward from an image

Computer vision infers properties of a scene from measured images. Computer graphics usually starts with scene geometry, materials, lighting, and a viewpoint to produce an image. Vision faces the inverse question: given the image, what could have produced it?

This distinction matters because the inverse need not be unique. A small nearby object and a larger distant object may occupy the same pixels. Surface color and illumination can also trade off: a dark pixel might belong to a dark material or a shadow. The ambiguous figures and lighting illusions on PDF pages 30–42 are reminders that an image is evidence, not a complete description of the world.

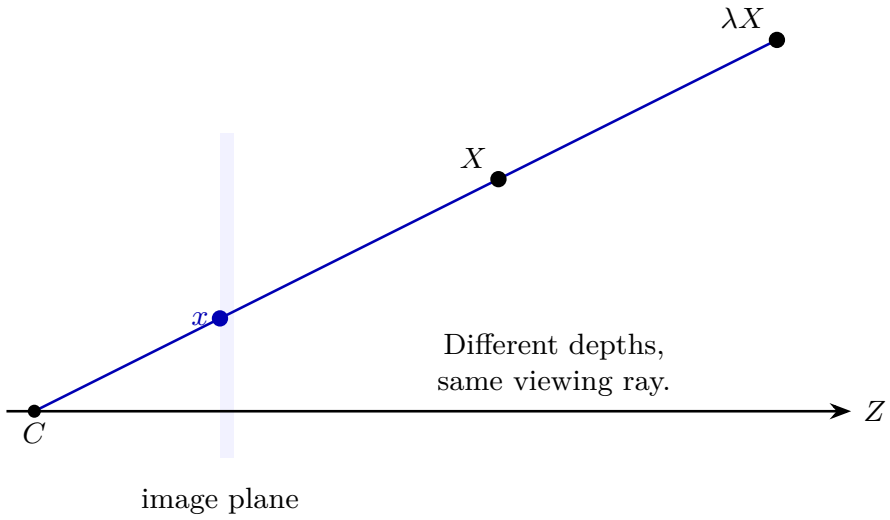


Fig. 1.1: One image point specifies a viewing ray, not a unique depth. Original diagram explaining the projection ambiguity illustrated on Lecture 1 PDF pages 37 and 105–107.

The picture gives us a useful distinction between **geometric information**—positions, shapes, and camera motion—and **semantic information**, such as whether a region depicts a building or vegetation. Both can help a vision system, but this book concentrates on geometry. A geometric reconstruction can have accurate surface positions without knowing the names of the objects.

1.1.2 A reconstruction is a chain of linked problems

The Colosseum sequence on PDF pages 19–24 motivates the following pipeline. The arrows are logical dependencies; practical systems may revisit earlier stages.

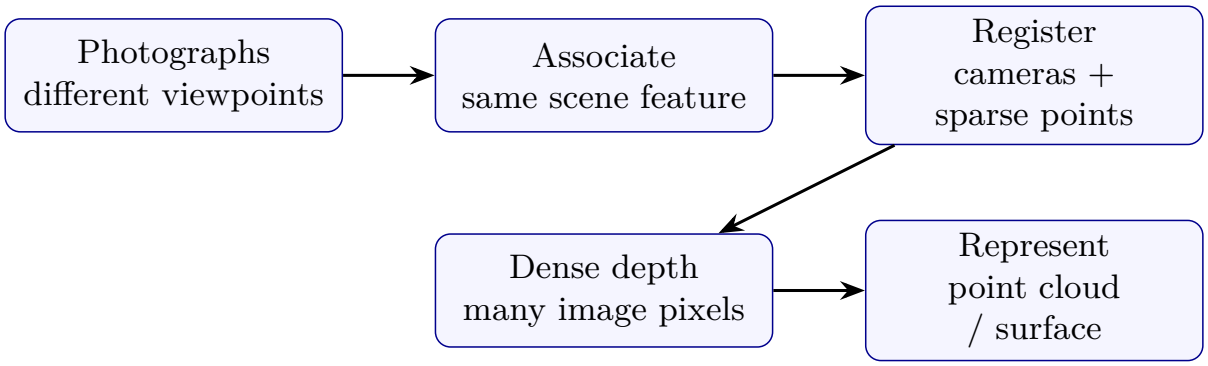


Fig. 1.2: From independent photographs to a geometric representation. Original redrawing of the lecture's reconstruction workflow, PDF pages 18–26. The underlying lecture figures use the Colosseum as an example.

1. **Associate:** identify measurements of the same scene feature across images. A similar-looking window is not automatically the same physical window.

2. **Register:** estimate camera positions/orientations and a sparse set of 3D points. **Structure from motion (SfM)** estimates scene structure and camera motion together from image correspondences.
3. **Estimate dense depth:** recover geometry for many pixels, not just a few distinctive corners.
4. **Represent:** organize the recovered geometry as a point cloud, surface, or another useful model. Dense depth, point-cloud fusion, and meshes are previews here; their full algorithms lie beyond these six lectures.

A ***depth map*** stores a depth value at each valid image location. A ***point cloud*** stores 3D point positions. They are different representations: a depth map still depends on its camera coordinates; a point cloud can express all points in one common frame.

1.1.3 Follow the fountain from pixels toward shape

PDF pages 50–51 show photographs, depth estimates, fusion, and colored geometry of a fountain. This is particularly useful because its decorative details are visible in both the photograph and geometric output.



Fig. 1.3: An input view from the fountain example, extracted from Lecture 1 PDF page 50, printed slide 51. Source: supplied Enrique Dunn lecture deck; the original photograph's creator is not identified on that slide.



Fig. 1.4: A depth output from the same example, extracted from PDF page 51, printed slide 52. The slide compares raw and fused estimates. Grayscale is an encoding of the source's depth data; its numerical scale is not specified, so do not infer metric depths from brightness alone. Source: supplied lecture deck; original figure creator not specified there.

Notice three things. First, matching must identify the same ornament or basin location in different views. Second, a depth estimate is a geometric hypothesis and can contain holes, noise, or mistakes near occlusion boundaries. Third, fusion must place estimates in a common reference frame; simply stacking pixel arrays would not align their 3D points.

The slides also show urban capture in Chapel Hill (PDF 47–49), large photo-collection reconstructions (52–54), visual SLAM (55), dynamic reconstruction (56), and remote sensing, autonomous systems, augmented reality, and geolocalization (57–61). These illustrate different data conditions, not extra algorithms to memorize in Lecture 1. **Visual SLAM** emphasizes estimating a moving camera's pose while building a map. A dynamic scene introduces an additional challenge: the same object may move between images.

1.1.4 Why the geometry is difficult

The lecture's examples identify several different obstacles:

- **Projection loses depth:** many 3D points can lie on one image ray.
- **Appearance is not geometry:** shadows, reflections, materials, and exposure change pixel intensities.
- **Measurements are imperfect:** noise, quantization, blur, occlusion, and camera-model errors affect the observations.
- **Association is ambiguous:** repeated textures and locally similar patches can produce incorrect matches.
- **More data creates more work:** a large photo collection can improve coverage but also increases matching and optimization costs.

The answer is not to abandon images. It is to state the model and assumptions clearly, use multiple sources of evidence, and check the resulting estimates.

Check 1.1 — is a correspondence already a 3D point?

Two images contain matching pixels on the same fountain ornament. Do those two pixel coordinates alone provide its position in meters?

Solution

No. We still need suitable camera geometry and a scale reference for metric units. Matching establishes data association. Reconstruction uses the image measurements and cameras to infer geometry. Even a valid monocular reconstruction generally has an unknown global scale unless additional information fixes it.

1.1.5 What to remember

The rest of Lecture 1 builds the language needed to describe this pipeline: homogeneous points and lines, transformations, planes, rotations, reference frames, and camera projection. The goal is to understand how a calculation relates to a geometric question, not to mistake a visually convincing output for a verified reconstruction.

1.2 Points, lines, and the meaning of infinity

Railway tracks are parallel in the world, yet their image traces appear to converge. Is that a contradiction? No: the world plane and the image plane are related by perspective, which does not preserve parallelism.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 64–71 (printed slides 65–72).



Fig. 1.5: The railway example from PDF page 68, printed slide 69. Source: supplied lecture deck; original photographer unspecified. The source image has limited resolution, so it is intentionally displayed at a modest size.

1.2.1 Add a coordinate, not an extra physical dimension

An ordinary point (u, v) can be represented by $\mathbf{x} = (u, v, 1)^T$. Multiplying all three components by a nonzero scalar gives the same projective point. The added coordinate makes certain geometric operations easier; it does not give a 2D point an extra physical dimension.

Recall from *Notation and coordinate conventions* that $\sim$ means equality up to nonzero scale. A finite point $(x_1, x_2, w)^T$ becomes $(x_1/w, x_2/w)$ when $we0$. For example, $(6, 9, 3)^T$ represents $(2, 3)$.

The ***projective plane*** $\mathbb{P}^2$ is the set of nonzero vectors in $\mathbb{R}^3$ **identified under scale**. This last clause matters: the slide's shorthand $\mathbb{R}^3 \setminus \{0\}$ must be understood with this identification. Three homogeneous entries have only two degrees of freedom because one overall scale is redundant.

Formula — point–line incidence

$$\ell^T \mathbf{x} = ax_1 + bx_2 + cw = 0. \tag{1.1}$$

Here $\ell = (a, b, c)^T \neq 0$ is a homogeneous line and $\mathbf{x} = (x_1, x_2, w)^T \neq 0$ is a point. For $w = 1$, this is the familiar $au + bv + c = 0$. The scalar zero tests ***incidence***: whether the point lies on the line.
Conditions and limits: This is exact. Both line and point coefficients may be rescaled. It tests incidence, not Euclidean distance; a nonzero residual has no scale-independent magnitude unless you normalize it. At least one of a, b must be nonzero for an ordinary finite line.

How to use it — point–line incidence

1. Express the point and line in the same coordinates.
2. Multiply corresponding entries and add them.
3. Check whether the result is zero (or numerically small under a specified normalization).

Worked connection: *Example 1.1*.

Why a cross product appears

The cross product of two 3-vectors is perpendicular to both. Therefore, if two vectors encode points, their cross product supplies line coefficients orthogonal to both points. If the vectors encode lines, their cross product supplies a point incident to both lines. The algebra is the same; the geometric interpretation changes.

Formula — join two points or intersect two lines

$$\ell \sim \mathbf{x} \times \mathbf{y}, \qquad \mathbf{p} \sim \ell_1 \times \ell_2,$$
$$\mathbf{a} \times \mathbf{b} = \begin{pmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{pmatrix}. \tag{1.2}$$

Each input and output is a 3×1 homogeneous vector. Joining two distinct points gives a line; intersecting two distinct lines gives a point. The components have coefficient-dependent units, so interpret the result after the appropriate homogeneous normalization.
Conditions and limits: The result must be nonzero. Equal points up to scale do not define a unique line; equal lines do not define a unique intersection. Reversing the order changes the sign and therefore preserves the represented object.

How to use it — join two points or intersect two lines

1. Decide whether the inputs represent points or lines.
2. Evaluate the three cross-product components.
3. If the result is zero, diagnose coincident/degenerate inputs.
4. For a point, divide by the last component only if it is nonzero. Verify incidence by dot products.

Worked connection: *Example 1.1*.

Worked example 1.1 — parallel lines have a projective intersection

Source: original numerical expansion of the railway/ideal-point example, PDF pages 67–71. The related Spring 2020 Quiz 1 is solved in *Foundation quiz practice*.

Take $y = 1$ and $y = 2$, represented by $\ell_1 = (0, 1, -1)^T$ and $\ell_2 = (0, 1, -2)^T$.

1. Their cross product is $(-1, 0, 0)^T \sim (1, 0, 0)^T$.
2. The last coordinate is zero, so this is an ***ideal point***, not a finite Euclidean position.
3. Check incidence: both $\ell_1^T(1, 0, 0)$ and $\ell_2^T(1, 0, 0)$ are zero.
4. Its first two coordinates identify the horizontal direction. Division by the last coordinate is neither required nor allowed.

For comparison, intersecting $x = 2$, or $(1, 0, -2)^T$, with $y = 3$, or $(0, 1, -3)^T$, gives $(2, 3, 1)^T$, a finite point.

Formula — ideal points and the line at infinity

$$\begin{aligned} \ell_1 \times \ell_2 &= (c_2 - c_1)(b, -a, 0)^T, \\ \ell_\infty &= (0, 0, 1)^T, \qquad \ell_\infty^T (d_x, d_y, 0)^T = 0. \end{aligned} \tag{1.3}$$

The parallel finite lines are $\ell_1 = (a, b, c_1)^T$ and $\ell_2 = (a, b, c_2)^T$. Their ideal intersection records direction $(b, -a)$. All points with last coordinate zero lie on the ***line at infinity***.

Conditions and limits: Require $(a, b) \neq (0, 0)$ and $c_1 \neq c_2$. The factor $c_2 - c_1$ may be dropped up to scale, but not when it is zero. "At infinity" is a projective classification, not a very large finite coordinate.

How to use it — ideal points and the line at infinity

1. Put distinct parallel lines into a shared (a, b) normalization.
2. Cross them and recognize the zero last component.
3. Read the direction from the first two components and check it is perpendicular to (a, b).

Worked connection: *Example 1.1*.

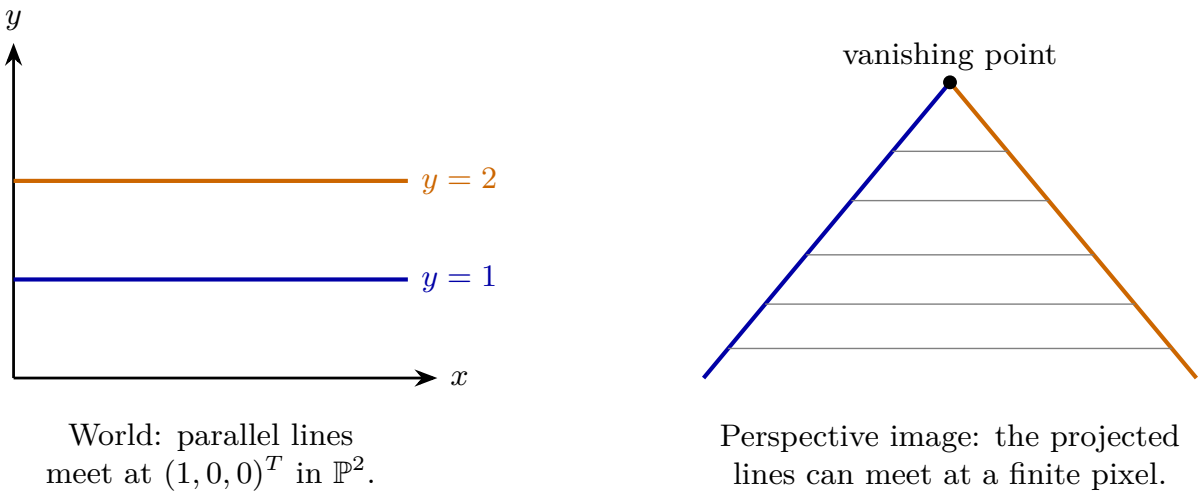


Fig. 1.6: An ideal point in one plane may map to a finite vanishing point in another. Original diagram developing PDF pages 68–71. The world and image panels are different coordinate spaces, not two claims about the same pair of coordinate lines.

1.2.2 A vanishing point is not automatically an ideal image point

A scene's parallel rails share a direction. Under a perspective mapping, that direction can project to a finite pixel location: the ***vanishing point*** in the photograph. The ***horizon*** of a scene plane is the image of that plane's line at infinity. It need not coincide with the image's own line at infinity.

This distinction explains the opening puzzle. The world lines remain parallel in world coordinates, while their images can intersect at an ordinary finite pixel. We will derive the camera mapping later in *Image formation: where does a 3D point appear?*.

Check 1.2 — does changing scale change the line?
Do $(2, -2, 4)^T$ and $(-1, 1, -2)^T$ describe different lines? Does $(0, 0, 0)^T$ describe every line?

Solution

The first two vectors differ by a factor of -2, so they describe the same line. The zero vector is invalid: its equation places no restriction on points and cannot identify a unique line.

1.2.3 What to remember

Use dot products for incidence and cross products for joins/intersections. Normalize a finite point only after checking its last coordinate. Track which space a point belongs to before calling it a vanishing point or a point at infinity.

1.3 Move points without losing the geometry

Suppose a point is one unit to the right of a pivot. A quarter-turn should put it one unit above that pivot. Rotating around the coordinate origin instead gives the wrong answer unless the pivot happens to be the origin. A small coordinate diagram makes this distinction easier than memorizing a large matrix.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 72–79 (printed slides 73–80).

Throughout the 2D rotation examples, x points right and y points **up** in a Cartesian diagram. This differs from the downward pixel v axis in the notation guide. We use active rotations of points and the right-hand rule; do not transfer a screen-clockwise interpretation without checking the axes.

1.3.1 Derive rotation from the basis vectors

A counterclockwise rotation by $heta$ sends $(1,0)^T$ to $(\cos heta, \sin heta)^T$. It sends $(0,1)^T$ to $(-\sin heta, \cos heta)^T$. A point $(x,y)^T$ is x times the first basis vector plus y times the second. Rotating both basis vectors and combining them gives the columns of the rotation matrix.

Formula — 2D rotation

$$\mathbf{p}' = R(\theta)\mathbf{p}, \qquad R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}. \tag{1.4}$$

$\mathbf{p}$ and $\mathbf{p}'$ are 2×1 positions relative to the rotation center; R is 2×2 and unitless. Thus $x' = x\cos\theta - y\sin\theta$ and $y' = x\sin\theta + y\cos\theta$.
Conditions and limits: Exact for a planar rotation. State angle units before evaluating trigonometric functions. This rotates around the origin and preserves lengths and angles; translation is a separate operation.

How to use it — 2D rotation

1. Choose a Cartesian axis convention and rotation center.
2. Convert the angle to the units expected by the computation.
3. Multiply the matrix by the point; check that its distance from the center is unchanged.

Worked connection: *Example 1.2*.

Formula — translation and rotation about a pivot

$$\begin{pmatrix} \mathbf{p}' \\ 1 \end{pmatrix} = \begin{pmatrix} R & \mathbf{t} \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \mathbf{p} \\ 1 \end{pmatrix}, \qquad \mathbf{p}' = \mathbf{c} + R(\mathbf{p} - \mathbf{c}), \qquad H_c = T(\mathbf{c})\,\mathrm{diag}(R, 1)T(-\mathbf{c}). \tag{1.5}$$

$\mathbf{p}, \mathbf{c}, \mathbf{t}$ are 2×1 vectors in the same length unit. $T(\mathbf{t})$ is the 3×3 homogeneous translation matrix with identity upper-left block and last column $(t_x, t_y, 1)^T$.
Conditions and limits: Exact rigid geometry; the rightmost matrix acts first. A direction has homogeneous last coordinate zero, so a translation does not change it. A position has last coordinate one.

How to use it — translation and rotation about a pivot

1. Subtract the pivot to express the point relative to it.
2. Apply the rotation.
3. Add the pivot back. In homogeneous form multiply in that same right-to-left order.
4. Check that the pivot itself stays fixed.

Worked connection: *Example 1.2*.

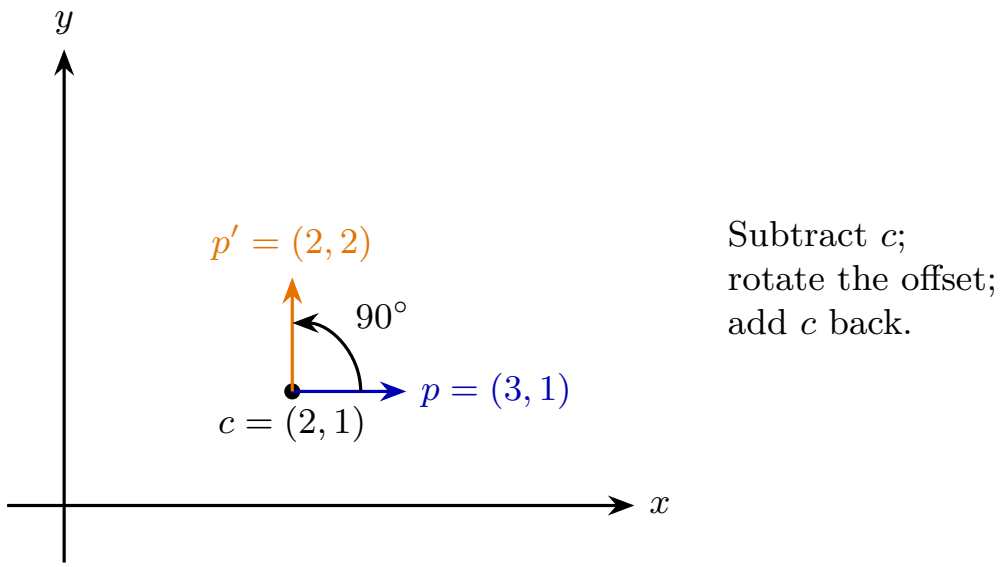


Fig. 1.7: Translate to the pivot, rotate the offset, then translate back. Original solution diagram for the hands-on question on PDF page 77, printed slide 78.

Worked example 1.2 — a quarter-turn around a chosen center

Source: numerical adaptation of the hands-on tasks on PDF page 77.
Let $p = (3, 1)^T$, $c = (2, 1)^T$, and $\theta = 90°$.

1. The offset is $p - c = (1, 0)^T$.
2. A quarter-turn sends that offset to $(0, 1)^T$.

3. Adding the center gives $p' = (2, 2)^T$.
4. Both offsets have length one, and applying the same operation to c gives c.

Rotating p directly around the origin would give $(-1, 3)^T$, demonstrating why the chosen center matters.

1.3.2 Transformation families: what stays unchanged?

The **degrees of freedom (DOF)** count independent parameters, not the number of printed matrix entries. A general invertible 3×3 homogeneous matrix has nine entries, but its overall nonzero scale is irrelevant, leaving eight DOF.

Formula — 2D transformation hierarchy

$$H_E = \begin{pmatrix} R & t \\ 0^T & 1 \end{pmatrix}, \quad H_S = \begin{pmatrix} sR & t \\ 0^T & 1 \end{pmatrix}, \quad H_A = \begin{pmatrix} A & t \\ 0^T & 1 \end{pmatrix}, \quad H_P = \begin{pmatrix} A & t \\ v^T & \alpha \end{pmatrix}. \tag{1.6}$$

Blocks R, A are 2×2; t, v are 2×1 and α, s are scalars. Euclidean, similarity, affine, and projective maps have respectively 3, 4, 6, and 8 DOF in 2D. Here $s > 0$ and R is a proper rotation.

Conditions and limits: Each full mapping must be invertible; affine A must be invertible. Projective division is required after multiplication and can map finite points to infinity. The Euclidean form here excludes reflections; allowing reflections adds a discrete choice, not another continuous DOF.

How to use it — 2D transformation hierarchy

1. Ask what the data require: rigid motion, uniform scale, shear, or perspective.
2. Identify the smallest valid transformation family.
3. Check its invariants using the table below and normalize transformed finite points.

Worked connection: *Example 1.3*.

Family	What it preserves	What it can change
Euclidean	Lengths, angles, incidence, parallelism	Position and orientation
Similarity	Angles and ratios of lengths	Overall scale
Affine	Incidence, parallelism, ratios along a line, ratios of areas	Angles and unequal directional scales
Projective	Incidence, straightness, cross-ratio where defined	Parallelism, lengths, angles, ordinary area ratios

The table summarizes PDF page 78. "Preserves ratios" for an affine map means the relevant ratio within the same transformation, not that every arbitrary pair of vector lengths is preserved. A projectivity also preserves tangency where defined; it need not preserve a visible square's appearance.

Worked example 1.3 — shear is not a rotation

Source: original computation developing the transformation hierarchy on PDF page 78.

Take $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$. It sends the basis vectors to $(1, 0)^T$ and $(1, 1)^T$.

1. Their original dot product is zero; their new dot product is one. The right angle changed.
2. Horizontal parallel lines remain horizontal and parallel.
3. The determinant is one, so oriented area is unchanged for this particular shear.

This is affine, but not Euclidean or similarity. General affine maps multiply every area by the same determinant magnitude, which explains why area ratios are preserved even when areas themselves change.

The hierarchy slide also names the ***circular points***: two complex points at infinity shared by all Euclidean circles in a plane. Their invariance under similarities is a projective way of encoding angle information; they are not finite pixels you could photograph. In 3D, the analogous ***absolute conic*** is a complex conic on the plane at infinity that encodes the Euclidean metric. These are vocabulary previews for reconstruction upgrades in Lecture 6, not additional numerical tasks for Lecture 1.

1.3.3 Lines transform differently from points

A line is a set of points satisfying $\ell^T x = 0$. If $x' = Hx$, then $x = H^{-1}x'$. Substituting gives $\ell^T H^{-1} x' = 0$. Transpose the coefficient product: $(H^{-T}\ell)^T x' = 0$. That is why multiplying a line by H generally gives the wrong result.

Formula — transform line and plane coefficients

$$\mathbf{x}' \sim H\mathbf{x}, \qquad \ell' \sim H^{-T}\ell; \qquad \mathbf{X}' \sim G\mathbf{X}, \quad \pi' \sim G^{-T}\pi. \tag{1.7}$$

H is an invertible 3×3 planar projectivity; G is an invertible 4×4 spatial projectivity. Lines and planes are **dual** coefficients of incidence equations, so they transform with the inverse transpose.

Conditions and limits: Exact projective incidence. The inverse does not exist for a singular transformation; such a map is not a projectivity. Coefficient normalization does not change the represented line or plane.

How to use it — transform line and plane coefficients

1. Confirm whether the object is a point/direction or an incidence coefficient vector.
2. Use the forward matrix for points and its inverse transpose for lines/planes.
3. Transform a known incident point and verify that its new incidence residual is zero.

Worked connection: *Example 1.4*.

Worked example 1.4 — translate a line

Source: original verification of PDF page 79's line-transformation rule.

Translate every point by $(2, 0)^T$. The line $x = 0$ should become $x' = 2$.

Its coefficients are $\ell = (1, 0, 0)^T$. With $H = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$, the correct result is $H^{-T}\ell = (1, 0, -2)^T$, or $x' - 2 = 0$. Applying H directly to ℓ would leave the line at $x' = 0$, which fails the geometric check.

Check 1.3 — reverse the operation

After the pivot rotation in Example 1.2, how do you recover the original point?

Solution

Subtract the same pivot, rotate by $-90°$, then add the pivot back. The inverse reverses the operation order: $(ABC)^{-1} = C^{-1}B^{-1}A^{-1}$. For this pivot sandwich the result is $T(c)R(-90°)T(-c)$.

1.4 Points, planes, and lines in 3D

A 2D line is described by one linear equation. In 3D, one linear equation describes a plane instead. A spatial line therefore needs more structure: it is the intersection of two planes, or a point together with a direction.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 80–87, printed slides 81–88.

1.4.1 Homogeneous 3D points

Represent a finite point (X, Y, Z) by $(X, Y, Z, 1)^T$. Divide a general homogeneous point $(X_1, X_2, X_3, X_4)^T$ by X_4 when $X_4 \neq 0$. When $X_4 = 0$, it represents an ideal direction. The zero vector is invalid.

An invertible 4×4 matrix G acts by $\mathbf{X}' \sim G\mathbf{X}$. It has $16 - 1 = 15$ projective DOF because one overall scale is redundant. A plane transforms by G^{-T}; the incidence derivation in *Move points without losing the geometry* applies without change.

1.4.2 A plane is a normal and an offset

A ***surface normal*** is perpendicular to the surface's local tangent directions. For a plane it is constant everywhere. If $n = (a, b, c)^T$, the equation $n^T q + d = 0$ describes points whose displacement from a point on the plane is perpendicular to n.

Formula — plane incidence and Euclidean distance

$$\pi^T X = 0, \quad \pi = (a, b, c, d)^T, \qquad D(q, \pi) = \frac{|n^T q + d|}{\|n\|}. \tag{1.8}$$

$X = (q^T, 1)^T$ is 4×1; q, n are 3×1. D is unsigned point-to-plane distance in the coordinate length unit. The signed version omits the absolute value and depends on normal orientation.

Conditions: $n \neq 0$ for a finite Euclidean plane. The plane at infinity $(0, 0, 0, 1)^T$ is projectively valid but has no finite distance formula. Exact geometry; normal scale is arbitrary.

How to use it — test incidence or measure distance

1. Put the point and plane in the same frame.
2. Evaluate $n^T q + d$; zero means incidence.
3. Divide its magnitude by $\|n\|$ for distance.
4. Check that rescaling all four coefficients leaves the distance unchanged.

See Example 1.5 below.

Formula — plane through three noncollinear points

$$n = (B - A) \times (D - A), \qquad d = -n^T A, \qquad \pi = (n^T, d)^T. \tag{1.9}$$

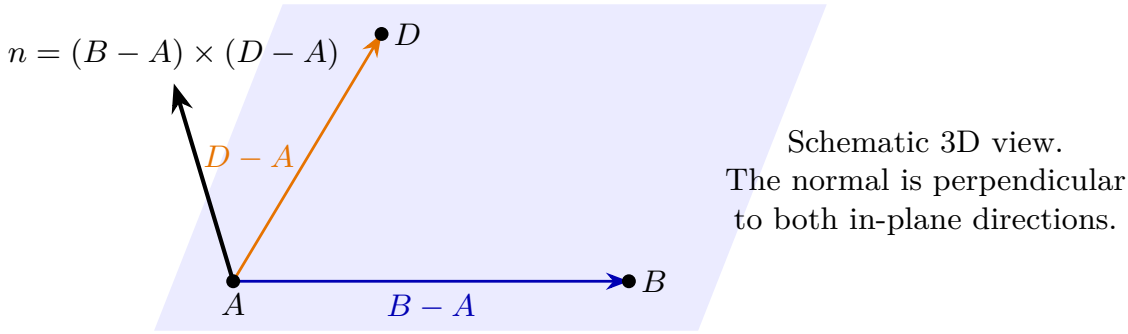
A, B, D are 3×1 Euclidean positions. Their differences lie in the plane, so their cross product is perpendicular to it. Substituting A fixes d.

Conditions: the points must be noncollinear. A zero cross product means no unique plane. Reversing its order reverses the normal but preserves the unoriented plane. If coordinates have length units, the unnormalized normal has squared-length units; normalize before metric interpretation.

How to use it — construct and verify a plane

1. Subtract the same anchor point to form two directions.
2. Cross them and check for a nonzero result.
3. Compute d and substitute all three points into the plane equation.
4. Normalize n only when a unit normal or signed metric distance is needed.

See *Example 1.5* and the camera/object quiz in *Reference frames and two worked quiz problems*.



Schematic 3D view. The normal is perpendicular to both in-plane directions.

Fig. 1.8: A normal comes from two in-plane directions. Original schematic developing PDF pages 82–84. Apparent angles in this drawing are not measurements of the 3D angles.

Worked example 1.5 — find a plane, then check a distance

Source: original numerical development of PDF pages 82–84.
Let $A = (1, 0, 0)$, $B = (0, 1, 0)$, and $D = (0, 0, 1)$.

1. $B - A = (-1, 1, 0)$ and $D - A = (-1, 0, 1)$.
2. Their cross product is $(1, 1, 1)$, so $d = -1$.
3. The plane is $x + y + z - 1 = 0$. All three input points satisfy it.
4. The origin's distance is $1/\sqrt{3}$ units, not one: the normal is not unit length.

1.4.3 The same geometry from a nullspace

A ***nullspace*** consists of vectors a matrix sends to zero. Stacking homogeneous input points as rows makes a plane coefficient vector orthogonal to every row. Conversely, stacking planes gives the point incident to all of them.

Formula — point/plane incidence systems

$$\underbrace{\begin{pmatrix} X_A^T \\ X_B^T \\ X_D^T \end{pmatrix}}_{M\,:\,3\times4} \pi = 0, \qquad \underbrace{\begin{pmatrix} \pi_1^T \\ \pi_2^T \\ \pi_3^T \end{pmatrix}}_{N\,:\,3\times4} X = 0. \tag{1.10}$$

Each unknown is a nonzero 4×1 vector, unique up to scale when the coefficient matrix has rank three. An SVD $M = U\Sigma V^T$ supplies the null vector as the last column of the full 4×4 matrix V.
Conditions: exact rank-three incidence. Lower rank means nonuniqueness. A reduced SVD of a 3×4 matrix can omit the null vector: request the full right-singular basis. An intersection with last coordinate zero is ideal, not finite.

How to use it — solve homogeneous incidence

1. Put point or plane coefficients in rows.
2. Check rank and solve for a nonzero right-null vector.
3. Normalize only when valid; substitute back into every equation.
4. For numerical data state a rank tolerance and inspect the residual.

See *Example 1.6*.

The determinant construction on PDF page 84 is another way to express the same plane. Four coplanar homogeneous point columns are linearly dependent, so their 4×4 determinant vanishes. Expanding along the variable point column yields the plane coefficients.

Formula — plane coefficients by cofactors

$$\det\begin{pmatrix} X & X_A & X_B & X_D \end{pmatrix} = 0, \qquad \pi = (D_{234}, -D_{134}, D_{124}, -D_{123})^T. \tag{1.11}$$

D_{ijk} is the 3×3 determinant formed from rows i, j, k of the last three columns. Expansion gives $X_1 D_{234} -$

$X_2 D_{134} + X_3 D_{124} - X_4 D_{123} = 0$.
Conditions: independent input point columns. This exact construction equals the nullspace result; near degeneracy can make direct cofactor evaluation poorly conditioned.

How to use it — evaluate plane cofactors

1. Form the 4×3 matrix of the input point columns.
2. Delete each row and calculate the remaining determinant.
3. Apply alternating signs and verify incidence at each input point.

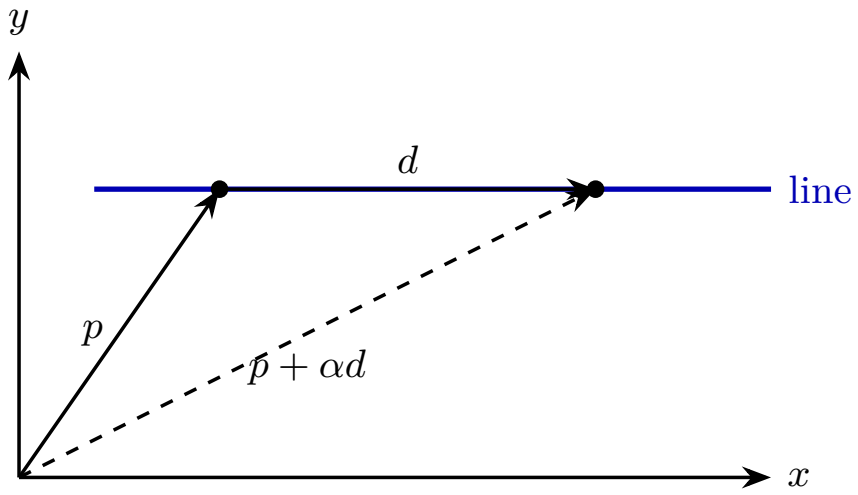
See *Example 1.6*.

Worked example 1.6 — duality in numbers

For Example 1.5 the point rows are $(1, 0, 0, 1)$, $(0, 1, 0, 1)$, and $(0, 0, 1, 1)$. Multiplying by $(1, 1, 1, -1)^T$ gives zero. The cofactors are $1, -1, 1, 1$ in the order above, giving the same plane after alternating signs. Conversely, the planes $x = 1$, $y = 2$, and $z = 3$ have rows $(1, 0, 0, -1)$, $(0, 1, 0, -2)$, and $(0, 0, 1, -3)$. Their null vector is $(1, 2, 3, 1)^T$, so their intersection is $(1, 2, 3)$.
Source: original numerical expansions of PDF pages 83–85.

1.4.4 Plücker coordinates: a line is a direction and a moment

Plücker coordinates encode a nonzero direction d and a moment $m = p \times d$, where p is any point on the line. Choosing a different point along the line should not change its representation.



Changing the chosen point along the line does not change $m = p \times d$, because $d \times d = 0$.

Fig. 1.9: The chosen point may move along the line without changing its moment. Original diagram developing PDF pages 86–87.

Formula — finite-line Plücker coordinates

$$L = (d, m), \qquad m = p \times d, \qquad d^T m = 0, \qquad p_\perp = \frac{d \times m}{\|d\|^2}. \tag{1.12}$$

All component vectors are 3×1. $p_\perp$ is the closest point on the line to the origin. With unitless $d, p, m, p_\perp$ have length units. A common nonzero scale on (d, m) represents the same line.
Conditions: $d \neq 0$. Six entries minus one scale and one orthogonality constraint leave four line DOF. Not every six-vector is valid. Lines entirely at infinity require a more general treatment than this finite-line form.

How to use it — encode or recover a spatial line

1. Choose a point and a nonzero direction; calculate $m = p \times d$.
2. Check $d^T m = 0$.
3. Recover $p_\perp$ if useful and check it belongs to the same line.
4. Keep the direction/moment order and cross-product sign consistent.

See *Example 1.7*.

Replacing p by $p + \alpha d$ leaves the moment unchanged because $d \times d = 0$. The closest-point formula follows from

$$d \times (p \times d) = \|d\|^2 p - (d^T p) d.$$

After division by $\|d\|^2$, the component along the line is removed, leaving the perpendicular displacement from the origin.

Formula — reciprocal product and line distance

$$\begin{aligned} \rho &= d_1^T m_2 + d_2^T m_1, \\ D &= \frac{|\rho|}{\|d_1 \times d_2\|} \quad (d_1 \times d_2 \neq 0), \\ D_{\parallel} &= \frac{\|(p_2 - p_1) \times d_1\|}{\|d_1\|}. \end{aligned} \tag{1.13}$$

$L_i = (d_i, m_i)$ denotes each valid finite line; p_i is any point on it. The ***reciprocal product*** ρ is zero for coplanar lines, including both intersecting and parallel lines. D is their shortest distance in the coordinate length unit.
Conditions: nonzero directions. Use the parallel expression when directions are parallel; the nonparallel formula is unstable near parallelism. A zero reciprocal product alone does not distinguish intersection from parallelism.

How to use it — measure separation

1. Check the direction cross product to classify the case.
2. For nonparallel lines, divide the reciprocal-product magnitude by the cross-product norm.
3. For parallel lines, measure the perpendicular part of the displacement between two points.
4. Check nonnegativity, units, and invariance under common rescaling of a line's coordinates.

See *Example 1.7*.

The nonparallel formula measures displacement along a common perpendicular. Substituting $m_i = p_i \times d_i$ gives $\rho = (p_1 - p_2)^T (d_1 \times d_2)$. The denominator turns the common perpendicular into a unit vector.

Worked example 1.7 — two skew lines are two units apart

Source: original numerical companion to PDF page 87; supplementary relative to the exam priorities.
Let $p_1 = (0, 1, 0)$, $d_1 = (1, 0, 0)$, $p_2 = (0, 0, 2)$, and $d_2 = (0, 1, 0)$.

1. $m_1 = (0, 0, -1)$ and $m_2 = (-2, 0, 0)$; each satisfies $d_i^T m_i = 0$.
2. $d_1 \times d_2 = (0, 0, 1)$ has norm one.
3. $\rho = -2$, so $D = 2$.
4. The first line lies at $z = 0$ and the second at $z = 2$. Their closest points have equal x, y, confirming the result.

Check 1.4 — does any triplet determine a plane?

What happens for $(0, 0, 0)$, $(1, 0, 0)$, and $(2, 0, 0)$?

Solution

The points are collinear. Their displacement cross product is zero, so infinitely many planes contain the common line. Do not normalize the zero vector or claim a unique solution.

1.5 Rotations: several descriptions of the same motion

A 3D orientation can be stored as a matrix, an axis and angle, three ordered angles, or a quaternion. These representations are not interchangeable lists of numbers. Ask **what rotates, about which axes, and in which order?** before using any of them.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 88–100, printed slides 89–101.

1.5.1 Rotation matrices preserve the dot product

We use active right-handed rotations of column vectors. Each elementary rotation keeps its named axis fixed and performs a 2D rotation in the perpendicular plane.

Formula — elementary 3D rotations

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{pmatrix}, \quad R_y(\theta) = \begin{pmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{pmatrix}, \quad R_z(\theta) = \begin{pmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad c = \cos\theta, \quad s = \sin\theta. \tag{1.14}$$

Each matrix is unitless and 3×3; the point it multiplies is 3×1. Angles follow the right-hand rule in a right-handed Cartesian frame.
Conditions: exact rotations about axes through the origin. A displaced axis needs translations as well. A passive change of coordinates uses the inverse. Specify degrees or radians explicitly.

How to use it — rotate around a named axis

1. Name the axis and state whether points or coordinates are rotating.
2. Substitute sine/cosine in consistent angle units.
3. Multiply by the point.
4. Check the fixed coordinate and preserved length.

See *Example 1.8* and the tetrahedron quiz in *Reference frames and two worked quiz problems*.

Formula — validate and invert a rotation

$$R^T R = I, \qquad \det R = +1, \qquad R^{-1} = R^T. \tag{1.15}$$

The columns of a proper rotation are orthonormal basis vectors. Dot products are preserved because $(Ra)^T(Rb) = a^T(R^T R)b = a^T b$; therefore lengths and angles are preserved. Nine entries with six independent orthonormality constraints leave three continuous DOF.
Conditions: a proper rotation, not an arbitrary matrix. An orthogonal matrix with determinant -1 includes a reflection. Floating-point checks need tolerances; transposing a noisy nonrotation is not generally its inverse.

How to use it — check a claimed rotation

1. Compare $R^T R$ with identity and $\det R$ with $+1$.
2. Use R^T as the inverse only when those conditions hold.
3. Rotate a test vector forward and backward and compare with the original.

See *Example 1.8*.

The slide example on PDF page 89 sends $(1,0,1)^T$ to $(0,1,1)^T$ with a positive quarter-turn about z. The unchanged last coordinate and equal lengths give two independent sanity checks.

1.5.2 Composition: order is part of the answer

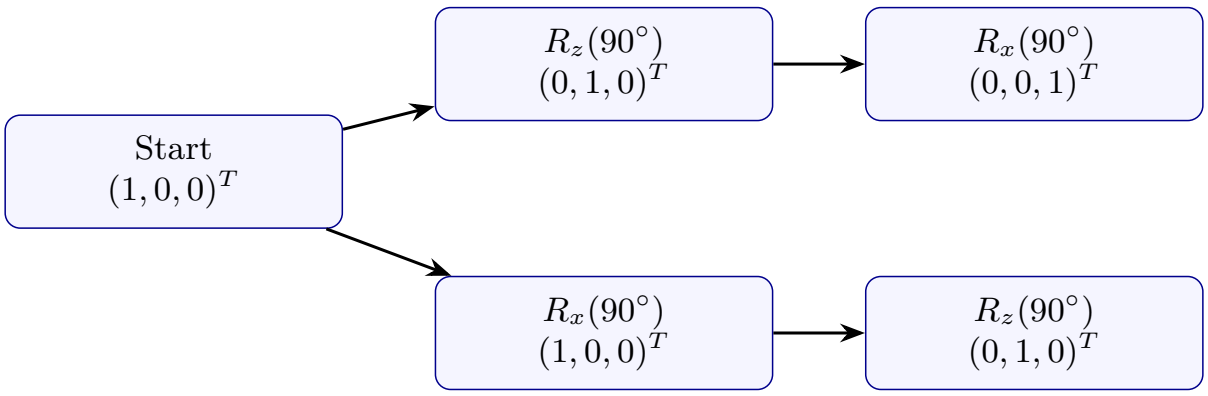


Fig. 1.10: The same two rotations in different orders give different results. Original numerical diagram developing PDF pages 93–100.

Worked example 1.8 — prove that rotation order matters

Source: original numerical expansion of the compositional-rotation slides, PDF pages 93–100.
Start with $v = (1,0,0)^T$.

1. Apply $R_z(90°)$: v becomes $(0,1,0)^T$.

2. Apply $R_x(90^\circ)$ next: it becomes $(0, 0, 1)^T$.
3. Reverse the order: the first x rotation leaves v unchanged, then the z rotation gives $(0, 1, 0)^T$.

Thus $R_x R_z \neq R_z R_x$. The rightmost matrix in each product acts first on the column vector.

1.5.3 Body axes versus fixed axes

An ***intrinsic rotation*** sequence uses axes attached to the moving body. An ***extrinsic rotation*** sequence uses fixed reference axes. Euler terminology varies: some sources reserve "Euler angles" for a repeated first/third axis, while others also include three-distinct-axis sequences. Write the axes and matrix product explicitly.

The lecture's body-axis sequence uses x, then the new y, then the new z (PDF 95–98). The fixed-axis sequence uses premultiplication (PDF 99–100). Equal angle values in these two sequences do not generally give equal orientations.

Formula — ordered-angle composition

$$R_{\text{body } xyz} = R_x(a)R_y(b)R_z(c), \qquad R_{\text{fixed } xyz} = R_z(c)R_y(b)R_x(a). \tag{1.16}$$

Angles a, b, c name successive x, y, z rotations. For an orientation matrix mapping body coordinates to world coordinates, a body-axis increment postmultiplies it; a world-axis increment premultiplies it. All factors are 3×3.

Conditions: the sequence and frame interpretation are essential. At $b = \pm 90^\circ$ these three-angle parameterizations become singular: two angle directions lose independent effect, called ***gimbal lock***. The physical orientation remains valid; its angle description becomes ambiguous.

How to use it — assemble an orientation

1. Specify body versus fixed axes and the full sequence.
2. Associate each angle with its named axis.
3. Multiply the elementary matrices in the stated order.
4. Test a basis vector instead of trusting roll/pitch/yaw names alone.

See *Example 1.8*.

For reference, multiplying the body-xyz factors gives

$$\begin{pmatrix} c_b c_c & -c_b s_c & s_b \\ s_a s_b c_c + c_a s_c & -s_a s_b s_c + c_a c_c & -s_a c_b \\ -c_a s_b c_c + s_a s_c & c_a s_b s_c + s_a c_c & c_a c_b \end{pmatrix},$$

with $c_a = \cos a$, $s_a = \sin a$, and similarly for b, c. This is an expansion of the boxed product, not a different rule. Recover it by multiplying two factors first and checking row–column dot products. Roll/pitch/yaw labels depend on axis convention; the lecture's fixed-axis figure uses its own assignment. An explicit product avoids that ambiguity.

1.5.4 Axis–angle: turn about one direction

An **axis–angle representation** uses a unit axis a and a turn angle θ. Components along the axis stay fixed. Components perpendicular to it undergo the familiar planar rotation.

Formula — extract axis and angle

$$\theta = \arccos\left(\frac{\operatorname{tr}(R) - 1}{2}\right), \qquad a = \frac{1}{2\sin\theta}\begin{pmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{pmatrix}. \tag{1.17}$$

R is a proper 3×3 rotation and $\theta \in [0, \pi]$ is its principal angle. a is a unit 3×1 direction. The trace is the diagonal sum: one unchanged axial component contributes one, while the perpendicular rotation contributes $2\cos\theta$.
Conditions: the axis expression requires $\sin\theta \neq 0$. At zero angle the axis is arbitrary. At π, recover the axis with a stable method, such as a unit eigenvector satisfying $Ra = a$. After validating R, clamp small numerical arccos overshoots to $[-1, 1]$.

How to use it — recover a geometric rotation

1. Validate R, calculate its trace, and recover the angle.
2. Handle near-zero and near-π cases separately.
3. Otherwise compute and normalize the axis.
4. Check $Ra = a$ and reconstruct a test rotation.

See *Example 1.9*.

An original derivation companion makes this representation computational. Split p into $p_{\parallel} = a(a^T p)$ and $p_{\perp} = p - p_{\parallel}$. Keep the first part, rotate the second by sine/cosine, and use $a \times p$ as its quarter-turn direction. Collecting terms yields **Rodrigues' formula**:

Formula — build a rotation from axis and angle

$$R = I\cos\theta + (1 - \cos\theta)aa^T + \sin\theta[a]_{\times}, \qquad [a]_{\times}p = a \times p. \tag{1.18}$$

Here $[a]_{\times} = \begin{pmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{pmatrix}$. Every matrix is 3×3 and unitless.
Conditions: $\|a\| = 1$, active right-handed rotation, and consistent angle units. This exact formula is an explanatory supplement to PDF page 91; it is not claimed to be printed on that slide.

How to use it — construct a rotation matrix

1. Normalize the axis and evaluate sine/cosine.
2. Form the outer product aa^T and cross-product matrix.

3. Add the three terms.
4. Check the axis stays fixed and the result is a proper rotation.

See *Example 1.9*.

1.5.5 Quaternions: four entries for a three-DOF rotation

A ***quaternion*** is a scalar–vector pair $q = (w, v)$ with a special multiplication rule. We use scalar-first Hamilton multiplication and active rotations. Unit length removes one DOF from four entries. The pair $q, -q$ represents the same rotation; this is a discrete ambiguity.

Formula — unit quaternion and active point rotation

$$\begin{aligned} q &= \left(\cos\frac{\theta}{2},\ a\sin\frac{\theta}{2}\right), \qquad P = (0, p), \\ (w_1, v_1)(w_2, v_2) &= (w_1 w_2 - v_1^T v_2,\ w_1 v_2 + w_2 v_1 + v_1 \times v_2), \\ q^{-1} &= (w, -v) \quad (\|q\| = 1), \qquad P' = qPq^{-1}. \end{aligned} \tag{1.19}$$

a, p, v are 3×1; a is unit length. The vector part of pure quaternion P' is the rotated point. Unit quaternion entries are dimensionless; point coordinates retain their length unit.
Conditions: exact under the stated convention. For a nonunit quaternion the inverse is $(w, -v)/(w^2 + v^T v)$, not just its conjugate. PDF page 92 prints $q^{-1}Pq$: that gives the inverse/passive rotation relative to our active convention. Do not mix sandwich orders.

How to use it — rotate a point with a quaternion

1. Build or normalize the scalar-first quaternion.
2. Embed the point as $(0, p)$.
3. Apply Hamilton multiplication twice in the stated order.
4. Extract the vector part; check zero scalar part and preserved vector length.

See *Example 1.9*.

Worked example 1.9 — a quarter-turn in three representations

Source: original numerical companion to PDF pages 88–92.
For 90° about $a = (0, 0, 1)^T$, use $R_z(90^\circ)$ and $q = (\sqrt{2}/2, 0, 0, \sqrt{2}/2)$.

1. The matrix trace is one: $\theta = \arccos(0) = 90^\circ$.
2. The skew-entry formula returns $a = (0, 0, 1)$.
3. With $P = (0, 1, 0, 0)$, the first Hamilton product is $qP = (0, \sqrt{2}/2, \sqrt{2}/2, 0)$.
4. Multiplying by q^{-1} gives $P' = (0, 0, 1, 0)$, so the rotated point is $(0, 1, 0)^T$.
5. Rodrigues' formula and the elementary matrix agree. The opposite sandwich $q^{-1}Pq$ instead gives $(0, -1, 0)^T$.

Check 1.5 — is negative identity a 3D rotation?

The matrix $-I_3$ preserves lengths. Is it a proper rotation?

Solution

No: its determinant is -1. It is orthogonal but reverses orientation. Proper rotations require both orthogonality and determinant $+1$.

1.6 Reference frames and two worked quiz problems

A camera pose answers a question only after its coordinate frames are named. "The translation is $(2, 0, 0)$" is incomplete: from which frame to which other frame? This section makes the direction explicit, then uses it to solve the lecture's most relevant quiz problems.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 101–103, printed slides 102–104; supporting plane/rotation material on PDF pages 82–89.

1.6.1 Rigid motion is rotation plus translation

A ***rigid transformation*** preserves distances and angles. With column vectors, rotate a position into the target axes, then add the source origin's position expressed in the target frame.

Formula — rigid coordinate transform and inverse

$$\begin{aligned} p_B &= R_{B\leftarrow A}p_A + t_{B\leftarrow A}, \\ T_{B\leftarrow A} &= \begin{pmatrix} R & t \\ 0^T & 1 \end{pmatrix}, \qquad T_{A\leftarrow B} = \begin{pmatrix} R^T & -R^T t \\ 0^T & 1 \end{pmatrix}. \end{aligned} \tag{1.20}$$

p_A, p_B, t are 3×1 and use one length unit; R is a proper 3×3 rotation; T is 4×4. The inverse follows by subtracting t from p_B and multiplying by R^T.
Conditions: exact rigid transform. The inverse translation is $-R^T t$, not generally $-t$. Directions and normals under a rigid transform rotate without translation; positions translate as well.

How to use it — move a coordinate description

1. Label the source and target frames on the matrix.
2. Rotate the source-frame position and add the translation in target coordinates.
3. To reverse, apply the inverse with rotated negative translation.
4. Check that a source-frame origin maps to t and that forward/inverse composition is identity.

See *Example 1.10*.

Formula — compose compatible frame transforms

$$T_{C\leftarrow A} = T_{C\leftarrow B}T_{B\leftarrow A}, \qquad R_{CA} = R_{CB}R_{BA}, \quad t_{CA} = R_{CB}t_{BA} + t_{CB}. \tag{1.21}$$

All T matrices are 4×4; all R blocks are 3×3 and translations 3×1. Intermediate frame B matches between adjacent factors. The subscripts without arrows abbreviate the same target/source ordering.
Conditions: frames, time, object convention, and length units must agree. A product of numerically compatible matrices can still be geometrically meaningless if the frame path does not connect.

How to use it — follow the frame path

1. Write the desired coordinate journey, for example $A \to B \to C$.
2. Place the first operation on the right.
3. Invert any supplied transform whose direction opposes the journey.
4. Multiply and test the result on a known point.

See *Example 1.10*.

1.6.2 Quiz: recover camera pose through a shared object

Assigned source: Fall 2025 Quiz 1, questions 3–4, `cs532f25_quiz_1_solution.pdf`, PDF page 2. Its printed title is "RANSAC and Two-view Geometry." The probability questions 1–2 belong beside RANSAC in Lecture 3; the geometry questions belong here according to `quizByLec.md`.

Each static camera estimates the object's pose in its own coordinates. Assume both estimates describe the same rigid object frame at the same instant and in the same physical units.

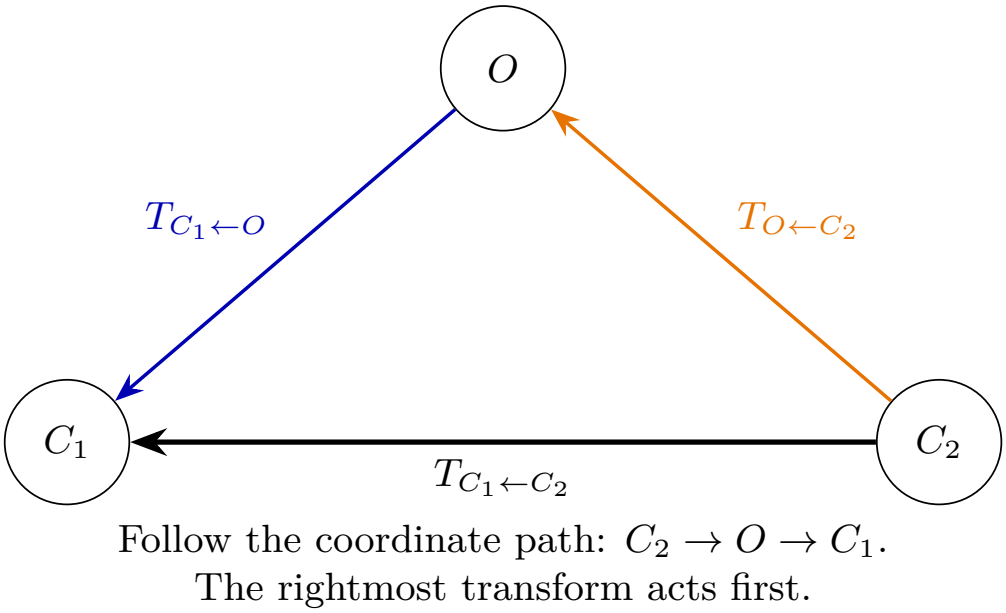


Fig. 1.11: Follow the transform directions through the shared object. This is a frame graph, not a drawing of physical camera positions. Original diagram for Fall 2025 Quiz 1, question 3.

Worked example 1.10 — Fall 2025 Quiz 1, question 3. *Read the original problem and every worked step.*

$$T_{C_1 \leftarrow C_2} = T_{C_1 \leftarrow O}(T_{C_2 \leftarrow O})^{-1}. \tag{1.22}$$

The boxed general composition formula is the canonical rule; the displayed quiz equation is its concrete specialization. Its 4×4 inputs and output have the same frame/unit conditions. Use the graph as the algorithm: invert the camera-2 object pose, multiply on the left by the camera-1 object pose, and verify on the object origin.

Worked example 1.11 — Fall 2025 Quiz 1, question 4. *Read the original problem and every worked step.*

1.6.3 Quiz: tetrahedron face normals and a rotated object

Assigned source: Fall 2017 Quiz 9, problem 2(a–b), `cs532_f17_quiz9.pdf`, PDF page 2. This is supplementary practice for Lecture 1; it is not a high-probability exam prediction. Its depth-map problem 1 is previewed in the camera section and retained for deeper treatment in Lectures 3–4.

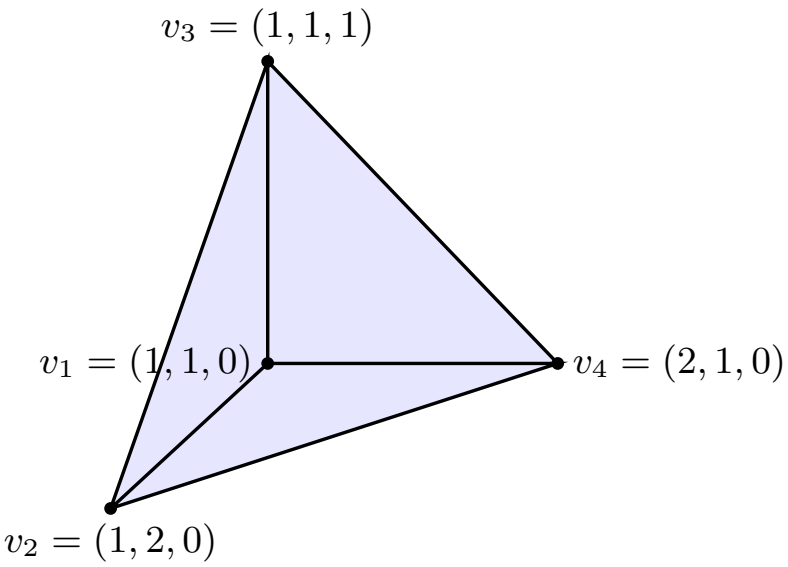


Fig. 1.12: The quiz tetrahedron. Coordinates are shown relative to the corner v_1 for drawing convenience; vertex labels give the actual quiz coordinates. Original schematic, not a metric perspective rendering.

Worked example 1.12 — Fall 2017 Quiz 9, problem 2(a). *Read the original problem and every worked step.*

Worked example 1.13 — Fall 2017 Quiz 9, problem 2(b). *Read the original problem and every worked step.*

1.6.4 Where rigid transformations sit in the 3D hierarchy

The last geometry slide, PDF page 103, extends the 2D hierarchy to 3D. The homogeneous forms in (1.6) extend by making R, A 3×3 and t, v 3×1.

Family	3D continuous DOF	Important preserved properties
Proper rigid/Euclidean	6	Distances, angles, volume
Similarity	7	Angles and ratios of lengths
Affine	12	Parallelism, collinear ratios, volume ratios
Projective	15	Incidence and tangency where defined

A rigid motion has three rotation and three translation parameters. Adding uniform scale gives seven. An affine map has nine linear and three translation parameters. A projectivity has sixteen entries minus one common scale. Its matrix must be invertible. Projective notions such as the absolute conic, named in the hierarchy figures, motivate metric invariants but are developed with reconstruction upgrades in Lecture 6.

Check 1.6 — why not add translation to a normal?

A plane is moved three meters sideways without rotation. Does its normal acquire that translation?

Solution

No. A normal describes orientation, not position. The plane's offset changes, while its direction remains fixed. The homogeneous plane coefficients change through the inverse transpose, which accounts for the offset correctly.

1.7 Image formation: where does a 3D point appear?

You move a camera toward a stationary object. Does the object's image move toward the principal point, or away from it? This is the lecture's final hands-on question, and it ties projection to reference frames.

Lecture source: `wk1_2_cs532s26_IntroToGeometry.pdf`, PDF pages 104–114, printed slides 105–115. The introductory image-formation slides credit John Oliensis.

1.7.1 Start with a pinhole and similar triangles

An ideal **pinhole camera** sends each visible scene point along a ray through the camera center to an image plane. The physical sensor behind the pinhole receives an inverted image. We instead use a **virtual image plane** in front of it, which gives a positive focal-length convention and avoids carrying that inversion into every equation.

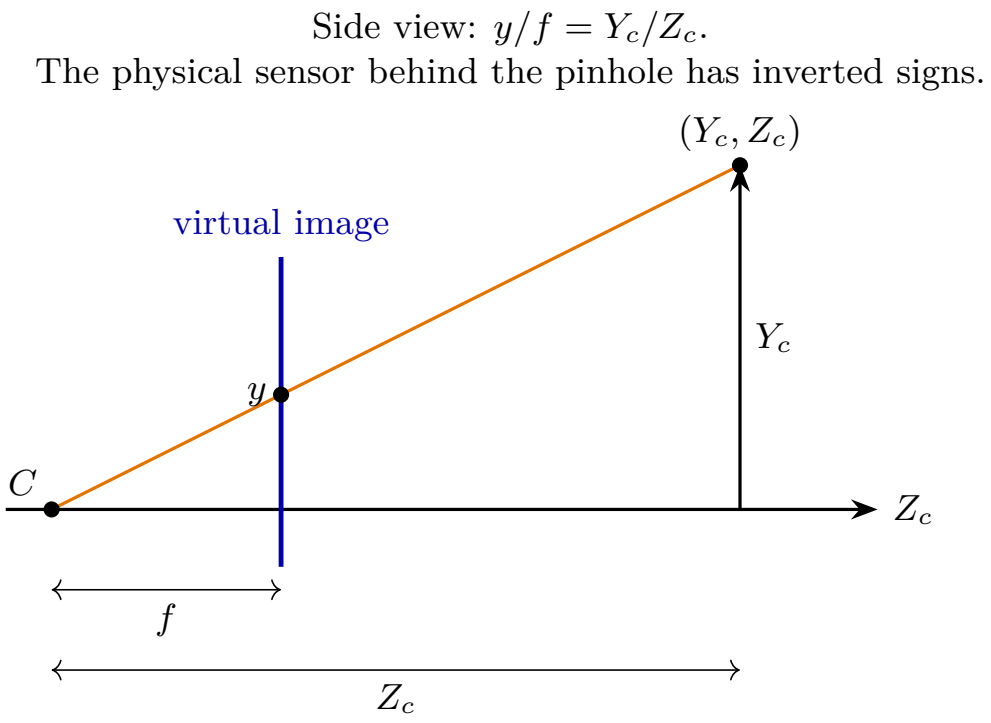


Fig. 1.13: A side-view similar-triangle construction. Original diagram developing PDF pages 105–107; those slide sections credit John Oliensis. The drawing's page orientation is schematic; the coordinate signs follow the labeled camera/image axes.

The small image triangle and large scene triangle share the ray angle. Their corresponding side lengths are proportional. The vertical ratio is $y/Y_c = f/Z_c$; the horizontal argument gives the same ratio for x.

Formula — perspective projection in camera coordinates

$$x = f\frac{X_c}{Z_c}, \qquad y = f\frac{Y_c}{Z_c}, \qquad \begin{pmatrix} x \\ y \\ f \end{pmatrix} = \frac{f}{Z_c}\begin{pmatrix} X_c \\ Y_c \\ Z_c \end{pmatrix}. \tag{1.23}$$

(X_c, Y_c, Z_c) is the point in camera coordinates. x, y, f are coordinates and focal length on the virtual image plane in the same length unit; the ratios are dimensionless. Normalized image coordinates are $(X_c/Z_c, Y_c/Z_c)$.
Conditions: exact ideal central projection, $f > 0$, and $Z_c \neq 0$. Visible points for this forward-facing model need $Z_c > 0$. Pixel coordinates require calibration; do not confuse physical focal length with focal length measured in pixels.

How to use it — project a point

1. Express the point in camera coordinates.
2. Check it is in front of the camera.
3. Divide the transverse coordinates by depth and multiply by focal length.
4. Check that points on one ray have the same image and that doubling depth at fixed transverse position halves the image displacement.

See *Example 1.14*.

Worked example 1.14 — perspective loses depth

Source: original numerical development of PDF pages 107–110.
Use a virtual-plane focal length of 2 length units and a point $(1, 0.5, 10)$ in camera coordinates.

1. $x = 2(1/10) = 0.2$ and $y = 2(0.5/10) = 0.1$ in image-plane length units.
2. The point $(2, 1, 20)$ gives the same $(0.2, 0.1)$ because every scene coordinate was scaled equally.
3. The point $(1, 0.5, 20)$ instead gives $(0.1, 0.05)$: it moved deeper without increasing its transverse displacement.

One image coordinate does not uniquely determine a 3D point. The inverse is a ray unless depth or another geometric constraint is supplied.

1.7.2 Projection properties and their exceptions

The lecture says that lines project to lines and polyhedra project to polygons (PDF 108–109). The underlying reason is that a 3D line together with the camera center defines a plane; that plane intersects the image plane in an image line.

There is a degenerate case: a 3D line through the camera center projects to a single image point, wherever its visible projection is defined. A point at the camera center has no valid projection at all. Occlusion also determines which projected edges are visible. These qualifications prevent an ideal geometric statement from being mistaken for a guarantee about every photograph.

The "distant objects are smaller" example on PDF page 110 is about equal physical objects at different depths: their projected sizes vary inversely with depth under the same camera. Changing focal length at the same time changes that comparison.

1.7.3 Separate camera intrinsics from camera pose

Intrinsics turn normalized camera coordinates into pixel coordinates. **Extrinsics** turn world coordinates into camera coordinates. They answer different questions and have different units.

Formula — calibrated pixel projection

$$K = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \qquad P = K \begin{pmatrix} R & t \end{pmatrix}, \tag{1.24}$$
$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = PX_w, \qquad X_c = RX_{w,\mathrm{E}} + t, \qquad C = -R^T t.$$

K is 3×3; f_x, f_y and skew s are in pixels, and (c_x, c_y) is the ***principal point*** in pixels. X_w is a homogeneous 4×1 world point; $X_{w,\mathrm{E}}$ is its Euclidean 3×1 form. R, t map world to camera, so P is 3×4. C is the camera center in world coordinates.
Conditions: ideal undistorted camera, finite valid point, and a declared world-to-camera convention. With a world point normalized to last coordinate one, this calibration gives $\lambda = Z_c$. P is defined up to nonzero scale. A calibrated finite camera has rank three; its nullspace is the homogeneous camera center.

How to use it — world point to pixel

1. Compute $X_c = RX_{w,\mathrm{E}} + t$ and check $Z_c > 0$.
2. Multiply by K, or multiply PX_w directly.
3. Divide the first two output entries by the third.
4. Check image bounds separately: a point in front can still project outside the sensor.
5. Verify the center relation by $RC + t = 0$.

See *Example 1.15*.

Expanding the first two rows makes the meaning explicit:

$$u = f_x X_c/Z_c + sY_c/Z_c + c_x, \qquad v = f_y Y_c/Z_c + c_y.$$

The offsets place the optical-axis intersection at (c_x, c_y), not necessarily at image coordinate zero. Pixel focal length is a physical focal length divided by pixel pitch; different pixel scales can give different f_x, f_y. Detailed calibration and homography estimation follow in Lecture 3.

Convention translation for PDF page 113: the slide writes $K[R^T \mid -R^T t]$ when R, t describe the camera's pose in world coordinates. In our notation call those $R_{w\leftarrow c}, C$. Then the world-to-camera map is $[R_{w\leftarrow c}^T \mid -R_{w\leftarrow c}^T C]$. It is the same geometry, expressed with a different meaning for the symbol R. The slide's "2-D to 3-D" description should not suggest that P uniquely inverts projection: P maps 3D homogeneous points to 2D homogeneous image points.

1.7.4 Work the actual camera-motion exercise

PDF page 114 asks us to choose visible points for a camera with focal length 500 and a 500×500 image, move the camera forward by two units, and then consider rotating/translating points. The slide leaves the points, principal point, and precise sequencing open. We must declare them.

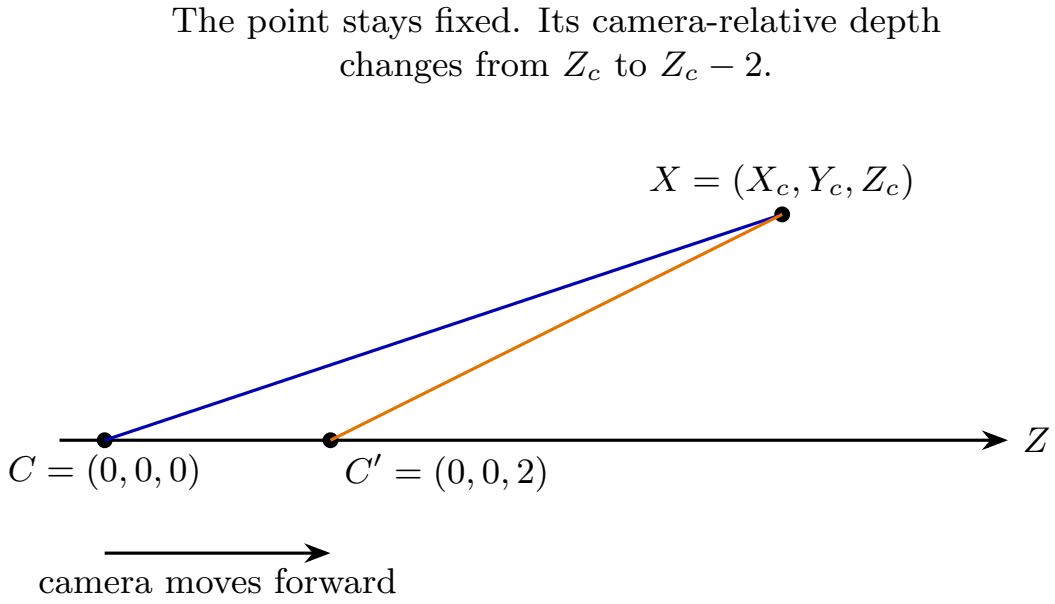


Fig. 1.14: Move the camera, not the object: the world point stays fixed, but its camera-relative depth decreases. Original solution diagram for PDF page 114, printed slide 115.

Worked example 1.15 — the lecture's camera-motion problem

Source: numerical solution adapted from PDF page 114. All values added to complete the setup are stated here.

Assume square pixels, zero skew, $f_x = f_y = 500$ pixels, and principal point $(250, 250)$. Initially the camera is at the world origin with $R = I$. Choose $X_w = (1, 0.5, 10)$ in an arbitrary consistent length unit. Its image is inside the 500×500 sensor.

1. Initially, $(u, v) = (250 + 500/10, 250 + 250/10) = (300, 275)$ pixels.
2. Move the camera center to $C' = (0, 0, 2)$ without rotating it. The stationary point now has camera coordinates $(1, 0.5, 8)$.
3. The new pixel is $(312.5, 281.25)$. Its displacement from the principal point increases because the point is closer.
4. For a separate branch starting again from the original setup, rotate the **point** by 45° about the camera's z axis and then translate it five units farther along z.
5. The new point is $(0.5/\sqrt{2}, 1.5/\sqrt{2}, 15)$. Projection gives $(261.785, 285.355)$ pixels after rounding.

If the operations were meant cumulatively after the camera move, the rotated/transformed point would have depth $8 + 5 = 13$ in that moved camera, giving approximately $(263.598, 290.795)$ instead. A complete answer must specify the sequence.

For a general forward camera displacement Δ, the normalized transverse displacement changes from X_c/Z_c to $X_c/(Z_c-\Delta)$; visible points need $Z_c-\Delta>0$. A point on the optical axis does not move away from the principal point. Translating the object farther away instead gives denominator $Z_c+\Delta$ and moves its image inward. Camera motion and scene motion require inverse coordinate relationships, as PDF page 111 emphasizes.

Explore camera motion and projection — print edition

The 3D explorer lets you move the point, move or turn the camera, and change focal length. Drag empty space to inspect the scene from another angle; that changes your viewing position, not the teaching camera's pose. The calculation follows **world point → camera coordinates → image pixel**, using (1.24). Turning the teaching camera by yaw α changes the world-to-camera rotation to $R_y(-\alpha)$ and the translation to $-RC$; the plane and camera axes turn with it.

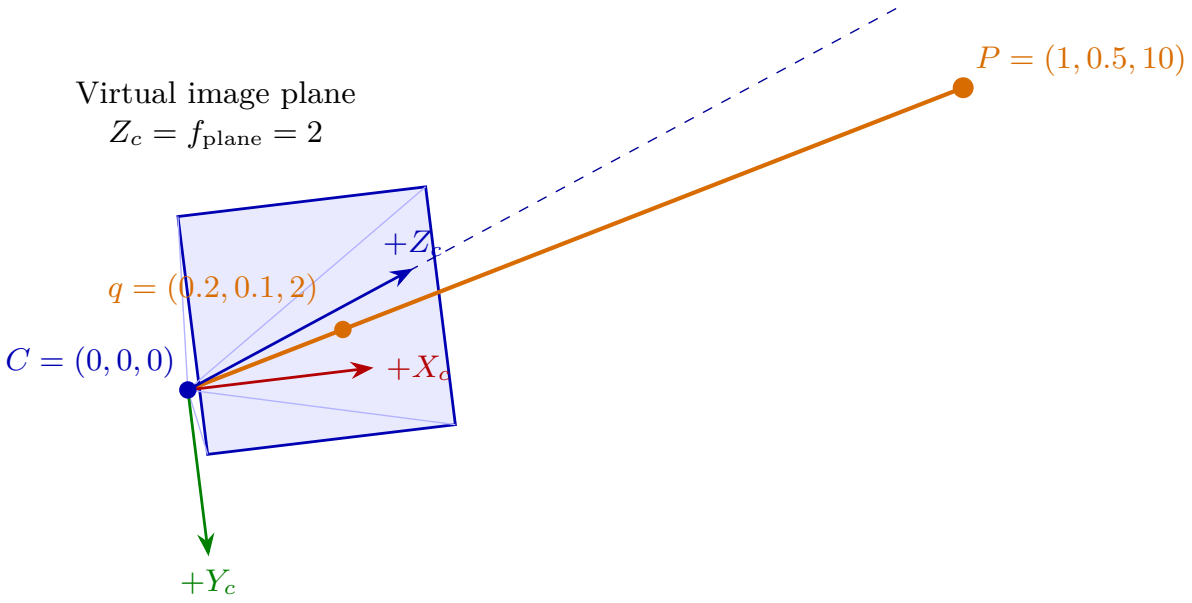


Fig. 1.15: Initial 3D scene: the orange ray from C through P crosses the blue virtual image plane at q. Camera $+Y_c$ points down and $+Z_c$ points forward. This is a static view of the rotatable web scene; depth is encoded geometrically rather than by the page's horizontal distance.

Connect the scene to the numbers: the sensor uses 0.004 world-length units per pixel, so $f = 500$ px places the virtual plane at distance $f_{\text{plane}} = 2$. Its width and height are each $500(0.004) = 2$ units. The ray meets it at $q = (2 \cdot 1/10, 2 \cdot 0.5/10, 2) = (0.2, 0.1, 2)$. Converting those plane coordinates to pixels gives $(u, v) = (250 + 0.2/0.004, 250 + 0.1/0.004) = (300, 275)$.

The table holds $P = (1, 0.5, 10)$ fixed. The image is 500×500 with principal point $(250, 250)$; the camera has no rotation, $f_x = f_y = f$, and skew is zero. Moving the point in the web scene changes its camera coordinates before this same projection is applied.

Predict, then compare: moving the camera forward makes the projected point move away from the principal point. Increasing focal length does the same without changing depth. Can a point be in front of the camera yet outside the image?

Camera C_z	f (px)	Depth	Pixel (u, v)	Visibility
0	500	10	$(300, 275)$	Inside
2	500	8	$(312.5, 281.25)$	Inside
2	1000	8	$(375, 312.5)$	Inside
8.5	500	1.5	$(583.333, 416.667)$	In front; off-image
10	500	0	No finite projection	Undefined
12	500	-2	$(0, 125)$ algebraically	Behind; not visible

The off-image row answers the question: positive depth is necessary but does not ensure sensor visibility. At zero depth division fails; negative-depth coordinates do not describe a visible point in this model. Values are rounded to three decimals.

1.7.5 A depth-map bridge to the assigned quiz

Source: Fall 2017 Quiz 9, problem 1, `cs532_f17_quiz9.pdf`, PDF page 1. The quiz asks how two depth maps can become one common-frame point cloud. This is a supplementary preview; depth estimation and fusion algorithms are beyond Lecture 1.

Formula — axial-depth backprojection and frame conversion

$$q = K^{-1}\begin{pmatrix} u \\ v \\ 1 \end{pmatrix}, \qquad X_c = \frac{Z_c}{q_3} q, \qquad X_w = R^T(X_c - t). \tag{1.25}$$

K is 3×3, q, X_c, X_w, t are 3×1, and Z_c is ***axial depth*** in the camera's forward direction. For the standard intrinsic matrix $q_3 = 1$. The final equation assumes R, t map world to camera.
Conditions: known calibration and camera poses in a common metric frame, invertible K, valid positive depth, and distortion handled. If depth means Euclidean range r along the ray, use $X_c = rq/\|q\|$ instead. Mixing range and axial depth produces incorrect geometry.

How to use it — turn depth maps into common-frame points

1. For each valid depth pixel, use that camera's calibration to recover a viewing direction.
2. Scale it according to the depth convention.
3. Transform the camera-coordinate point into the shared world frame.
4. Repeat for both cameras and combine the points. Combining does not automatically denoise or resolve occlusion.

See *Example 1.16*.

Worked example 1.16 — Fall 2017 Quiz 9, problem 1: preview solution. *Read the original problem and every worked step.*

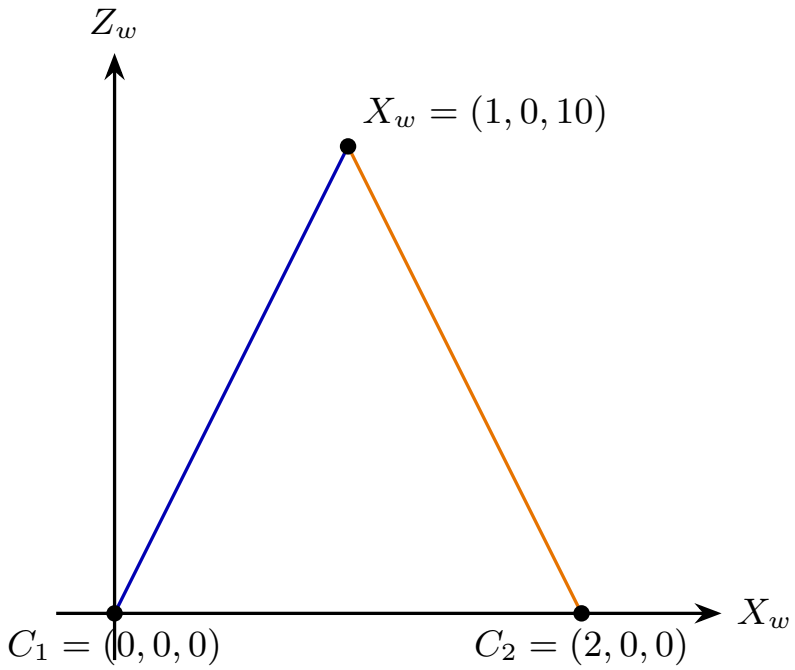


Fig. 1.16: Both backprojected points coincide after conversion to world coordinates. Original schematic for Fall 2017 Quiz 9, problem 1; the optical-axis depth and coordinates come from Example 1.16.

Check 1.7 — is an off-image point behind the camera?

A point has positive camera depth but projects to $u = 700$ in a 500-pixel-wide image. Which condition failed?

Solution

It is in front of the camera but outside the image bounds. Positive depth and sensor visibility are separate tests. A valid homogeneous projection alone does not establish that the camera actually measured the point.

1.8 Foundation quiz practice

Apply the geometry and camera formulas from this chapter to **Spring 2020 Quiz 1**. The supplied quiz map assigns it to Lecture 2. Its *complete quiz workbook* now contains the original problems and colored step-by-step solutions; this page connects them to the foundational rules.

Use (1.1) and (1.2) for Problems 1 and 3, and (1.24) for Problem 2.

1.8.1 Quiz Problem 1: are three points collinear?

Combining the two referenced rules gives the quiz's compact three-point test.

Formula — collinearity test

$$s = c^T(a \times b), \qquad s = 0 \quad \text{for collinear points.} \tag{1.26}$$

a, b, c are nonzero homogeneous 3×1 point vectors. The zero/nonzero result is unchanged by rescaling them. This is an exact determinant test. For measured points, a small unnormalized s is not a meaningful distance threshold.

Check collinearity without accepting an invalid line

1. Check that all point vectors are nonzero.
2. Form $a \times b$. If it vanishes, choose another distinct pair.
3. Dot that line with the third point; zero means incidence.

If all three represent the same point, no unique joining line exists. Any line through that point contains the repeated inputs. See Example 1.17.

Worked example 1.17 — the dot/cross quiz. *Read the original problem and every worked step.*

1.8.2 Quiz Problem 2: what size is a camera matrix?

Worked example 1.18 — dimensions before arithmetic. *Read the original problem and every worked step.*

1.8.3 Quiz Problem 3: how many intersections?

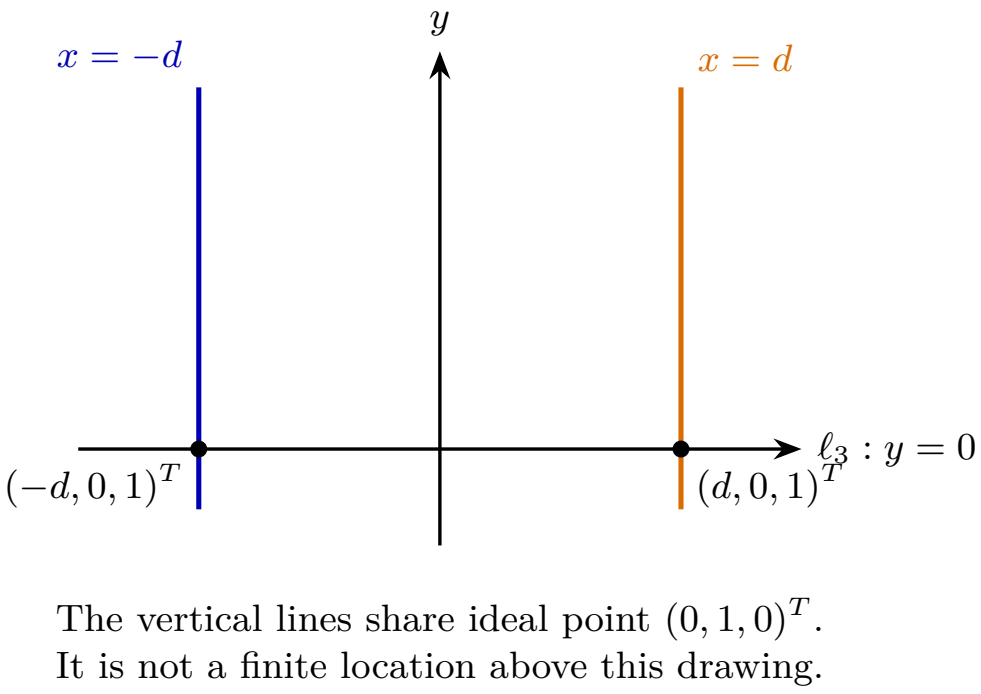


Fig. 1.17: The third intersection is at infinity. It has no ordinary position on this finite drawing. Original diagram of Quiz Problem 3.

Worked example 1.19 — count in the projective plane. *Read the original problem and every worked step.*

Check 1.8 — test a point off the line

Replace c in Example 1.17 by $(5, 1, 1)^T$. What changes?

Answer

The dot product is 2: the three points are not collinear.

1.9 Original quiz questions and worked solutions

Read the problem in black. Follow the numbered red steps. Work through the blue mathematics. Every step explains why it is needed. Matrix calculations show the inputs and intermediate results; written-reasoning questions explain the decision instead of inventing numerical data.

Source: the supplied lecture-organized quiz booklet. Full problems follow its wording and givens, with typography normalized; the historical PDF filenames appear below. Source-key corrections and added numerical companions are marked.

Retests have their own questions and full solutions. Problems labeled **Original practice** are the booklet's seven supplemental examples, separate from the 18 historical quizzes. Colored steps remain numbered and readable in grayscale.

1.9.1 Jump to a quiz

- *Fall 2025 Quiz 1 — Probability and rigid camera poses*
- *Fall 2017 Quiz 9 — Depth maps and tetrahedron normals*

1.9.2 Fall 2025 Quiz 1 — Probability and rigid camera poses

Historical source: `3dcvMid/quizzesf25sol/cs532f25_quiz_1_solution.pdf`.

Question 1

Original problem · quiz booklet

A parlay succeeds only if all three independent fair coin flips succeed. What is the chance of winning one parlay?

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Identify independent events

Why: Independent outcomes let us multiply their probabilities.

Each fair flip succeeds with probability $1/2$. The three flip outcomes are independent.

Step 2 — Multiply the three chances

Why: A parlay wins only when every flip wins.

Multiply probabilities for the joint event: $P(\text{all } 3) = (1/2)^3 = 1/8$.

Step 3 — Interpret the probability

Why: A fraction and a percentage describe the same chance.

$\boxed{P(\text{win}) = 1/8 = 12.5\%}$. This is the same product rule used for all-inlier RANSAC samples.

Question 2

Original problem · quiz booklet

You attempt one three-coin-flip parlay each day for seven independent days. What is the probability of winning at least one?

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Find one day's failure chance

Why: Counting no wins is easier than counting every possible pattern of wins.

One parlay win probability is $q = 1/8$. Its loss probability is $1 - q = 7/8$.

Step 2 — Multiply seven failures

Why: The daily attempts are assumed independent.

All seven attempts fail with probability $P(0 \text{ wins}) = (7/8)^7$.

Step 3 — Take the complement

Why: At least one win and no wins are complementary events.

Use the complement of no wins:

$$P(\geq 1 \text{ win}) = 1 - (7/8)^7 = 1 - 823543/2097152.$$

Step 4 — Convert to a percentage

Why: The result should be a probability between zero and one.

Therefore $\boxed{P(\geq 1 \text{ win}) \approx 0.607304 \approx 60.73\%}$.

Question 3

Original problem · quiz booklet

Two static cameras concurrently estimate the 3D rigid pose of the same object, as $[R_1, t_1]$ and $[R_2, t_2]$ in their own camera frames. Can one recover the relative camera pose? Explain and derive a transformation.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Define the coordinate maps

Why: Matrix order is meaningful only after we know what each map does.

Define the direction before multiplying. For $i = 1, 2$, the object-to-camera pose is

$$T_i = \begin{pmatrix} R_i & t_i \\ 0 & 1 \end{pmatrix}, \qquad X_{C_i} = T_i X_O.$$

Here R_i is 3×3, t_i is 3×1, and T_i is 4×4. The same physical object frame, time and length scale must be used in both estimates.

Step 2 — Invert camera 2's object pose

Why: We must return to the shared object frame first.

Undo the second camera's map to return to object coordinates:

$$X_O = T_2^{-1} X_{C_2}, \qquad T_2^{-1} = \begin{pmatrix} R_2^T & -R_2^T t_2 \\ 0 & 1 \end{pmatrix}.$$

A rotation's inverse is its transpose, and its translation must be undone in the rotated frame.

Step 3 — Compose the frame path

Why: The object frame connects the two camera frames.

Follow the frame path camera 2 → object → camera 1:

$$X_{C_1} = T_1(T_2^{-1} X_{C_2}) = (T_1 T_2^{-1}) X_{C_2}.$$

The rightmost matrix acts first; reversing the product changes the mapping direction.

Step 4 — Multiply the block matrices

Why: This exposes the recovered rotation and translation separately.

Multiply the complete block matrices:

$$\begin{aligned} T_{C_1 \leftarrow C_2} &= \begin{pmatrix} R_1 & t_1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} R_2^T & -R_2^T t_2 \\ 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} R_1 R_2^T & t_1 - R_1 R_2^T t_2 \\ 0 & 1 \end{pmatrix}. \end{aligned}$$

Thus $R_{12} = R_1 R_2^T$ and $t_{12} = t_1 - R_1 R_2^T t_2$.

Step 5 — Insert a numerical example

Why: Full matrices make the abstract composition concrete.

Original numerical companion, not extra quiz givens: let $R_1 = R_2 = I$, $t_1 = (1, 1, 4)^T$, $t_2 = (-1, 1, 4)^T$. Then

$$T_1 = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 4 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad T_2^{-1} = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -4 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Their product is

$$T_{12} = \begin{pmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Step 6 — Check the object origin

Why: A known point tests both the matrix order and translation sign.

Check the direction on the observed object origin:

$$\begin{pmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} -1 \\ 1 \\ 4 \\ 1 \end{pmatrix} = \begin{pmatrix} -1+2 \\ 1 \\ 4 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 4 \\ 1 \end{pmatrix}.$$

This is exactly camera 1's observation t_1. Independent unknown pose scales or inconsistent object-frame orientations prevent this metric recovery.

Source check / assumptions: The uploaded key's superscript/subscript arrows are inconsistent in places. The formula above is unambiguous because its mapping direction is defined explicitly. If the poses are only known up to unknown scale, relative metric translation is not determined absolutely.

Question 4

Original problem · quiz booklet

Can one determine the 3D plane passing through both camera centers and the tracked object's center? Derive it.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Choose one working frame

Why: Points can share a plane equation only when their coordinates use the same frame.

Use camera-1 coordinates as the world frame: $C_1 = (0,0,0)^T$. The object-frame origin is observed at $O = t_1$.

Step 2 — Locate camera 2

Why: Its origin maps to the translation column of the relative camera pose.

Find camera-2 center C_2 as the translation part of the transform ${}^{C_1}T_{C_2}$ from Question 3.

Step 3 — Cross two in-plane vectors

Why: The cross product is perpendicular to both directions.

Two vectors lying in the plane are $a = O - C_1$ and $b = C_2 - C_1$. Take the cross product to find a plane normal: $\boxed{n = a \times b}$.

Step 4 — Write the plane equation

Why: Every displacement inside the plane is perpendicular to its normal.

The plane equation is $n^T(X - C_1) = 0$ or $n^T X + d = 0$ with $d = -n^T C_1$. Because $C_1 = 0$, $\boxed{n^T X = 0}$.

Step 5 — Check for collinearity

Why: Three collinear points do not determine a unique plane.

Require $n \neq 0$, i.e. the three points are noncollinear. If cameras and object center lie on one line, infinitely many planes contain them. Draw a triangle joining C_1, C_2, O with a perpendicular arrow n.

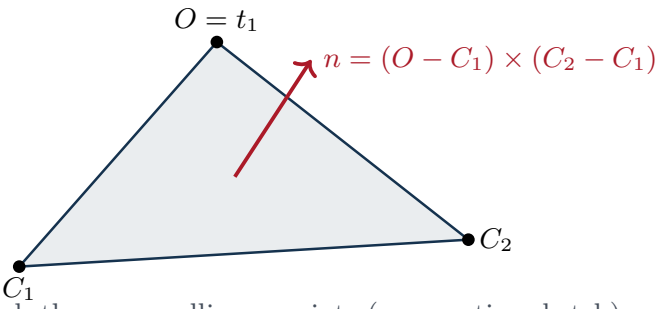
Step 6 — Expand and verify a numerical plane

Why: Substitution checks that all three required points actually lie on the answer.

Original numerical companion: the preceding question gave $C_1 = (0,0,0)^T$, $C_2 = (2,0,0)^T$, $O = (1,1,4)^T$. Expand the cross product:

$$n = \begin{pmatrix}1\\1\\4\end{pmatrix} \times \begin{pmatrix}2\\0\\0\end{pmatrix} = \begin{pmatrix}1(0) - 4(0)\\4(2) - 1(0)\\1(0) - 1(2)\end{pmatrix} = \begin{pmatrix}0\\8\\-2\end{pmatrix} \sim \begin{pmatrix}0\\4\\-1\end{pmatrix}.$$

The plane is $4Y - Z = 0$. Substitution gives 0 for both camera centers and $4(1) - 4 = 0$ for the object origin. An extended object need not lie entirely on this plane.



Plane through three noncollinear points (perspective sketch)

Fig. 1.18: The two camera centers and object origin determine a plane when noncollinear. Schematic from the supplied quiz booklet.

1.9.3 Fall 2017 Quiz 9 — Depth maps and tetrahedron normals

Historical source: `3dcvMid/olderquizzes/cs532_f17_quiz9.pdf`.

Problem 1

Original problem · quiz booklet

Explain how to convert two depth maps into a single 3D point cloud in a common reference frame. What information is required? Make a sketch.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Collect calibration and poses

Why: Depth alone does not identify a point's coordinates in a shared world frame.

For camera $i \in \{1, 2\}$ obtain intrinsic matrix K_i, per-pixel depth $Z_i(u, v)$, and a known camera-to-world rigid transform

$$ {}^{W}T_{C_i} = \begin{bmatrix} R_i & t_i \\ 0 & 1 \end{bmatrix} $$

. First correct lens distortion if depth is reported in distorted coordinates.

Step 2 — Backproject each depth pixel

Why: Intrinsics turn a pixel into a viewing ray; axial depth sets its scale.

For pixel $p_i = (u, v, 1)^T$, compute normalized camera ray $r_i = K_i^{-1} p_i$, whose last component is 1 under standard intrinsics. If depth means camera-axis Z, backproject: $\boxed{X_{C_i} = Z_i(u, v)\, K_i^{-1} p_i}$.

Step 3 — Expand the camera coordinates

Why: This makes the pixel offsets and focal-length divisions explicit.

For standard intrinsics with zero skew this becomes $X_{C_i} = ((u - c_x)Z/f_x, (v - c_y)Z/f_y, Z)^T$. Skip missing, zero, invalid, or occluded depth pixels.

Step 4 — Transform both clouds into one frame

Why: Coordinates from different cameras cannot be merged directly.

Map every 3D point into the *same* world frame: $\boxed{X_W = R_i X_{C_i} + t_i}$. Combine both transformed sets; optional voxel fusion/outlier removal prevents duplicates or inconsistent observations.

Step 5 — Check the meaning of depth

Why: Range along a ray and depth along the optical axis require different scaling.

If depth means *Euclidean range along ray*, use $X_{C_i} = \rho_i r_i / \|r_i\|$, not $Z_i r_i$. The question needs a depth convention, the two sets of intrinsics, and the relative/world poses.

Step 6 — Multiply a complete numerical example

Why: Both camera observations should land on the same world point.

Original numerical companion: use

$$K = \begin{pmatrix} 500 & 0 & 250 \\ 0 & 500 & 250 \\ 0 & 0 & 1 \end{pmatrix}, \quad K^{-1} = \begin{pmatrix} 1/500 & 0 & -1/2 \\ 0 & 1/500 & -1/2 \\ 0 & 0 & 1 \end{pmatrix}.$$

Both axial depths are 10. Backproject each measurement explicitly:

$$X_{C_1} = 10K^{-1} \begin{pmatrix} 300 \\ 250 \\ 1 \end{pmatrix} = 10 \begin{pmatrix} 300/500 - 1/2 \\ 250/500 - 1/2 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 10 \end{pmatrix},$$

$$X_{C_2} = 10K^{-1} \begin{pmatrix} 200 \\ 250 \\ 1 \end{pmatrix} = 10 \begin{pmatrix} 200/500 - 1/2 \\ 250/500 - 1/2 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \\ 10 \end{pmatrix}.$$

Camera 2 is at world $(2, 0, 0)$ with identity orientation. Its camera-to-world map gives

$$\begin{pmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} -1 \\ 0 \\ 10 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 10 \\ 1 \end{pmatrix}.$$

The two observations now represent the same world point.

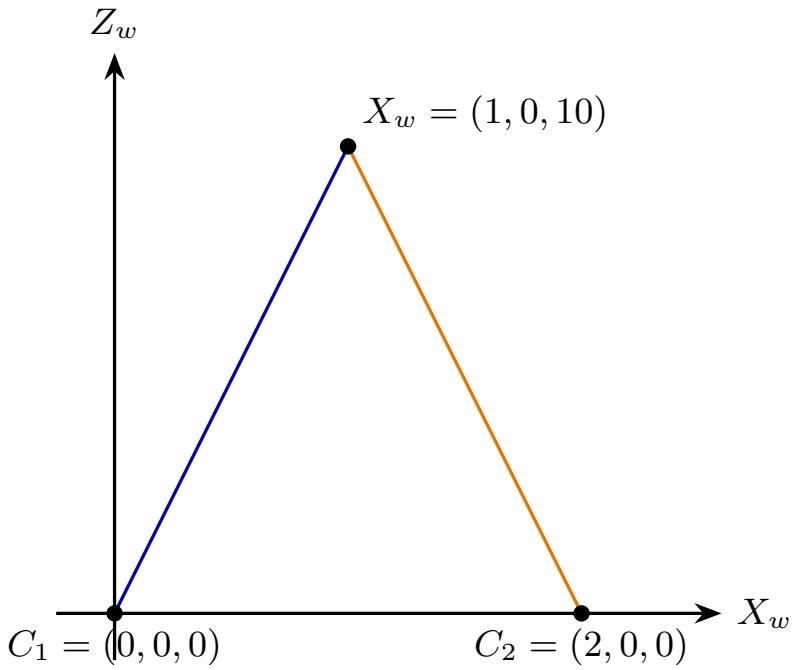


Fig. 1.19: Two camera rays meet at the same point after both depth maps are transformed into one frame.

Problem 2(a)

Original problem · quiz booklet

A tetrahedron has vertices $v_1 = (1, 1, 0)$, $v_2 = (1, 2, 0)$, $v_3 = (1, 1, 1)$, $v_4 = (2, 1, 0)$. Find one *outward* surface normal per face, and sketch the result.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Choose faces and an orientation rule

Why: A cross product gives two possible normal directions depending on edge order.

Use the four faces 123, 124, 134 and 234. For each face take two edge vectors and cross them. Choose outward unit normals for a consistent answer; the opposite vertex lies on the inward side. A candidate n is outward when $n^T(v_{opposite} - v_{face}) < 0$.

Step 2 — Compute face 123

Why: The opposite vertex tells us whether to flip the candidate normal.

Face 123: subtract the vertices and expand:

$$a = v_2 - v_1 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad b = v_3 - v_1 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix},$$

$$a \times b = \begin{pmatrix} 1(1) - 0(0) \\ 0(0) - 0(1) \\ 0(0) - 1(0) \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}.$$

The opposite vertex is toward $+X$, so this candidate points inward. Flip it: $n_{123} = (-1, 0, 0)^T$.

Step 3 — Compute face 124

Why: The cross product must point away from the tetrahedron's interior.

Face 124 uses $a = (0, 1, 0)^T$ and $b = v_4 - v_1 = (1, 0, 0)^T$:

$$a \times b = \begin{pmatrix} 1(0) - 0(0) \\ 0(1) - 0(0) \\ 0(0) - 1(1) \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix}.$$

It points away from the opposite vertex above $Z = 0$, so $n_{124} = (0, 0, -1)^T$.

Step 4 — Compute face 134

Why: A candidate pointing toward the opposite vertex is inward.

Face 134 uses $a = (0, 0, 1)^T$, $b = (1, 0, 0)^T$:

$$a \times b = \begin{pmatrix} 0(0) - 1(0) \\ 1(1) - 0(0) \\ 0(0) - 0(1) \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}.$$

The opposite vertex lies toward $+Y$, so flip this inward candidate: $n_{134} = (0, -1, 0)^T$.

Step 5 — Compute face 234

Why: The slanted face also needs normalization to unit length.

Face 234 uses $a = v_3 - v_2 = (0, -1, 1)^T$, $b = v_4 - v_2 = (1, -1, 0)^T$:

$$a \times b = \begin{pmatrix} (-1)(0) - 1(-1) \\ 1(1) - 0(0) \\ 0(-1) - (-1)(1) \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}.$$

Its dot product with $v_1 - v_2 = (0, -1, 0)^T$ is -1, so it already points outward. Divide by $\sqrt{1+1+1} = \sqrt{3}$: $n_{234} = (1, 1, 1)^T / \sqrt{3}$.

Step 6 — Collect and check all normals

Why: Unit length, perpendicularity and outward direction are separate checks.

The four answers are

$$n_{123} = \begin{pmatrix} -1 \\ 0 \\ 0 \end{pmatrix}, \quad n_{124} = \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix}, \quad n_{134} = \begin{pmatrix} 0 \\ -1 \\ 0 \end{pmatrix}, \quad n_{234} = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}.$$

Each has unit length and zero dot product with both edges of its own face. Draw arrows outward from the four face interiors.

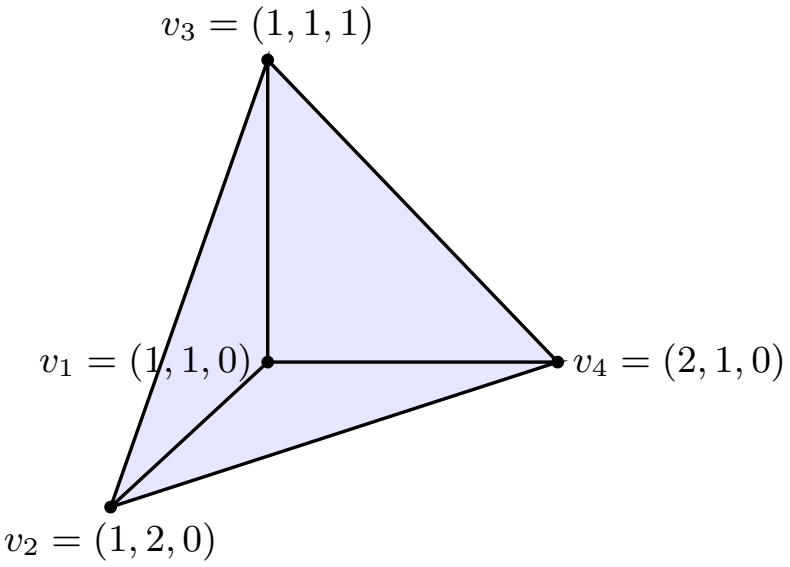


Fig. 1.20: The tetrahedron from the problem, with all four given vertices.

Problem 2(b)

Original problem · quiz booklet

Rotate the complete tetrahedron by $+90^\circ$ about the global Z axis. Find the modified face normals.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Build the rotation matrix

Why: The active right-handed convention fixes the signs.

For an *active*, right-handed $+90^\circ$ rotation,

$$R_z(90^\circ) = \begin{bmatrix} \cos 90^\circ & -\sin 90^\circ & 0 \\ \sin 90^\circ & \cos 90^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Step 2 — Apply the normal transformation rule

Why: A rigid rotation rotates a normal without adding translation.

Pure rigid rotations rotate normals by the same matrix: $n'_{ijk} = R_z n_{ijk}$. (General nonrigid affine transforms use an inverse-transpose rule instead.)

Step 3 — Multiply the three axis normals

Why: Writing every matrix-vector product exposes the changed directions.

Multiply the rotation by each axis-aligned normal:

$$n'_{123} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} -1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ -1 \\ 0 \end{pmatrix},$$

$$n'_{124} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix},$$

$$n'_{134} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ -1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}.$$

Step 4 — Multiply the slanted normal

Why: Its normalization must be preserved too.

For the slanted face, retain the normalization:

$$n'_{234} = \frac{1}{\sqrt{3}} \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{3}} \begin{pmatrix} 0(1) - 1(1) + 0(1) \\ 1(1) + 0(1) + 0(1) \\ 0(1) + 0(1) + 1(1) \end{pmatrix} = \frac{1}{\sqrt{3}} \begin{pmatrix} -1 \\ 1 \\ 1 \end{pmatrix}.$$

Step 5 — Check geometric consistency

Why: A rotation preserves lengths and perpendicularity.

Check the rotation rather than adding translation: $R_z^T R_z = I$ preserves unit lengths and dot products. Rotating the vertices and recomputing their outward normals gives these same answers. Under a general nonrigid affine transform, normals instead use an inverse transpose.

1.10 Review: choose the right geometric operation

The most useful formula is the one whose inputs, coordinate conventions, and assumptions match the problem. Before calculating, say what geometric object you have and what you want to recover.

1.10.1 Formula and source checklist

The PDF-page ranges refer to `wk1_2_cs532s26_IntroToGeometry.pdf`. Equation links lead to the canonical formula boxes and application procedures.

Topic / operation	Formula or explanation	Source PDF pages
Motivation, reconstruction pipeline, ambiguity, application examples	*Why reconstruct a world from photographs?*	2–61
Homogeneous scale, finite/ideal points	*Points, lines, and the meaning of infinity* and (1)	65–71
Point–line incidence	(1.1)	65
Joins and intersections	(1.2)	67
Parallel lines and infinity	(1.3)	68–71
2D rotation and pivot transforms	(1.4), (1.5)	72–77
Transformation families and invariants	(1.6)	78, 103
Point, line, and plane transforms	(1.7)	79, 81–82
Planes, distance, three-point construction	(1.8), (1.9)	82–84
Point/plane nullspaces and cofactors	(1.10), (1.11)	83–85
Plücker representation and line distances	(1.12), (1.13)	86–87
Axis rotations and inverse	(1.14), (1.15)	88–90
Ordered angles and fixed/body axes	(1.16)	93–100
Axis–angle and computational supplement	(1.17), (1.18)	91
Quaternion operations and conventions	(1.19)	92
Rigid transforms and frame composition	(1.20), (1.21)	101–102
Virtual-plane projection	(1.23)	105–110
Extrinsics, intrinsics, projection matrix	(1.24)	111–113
Camera-motion hands-on task	*Example 1.15*	114
Depth-map backprojection: quiz extension	(1.25)	Quiz, not a slide formula

The workflow previews on PDF pages 25–26 point ahead to *epipolar constraints* and *stereo depth*, where their variables and assumptions are explained.

The administrative title, policies, logistics, and reading-list slides are recorded in the coverage ledger but do not become mathematical exercises. The rank/SVD and Rodrigues explanations fill explanatory steps needed to understand the lecture; they are identified as supplements where they go beyond its printed formulas.

1.10.2 Rework the assigned quizzes without the solutions

Source	Location	What to check
Fall 2025 Quiz 1, Q3	*Example 1.10*	Frame path, inversion, shared object frame/scale/time
Fall 2025 Quiz 1, Q4	*Example 1.11*	Camera centers in one frame; noncollinearity
Fall 2017 Quiz 9, P2(a)	*Example 1.12*	Cross products, orientation, unit normalization
Fall 2017 Quiz 9, P2(b)	*Example 1.13*	Declared active rotation; normals remain perpendicular
Fall 2017 Quiz 9, P1	*Example 1.16*	Supplement: calibration, axial depth/range, common frame

Fall 2025 Quiz 1, Q1–Q2, remains assigned to Lecture 3's RANSAC/probability explanation. The Spring 2020 homogeneous-geometry quiz remains in Lecture 2; use its questions later as a transfer test rather than counting them as newly introduced Lecture 1 quiz material.

1.10.3 Common mistakes to catch before you calculate

- Dividing a homogeneous coordinate by zero or treating the zero vector as a point.
- Calling an image vanishing point an ideal image point without checking its coordinates.
- Rotating about the origin when a different pivot was specified.
- Applying the point matrix directly to line/plane coefficients.
- Inferring a unique plane from collinear points.
- Treating every length-preserving matrix as a proper rotation.
- Using ordered-angle names without declaring axes and multiplication order.
- Mixing active/passive quaternion sandwiches.
- Negating a translation without rotating it when inverting a rigid transform.
- Treating camera pose and world-to-camera extrinsics as identical arrays.
- Assuming image-plane length, pixel coordinates, axial depth, and range use interchangeable units.

1.10.4 A short transfer problem

Exercise 1.1 — combine frames, planes, and projection

A world point is $X = (2, 1, 6)^T$. A camera has center $C = (1, 0, 2)^T$, identity orientation, $f_x = f_y = 400$ pixels, zero skew, and principal point $(200, 150)$.

1. Write the world-to-camera translation and camera coordinates of X.
2. Project X to pixels.
3. Find a plane through the camera center, the world origin, and X.
4. State when the plane-construction step would fail.

Solution

1. $t = -C = (-1, 0, -2)^T$, so $X_c = (1, 1, 4)^T$.
2. $u = 200 + 400(1/4) = 300$ and $v = 150 + 400(1/4) = 250$ pixels.
3. Use the world origin as anchor: $C \times X = (-2, -2, 1)^T$. Thus $-2x - 2y + z = 0$. Both C and X satisfy it; the origin does too.
4. It fails to determine a unique plane if the origin, camera center, and point are collinear, or if repeated points remove an independent direction.

You are ready to move on when you can explain the geometry before choosing a formula, solve these examples without copying their steps, and state each answer's assumptions and degeneracies. Lecture 2 will revisit the projective and rotation language and extend it toward image transformations and estimation.

Finish with the *TLDR and short quiz* for a quick self-check.

1.11 TLDR

Lecture 1 in one sentence: images lose depth, so reconstructing a scene requires geometry, coordinate frames, and careful use of multiple measurements.

1.11.1 The big picture

The reconstruction pipeline is **match features → estimate cameras and 3D points → estimate dense depth → build a representation**. A convincing picture is not proof of correct geometry: lighting, occlusion, repeated patterns, and noise can mislead us.

1.11.2 The rules worth remembering

1. **Homogeneous coordinates ignore overall scale.** $(2, 4, 2)^T$ and $(1, 2, 1)^T$ represent the same point. Divide by the last coordinate to get ordinary coordinates only when it is nonzero. A zero last coordinate means an ideal point; the all-zero vector is invalid. A scene direction can project to a finite image vanishing point. See *Points, lines, and the meaning of infinity*.

2. **Dot products test incidence; cross products construct geometry.** A point lies on a line when $\ell^T x = 0$. Join two distinct image points with $x \times y$; intersect two distinct image lines with $\ell_1 \times \ell_2$. Parallel lines meet at an ideal point. See (1.2).

3. **Choose the transformation you actually need.** Rigid motion preserves distances and angles; similarity adds uniform scale; affine maps preserve parallelism; projective maps preserve incidence but can change parallelism. Points transform with H; line and plane coefficients use its inverse transpose. To rotate about a pivot, **subtract the pivot → rotate → add it back**. See *Move points without losing the geometry*.

4. **Three noncollinear points determine a plane.** Use $n = (B - A) \times (D - A)$ and $d = -n^T A$ in $n^T p + d = 0$ for a point p. Collinear points give no unique plane. A unit normal gives orientation; its sign depends on your chosen inward/outward convention. Under a rigid rotation, normals rotate without translation. See *Points, planes, and lines in 3D*.

5. **A rotation is more than a square matrix.** A proper rotation satisfies $R^T R = I$ and $\det R = +1$, so $R^{-1} = R^T$. Rotation order matters. Matrices, ordered angles, axis–angle, and unit quaternions describe rotations, but their conventions must agree. Here active quaternion rotation is qPq^{-1}. See *Rotations: several descriptions of the same motion*.

6. **Label every coordinate frame.** For $p_B = Rp_A + t$, undo the motion with $p_A = R^T(p_B - t)$. In a matrix product the rightmost transform acts first. To go from camera 2 through a shared object to camera 1, use $T_{C_1 \leftarrow O}(T_{C_2 \leftarrow O})^{-1}$. See *Reference frames and two worked quiz problems*.

7. **Projection divides by depth.** In camera coordinates, normalized image coordinates are $(X_c/Z_c, Y_c/Z_c)$. Intrinsics convert these to pixels; extrinsics convert world coordinates to camera coordinates. Together,

$x \sim K[R \mid t]X_w$. Require positive depth for a point in front, then check image bounds separately. See *Image formation: where does a 3D point appear?*.

8. **Camera motion changes the point's camera coordinates.** Moving the camera forward by Δ changes a stationary point's depth to $Z_c - \Delta$. Off-axis points move farther from the principal point while they remain in front. Moving the object farther away does the opposite. Backprojection needs calibration and a specified depth convention: axial depth and distance along the ray are different. See *Example 1.15*.

Also recognize: a nullspace solves homogeneous incidence equations; a finite Plücker line stores direction d and moment $p \times d$. The *formula checklist* links to these less frequently used constructions and their conditions.

Before calculating: name the frames, check units, choose the formula, and test whether the answer makes geometric sense.

1.11.3 Short quiz

Try these without looking at the answers. These are original practice questions based on Lecture 1; the assigned course quizzes are worked through in *Reference frames and two worked quiz problems*.

Each question has its own complete worked answer below. Open it after trying the problem; all answers are visible in print.

Question 1

Original practice question

Depth: why can one image pixel correspond to many 3D points?

Worked solution

Step 1 — Write a camera ray

Why: A pixel fixes a direction but leaves its distance unknown.

For a calibrated pixel $p = (u, v, 1)^T$, let $r = K^{-1}p$. Points on its ray have $X_C = Zr$ when $r_3 = 1$ and depth is axial.

Step 2 — Check two depths

Why: Perspective division cancels the common ray scale.

For normalized direction $r = (a, b, 1)^T$, both points $(a, b, 1)^T$ and $(2a, 2b, 2)^T$ project to (a, b). Thus one pixel can represent many positive depths.

Question 2

Original practice question

Coordinates: what ordinary point does $(6, 9, 3)^T$ represent? Can you similarly normalize $(1, 0, 0)^T$?

Worked solution

Step 1 — Normalize the finite point

Why: Ordinary coordinates are ratios to the final homogeneous coordinate.

$$\begin{pmatrix} 6 \\ 9 \\ 3 \end{pmatrix} /3 = \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix}, \qquad (u, v) = (2, 3).$$

Step 2 — Check the other denominator

Why: A zero final coordinate cannot be divided out.

The nonzero vector $(1, 0, 0)^T$ is an ideal point. It has no finite ordinary coordinate pair.

Question 3

Original practice question

Intersection: find the homogeneous intersection of $x = 2$ and $y = 3$, using line vectors $(1, 0, -2)^T$ and $(0, 1, -3)^T$.

Worked solution

Step 1 — Expand the line cross product

Why: The cross product intersects two homogeneous image lines.

$$\begin{pmatrix} 1 \\ 0 \\ -2 \end{pmatrix} \times \begin{pmatrix} 0 \\ 1 \\ -3 \end{pmatrix} = \begin{pmatrix} 0(-3) - (-2)(1) \\ (-2)(0) - 1(-3) \\ 1(1) - 0(0) \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix}.$$

Step 2 — Verify both incidences

Why: The answer must lie on both original lines.

$$\begin{pmatrix} 1 & 0 & -2 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix} = 2 - 2 = 0, \qquad \begin{pmatrix} 0 & 1 & -3 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix} = 3 - 3 = 0.$$

The ordinary intersection is $(2, 3)$.

Question 4

Original practice question

Pivot: rotate $(3, 1)$ by $+90^\circ$ about $(2, 1)$ in Cartesian axes with y upward. Where does it land?

Worked solution

Step 1 — Rotate the offset

Why: Rotating about a pivot requires coordinates relative to that pivot.

$$p - c = \begin{pmatrix} 3 \\ 1 \end{pmatrix} - \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad R(p-c) = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Step 2 — Add the pivot back

Why: The final answer must use the original coordinate frame.

$$p' = \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}.$$

Question 5

Original practice question

Plane: find a normal and plane equation through $(0, 0, 0)$, $(1, 0, 0)$, and $(0, 1, 0)$. When would the three-point method fail?

Worked solution

Step 1 — Cross the two edges

Why: Their cross product is perpendicular to the desired plane.

$$n = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \times \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

Step 2 — Set the plane offset and check uniqueness

Why: The given origin lies on the plane, fixing its offset to zero.

$$d = -n^T \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} = 0, \qquad n^T \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} + d = Z = 0.$$

Collinear or repeated inputs give a zero edge cross product and no unique plane. Negating the normal describes this same unoriented plane.

Question 6

Original practice question

Rotation: a matrix satisfies $R^T R = I$ but has determinant -1. Is it a proper rotation?

Worked solution

Step 1 — Check orthogonality

Why: The first condition preserves lengths and angles.

$R^T R = I$ makes R orthogonal. Orthogonal matrices can have determinant $+1$ or -1.

Step 2 — Check orientation

Why: A proper rotation must preserve handedness too.

The supplied $\det R = -1$ fails the condition $\det R = +1$. Therefore this is not a proper rotation; it includes an orientation reversal.

Question 7

Original practice question

Frames: two object-to-camera poses have identity rotations and translations $t_1 = (1, 1, 4)^T$, $t_2 = (-1, 1, 4)^T$. What is the translation from camera 2 to camera 1?

Worked solution

Step 1 — Subtract the translations

Why: Both rotations are identity, so the general relative-pose formula simplifies.

$$t_{12} = t_1 - t_2 = \begin{pmatrix} 1 \\ 1 \\ 4 \end{pmatrix} - \begin{pmatrix} -1 \\ 1 \\ 4 \end{pmatrix} = \begin{pmatrix} 2 \\ 0 \\ 0 \end{pmatrix}.$$

Step 2 — Test the observed origin

Why: The translation should map camera 2's object coordinates into camera 1's.

$$\begin{pmatrix} -1 \\ 1 \\ 4 \end{pmatrix} + \begin{pmatrix} 2 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 4 \end{pmatrix} = t_1.$$

Question 8

Original practice question

Camera: use zero skew, $f_x = f_y = 100$ pixels, and principal point $(0, 0)$. Project camera-coordinate point $(2, 1, 5)^T$. Then move the camera one unit forward without rotating it while the point stays fixed. What is the new pixel?

Worked solution

Step 1 — Project the initial point

Why: Intrinsics multiply the camera point before division by depth.

$$q = \begin{pmatrix} 100 & 0 & 0 \\ 0 & 100 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \\ 5 \end{pmatrix} = \begin{pmatrix} 200 \\ 100 \\ 5 \end{pmatrix}, \qquad (u, v) = (200/5, 100/5) = (40, 20).$$

Step 2 — Update camera coordinates and reproject

Why: A forward camera motion reduces a stationary point's camera depth.

$$X'_C = \begin{pmatrix} 2 \\ 1 \\ 5 \end{pmatrix} - \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 4 \end{pmatrix}, \quad q' = \begin{pmatrix} 100 & 0 & 0 \\ 0 & 100 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \\ 4 \end{pmatrix} = \begin{pmatrix} 200 \\ 100 \\ 4 \end{pmatrix}.$$

Divide by four: the new pixel is $(50, 25)$.

CHAPTER

TWO

LECTURE 2: PROJECTIVE GEOMETRY AND IMAGE WARPING

Original quiz questions + fully worked solutions — complete problem statements, numbered red steps with a plain-English reason, and blue mathematics showing intermediate operations.

How can a tilted photograph become a view from above? The basketball-court task turns geometry into a practical tool: find a mapping, then use it to create a new image.

This chapter teaches the material added beyond Lecture 1: quaternion conversions and interpolation, planar rectification, image sampling, robust estimation, lens correction, and two-view constraints.

Prerequisites: *Lecture 1* owns homogeneous geometry, transformation families, rotations, frames, and camera projection. The source slides repeat those topics; the textbook links to their original explanations. Spring 2020 Quiz 1 remains attributed to Lecture 2 in the *quiz index*, with its foundation solutions collected in *Lecture 1*.

2.1 Reading route

1. *Quaternion conversions*: convert rotation descriptions and interpolate orientations.
2. *Homographies*: straighten a photographed plane and calculate cross-ratios.
3. *Image warping*: sample the source image to fill a destination grid.
4. *Estimate a reliable mapping*: DLT, normalization, and RANSAC.
5. *Distortion and two-view geometry*: handle lens bending and constrain matches across views.
6. *Review and formula checklist*, then the final *TLDR and short quiz*.

Source: `cs532_f25_Week2.pdf`, 129 PDF pages. Printed labels equal PDF pages through page 43; pages 44–49 have no visible label; from PDF page 50 onward, printed labels are one greater. Notes below use **PDF pages** unless stated otherwise. The authoring coverage register records which repeated slides are already covered in Lecture 1.

2.2 Quaternions: convert without changing the rotation

Prerequisite: (1.19) defines the quaternion convention. This section adds conversion and interpolation.

Source: PDF pages 42–49. Page 48 uses the transpose/passive matrix relative to our convention. Page 49 omits a necessary $+1$ inside a square root. The formulas below use one consistent convention; see the *Errata and assumptions*.

2.2.1 From a quaternion to a matrix

Write the scalar-first unit quaternion as $q = (w, x, y, z)$. Collecting the point-coordinate coefficients in Lecture 1's quaternion rotation gives the matrix below. Here x, y, z name quaternion components.

Formula — quaternion to active rotation matrix

$$R(q) = \begin{pmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{pmatrix}. \tag{2.1}$$

q has four unitless entries; R is 3×3. This form assumes **unit length**.

Turn an orientation into a matrix

1. Reject a zero quaternion; otherwise normalize it.
2. Substitute its scalar-first components.
3. Check $R^T R = I$, $\det R = 1$, and a familiar vector.

Use Example 2.1 to check the signs.

2.2.2 From a matrix back to a quaternion

Taking the trace gives $\operatorname{tr} R = 4w^2 - 1$. That explains the square root and the missing $+1$ in the slide.

Formula — matrix to quaternion, trace branch

$$\begin{aligned} w &= \tfrac{1}{2}\sqrt{1 + \operatorname{tr} R}, \\ (x, y, z) &= \frac{(R_{32} - R_{23},\ R_{13} - R_{31},\ R_{21} - R_{12})}{4w}. \end{aligned} \tag{2.2}$$

Indices are one-based. R must be a proper active rotation. This branch chooses $w \geq 0$ and requires w safely away from zero. Near a half-turn, division by w is unstable.

Use the largest vector component instead. For example, when x is largest,

$$x = \tfrac{1}{2}\sqrt{1 + R_{11} - R_{22} - R_{33}}, \quad (w, y, z) = \frac{(R_{32} - R_{23},\ R_{12} + R_{21},\ R_{13} + R_{31})}{4x}.$$

The y and z branches follow by cyclic permutation. Normalize the result; the overall sign is arbitrary.

Recover an orientation safely

1. Validate the matrix as a proper rotation.
2. Use the trace branch away from a half-turn; near a half-turn, use the largest vector-component branch.
3. Normalize and rebuild $R(q)$; compare matrices rather than quaternion signs.

See Example 2.1. Clamp small rounding errors only after validating the input.

Worked example 2.1 — cycle the axes, then test a half-turn

Source: original numerical companions to PDF pages 48–49.

For 120° about $a = (1,1,1)^T/\sqrt{3}$, the quaternion is $q = (1/2, 1/2, 1/2, 1/2)$. Substitution gives $R = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$. It cycles the coordinate axes: $e_x \to e_y \to e_z \to e_x$.

The trace is zero. Recover $w = 1/2$ and $x = y = z = 1/(4w) = 1/2$. Omitting $+1$ would give $w = 0$ and make the remaining divisions fail. This tests a different rotation from Lecture 1's quarter-turn.

For $R = \mathrm{diag}(1, -1, -1)$, the trace branch gives $w = 0$, so do not divide by it. The largest-component branch gives $x = 1$, $w = y = z = 0$: a 180° turn about x.

2.2.3 Interpolation

The slides mention smooth interpolation. **SLERP** (spherical linear interpolation) follows the unit-quaternion sphere instead of independently changing Euler angles:

Formula — interpolate two orientations

$$q(t) = \frac{\sin((1-t)\Omega)}{\sin\Omega} q_a + \frac{\sin(t\Omega)}{\sin\Omega} q_b, \qquad \cos\Omega = q_a^T q_b, \quad 0 \le t \le 1. \tag{2.3}$$

This is an explanatory companion to PDF page 45, not a printed formula. Both endpoints are unit 4-vectors. If their dot product is negative, negate one endpoint first to select a shortest rotation path. Constant change in t gives constant angular speed. For nearly equal endpoints, use normalized linear interpolation to avoid a small $\sin\Omega$ denominator.

Find an in-between orientation

1. Normalize endpoints, align signs, and clamp their computed dot product to $[-1, 1]$.
2. Calculate Ω and the weights, or use the near-equal fallback.
3. Normalize and check the endpoints at $t = 0, 1$.

Between identity $(1,0,0,0)$ and a 90° z rotation, $t = 1/2$ gives $(\cos 22.5^\circ, 0, 0, \sin 22.5^\circ)$: a 45° rotation.

Check 2.1 — do opposite quaternions cancel?

What happens if you average q and $-q$ directly?

Answer

You get the zero vector, which cannot represent an orientation. Align signs first; both endpoints describe the same rotation.

2.3 Homographies: straighten a photographed plane

A basketball court is rectangular, but its photograph is not. A ***homography*** maps one projective plane to another, letting us turn the photographed floor into a rectangle. It also preserves straight lines, so the court markings stay straight.

Source: PDF pages 62–71, printed slides 63–72. The example below develops the actual court-warping assignment.

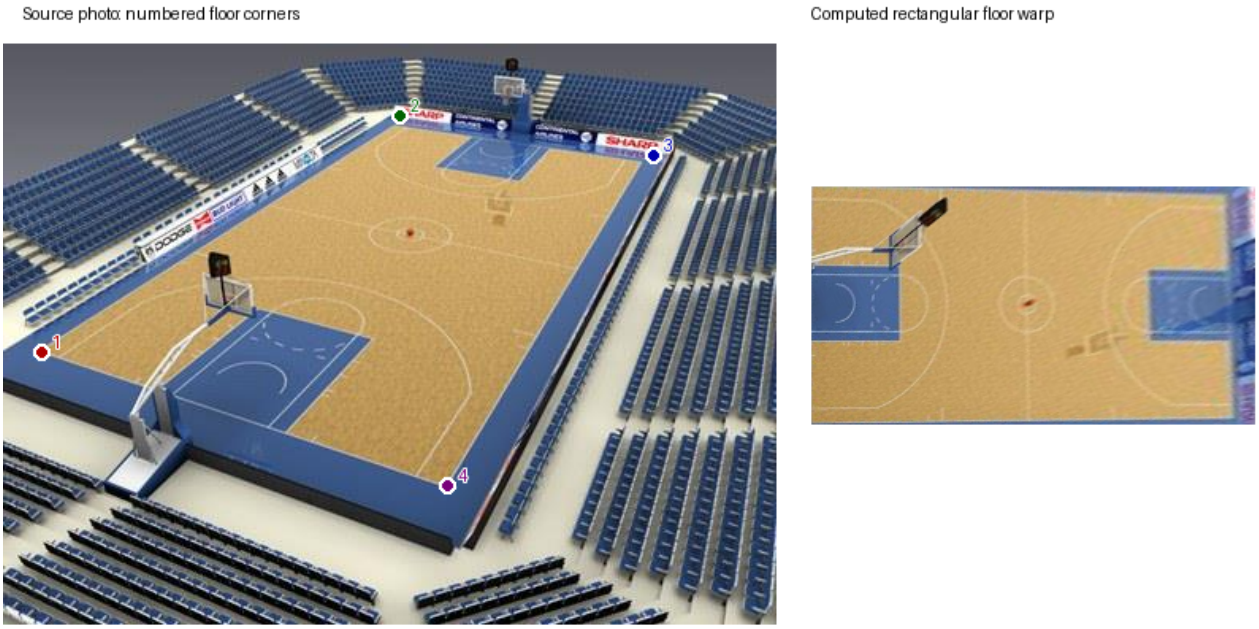


Fig. 2.1: Left: the lecture's court photograph, with four approximate floor corners added. Right: a computed rectangular warp of that region. The floor straightens; raised baskets and seating do not become a true overhead view. Source photograph: PDF page 71, printed slide 72; individual photographer uncredited. Warp and annotations are original.

2.3.1 One matrix, followed by a division

Formula — planar projective mapping

$$\begin{aligned} x' &\sim Hx, \qquad H \in \mathbb{R}^{3\times 3}, \quad \det H \neq 0, \\ u' &= \frac{h_{11}u + h_{12}v + h_{13}}{h_{31}u + h_{32}v + h_{33}}, \qquad v' = \frac{h_{21}u + h_{22}v + h_{23}}{h_{31}u + h_{32}v + h_{33}}. \end{aligned} \tag{2.4}$$

$x = (u, v, 1)^T$ and $x' = (u', v', 1)^T$ are image or plane coordinates. Use consistent units within each plane. The matrix scale follows the convention in (1.6).
The ordinary-coordinate fractions require a nonzero denominator. Zero means an ideal destination point. A single H models corresponding images of one plane, or a camera rotating about a fixed center. General camera translation with objects at different depths requires more than one homography.

Map a point or rectify a plane

1. Specify the direction: here H maps source coordinates to destination coordinates.
2. Multiply $H(u, v, 1)^T$ and divide by its last entry.
3. Check the result against known correspondences and image bounds.

To recover H, use at least four corresponding points in general position: no three collinear in either four-point set. See *estimation* and *pixel sampling*.

Worked example 2.2 — perspective changes spacing

Source: original small calculation for PDF pages 62–64.

Let $H = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1/2 & 0 & 1 \end{pmatrix}$. The source points $(0, 0)$, $(1, 0)$, $(2, 0)$ map to $(0, 0)$, $(2/3, 0)$, $(1, 0)$.
They stay on a line, but equal source intervals become unequal destination intervals: $2/3$ and $1/3$. The denominator depends on position. This is what an affine map cannot do along a line.
The finite point $(-2, 1)$ maps to $(-2, 1, 0)^T$, an ideal point. The homography remains invertible; only ordinary coordinates fail there.

The transformation families and their invariants are taught in *Move points without losing the geometry*. Here we use their projective case for an actual plane-to-image mapping.

2.3.2 A new detail: separate an affine map into rotations and scales

PDF page 68 adds a factorization of the affine linear part. It reveals the directions in which the map stretches the image.

Formula — directional scaling inside an affine map

$$A = R(\theta)R(-\phi)DR(\phi), \qquad D = \operatorname{diag}(s_1, s_2). \tag{2.5}$$

All matrices are 2×2; R is the planar rotation from (1.4). The angles orient the scaling axes and the final rotation. Nonzero signed scales allow any invertible real A, including orientation reversal. Translation is handled by the existing affine form in (1.6).

Read a directional stretch

1. Apply the rightmost rotation to express a vector in the scaling axes.
2. Scale the two components by s_1, s_2, then rotate back by $-\phi$.
3. Apply the final rotation by θ.

For $\theta = 0$, $\phi = 45^\circ$, and $D = \operatorname{diag}(2, 1)$, multiplication gives $A = \begin{pmatrix} 3/2 & -1/2 \\ -1/2 & 3/2 \end{pmatrix}$. It doubles the component along $(1, -1)^T$ and leaves the component along $(1, 1)^T$ unchanged. This is an original calculation of the slide's added factorization.

2.3.3 Cross-ratio: a ratio perspective cannot change

Ordinary distances change in perspective, but a particular combination of four collinear locations survives.

Formula — cross-ratio of four collinear points

$$\mathrm{CR}(a, b; c, d) = \frac{(c - a)(d - b)}{(c - b)(d - a)}. \tag{2.6}$$

a, b, c, d are distinct scalar positions along one line, in a common coordinate. The result is unitless and depends on the chosen ordering. This finite-coordinate expression assumes nonzero denominators; ideal points are handled by limits or homogeneous coordinates. An invertible projective mapping of the line preserves the result.

Check a projective invariant

1. Keep the same ordering of four distinct collinear points before and after mapping.
2. Compute the two cross-ratios and compare.
3. Do not substitute ordinary image distances for directed line coordinates.

For $a, b, c, d = 0, 1, 2, 3$, the result is $4/3$. Under Example 2.2's map, the positions become $0, 2/3, 1, 6/5$;

substitution still gives $4/3$.

Explore a perspective warp — print edition

Vary the perspective coefficient p in $H = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ p & 0 & 1 \end{pmatrix}$. The destination coordinates are $(u/(1+pu), v/(1+pu))$; source coordinates are dimensionless. The square occupies $-1 \le u, v \le 1$. Restricting $-0.8 \le p \le 0.8$ keeps its denominators positive.

Predict, then compare: at $p = 0$ the square stays unchanged. At $p = 0.5$, its corners become $(-2, -2)$, $(2/3, -2/3)$, $(2/3, 2/3)$, and $(-2, 2)$ in cyclic source order. Equal spacings change, but the grid lines remain straight.

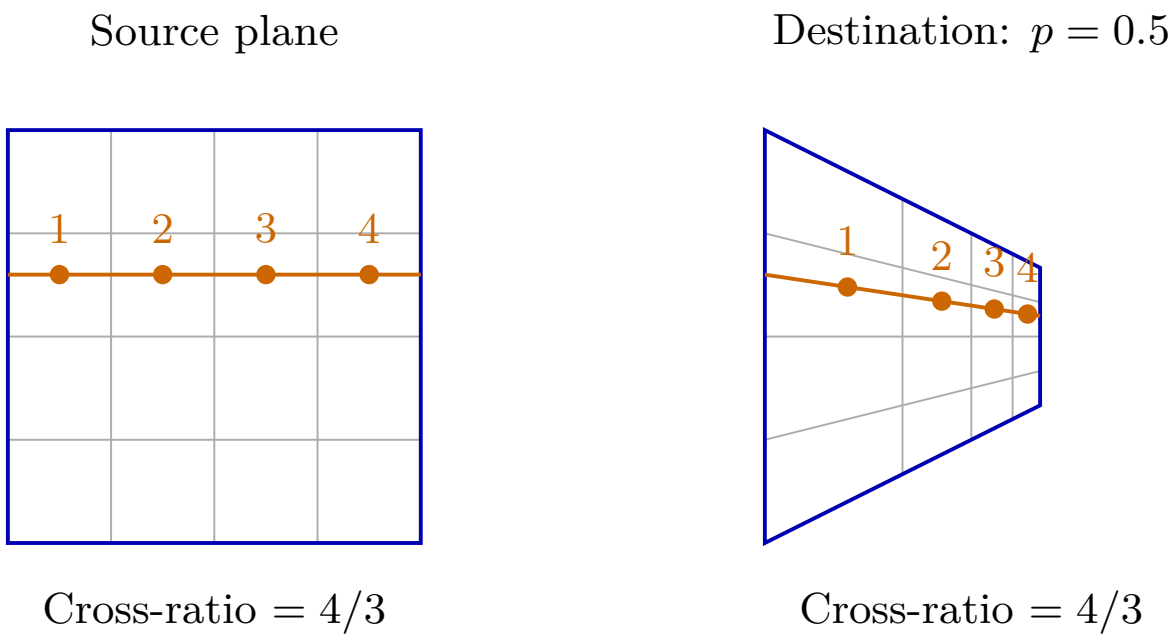


Fig. 2.2: Static view at $p = 0.5$. Each panel is fitted separately to its view, so compare shape and spacing rather than displayed scale. The four source points are $(-0.75, 0.3)$, $(-0.25, 0.3)$, $(0.25, 0.3)$, $(0.75, 0.3)$; they lie on one line before and after the warp.

The mapped horizontal coordinates are -1.2, $-2/7$, $2/9$, and $6/11$. Substituting either ordered set into (2.6) gives $4/3$. This is what stays fixed while the shape changes.

2.3.4 What is missing from the court assignment?

We need **correspondences and a target shape**. Select four floor corners in cyclic order, match them to a rectangle, then estimate H. Its width-to-height ratio must come from known court geometry if we want metric proportions. An arbitrary rectangle can look tidy while having the wrong scale ratio.

Our figure uses approximate source pixels $(24, 194)$, $(250, 45)$, $(410, 70)$, $(280, 278)$, matched to $(0, 0)$, $(279, 0)$, $(279, 149)$, $(0, 149)$. The chosen aspect ratio is illustrative, not a measurement of the court. Boundary pixels are approximate because the supplied photo is small. **A floor homography cannot reveal hidden floor or flatten raised objects correctly.**

2.4 Image warping: where do the new pixels come from?

Mapping a point is not enough to create an image. Source pixels lie on a grid; transformed positions usually fall between destination pixels. We need a rule for filling the destination grid.

Source: PDF pages 72–81, printed slides 73–82; the section credits Steve Seitz.

2.4.1 Move colors, or look them up?

Image filtering changes pixel values at fixed coordinates, such as darkening a picture. **Image warping** changes where the content appears. A global parametric warp uses the same coordinate function everywhere, though its displacement can vary with position.

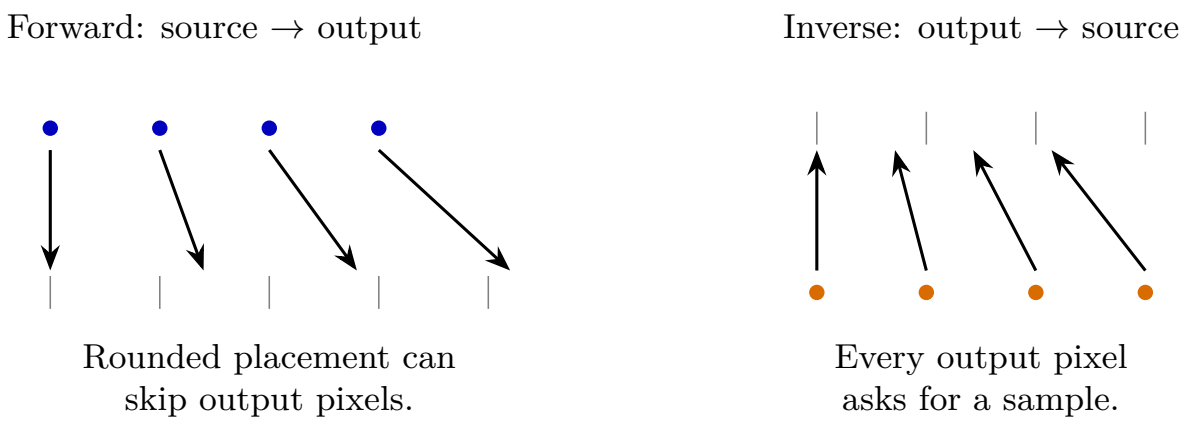


Fig. 2.3: Forward mapping asks where each source pixel lands. Inverse mapping asks where each output pixel should sample. Original diagram of the distinction on PDF pages 76–79.

Formula — inverse image sampling

$$g(u', v') = f\big(\text{dehom}\big(H^{-1}(u', v', 1)^T\big)\big)\,. \tag{2.7}$$

f is the source image, g the output; H maps **source to destination**. The operator dehom divides the first two coordinates by the last. Pixel values may be grayscale scalars or color vectors. This describes continuous sampling; discrete images require interpolation and a boundary rule.

Fill an output image

1. Visit every output pixel center and multiply by H^{-1}.
2. Divide by the homogeneous denominator; mark invalid/out-of-source samples as background.
3. Interpolate the source image and store that value.

This avoids holes caused by rounded forward placement. Forward warping is still useful when samples must be projected or splatted. When shrinking an image substantially, prefilter it to reduce aliasing; interpolation alone is insufficient.

2.4.2 Bilinear interpolation: blend four neighbors

Suppose inverse mapping returns source location $(i + a, j + b)$, where i, j are integers and $0 \le a, b < 1$. First interpolate horizontally in each row, then vertically between those results.

Formula — bilinear interpolation

$$\begin{aligned} f(i+a, j+b) \approx (1-a)(1-b) f(i,j) + a(1-b) f(i+1,j) \\ + (1-a) b f(i,j+1) + ab f(i+1,j+1). \end{aligned} \tag{2.8}$$

a, b are unitless fractions of a pixel; the output has the same intensity units as the samples. We write coordinates as (column, row); an image array usually indexes $f[\text{row}, \text{column}]$. The four weights are nonnegative and sum to one. Specify how samples outside the image are handled.

Calculate a sampled intensity

1. Take the floor of each continuous coordinate to find i, j.
2. Compute a, b and the four weights.
3. Blend each image channel separately.

Check that a constant patch remains constant and that integer coordinates reproduce the stored pixel. See Example 2.3.

Worked example 2.3 — a pixel between pixels

Source: original numerical companion to PDF page 80, printed slide 81.
The four intensities at $(i, j), (i + 1, j), (i, j + 1), (i + 1, j + 1)$ are $10, 30, 50, 70$. At $a = 1/4$, $b = 1/2$, the weights are $3/8, 1/8, 3/8, 1/8$.
The sampled value is $(3 \cdot 10 + 30 + 3 \cdot 50 + 70)/8 = 35$. Equivalently, the two horizontal interpolations give 15 and 55, whose halfway blend is 35. It lies within the range of the four inputs.

Check 2.2 — which inverse?

If H shifts source content ten pixels right, where does output pixel $(20, 8)$ sample?

Answer

At source $(10, 8)$. Using H instead of H^{-1} would sample $(30, 8)$ and move content in the wrong direction.

2.5 Estimate a reliable mapping

Four matches can determine a homography, but real matches are noisy and some are wrong. **DLT** solves for the matrix; **RANSAC** finds matches that agree with one matrix. They solve different problems.

Source: PDF pages 82–104, printed slides 83–105. These sections credit R. Hartley, A. Zisserman, and M. Pollefeys. *Lecture 3* adds the assigned homography/probability quizzes and a full projective numerical walk-through.

2.5.1 DLT: turn matches into linear equations

The **direct linear transformation (DLT)** starts from $x' \sim Hx$. Their cross product is zero because the vectors represent the same projective point: $x' \times Hx = 0$.

For finite matches $(u, v) \leftrightarrow (u', v')$, multiply out the denominators in (2.4). Each match supplies two independent equations in the nine entries of H.

Formula — homography DLT system

$$
\begin{aligned}
A_i &= \begin{pmatrix} u & v & 1 & 0 & 0 & 0 & -u'u & -u'v & -u' \\ 0 & 0 & 0 & u & v & 1 & -v'u & -v'v & -v' \end{pmatrix}, \\
h &= (h_{11}, h_{12}, h_{13}, h_{21}, h_{22}, h_{23}, h_{31}, h_{32}, h_{33})^T, \\
A &= \begin{pmatrix} A_1 \\ \vdots \\ A_n \end{pmatrix}, \qquad \min_{\|h\|=1} \|Ah\|^2.
\end{aligned}
\tag{2.9}
$$

A is $2n \times 9$, h is 9×1. The constraint excludes the useless zero solution. Four nondegenerate exact matches give rank eight and a one-dimensional nullspace; with noisy extra matches, minimize the residual instead of expecting exact zero.

The finite-coordinate rows assume each destination's last coordinate is nonzero. For ideal points, retain appropriate independent rows of the full cross-product system. Three cross-product equations have only two independent constraints; dropping the same row blindly can lose information.

Estimate a homography with SVD

1. Normalize coordinates as described below; reject a degenerate four-point sample.
2. Stack two rows per match to form A.
3. Compute $A = U\Sigma V^T$ using singular value decomposition. Choose the **right singular vector** with smallest singular value and reshape it row by row into H.
4. Undo coordinate normalization and check projected matches.

For an 8×9 system, request the full right-singular-vector basis: a reduced SVD can omit the ninth vector containing the nullspace. Do not confuse the last row of V^T with the last column of V^T.

Fixing $h_{33} = 1$ is a tempting shortcut, but valid homographies can have $h_{33} = 0$. Unit-vector normalization avoids privileging that entry. DLT minimizes **algebraic error**, which depends on coordinate scaling. Pixel reprojection error is a more interpretable final accuracy measure.

Worked example 2.4 — recover a simple planar map

Source: original numerical companion to PDF pages 83–94.

Match $(0,0), (1,0), (1,1), (0,1)$ to $(2,1), (4,1), (4,4), (2,4)$ in that order. The first correspondence produces rows

$$(0,0,1,0,0,0,0,0,-2), \qquad (0,0,0,0,0,1,0,0,-1).$$

For $(1,0) \leftrightarrow (4,1)$ the rows are

$$(1,0,1,0,0,0,-4,0,-4), \qquad (0,0,0,1,0,1,-1,0,-1).$$

The other two matches fill the remaining four rows. The resulting matrix has rank eight. Its null vector, reshaped and rescaled, gives $H = \begin{pmatrix} 2 & 0 & 2 \\ 0 & 3 & 1 \\ 0 & 0 & 1 \end{pmatrix}$.

Check the unmeasured center: $H(1/2, 1/2, 1)^T = (3, 5/2, 1)^T$, the center of the destination rectangle. All four measured corners reproject exactly. Three collinear source points would invalidate this four-point recovery.

2.5.2 Normalize: put the coordinates on a sensible scale

Raw pixel coordinates may be hundreds, while their products in A reach tens of thousands. **Normalized DLT** recenters and rescales each image's points independently, reducing this imbalance.

Formula — coordinate normalization and recovery

$$
\begin{aligned}
c &= \frac{1}{n}\sum_i p_i, \quad \bar{r} = \frac{1}{n}\sum_i \|p_i - c\|, \quad s = \frac{\sqrt{2}}{\bar{r}}, \\
T &= \begin{pmatrix} s & 0 & -sc_u \\ 0 & s & -sc_v \\ 0 & 0 & 1 \end{pmatrix}, \qquad \tilde{x} = Tx, \quad \tilde{x}' = T'x', \\
H &= (T')^{-1}\widetilde{H}T.
\end{aligned}
\tag{2.10}
$$

$p_i = (u_i, v_i)^T$ are finite 2D points; T, T', H are 3×3. The mean distance from the normalized centroid becomes $\sqrt{2}$. Require $\bar{r} > 0$; coincident points provide no spatial information. Normalization improves conditioning, not incorrect matches or degeneracy.

Use normalized DLT

1. Compute separate T, T' from the source and destination matches.
2. Fit $\widetilde{H}$ to their normalized coordinates using (2.9).
3. Recover $H = (T')^{-1}\widetilde{H}T$ and evaluate error in original coordinates.

The source square in Example 2.4 has centroid $(1/2, 1/2)$, mean radius $1/\sqrt{2}$, and scale $s = 2$. Its normalized corners are $(\pm 1, \pm 1)$, at radius $\sqrt{2}$.

Why this recovery order? The normalized mapping is $T'x' \sim \widetilde{H}Tx$. Multiply by $(T')^{-1}$ to isolate the original destination coordinates. PDF page 92 also shows a simpler image-size scaling; it is an alternative heuristic, not the same transformation as centroid/mean-distance normalization.

2.5.3 RANSAC: do not let wrong matches vote as measurements

An ***inlier*** agrees with the model within a chosen error threshold. An **outlier** does not. ***RANSAC*** means random sample consensus: fit small random subsets, then count how many matches support each hypothesis.

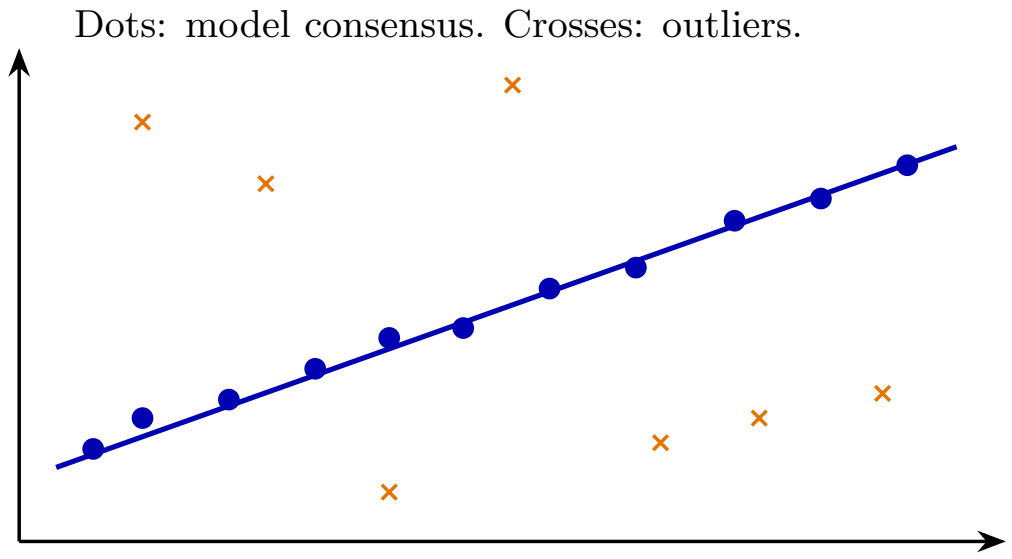


Fig. 2.4: The idea behind the lecture's robust-line example: a large consistent group supports the correct model despite scattered bad measurements. Original schematic; homography hypotheses use four matches rather than two line points.

Formula — RANSAC trial count and inlier score

$$N = \left\lceil \frac{\log(1-p)}{\log(1-w^s)} \right\rceil, \qquad r_i = \|p'_i - \mathrm{dehom}(Hx_i)\|_2, \qquad n_{\text{in}} = \#\{i : r_i < \tau\}. \tag{2.11}$$

p is the desired probability of at least one all-inlier sample, $w = 1 - e$ the inlier fraction, and s the sample size (four for a homography). Here residuals and threshold τ are in destination pixels. N is a dimensionless integer.

The usual w^s model assumes independent draws, or a large pool approximating them, and nondegenerate all-inlier samples. Trials are independent. For sampling s distinct matches from a small pool of n containing I true inliers, the exact all-inlier probability is $\binom{I}{s}/\binom{n}{s}$. Neither probability guarantees the selected model is correct.

The derivation is short: a trial is all-inlier with probability w^s. All N trials fail with probability $(1 - w^s)^N$. Set that failure probability at most $1 - p$, take logarithms, and round up.

Fit a homography robustly

1. Sample four matches; reject collinear or otherwise degenerate configurations; estimate H.
2. Score every match against H using a fixed error definition and threshold.
3. Keep the best consensus and repeat to the trial budget. Update the budget from a better observed inlier ratio, while recognizing that it is an estimate.
4. Refit to the best inliers, then refine pixel reprojection error if needed.

For $w = 0$ no finite budget succeeds; for $w = 1$ one nondegenerate trial suffices. A large consensus of repeated patterns can still support the wrong alignment. See Example 2.5.

Worked example 2.5 — why a better inlier fraction saves work

Source: numerical expansion of PDF page 98's trial table; application figures from PDF page 104.
With $p = 0.99$, $w = 0.5$, and $s = 4$, the continuous budget is $\log(0.01)/\log(15/16) \approx 71.36$. Round **up** to 72 trials. At $w = 0.8$, it falls to 9.
The lecture's building-image example starts with 268 putative matches. A 151-match consensus corresponds to roughly 56% inliers; using that rounded ratio gives about 45 trials. The slide reports 43 iterations: the simple probability model is a budget estimate, not a reproduction of the implementation's exact stopping history. Guided matching later finds 262 final inliers; this is a new matching stage, not simply the original consensus.

An expected consensus threshold is approximately wn (rounded as needed), but it is not a statistical guarantee. **LMedS**, least median of squares, instead minimizes median squared residual; it is a different robust objective. Ordinary least squares can be dominated by gross outliers. Iterative refinement is useful **after** robust selection; iteration alone does not make an estimator robust.

2.5.4 The complete slide pipeline

Detect interest points → propose matches → robustly estimate H → refine using inliers → use H to guide more matches. The slides name sum of absolute differences (SAD), sum of squared differences (SSD), and zero-mean normalized cross-correlation (ZNCC) as patch comparisons. SAD/SSD compare pixel intensities directly; ZNCC compares mean-centered, normalized patterns and needs nonzero patch variance. Lecture 5 develops matching and exposure changes.

When motion is known to be small, restrict candidate locations; do not impose that restriction on arbitrary viewpoint changes. Nonlinear refinement may use Levenberg–Marquardt to minimize an image-error objective, rather than the initial algebraic DLT objective.

Check 2.3 — can more points rescue a bad model?

Would fitting one homography to hundreds of matches on several depths always work better than using four floor corners?

Answer

No. Extra measurements cannot repair the wrong geometric model. With translation, a single homography generally fits one plane, not objects at many depths.

2.6 Distortion and two-view geometry

A planar warp is powerful, but it cannot explain curved lens distortion or general 3D parallax. The final slides introduce the tools for those cases.

Source: PDF pages 105–129, printed slides 106–130; the section credits R. Hartley, A. Zisserman, and M. Pollefeys. The assigned epipolar/stereo quizzes and a fuller estimation treatment belong to Lecture 4.

2.6.1 Straight lines first: correct lens distortion

Radial distortion moves points along directions from a distortion center. Straight scene edges can become curved. A homography preserves lines, so it cannot generally undo this bending.

Formula — radial correction model

$$r^2 = (u - c_u)^2 + (v - c_v)^2, \qquad \begin{pmatrix} \hat{u} \\ \hat{v} \end{pmatrix} = \begin{pmatrix} c_u \\ c_v \end{pmatrix} + (1 + k_1 r^2 + k_2 r^4 + \cdots) \begin{pmatrix} u - c_u \\ v - c_v \end{pmatrix}. \tag{2.12}$$

Here (u, v) are measured distorted coordinates and hats denote corrected coordinates. The coefficients belong to **this correction direction**; a forward-distortion model generally needs different coefficients or numerical inversion. Use calibrated units: if r is in pixels, k_1 has units pixel^{-2} and k_2 pixel^{-4}; normalized coordinates are also common.
This low-order model describes radial effects, not every lens defect. The distortion center may differ from the principal point. PDF page 109 mixes radius and squared-radius notation; writing r^2 explicitly avoids that ambiguity.

Correct a point before geometric estimation

1. Use the calibrated center, units, coefficients, and mapping direction.
2. Compute r^2 and the radial factor, then correct the coordinate.
3. Check that straight scene edges become straighter; resample images with an appropriate inverse map.

For center $(0, 0)$, point $(100, 0)$ pixels, $k_1 = 10^{-6}$ pixel^{-2}, and no higher terms, the factor is 1.01 and the corrected point is $(101, 0)$. A positive coefficient here expands radii; never transfer that sign interpretation to the inverse convention blindly.

The slides suggest estimating distortion jointly with other camera parameters or by straightening known straight edges. Their room-image before/after sequence illustrates why this matters: ideal projective geometry should be applied to undistorted coordinates.

2.6.2 One point becomes a line in the other image

With two camera centers C, C' and a scene point X, the three define an **epipolar plane**. It cuts each image plane in an ***epipolar line***. If the first image point is known but its depth is not, its possible matches lie along that line in the second image.

The ***epipole*** is the image of the other camera center; every epipolar line in an image passes through its epipole. The line between camera centers is the **baseline**.

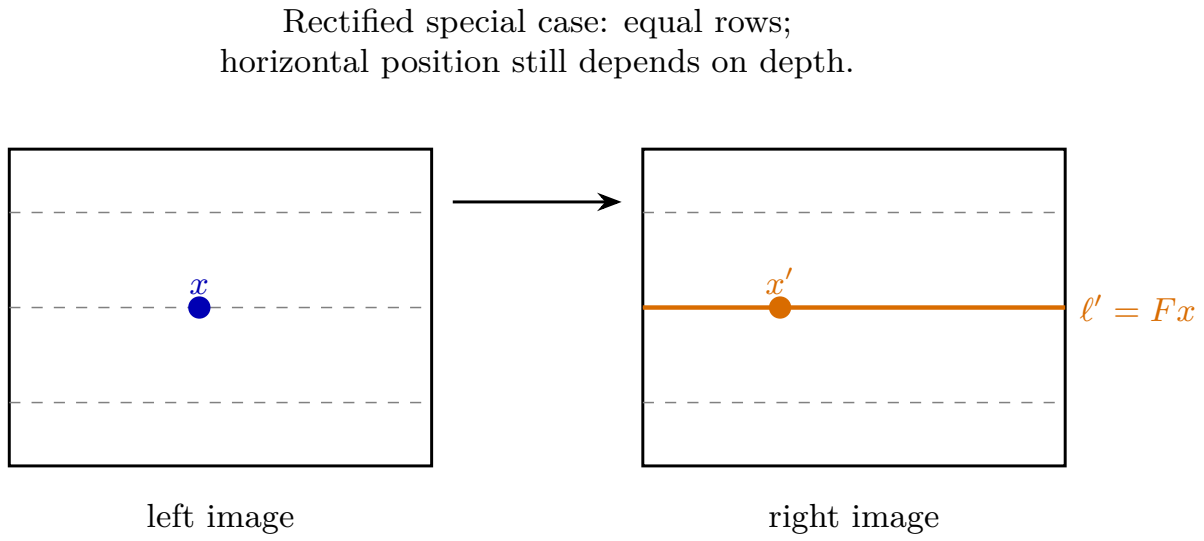


Fig. 2.5: A special, rectified case: the right match must lie on the corresponding row. Its location along that row still depends on depth. Original schematic developing PDF pages 113–119.

2.6.3 The fundamental matrix maps points to lines

Formula — epipolar constraint and epipoles

$$\begin{aligned} \ell' &= Fx, \qquad \ell = F^T x', \qquad x'^T F x = 0, \\ Fe &= 0, \qquad F^T e' = 0, \qquad \operatorname{rank} F = 2. \end{aligned} \tag{2.13}$$

x, x' are homogeneous image 3×1 points; ℓ, ℓ' are line coefficients, e, e' epipoles. F is a 3×3 ***fundamental matrix***, defined up to nonzero scale. It has seven DOF: nine entries minus scale minus the rank-two constraint. Reversing the image order transposes F.
This ideal constraint assumes corresponding undistorted points and distinct camera centers. At an epipole, the corresponding line vector can vanish. Pure rotation has no nonzero baseline and does not determine a nonzero fundamental matrix from the camera pair. The equation constrains correspondence; it does not identify a unique match.

Turn a point into a correspondence constraint

1. Multiply F by the first image point to obtain the second image's line.
2. Test a candidate match by its dot product with that line.
3. For noisy data, use a scale-normalized geometric error instead of comparing raw $x'^T F x$ values across arbitrary scalings.

Check matrix rank and image order. See Example 2.6.

Worked example 2.6 — a horizontal epipolar line

Source: original numerical companion to the parallel-camera example, PDF page 119.

Let $F = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$ and $x = (2, 3, 1)^T$. Then $Fx = (0, -1, 3)^T$: candidate matches satisfy $-v' + 3 = 0$.

Both $(5, 3, 1)^T$ and $(8, 3, 1)^T$ satisfy the constraint; $(5, 4, 1)^T$ does not. The two valid candidates show why an epipolar line is not a point prediction. The matrix has rank two and both epipoles are $(1, 0, 0)^T$, the horizontal ideal direction.

2.6.4 Where does the matrix come from?

The source derives F by projecting two points on the first camera's viewing ray into the second image. Their join is the epipolar line.

Formula — construct F from two camera matrices

$$PC = 0, \qquad e' = P'C, \qquad F = [e']_\times P' P^+. \tag{2.14}$$

P, P' are rank-three 3×4 camera matrices; C is the first camera center as a homogeneous 4-vector; P^+ is a 4×3 right pseudoinverse with $PP^+ = I_3$. The matrix $[e']_\times$ takes a cross product with e'.

For distinct centers, $P^+ x$ is one 3D point projecting to x; adding λC stays on its projective ray. Its second image lies on the line joining $P'P^+x$ to $P'C$, producing the formula. Coincident centers give $P'C = 0$, so this construction gives zero, not a valid rank-two F.

Check a camera-derived epipolar geometry

1. Find a nonzero null vector C of P and compute $e' = P'C$.
2. Form the pseudoinverse and cross-product matrix, then multiply in the stated order.
3. Project a test 3D point in both cameras and verify the scalar constraint.

The cameras $P = [I \mid 0]$, $P' = [I \mid (-1, 0, 0)^T]$ give Example 2.6's matrix up to sign. A scene point $(2, 3, 1, 1)^T$ projects to $(2, 3, 1)^T$ and $(1, 3, 1)^T$, on the same row.

For a fixed nondegenerate camera pair, F is unique **up to scale**. Estimating it from a finite set of matches can be ambiguous, particularly with planar scene points.

Formula — linear equations for F from matches

$$\begin{pmatrix} u'u & u'v & u' & v'u & v'v & v' & u & v & 1 \end{pmatrix} f = 0, \qquad Af = 0. \tag{2.15}$$

$f = (F_{11}, F_{12}, F_{13}, F_{21}, \ldots, F_{33})^T$ is 9×1; one finite match contributes one row. At least eight suitably independent matches give the linear estimate. This differs from homography DLT's **two** equations per match. Noisy linear solutions need a rank-two correction.

Recognize the eight-point estimation route

1. Normalize both sets of image coordinates and stack the rows.
2. Find the smallest right singular vector and reshape it into a matrix.
3. Enforce rank two by setting its smallest singular value to zero.
4. Undo normalization: if normalized points are $Tx, T'x'$, then $F = T'^T \widetilde{F} T$; verify on held-out matches.

For Example 2.6 and match $(u, v) = (2, 3)$, $(u', v') = (5, 3)$, the row is $(10, 15, 5, 6, 9, 3, 2, 3, 1)$. Dotting it with the row-stacked F gives $-3 + 3 = 0$. *Lecture 4* develops noisy estimation and the assigned quiz calculations.

2.6.5 Motion patterns and rectification

The three slide examples show different locations for the epipoles:

- **Converging cameras:** epipolar lines fan toward finite epipoles, possibly outside the images.
- **Translation parallel to the image plane:** epipoles are ideal; for a horizontal baseline and aligned cameras, epipolar lines are horizontal.
- **Forward translation:** epipoles can lie inside the images, with radial epipolar lines. Ordinary rectangular planar rectification becomes problematic near that epipole.

Rectification warps two views so corresponding epipolar lines become matching horizontal rows. It simplifies the search from two dimensions to one; it does not eliminate matching ambiguity or depth variation.

Formula — rectified coordinates and line geometry

$$\bar{x} \sim Hx, \quad \bar{x}' \sim H'x', \quad \bar{F} = H'^{-T} F H^{-1}, \qquad He \sim H'e' \sim (1, 0, 0)^T. \tag{2.16}$$

H, H' are nonsingular 3×3 image warps. Moving both epipoles to the horizontal ideal point makes epipolar lines horizontal; the two warps must also align the corresponding row coordinates. A finite epipole is sent to infinity, so its neighborhood cannot retain a finite undistorted rectangular image.

Interpret a rectified pair

1. Estimate camera/epipolar geometry and choose compatible rectifying warps.

2. Resample both images by inverse warping.
3. Check that known matches share the same row; then search along that row.

Example 2.6 is already rectified: identity warps preserve its horizontal lines and equal-row constraint. The final slide's paired objects illustrate the visual goal. *Stereo depth and error* follow in Lecture 4.

2.7 Original quiz questions and worked solutions

Read the problem in black. Follow the numbered red steps. Work through the blue mathematics. Every step explains why it is needed. Matrix calculations show the inputs and intermediate results; written-reasoning questions explain the decision instead of inventing numerical data.

Source: the supplied lecture-organized quiz booklet. Full problems follow its wording and givens, with typography normalized; the historical PDF filenames appear below. Source-key corrections and added numerical companions are marked.

Retests have their own questions and full solutions. Problems labeled **Original practice** are the booklet's seven supplemental examples, separate from the 18 historical quizzes. Colored steps remain numbered and readable in grayscale.

2.7.1 Jump to a quiz

- *Spring 2020 Quiz 1 — Projective points and camera projection*

2.7.2 Spring 2020 Quiz 1 — Projective points and camera projection

Historical source: `3dcvMid/olderquizzes/cs532s20_quiz1.pdf`.

Problem 1

Original problem · quiz booklet

For projective-plane points a, b, c, use only dot and cross products to test whether they are collinear. In what case does the test fail?

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Join two projective points

Why: The cross product constructs their homogeneous line.

Recall that the homogeneous *line through two distinct points* is their cross product: $\ell = a \times b$.

Step 2 — Test the third point

Why: A point is on a line exactly when their dot product is zero.

A third point c lies on ℓ exactly when its dot product with ℓ is zero. Thus test $\boxed{c^T(a \times b) = 0}$. This equals the determinant of the matrix with columns a, b, c.

Step 3 — Handle repeated points

Why: A zero cross product cannot identify a unique joining line.

If $a \sim b$, then $a \times b = 0$, so it does not construct a unique line. Choose a distinct pair instead. The determinant still correctly says the three representatives are linearly dependent: any repeated point together with a third point lies on at least one line. A zero homogeneous point itself is invalid.

Step 4 — Expand a numerical test

Why: Actual products show both a passing and a failing candidate.

Original numerical companion: take $a = (0, 0, 1)^T$, $b = (2, 0, 1)^T$, $c = (5, 0, 1)^T$. Then

$$a \times b = \begin{pmatrix} 0(1) - 1(0) \\ 1(2) - 0(1) \\ 0(0) - 0(2) \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \\ 0 \end{pmatrix}, \qquad c^T(a \times b) = \begin{pmatrix} 5 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 2 \\ 0 \end{pmatrix} = 0.$$

The points lie on $Y = 0$. Replacing c by $(5, 1, 1)^T$ gives $5(0) + 1(2) + 1(0) = 2$, so that point is off the line.

Problem 2(a–b)

Original problem · quiz booklet

Let $X = (X, Y, Z, W)^T$ be a homogeneous 3D point and $x \sim MX$ a homogeneous 2D image point. Find (a) the dimensions of M; (b) explicit inhomogeneous image coordinates from the entries m_{ij} and X.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Count input and output entries

Why: Matrix multiplication determines the camera matrix's dimensions.

A 3D homogeneous point has four components; a 2D homogeneous image point has three. Therefore $\boxed{M \in \mathbb{R}^{3\times 4}}$ and Mx is *not* the mapping; the correct product is $MX \in \mathbb{R}^{3\times 1}$.

Step 2 — Multiply the full projection matrix

Why: Every output entry is one row dotted with the world point.

Show the full matrix-vector product:

$$q = \begin{pmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix} = \begin{pmatrix} m_{11}X + m_{12}Y + m_{13}Z + m_{14}W \\ m_{21}X + m_{22}Y + m_{23}Z + m_{24}W \\ m_{31}X + m_{32}Y + m_{33}Z + m_{34}W \end{pmatrix}.$$

Step 3 — Divide by the last coordinate

Why: Homogeneous image coordinates represent ratios.

A finite homogeneous point represents ratios, so divide the first two entries by the third:

$$u = \frac{m_{11}X + m_{12}Y + m_{13}Z + m_{14}W}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}W},$$
$$v = \frac{m_{21}X + m_{22}Y + m_{23}Z + m_{24}W}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}W}.$$

For a finite world point choose $W = 1$. If $q_3 = 0$ and $q \neq 0$, the result is an ideal image point; if $q = 0$, projection is undefined.

Problem 3(a–b)

Original problem · quiz booklet

Let ℓ_1, ℓ_2 be two distinct vertical lines each at distance $d > 0$ from the 2D origin, and ℓ_3 the horizontal line through the origin. Give their homogeneous line vectors and count the possible pairwise intersections.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Represent the vertical lines

Why: Their equations fix the signs of the constant coefficients.

Choose $\ell_1 : x = d$ and $\ell_2 : x = -d$; as $ax+by+c = 0$, their vectors are $\boxed{\ell_1 = (1, 0, -d)^T}$, $\boxed{\ell_2 = (1, 0, d)^T}$.

Step 2 — Represent the horizontal line

Why: The line through the origin has no constant offset.

For the horizontal line $y = 0$, use $\boxed{\ell_3 = (0, 1, 0)^T}$. Line vectors are homogeneous, so nonzero multiples are equivalent.

Step 3 — Compute both finite intersections

Why: Cross products intersect homogeneous lines.

Compute both finite intersections component by component:

$$\ell_1 \times \ell_3 = \begin{pmatrix} 0(0) - (-d)(1) \\ (-d)(0) - 1(0) \\ 1(1) - 0(0) \end{pmatrix} = \begin{pmatrix} d \\ 0 \\ 1 \end{pmatrix},$$

$$\ell_2 \times \ell_3 = \begin{pmatrix} 0(0) - d(1) \\ d(0) - 1(0) \\ 1(1) - 0(0) \end{pmatrix} = \begin{pmatrix} -d \\ 0 \\ 1 \end{pmatrix}.$$

Their final coordinates are already one, giving ordinary points $(d, 0)$ and $(-d, 0)$.

Step 4 — Intersect the parallel lines

Why: A zero last coordinate represents a point at infinity.

The parallel vertical lines give

$$\ell_1 \times \ell_2 = \begin{pmatrix} 0(d) - (-d)(0) \\ (-d)(1) - 1(d) \\ 1(0) - 0(1) \end{pmatrix} = \begin{pmatrix} 0 \\ -2d \\ 0 \end{pmatrix} \sim \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}.$$

The last coordinate is zero, so do not divide: this is a point at infinity in the vertical direction.

Step 5 — Count in the stated geometry

Why: Projective and finite-plane intersection counts differ.

Thus 3 projective intersections among the 3 line pairs, of which 2 are finite. Say both numbers to clarify the geometric convention.

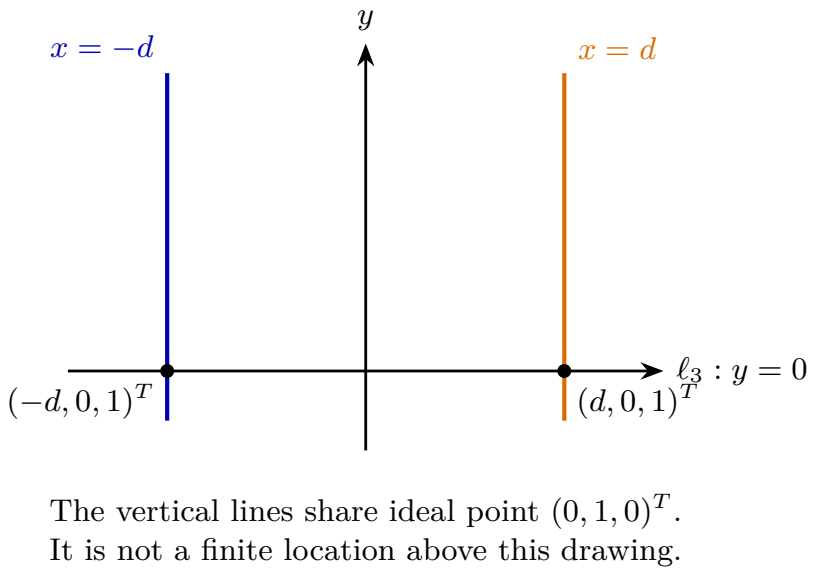


Fig. 2.6: Two finite intersections and one vertical point at infinity.

2.8 Review and formula checklist

Check the new tools below. Foundation explanations and Spring 2020 Quiz 1 solutions are in *Foundation quiz practice*. Source ranges below are PDF pages in `cs532_f25_Week2.pdf`.

Topic and source pages	Canonical formula or explanation	Practice
Quaternion conversion, 44–49	(2.1); (2.2)	*Example 2.1*
Smooth rotation interpolation, 45	(2.3) (explanatory companion)	Identity to quarter-turn midpoint
Directional affine scaling, 68	(2.5)	Stretch along a diagonal
Planar rectification and cross-ratio, 62–71	(2.4); (2.6)	*Example 2.2*; court warp
Warping and interpolation, 72–81	(2.7); (2.8)	*Example 2.3*
Homography estimation, 82–94	(2.9); (2.10)	*Example 2.4*
Robust estimation and matching, 95–104	(2.11); *Estimate a reliable mapping*	*Example 2.5*
Radial correction, 105–110	(2.12)	Correct $(100, 0)$ to $(101, 0)$
Epipolar geometry, 111–126	(2.13); (2.14); (2.15)	*Example 2.6*
Rectification, 127–129	(2.16)	Equal-row and epipole checks

2.8.1 Can you do these without notes?

- Convert a rotation matrix to a quaternion and verify the round trip.
- Check quaternion signs with a cyclic-axis rotation and handle a half-turn without dividing by zero.
- Explain why a homography has eight DOF and needs four nondegenerate matches.
- Write two DLT rows for a match and identify the correct right singular vector.
- Undo coordinate normalization in the correct order.
- Explain why inverse warping still needs interpolation and a boundary rule.
- Distinguish noise, outliers, and the wrong geometric model.
- Map an image point to an epipolar line, then test a possible match.

Choose the right image model: a homography maps planar points to points; a fundamental matrix maps a point to a correspondence line in another view.

Finish with the *TLDR and short quiz*. *Lecture 3* adds camera calibration and projective homography practice. *Lecture 4* adds noisy epipolar estimation, calibrated poses, stereo, and the assigned quizzes.

2.9 TLDR

Lecture 2 in one sentence: estimate planar image mappings from trustworthy matches, resample them correctly, and recognize when distortion or 3D depth requires another model.

2.9.1 The important things

1. **Convert quaternions consistently.** This chapter adds quaternion-to-matrix and matrix-to-quaternion formulas. The scalar extraction is $w = \frac{1}{2}\sqrt{1 + \operatorname{tr} R}$; near a half-turn, use a largest-component branch. See *Quaternions: convert without changing the rotation*.
2. **SLERP interpolates orientations.** Align endpoint quaternion signs, then interpolate along the unit sphere. For nearly identical orientations, normalized linear interpolation avoids an unstable denominator.
3. **A plane can be rectified with four nondegenerate matches.** A homography supplies the mapping; the destination rectangle's aspect ratio needs known geometry for metric accuracy. Raised objects and hidden floor cannot be reconstructed by a floor warp. See *Homographies: straighten a photographed plane*.
4. **Cross-ratio survives perspective.** For four ordered collinear coordinates, $\mathrm{CR}(a, b; c, d) = (c - a)(d - b)/((c - b)(d - a))$. Ordinary spacings may change while this value stays fixed.
5. **Inverse warping fills the output grid.** For each destination pixel, use H^{-1} to locate its source sample. Bilinear interpolation blends four neighbors with weights summing to one; strong shrinking also needs antialiasing. See *Image warping: where do the new pixels come from?*.
6. **DLT estimates the mapping; normalization stabilizes it.** Each homography match contributes two rows to a $2n \times 9$ system. The smallest right singular vector supplies the matrix entries. Normalize each image independently, then recover $H = (T')^{-1}\widetilde{H}T$. See *Estimate a reliable mapping*.
7. **RANSAC rejects inconsistent matches.** Sample → fit → score → repeat → refit. Under its sampling assumptions, use $N = \lceil \log(1 - p) / \log(1 - w^s) \rceil$, where w is the inlier fraction, s the sample size, and p the desired sampling success probability. Consensus is not proof of a correct model.
8. **Distortion and depth variation require different tools.** Radial correction straightens lens-bent edges. The fundamental matrix constrains correspondence by $\ell' = Fx$ and $x'^T F x = 0$; it has rank two and seven DOF. Rectification aligns corresponding epipolar lines with matching rows. See *Distortion and two-view geometry*.

2.9.2 Short quiz

These original questions test the additions in this chapter. Foundation quiz solutions are in *Foundation quiz practice*.

Each question has its own complete worked answer below. Open it after trying the problem; all answers are visible in print.

Question 1

Original practice question

For a proper rotation with trace zero, what scalar quaternion component does the trace branch give?

Worked solution

Step 1 — Insert the trace

Why: This branch extracts the nonnegative quaternion scalar from a proper rotation.

$$w = \frac{1}{2}\sqrt{1 + \operatorname{tr} R} = \frac{1}{2}\sqrt{1 + 0} = \frac{1}{2}.$$

Step 2 — State the sign convention

Why: A quaternion and its negative encode the same rotation.

The trace branch chooses $w = +1/2$; negating the entire quaternion gives an equivalent representation.

Question 2

Original practice question

Before SLERP, what should you do if the endpoint quaternions have a negative dot product?

Worked solution

Step 1 — Inspect the quaternion dot product

Why: Opposite quaternion signs can describe equivalent rotations.

If $q_0^T q_1 < 0$, replace one endpoint by its negative: $q_1 \leftarrow -q_1$.

Step 2 — Check the resulting path

Why: Sign alignment avoids taking the unnecessarily long interpolation arc.

$$q_0^T(-q_1) = -q_0^T q_1 > 0.$$

SLERP then uses sign-aligned endpoints for a shortest rotation path; handle nearly equal endpoints with a stable interpolation branch.

Question 3

Original practice question

What is $\mathrm{CR}(0, 1; 2, 3)$? Should a projective warp of the line change it?

Worked solution

Step 1 — Substitute all four coordinates

Why: The ordering fixes which cross-ratio is being evaluated.

$$\mathrm{CR}(0, 1; 2, 3) = \frac{(2-0)(3-1)}{(2-1)(3-0)} = \frac{2 \cdot 2}{1 \cdot 3} = \frac{4}{3}.$$

Step 2 — Apply projective invariance

Why: A valid projectivity preserves this ordered collinear ratio.

The warped points have the same cross-ratio $4/3$, even when their ordinary distances change. Keep their order and handle ideal coordinates homogeneously if needed.

Question 4

Original practice question

How many nondegenerate matches and how many DLT rows are needed for a minimal homography estimate?

Worked solution

Step 1 — Count the constraints

Why: Each homography correspondence supplies two independent linear rows.

A homography has nine entries up to one common nonzero scale, leaving eight degrees of freedom. Four pairs give $2(4) = 8$ equations.

Step 2 — State the system and its rank

Why: A unique homogeneous solution needs a one-dimensional nullspace.

$$A \in \mathbb{R}^{8\times 9}, \qquad \operatorname{rank} A = 8, \qquad Ah = 0.$$

Four nondegenerate matches are minimal; degenerate arrangements need additional usable information.

Question 5

Original practice question

A source-to-destination warp translates content by $(6, -2)$ pixels. Where does destination pixel $(10, 10)$ sample?

Worked solution

Step 1 — Invert the translation

Why: An output pixel must look backward to its source location.

$$H^{-1} = \begin{pmatrix} 1 & 0 & -6 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{pmatrix}.$$

Step 2 — Multiply the output pixel

Why: The inverse warp identifies which source pixel to sample.

$$H^{-1} \begin{pmatrix} 10 \\ 10 \\ 1 \end{pmatrix} = \begin{pmatrix} 10 - 6 \\ 10 + 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 12 \\ 1 \end{pmatrix}.$$

Sample source $(4, 12)$.

Question 6

Original practice question

Four neighboring intensities are $0, 20, 40, 60$. What is their bilinear value at the center of the square?

Worked solution

Step 1 — Find the four weights

Why: The center is halfway along both axes of the square.

Each bilinear weight is $(1/2)(1/2) = 1/4$. The weights sum to one.

Step 2 — Take the weighted sum

Why: Bilinear interpolation combines all four neighboring values.

$$I = \frac{1}{4}(0) + \frac{1}{4}(20) + \frac{1}{4}(40) + \frac{1}{4}(60) = \frac{120}{4} = 30.$$

Question 7

Original practice question

For $p = 0.99$, $w = 0.8$, $s = 4$, is the standard RANSAC budget 8 or 9 trials? Does it guarantee the correct model?

Worked solution

Step 1 — Compute one clean-sample probability

Why: The standard model treats the four sampled inlier statuses as independent.

$$q = w^s = 0.8^4 = 0.4096.$$

Step 2 — Round the trial budget upward

Why: A fractional number of trials must be rounded up to meet the target probability.

$$N = \left\lceil \frac{\ln(1 - 0.99)}{\ln(1 - 0.4096)} \right\rceil = \lceil 8.7392 \ldots \rceil = 9.$$

Step 3 — Interpret the guarantee correctly

Why: Sampling success alone cannot certify the final model.

The calculation concerns at least one clean sample under the stated assumptions. Degeneracy, an unsuitable threshold or ambiguous image content can still spoil the estimated model.

Question 8

Original practice question

If $Fx = (0, -1, 4)^T$, what must a matching ordinary point (u', v') satisfy? Is u' determined?

Worked solution

Step 1 — Test point-line incidence

Why: The three supplied numbers are line coefficients.

$$\begin{pmatrix} u' & v' & 1 \end{pmatrix} \begin{pmatrix} 0 \\ -1 \\ 4 \end{pmatrix} = 0u' - v' + 4 = 0.$$

Step 2 — Read the free coordinate

Why: A line imposes one equation on two ordinary coordinates.

Thus $v' = 4$, while u' is not determined. Image evidence must select the actual match along the line.

CHAPTER

THREE

LECTURE 3: PUT CAMERA GEOMETRY TO WORK

Original quiz questions + fully worked solutions — complete problem statements, numbered red steps with a plain-English reason, and blue mathematics showing intermediate operations.

Can you turn an image size into a camera matrix, or a pixel into a 3D point? This chapter puts the earlier geometry to work on the assigned quizzes, then shows why even thousands of random trials cannot resolve an ambiguous match.

The Lecture 3 deck repeats theory already taught in Lectures 1–2. Those explanations keep their original homes. Here we add **new applications**, a complete projective calculation, and a sampling experiment.

3.1 Reading route

1. *Calibration from field of view*: solve the camera quiz and keep pixels separate from physical lengths.
2. *Find depth on a known plane*: intersect a viewing ray with geometry you know.
3. *A complete projective DLT calculation*: normalize, solve, undo normalization, and check a new point.
4. *Probability and the pentagon puzzle*: count matches, choose a trial budget, and experiment with RANSAC sampling.
5. *Review and source checklist*, then the final *TLDR and short quiz*.

Prerequisites: *camera projection*, *homographies*, *image warping*, and *DLT and RANSAC*.

Source: `wk3_cs532s26_ImageFormation+Homographies-1.pdf`, 80 PDF pages. Visible printed slide numbers match PDF page numbers on pages 2–80; the title page is unnumbered. References below use one-based **PDF pages**. Its pages 2–80 repeat Lecture 2's PDF pages 51–129. The coverage register maps every page to its teaching location.

3.2 From field of view to a camera matrix

A wider view fits more of the scene into the same image. That means a shorter focal length **in pixels**. A right triangle tells us exactly how much shorter.

Sources: lecture PDF pages 3–12; Spring 2019 Quiz 2, `cs532s19_quiz2-2.pdf`, P1–P4, PDF page 1. Spring 2020 Quiz 3, `cs532s20_quiz3.pdf`, page 1, repeats these questions under a mixed Spring 2019/Spring 2020 header.

3.2.1 Fit the view into the image

The **horizontal field of view** θ is the angle between the rays through the left and right image boundaries. For a centered, undistorted pinhole image, half the image width and the focal length form a right triangle.

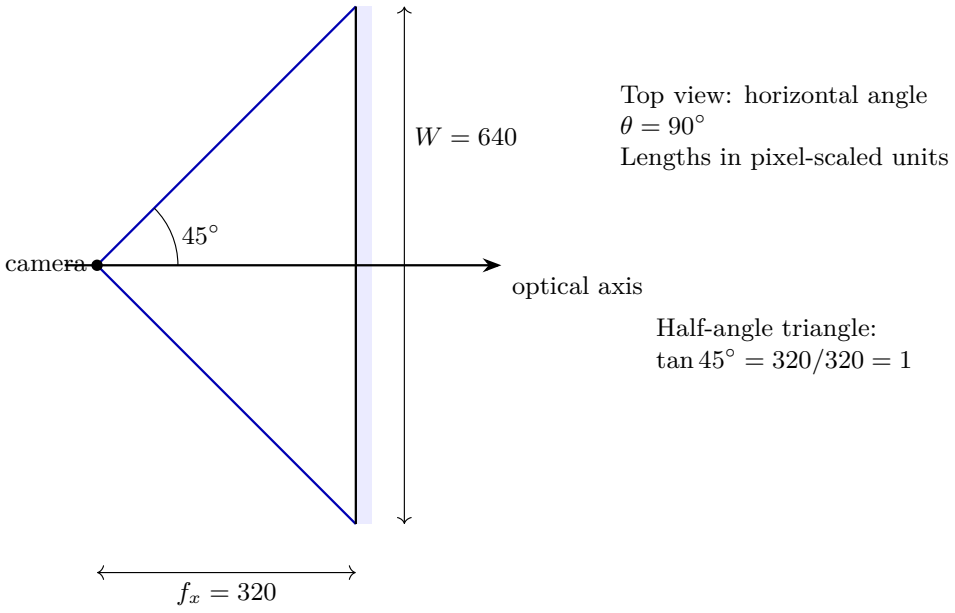


Fig. 3.1: Half the view makes a right triangle: opposite side $W/2$, adjacent side f_x. Both lengths must use the same units.

Formula — focal length from centered horizontal field of view

$$\tan\frac{\theta}{2} = \frac{W/2}{f_x}, \qquad f_x = \frac{W}{2\tan(\theta/2)}. \tag{3.1}$$

W is image width in pixels, f_x is horizontal focal length in pixels, and $0 < \theta < 180°$. Use a centered principal point and a rectilinear pinhole model. An off-center principal point needs separate left/right angles; strong lens distortion needs its own model.

Build a calibration estimate

1. **Input:** image width and horizontal FOV. Set the calculator's angle units correctly.
2. Compute f_x with the boxed equation.
3. To form K, supply the remaining assumptions: vertical focal length, skew, and principal point.
4. **Check:** doubling image resolution at unchanged FOV doubles focal length in pixels. A wider FOV reduces it.

FOV alone does not determine every entry of K.

Worked example 3.1 — the 640 × 480 camera. *Read the original problem and every worked step.*

3.2.2 A principal point has several descriptions

The **principal point** is where the optical axis meets the image plane. Its pixel coordinates, normalized coordinates, and physical 3D position describe related things in different units.

Worked example 3.2 — finish the calibration quiz. *Read the original problem and every worked step.*

Retest without duplicating the lesson

Use Spring 2020 Quiz 3 as a closed-notes retest. Answer all four problems, then check Examples 3.1–3.2. Before writing any coordinate, label it **pixel**, **normalized**, or **physical 3D**.

3.3 Find depth when the point lies on a known plane

One pixel supplies a ray, not a distance. A known plane supplies the missing condition: **keep moving along the ray until you reach the plane**.

Sources: lecture PDF pages 3–12; Spring 2020 Quiz 2, `cs532s20_quiz2.pdf`, P1–P2, PDF page 1; `cs532f15_quiz9.pdf`, P1, PDF page 1, printed **Spring 2019 Quiz 9** despite the filename.

3.3.1 First, keep the two kinds of coordinates straight

Worked example 3.3 — two points on one viewing ray. *Read the original problem and every worked step.*

3.3.2 Intersect the ray with a plane

Suppose the known plane is $AX + BY + CZ + D = 0$. Here A, B, C are plane coefficients, not the point named A in the previous example. Work entirely in the **camera frame**.

The image coordinate (x', y') lies on a virtual plane at focal length f. These three values use the same physical length unit. The ray direction with third component one is $q = (x'/f, y'/f, 1)^T$. If you start with a pixel (u, v) instead, first obtain $q = K^{-1}(u, v, 1)^T$ under the pinhole model; correct distortion before this step when necessary.

A scene point at axial depth Z is $P = Zq$. Substitute its coordinates into the plane:

$$A\frac{Zx'}{f} + B\frac{Zy'}{f} + CZ + D = 0 \quad \Rightarrow \quad Z\left(A\frac{x'}{f} + B\frac{y'}{f} + C\right) = -D.$$

Formula — depth from a ray and a plane

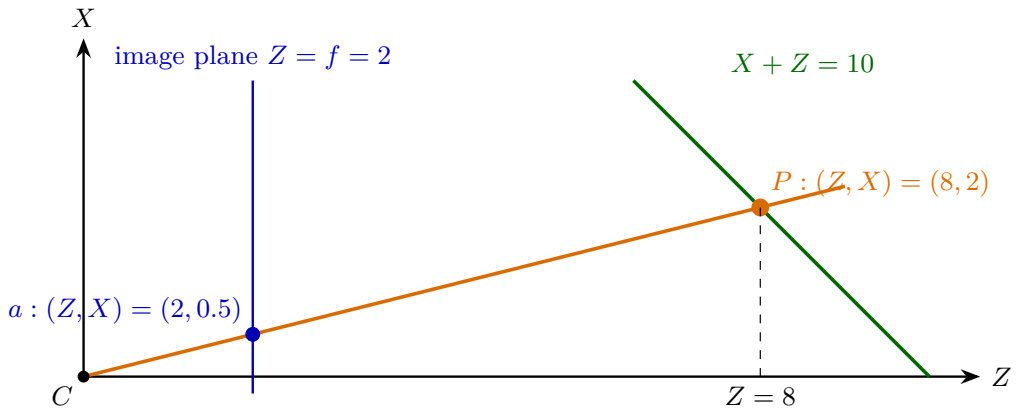
$$\gamma = (A, B, C)\, q, \qquad Z = -\frac{D}{\gamma} = -\frac{Df}{Ax' + By' + Cf}, \qquad P = Zq. \tag{3.2}$$

Require $(A, B, C) \neq \mathbf{0}$ and $\gamma \neq 0$. Z is **axial depth**, not Euclidean range. For dimensionless normal coefficients and D in meters, Z is in meters. Rescaling the plane coefficients leaves the result unchanged.

Recover a 3D point on the known plane

1. **Input:** a calibrated image coordinate and plane coefficients in the camera frame.
2. Form q with third component one. Compute $\gamma = (A, B, C)q$.
3. If $\gamma \neq 0$, calculate $Z = -D/\gamma$ and $P = Zq$.
4. **Check:** substitute P into the plane and reproject it. For a point visible in front of this camera, require $Z > 0$.

Near-parallel rays amplify input errors. Judge this using the angle between the ray and plane, rather than an unnormalized denominator.



Side view; Y is suppressed. Scene plane fixes the depth.

Fig. 3.2: A side view of Example 3.4. The image-plane intersection fixes the ray; the scene-plane intersection fixes its depth. The Y coordinate is suppressed only in the drawing.

Worked example 3.4 — put numbers into the depth answer

Adapted from: printed Spring 2019 Quiz 9, P1, whose requested symbolic answer is (3.2). The following numbers are an original companion.

Given: plane $X + Z = 10$, $f = 2$, $(x', y') = (0.5, 0.25)$, all lengths in the same unit.

1. Plane coefficients: $(A, B, C, D) = (1, 0, 1, -10)$.
2. Ray: $q = (0.25, 0.125, 1)^T$.
3. Denominator: $\gamma = 1(0.25) + 0(0.125) + 1 = 1.25$.
4. Depth: $Z = 10/1.25 = 8$; point $P = (2, 1, 8)^T$.
5. Check the plane: $2 + 8 = 10$. Check projection: $(fX/Z, fY/Z) = (0.5, 0.25)$.

The point is eight units forward along the optical axis. Its distance from the camera is $\sqrt{2^2 + 1^2 + 8^2} = \sqrt{69}$, a little larger.

3.3.3 When there is no useful visible intersection

Condition	Meaning	Action
$\gamma = 0$, $D \neq 0$	Ray is parallel to the plane and never reaches it	No finite solution
$\gamma = 0$, $D = 0$	The entire ray lies in the plane	Depth is undetermined
$\gamma \neq 0$, $D = 0$	Only intersection is the camera center, $Z = 0$	Cannot project this point
$Z < 0$	Intersection is behind the camera	Reject as a forward visible point

Keep the units honest: substituting raw pixel columns for x' while using a focal length in millimeters breaks the calculation before the algebra begins.

3.4 Recover a genuinely projective homography

The earlier DLT example recovered a shifted, stretched rectangle. This one adds a **changing denominator**: a square becomes a trapezoid, and its center does not map to the average of the destination corners.

Source: original numerical companion to lecture PDF pages 34–45, addressing the DLT/SVD priority in `ExamToDo.md`. Use the canonical *DLT rows and SVD procedure*; this section follows those steps with complete projective data.

Worked example 3.5 — normalize, solve, and recover

Given: the following four exact correspondences, in the same order.

Source (u, v)	Destination (u', v')
$(0, 0)$	$(0, 0)$
$(1, 0)$	$(2, 0)$
$(1, 1)$	$(2, 1)$
$(0, 1)$	$(0, 2)$

1. Normalize each image independently. For the source square, centroid $(0.5, 0.5)$ and mean radius $1/\sqrt{2}$ give scale 2. For the destination, centroid $(1, 0.75)$ and mean radius 1.28288937 give scale

1.10236596. Thus

$$T = \begin{pmatrix} 2 & 0 & -1 \\ 0 & 2 & -1 \\ 0 & 0 & 1 \end{pmatrix}, \qquad T' \approx \begin{pmatrix} 1.102366 & 0 & -1.102366 \\ 0 & 1.102366 & -0.826774 \\ 0 & 0 & 1 \end{pmatrix}.$$

The destination mean radius is the average of $1.25, 1.25, \sqrt{1.0625}, \sqrt{2.5625}$. This is **mean distance**, not root-mean-square distance.

2. Fill the normalized DLT rows. For the first pair, $\tilde{x} = (-1, -1, 1)^T$ and $\tilde{x}' \approx (-1.102366, -0.826774, 1)^T$. Its two rows, using (2.9), are approximately

$$(-1, -1, 1, 0, 0, 0, -1.102366, -1.102366, 1.102366),$$
$$(0, 0, 0, -1, -1, 1, -0.826774, -0.826774, 0.826774).$$

The other three pairs supply six more rows. The stacked matrix $\widetilde{A}$ is 8×9 with rank eight.

3. Read the full right null vector. A full SVD returns a 9×9 right-vector basis. One unit null vector, rounded here, is

$$h \approx (0.627538, 0, 0.209179, -0.156884, 0.418359, -0.052295, 0.189755, 0, 0.569265)^T.$$

Its negative is equally valid. Reshape **row by row**, then rescale for display so the last entry is one:

$$\widetilde{H} \approx \begin{pmatrix} 1.102366 & 0 & 0.367455 \\ -0.275591 & 0.734911 & -0.091864 \\ 1/3 & 0 & 1 \end{pmatrix}.$$

Request the full basis: the eight singular values returned for an 8×9 matrix do not themselves include a ninth listed zero. The extra right vector supplies its exact nullspace.

4. Undo normalization. Apply (2.10), using unrounded numbers in the calculation:

$$H = (T')^{-1} \widetilde{H} T \sim \begin{pmatrix} 4 & 0 & 0 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \end{pmatrix}.$$

5. Check a corner and a point you did not fit. For $(1, 1)$, the homogeneous result is $(4, 2, 2)^T$, hence $(2, 1)$: correct. The unmeasured square center gives

$$H(1/2, 1/2, 1)^T = (2, 1, 3/2)^T \quad \Rightarrow \quad (u', v') = (4/3, 2/3).$$

That differs from the destination vertex average $(1, 3/4)$. Perspective does not preserve ordinary averages. The nonzero h_{31} makes the denominator depend on u.

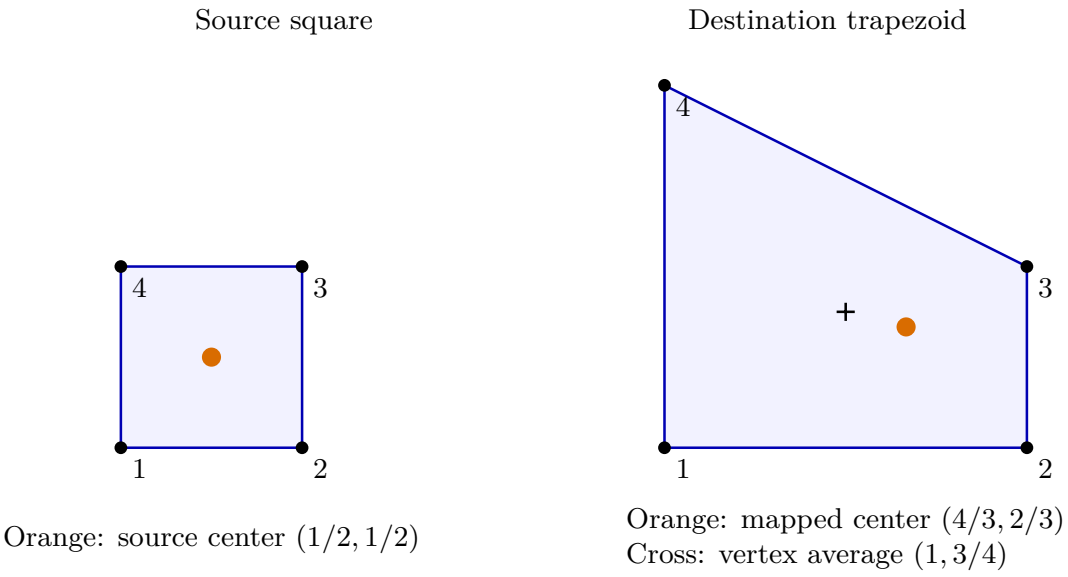


Fig. 3.3: The orange point is the mapped source center. The destination's marked vertex average is elsewhere; averaging coordinates is not a projective operation.

3.4.1 Use the matrix to make an image

The *court-rectification example* is the slide deck's real-image application, repeated here on PDF page 22. Its *inverse-warping procedure* tells you how to fill output pixels after estimating H. Keep the directions separate: the matrix above sends source coordinates to destination coordinates; filling a destination pixel requires the reverse map.

Exam check: can you explain where each of the nine entries came from, why four matches give eight independent equations, and why the coordinate transforms must be undone in that order? The answer should describe the calculation, not just name "SVD."

3.5 Probability and the pentagon puzzle

How many random tries are enough? First ask what counts as a successful try. RANSAC's trial budget depends on that definition and on how samples are drawn.

Sources: lecture PDF pages 46–55; Fall 2025 Quiz 1, `cs532f25_quiz_1_solution.pdf`, Q1–Q2, PDF page 1; Spring 2019 Quiz 3, `cs532s19-quiz3.pdf`, P1's four bullets, PDF page 1. The general RANSAC algorithm and trial-count formula remain in *Lecture 2*.

3.5.1 Start with the coin quiz

Worked example 3.6 — a winning streak, then repeated attempts. *Read the original problem and every worked step.*

3.5.2 Five corners, twenty-five possible matches

Assume two pictures show only a generic irregular pentagon. All five corners are detected, but their identities are unknown. The quiz asks you to try **every possible pairing**.

Worked example 3.7 — count matches and budget the search. *Read the original problem and every worked step.*

The small-pool qualification

Here the pool is unusually small. Four independent draws are an approximation to a real four-correspondence sampler. If asked for an **exact** count, specify the sampling rule.

Let q denote the probability that **one counted trial** is clean. Assuming independent trials and a generic pentagon with no three corners collinear:

Sampling rule for one counted trial	Clean-sample probability q	Trials for 99%
Independent inlier-status approximation	$(5/25)^4 = 0.0016$	2,876
Four distinct pairs drawn uniformly from all 25; count even invalid samples	$\binom{5}{4}/\binom{25}{4} = 1/2530$	11,649
Reject repeated source or destination corners; count only valid four-pair samples	$5/[\binom{5}{4}^2 4!] = 1/120$	551

Why the last denominator? Choose four of five source corners, four of five destination corners, then one of 4! bijections: $5 \times 5 \times 24 = 600$ valid sets. Only five are subsets of the intended five matches. These are different experiments, so their trial counts need not agree. If rejected draws consume runtime, include their cost separately.

For an exam: state the standard w^4 assumption before giving 2,876. If the sampling procedure is specified more precisely, use its corresponding q in the same complement-probability calculation.

3.5.3 Try the sampling experiment

Change the inlier percentage and trial count. Blue dots are good matches; orange dots are bad. Four blue dots make a clean hypothesis. The display compares the predicted chance of **at least one** clean sample with results from 200 simulated experiments.

Explore RANSAC sampling probability — print edition

Static experiment: assume four independently drawn match statuses per hypothesis. Then $q = w^4$ and $P = 1 - (1 - q)^N$. The simulator uses this approximation, not the exact finite-pool pentagon samplers above.

Inlier fraction w	Trials N	Chance of at least one clean sample
20%	100	$1 - (1 - 0.2^4)^{100} = 14.80\%$
20%	2,876	$1 - (1 - 0.2^4)^{2876} = 99.0001\%$
50%	72	$1 - (1 - 0.5^4)^{72} = 99.04\%$

A single experiment can succeed or fail. Across many independent experiments, the observed success fraction tends toward the predicted probability; it need not equal it exactly. A clean sample is a sampling event, not proof of a correct model.

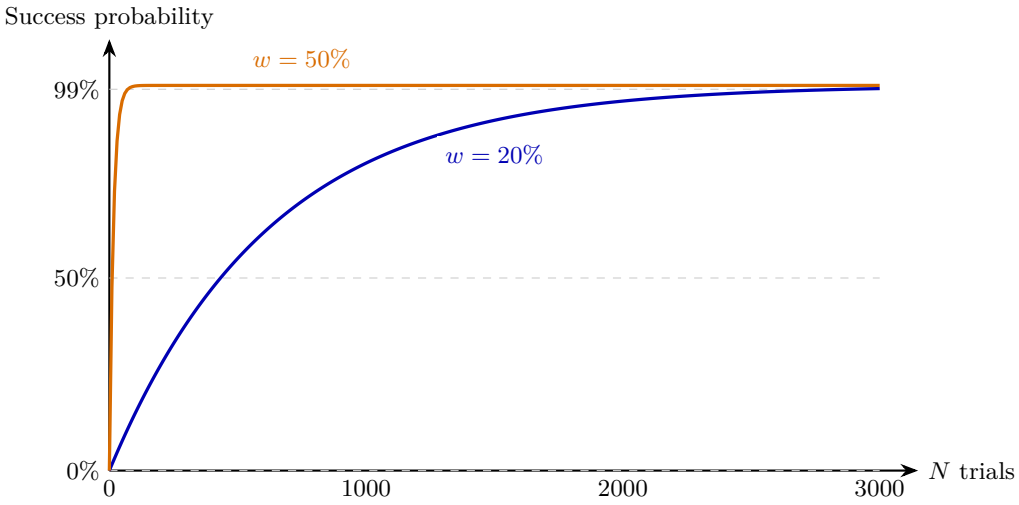


Fig. 3.4: More inliers can reduce the required trial count dramatically. The vertical axis measures sampling success, not geometric accuracy.

3.5.4 More trials cannot solve a symmetry

Worked example 3.8 — replace the shape with a regular pentagon. *Read the original problem and every worked step.*

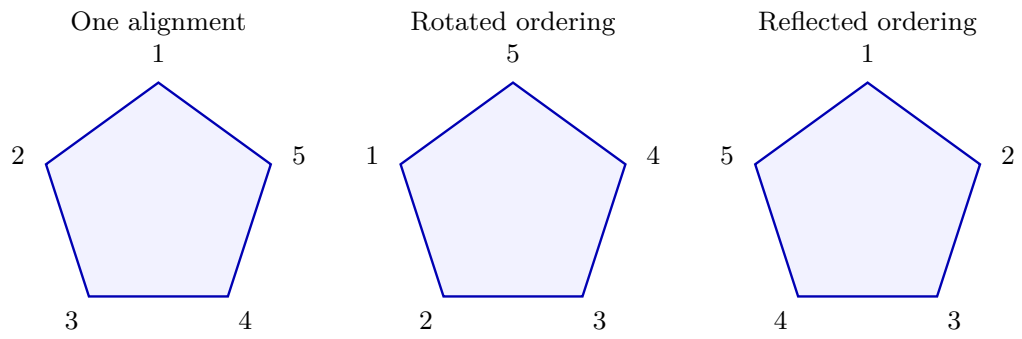


Fig. 3.5: The numbers are explanatory labels, not visible features in the quiz images. The identical outlines cannot tell you which ordering is intended.

What does your confidence actually mean?

Before quoting "99%," finish the sentence: "a 99% chance of drawing at least one ___, under the sampling assumption ___." If your answer says "a 99% chance the reconstruction is right," you have claimed more than the calculation supports.

3.6 Original quiz questions and worked solutions

Read the problem in black. Follow the numbered red steps. Work through the blue mathematics. Every step explains why it is needed. Matrix calculations show the inputs and intermediate results; written-reasoning questions explain the decision instead of inventing numerical data.

Source: the supplied lecture-organized quiz booklet. Full problems follow its wording and givens, with typography normalized; the historical PDF filenames appear below. Source-key corrections and added numerical companions are marked.

Retests have their own questions and full solutions. Problems labeled **Original practice** are the booklet's seven supplemental examples, separate from the 18 historical quizzes. Colored steps remain numbered and readable in grayscale.

3.6.1 Jump to a quiz

- *Spring 2019 Quiz 3 — Pentagon, homography, RANSAC*
- *Spring 2019 Quiz 2 — FOV, calibration, homogeneous scaling*
- *Spring 2020 Quiz 3 — Duplicate camera model*
- *Spring 2020 Quiz 2 — Pinhole ray and cross product*
- *Older Quiz 9 — Pinhole ray intersects a 3D plane*

- *A1. Homography, DLT and SVD (L3, slides 34–45)*

3.6.2 Spring 2019 Quiz 3 — Pentagon, homography, RANSAC

Historical source: `3dcvMid/olderquizzes/cs532s19-quiz3.pdf`.

Problem 1(a–d)

Original problem · quiz booklet

Two images show only the same *irregular five-sided polygon*, related by an unknown homography. All five corners are detected in each image. You create every possible putative corner correspondence. (a) How many? (b) What inlier rate? (c) How many RANSAC iterations are needed to reliably estimate the homography? (d) What changes for a regular/equilateral pentagon?

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Count all candidate pairs

Why: Every corner in the first image can pair with every corner in the second.

There are 5 features in image A and 5 in image B. Exhaustive matching forms all ordered cross-image pairs, so $\boxed{5 \times 5 = 25}$ putative correspondences.

Step 2 — Count the true inliers

Why: The inlier ratio is measured relative to one intended corner assignment.

Assume one unique correct correspondence for each of the 5 image-A corners. Then 5 of the 25 pairs are inliers, so $\boxed{w = 5/25 = 0.2}$ and outlier rate $e = 0.8$.

Step 3 — Set up the RANSAC probability

Why: A homography needs four nondegenerate correspondences in one clean sample.

The minimal homography solver requires $s = 4$ nondegenerate correspondences. The usual RANSAC independent-sample approximation sets

$$1 - (1 - w^s)^N \geq p \quad \Rightarrow \quad N \geq \frac{\ln(1-p)}{\ln(1-w^s)}.$$

Step 4 — Insert a confidence level

Why: The question needs a chosen confidence before a numerical trial count exists.

For the *illustrative* common confidence $p = 0.99$, $w^4 = 0.2^4 = 0.0016$, hence

$$\boxed{N_{\text{approx}} = \left\lceil \frac{\ln(0.01)}{\ln(0.9984)} \right\rceil = 2876}.$$

The quiz does not specify a confidence; the general formula is the source-supported answer. Sampling all putative matches independently also ignores the one-to-one matching constraint, so these iteration counts describe a simplified model.

Step 5 — Check the finite sampling model

Why: Drawing distinct pairs from a small pool changes the clean-sample probability.

Because there are only 25 candidate pairs and each RANSAC *sample selects 4 distinct pairs*, the exact probability of four inliers in a uniform draw without replacement is smaller:

$$q_{\text{exact}} = \frac{\binom{5}{4}}{\binom{25}{4}} = \frac{5}{12650} \approx 0.00039526.$$

If *each trial samples afresh* and we desire $p = 0.99$,

$$\boxed{N_{\text{finite}} = \left\lceil \frac{\ln(.01)}{\ln(1 - 5/12650)} \right\rceil = 11649}.$$

This is an optional exact finite-population refinement; the standard lecture approximation uses w^s.

Step 6 — Explain the regular-pentagon ambiguity

Why: More trials cannot distinguish equally supported symmetries.

For a regular pentagon, rotational/reflection symmetry makes the corner identities ambiguous: several match permutations may fit geometrically similar configurations. Without texture/orientation/other cues, a unique physical correspondence is less secure. Explain that good geometric fits need not identify the *intended* corner labeling. For a genuinely regular pentagon, each of its five corners can serve as the first aligned corner. Each choice permits a clockwise or counterclockwise ordering, giving $5 \times 2 = 10$ equally supported alignments: five rotations and five reflections. If an external orientation constraint excludes reflections, five remain. The 20% inlier rate refers to one selected alignment, not a union of different models. Equal side lengths alone do not imply these symmetries; the answer assumes a regular pentagon with equal angles too.

Source check / assumptions: The irregular-pentagon problem still requires nondegenerate 4-tuples (e.g. no three source points collinear). The exact without-replacement refinement above does not imply that *every* four-inlier subset yields a uniquely acceptable model under all degeneracies.

3.6.3 Spring 2019 Quiz 2 — FOV, calibration, homogeneous scaling

Historical source: `3dcvMid/olderquizzes/cs532s19_quiz2-2.pdf`.

Problem 1

Original problem · quiz booklet

A digital camera has image resolution 640×480 pixels and horizontal field of view 90°. Determine the focal length in pixels and draw a sketch.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Draw the half-angle triangle

Why: The symmetric horizontal field of view splits into two equal angles.

Take a horizontal cross-section through the optical axis. The full horizontal FOV is 90°, so each edge ray is 45° from the optical axis. The image half-width is $W/2 = 320$ pixels.

Step 2 — Rearrange the tangent relation

Why: Similar triangles connect pixel half-width and focal length.

In the pinhole triangle, $\tan(\theta/2) = (W/2)/f_x$. Rearrange: $f_x = (W/2)/\tan(\theta/2)$.

Step 3 — Substitute the camera's numbers

Why: The requested focal length is in pixels.

Substitute $W = 640$, $\theta = 90^\circ$:

$$\boxed{f_x = \frac{320}{\tan 45^\circ} = 320 \text{ px}}.$$

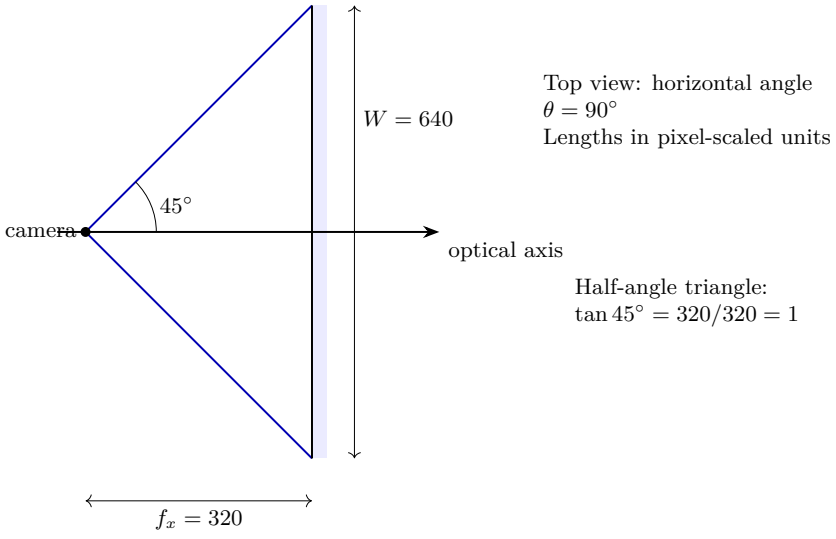


Fig. 3.6: The half-width and half field of view determine pixel focal length.

Problem 2

Original problem · quiz booklet

Write the intrinsic calibration matrix K for the camera.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Write the intrinsic matrix

Why: Its entries separate focal lengths, skew and principal point.

Use standard intrinsic form

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

.

Step 2 — State the missing assumptions

Why: Horizontal field of view alone does not determine every entry.

The horizontal FOV determines $f_x = 320$ px only. To fill the other terms, *assume* square pixels ($f_y = f_x$), zero skew ($s = 0$), and centered principal point ($(c_x, c_y) = (320, 240)$).

Step 3 — Fill in all matrix entries

Why: Only the stated extra assumptions justify a numerical K.

Under those common assumptions,

$$\boxed{K = \begin{bmatrix} 320 & 0 & 320 \\ 0 & 320 & 240 \\ 0 & 0 & 1 \end{bmatrix}}.$$

Without them, f_y, s, c_x, c_y cannot all be inferred from the given horizontal FOV alone.

Problem 3(a–b)

Original problem · quiz booklet

In $x = K[R \mid t]X$, give dimensions of every matrix/vector. If the right-hand side is multiplied by 2 to obtain $x' = 2K[R \mid t]X$, how are x' and x related?

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — List matrix dimensions

Why: Adjacent matrix dimensions must agree in the projection product.

$K : 3 \times 3$, $R : 3 \times 3$, $t : 3 \times 1$, $[R \mid t] : 3 \times 4$, $X : 4 \times 1$, $x : 3 \times 1$; total projection matrix $P = K[R \mid t]$ is 3×4.

Step 2 — Apply the common scale

Why: Homogeneous coordinates permit a nonzero common multiplier.

Homogeneous images are defined up to nonzero scale: $x' = (2q_1, 2q_2, 2q_3)^T$ vs $x = (q_1, q_2, q_3)^T$.

Step 3 — Divide out that scale

Why: Ordinary image coordinates depend on ratios, so the location stays fixed.

Dehomogenize: $u' = 2q_1/(2q_3) = q_1/q_3 = u$ and $v' = 2q_2/(2q_3) = v$. So $\boxed{x' \sim x}$: identical physical image points.

Problem 4

Original problem · quiz booklet

What are the 3D coordinates of the image principal point in the camera coordinate system?

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Locate the optical-axis intersection

Why: That intersection defines the principal point geometrically.

The principal point is the intersection of the camera optical axis with the image plane. In a virtual plane $z = f_{\text{physical}}$, the optical axis is $X = Y = 0$.

Step 2 — Give its camera-frame position

Why: The virtual image plane is at physical focal depth.

Therefore its camera-frame position is $\boxed{(0, 0, f_{\text{physical}})}$. If the image-plane coordinate units are *chosen as pixel-equivalent units*, one may write $(0, 0, 320)$ using the computed focal length in pixels.

Step 3 — Separate physical and pixel coordinates

Why: Pixel focal length does not reveal millimeters without pixel pitch.

Distinguish this 3D position from the image-array pixel coordinates $\boxed{(c_x, c_y) = (320, 240)}$ under the centered-principal-point assumption. Physical f in meters/mm is not determined by pixel focal length alone without pixel pitch.

3.6.4 Spring 2020 Quiz 3 — Duplicate camera model

Historical source: `3dcvMid/olderquizzes/cs532s20_quiz3.pdf`.

Header note: the supplied PDF contains mixed/overprinted year labels. The filename and separate retest entry are retained.

Problem 1

Original problem · quiz booklet

A camera is 640 pixels wide by 480 tall with horizontal FOV 90°. Find pixel focal length and sketch the rays.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Split the field of view

Why: The centered optical axis produces two 45-degree half angles.

Half-angle is $90^\circ/2 = 45^\circ$, and half-width is $640/2 = 320$ pixels. Draw the camera center, optical axis and the rays to the two horizontal ends of the image plane.

Step 2 — Write the tangent equation

Why: The opposite side is the 320-pixel half-width.

$\tan 45^\circ = (320\,\text{px})/f_x$.

Step 3 — Solve the focal length

Why: The tangent of 45 degrees is one.

Since $\tan 45^\circ = 1$, $\boxed{f_x = 320\,\text{px}}$. The half-angle/FOV diagram on the preceding quiz applies.

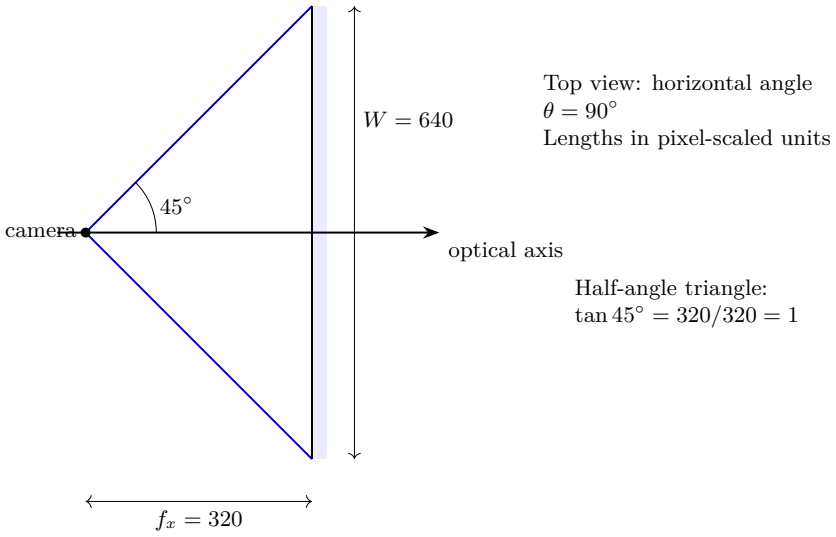


Fig. 3.7: The same half-angle construction applies to this separate retest.

Problem 2

Original problem · quiz booklet

Write the camera intrinsic matrix K.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Write the general matrix

Why: This shows which intrinsic parameters remain unknown.

Use

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

. The supplied FOV fixes only f_x.

Step 2 — Declare the calibration assumptions

Why: Centering, square pixels and zero skew are additional information.

Explicitly assume square pixels $f_y = f_x = 320$, zero skew $s = 0$, and principal point at $(320, 240)$.

Step 3 — Substitute every entry

Why: The assumptions now determine a full numerical matrix.

Then

$$K = \begin{bmatrix} 320 & 0 & 320 \\ 0 & 320 & 240 \\ 0 & 0 & 1 \end{bmatrix}$$

.

Problem 3(a–b)

Original problem · quiz booklet

Give dimensions in $x = K[R \mid t]X$ and explain what doubling the full homogeneous right-hand side changes.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Check every matrix shape

Why: The product maps four world entries to three image entries.

Identify $K \in \mathbb{R}^{3\times3}$, $R \in \mathbb{R}^{3\times3}$, $t \in \mathbb{R}^{3\times1}$, $[R \mid t] \in \mathbb{R}^{3\times4}$, $X \in \mathbb{R}^{4\times1}$, $x \in \mathbb{R}^{3\times1}$.

Step 2 — Double the homogeneous column

Why: The question rescales all three components together.

For projected $x = (a, b, c)^T$, the doubled homogeneous vector $x' = 2x = (2a, 2b, 2c)^T$.

Step 3 — Dehomogenize both columns

Why: The common multiplier cancels from both ratios.

The dehomogenized coordinates are the same: $(a/c, b/c) = (2a/2c, 2b/2c)$. Answer: no image location change.

Problem 4

Original problem · quiz booklet

Find the 3D coordinates of the principal point in the camera reference frame.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Write the optical-axis ray

Why: Its two lateral camera coordinates are zero.

Set the optical-axis ray to $\{(0, 0, Z) : Z > 0\}$; intersect with virtual image plane $z = f_{\text{physical}}$.

Step 2 — Intersect with the virtual image plane

Why: The plane fixes the third coordinate to the focal length.

The point is $\boxed{(0, 0, f_{\text{physical}})}$ in camera length units; in pixel-normalized plane units $f = 320$ gives the coordinate triple $(0, 0, 320)$.

Step 3 — Distinguish the pixel-array answer

Why: Camera-frame positions and pixel indices are different quantities.

If the question instead asks for image-array pixel coordinates, use $(c_x, c_y) = (320, 240)$ under the stated centering assumption.

3.6.5 Spring 2020 Quiz 2 — Pinhole ray and cross product

Historical source: `3dcvMid/olderquizzes/cs532s20_quiz2.pdf`.

Problem 1

Original problem · quiz booklet

Let $A = (X, Y, Z)$ be a camera-frame 3D point. Its viewing ray passes through image point a on the image plane. Given $f = Z/10$, determine the 3D camera-frame coordinates of a; sketch.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Use the common viewing ray

Why: A pinhole camera center, image-plane point and scene point are collinear.

For a virtual pinhole image plane at $z = f$, the 3D point and pixel lie on the same ray from the origin. Similar triangles give $x_a/f = X/Z$ and $y_a/f = Y/Z$.

Step 2 — Write all three coordinates

Why: The image point lies on the plane with third coordinate f.

Thus $a = (fX/Z, fY/Z, f)$.

Step 3 — Substitute the supplied focal length

Why: The common scale becomes one tenth.

Substitute $f = Z/10$, assuming $Z \neq 0$:

$$\boxed{a = (X/10, Y/10, Z/10) = A/10}.$$

Step 4 — Sketch the geometry

Why: The drawing should show the same ray through both points.

Draw origin O, the point A, the intermediate image plane $z = f$, and a on the segment of ray OA (for $Z > 0$ and $f = Z/10$). The vectors Oa and OA point in the same direction.

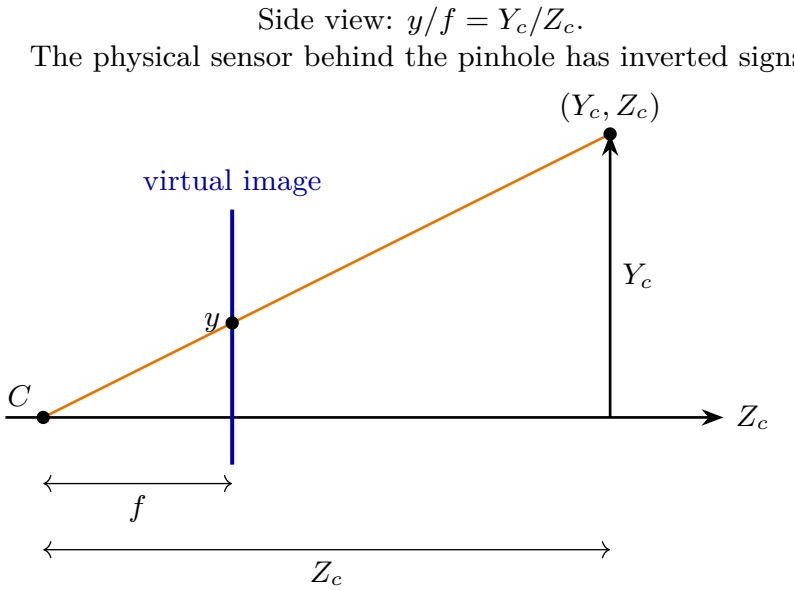


Fig. 3.8: Pinhole viewing-ray sketch, shown as a side view. In this quiz the image-plane point is one tenth of the scene vector; the schematic is not drawn to that scale.

Problem 2

Original problem · quiz booklet

Develop the expression for $A \times a$.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Insert the two vectors

Why: The preceding part fixes every coordinate of the image-plane point.

Set $A = (X, Y, Z)^T$ and $a = (X/10, Y/10, Z/10)^T$.

Step 2 — Set up the cross product

Why: Its determinant representation organizes the three component calculations.

Use the cross-product determinant:

$$A \times a = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ X & Y & Z \\ X/10 & Y/10 & Z/10 \end{vmatrix}.$$

Step 3 — Expand each component

Why: Equal scalar products cancel pairwise.

Expand components: $(YZ/10 - ZY/10,\ ZX/10 - XZ/10,\ XY/10 - YX/10)$.

Step 4 — Interpret the zero vector

Why: Parallel Euclidean 3D directions have zero cross product.

Every component is zero, so $\boxed{A \times a = (0,0,0)^T}$. Geometrically, collinear vectors have zero cross product.

3.6.6 Older Quiz 9 — Pinhole ray intersects a 3D plane

Historical source: `3dcvMid/olderquizzes/cs532f15_quiz9.pdf`.

Header note: this filename contains "2015"; the PDF is printed Spring 2019.

Problem 1

Original problem · quiz booklet

A 3D plane has homogeneous parameters $\pi = (A, B, C, D)^T$, i.e. $AX + BY + CZ + D = 0$. A pinhole camera has center at the 3D origin, orientation I, focal length f, and observes pixel image-plane coordinates (x', y'). Find the 3D depth of the intersection of the viewing ray with the plane. Draw a sketch.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Parameterize the viewing ray

Why: Every candidate 3D point must lie along that ray.

The ray begins at camera origin and passes through (x', y', f) on the virtual image plane. Its parametric form is $r(\lambda) = \lambda(x', y', f) = (\lambda x', \lambda y', \lambda f)$.

Step 2 — Express X and Y using depth

Why: Pinhole ratios reduce the unknown point to one unknown Z.

Using pinhole geometry, $x'/f = X/Z$, $y'/f = Y/Z$, hence $X = (x'/f)Z$ and $Y = (y'/f)Z$.

Step 3 — Substitute into the plane

Why: The requested intersection must satisfy the plane equation.

Substitute into plane equation:

$$A\frac{x'Z}{f} + B\frac{y'Z}{f} + CZ + D = 0.$$

Step 4 — Collect the depth terms

Why: Factoring Z isolates the quantity we want.

Factor out Z: $Z(Ax'/f + By'/f + C) = -D$.

Step 5 — Solve the axial depth

Why: Division is valid only when the ray-plane denominator is nonzero.

Solve the optical-axis depth:

$$\boxed{Z = -\frac{Df}{Ax' + By' + Cf}}.$$

Step 6 — Recover the full point

Why: Once depth is known the ray determines both lateral coordinates.

Recover the complete intersection point, if requested:

$$\boxed{X = \frac{x'Z}{f}, \quad Y = \frac{y'Z}{f}, \quad Z = -\frac{Df}{Ax' + By' + Cf}}.$$

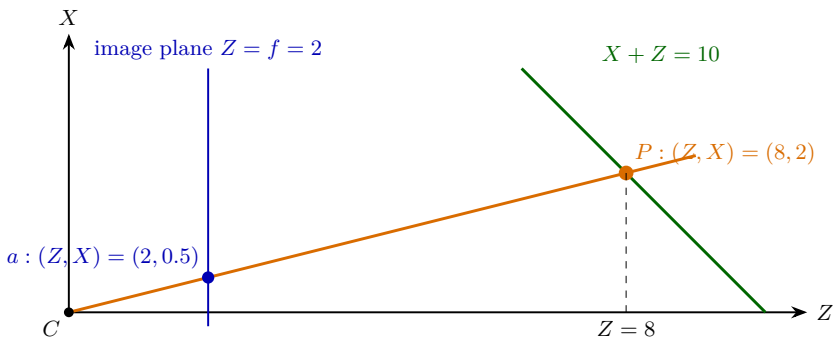
Equivalently $\lambda = -D/(Ax' + By' + Cf)$ and $r(\lambda)$ gives all three coordinates.

Step 7 — Check exceptional cases

Why: Parallel, contained and behind-camera cases need different interpretations.

Check exceptions: if $Ax' + By' + Cf = 0$ and $D \neq 0$, the viewing ray is parallel to the plane (no finite intersection). If both equal zero, the entire ray lies in the plane (no unique depth). A visible front-of-camera intersection normally requires $Z > 0$.

Source check / assumptions: The formula gives Z (depth along optical axis), not necessarily Euclidean camera-to-point range $\sqrt{X^2 + Y^2 + Z^2}$.



Side view; Y is suppressed. Scene plane fixes the depth.

Fig. 3.9: A ray–plane illustration. Its numerical labels belong to the separate Example 3.4 companion in the teaching section.

3.6.7 A1. Homography, DLT and SVD (L3, slides 34–45)

Original practice from the supplied booklet; not a historical quiz question.

Illustrative problem

Original problem · quiz booklet

Find the homography mapping $(0,0) \mapsto (0,0)$, $(1,0) \mapsto (1/2,0)$, $(0,1) \mapsto (0,1)$, $(1,1) \mapsto (1/2,1/2)$. Explain where SVD would enter for noisy pairs.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Write the projective mapping

Why: Homogeneous scaling makes the image coordinates ratios.

Write the unknown homography $H = [h_{ij}]$, with $h_{33} = 1$ for this convenient example. Use $u = (h_{11}x + h_{12}y + h_{13})/(h_{31}x + h_{32}y + 1)$ and $v = (h_{21}x + h_{22}y + h_{23})/(h_{31}x + h_{32}y + 1)$.

Step 2 — Use the origin match

Why: It isolates the translation-like numerator entries.

The origin maps to itself, so at $(x,y) = (0,0)$: $h_{13} = h_{23} = 0$.

Step 3 — Use the horizontal match

Why: Its nonzero input coordinate constrains one denominator term.

At $(1,0) \to (1/2,0)$: $h_{11}/(h_{31}+1) = 1/2$ and $h_{21}/(h_{31}+1) = 0$. Therefore $2h_{11} = h_{31} + 1$ and $h_{21} = 0$.

Step 4 — Use the vertical match

Why: This constrains the remaining numerator and denominator entries.

At $(0,1) \to (0,1)$: $h_{12}/(h_{32}+1) = 0$ and $h_{22}/(h_{32}+1) = 1$. Thus $h_{12} = 0$ and $h_{22} = h_{32} + 1$.

Step 5 — Use the final corner

Why: Combining previous constraints determines the projective denominator.

At $(1,1) \to (1/2,1/2)$: $h_{11}/(h_{31}+h_{32}+1) = 1/2$ and $h_{22}/(h_{31}+h_{32}+1) = 1/2$. Compare with step 3 to get $h_{32} = 0$, hence $h_{22} = 1$. Then $h_{31} + 1 = 2$, so $h_{31} = h_{11} = 1$.

Step 6 — Assemble and project a check point

Why: A held-out point tests the resulting matrix.

Assemble and verify:

$$H = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}.$$

For $(x, y) = (2, 1)$, $H(2, 1, 1)^T = (2, 1, 3)^T$, so the new image coordinates are $(2/3, 1/3)$.

Step 7 — Build the general DLT equations

Why: Each correspondence supplies two linear constraints on nine homogeneous entries.

In general, each match $(x, y) \to (u, v)$ yields two rows of $Ah = 0$:

$$A_i = \begin{bmatrix} x & y & 1 & 0 & 0 & 0 & -ux & -uy & -u \\ 0 & 0 & 0 & x & y & 1 & -vx & -vy & -v \end{bmatrix}.$$

With more than four noisy pairs, choose the *last right singular vector* of $A = U\Sigma V^T$ (smallest singular value) and reshape it to H. Coordinate normalization before DLT improves conditioning: $H = T'^{-1}\widetilde{H}T$.

Step 8 — Show the complete numerical system

Why: The actual eight rows expose the nullspace that SVD must recover.

For these four actual matches, the full 8×9 homogeneous DLT system is $Ah = 0$, where

$$A = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & -1/2 & 0 & -1/2 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & -1 & -1 \\ 1 & 1 & 1 & 0 & 0 & 0 & -1/2 & -1/2 & -1/2 \\ 0 & 0 & 0 & 1 & 1 & 1 & -1/2 & -1/2 & -1/2 \end{pmatrix},$$

$$h = \begin{pmatrix} h_{11} & h_{12} & h_{13} & h_{21} & h_{22} & h_{23} & h_{31} & h_{32} & h_{33} \end{pmatrix}^T.$$

The solution $h = (1, 0, 0, 0, 1, 0, 1, 0, 1)^T$ makes each row zero. For four pairs, use a full right-singular basis: the null vector is the ninth right singular vector of this wide matrix, not the last vector of a reduced eight-vector basis.

Source check / assumptions: Fixing $h_{33} = 1$ is only valid if the actual h_{33} is nonzero; the homogeneous SVD formulation avoids relying on that choice.

3.7 Review and source checklist

This chapter adds applications to the theory already taught. Use this checklist to find the right calculation without reading repeated explanations.

3.7.1 New formula boxes

Task	Canonical formula and application
Turn centered horizontal FOV into pixel focal length	(3.1), *calibration*
Intersect a calibrated ray with a known plane	(3.2), *ray–plane depth*

For DLT rows, coordinate normalization, and RANSAC's trial budget, use the original *Lecture 2 boxes*. This chapter's projective numerical example and sampling cases apply those formulas to new data.

3.7.2 Worked examples and quiz coverage

Example	Skill	Source
3.1	FOV and calibration matrix	Spring 2019 Quiz 2 P1–P2; Spring 2020 Quiz 3 retest
3.2	Dimensions, homogeneous scale, physical principal point	Same quizzes P3–P4; foundation dimensions linked to L1
3.3	Image-plane intersection and Euclidean cross product	Spring 2020 Quiz 2 P1–P2
3.4	Ray–plane depth and visibility	Printed Spring 2019 Quiz 9 P1; original numerical companion
3.5	Full normalized projective DLT and held-out check	Original companion to slides 34–45
3.6	Independent events and repeated attempts	Fall 2025 Quiz 1 Q1–Q2
3.7	Candidate counts, inliers, trial budget	Spring 2019 Quiz 3 P1 bullets 1–3
3.8	Indistinguishable regular-polygon alignments	Same quiz P1 bullet 4

3.7.3 Repeated slide material: where it lives

The entire 80-page deck was checked, including diagrams and formulas. Page 1 is the title; pages 2–80 repeat Lecture 2's PDF pages 51–129.

Lecture 3 PDF pages	Teaching location
2–12: image formation and camera motion	*Lecture 1 cameras*, including the camera-motion slide exercise
13–22: homographies and transformation hierarchy	*Lecture 1 transformations*, *Lecture 2 homographies and court task*
23–32: forward/inverse warping and interpolation	*Lecture 2 warping*
33–45: estimation, DLT, normalization	*Lecture 2 estimation*, plus the *new numerical walkthrough*
46–55: RANSAC, consensus, matching pipeline	*Lecture 2 estimation*, plus the *new quiz applications*
56–80: radial distortion, epipolar geometry, fundamental matrix, rectification	*Lecture 2 distortion and two-view introduction*; later two-view problems belong to Lecture 4

3.7.4 Check before handing in a solution

- **Coordinates:** are these pixels, physical image-plane coordinates, normalized rays, or homogeneous points?
- **Assumptions:** did you supply square pixels and a centered principal point before writing the full K?
- **Depth:** does the intersection satisfy both the plane and the image ray, with $Z > 0$?
- **DLT:** did you use the full right-vector basis and undo both normalizations?
- **Probability:** what does one counted trial sample, and what event does the confidence describe?

Continue to the final *TLDR and short quiz*.

3.8 TLDR

Lecture 3 in one sentence: use camera geometry to solve concrete calibration and depth problems, then distinguish a probable clean sample from an identifiable correct model.

3.8.1 The important things

1. **FOV determines horizontal focal length only after stating the model.** A centered 640-pixel-wide, 90° pinhole view gives $f_x = 320$ pixels. Square pixels, zero skew, and a centered principal point supply the remaining assumptions for the quiz's K. See *calibration*.
2. **Pixel focal length is not physical focal length.** Multiply by physical pixel pitch to change units. The principal point is (c_x, c_y) in pixels but $(0, 0, f_{\text{phys}})$ on the positive virtual image plane.
3. **A scaled Euclidean 3D vector stays on the same camera ray.** The image-plane intersection is one tenth of the scene vector when $f = Z/10$. Its cross product with that vector is zero. See *the ray quiz*.
4. **A known plane can supply missing depth.** Use (3.2), then check plane incidence, reprojection, and positive depth. A parallel ray may have no intersection or lie entirely in the plane.
5. **Projective mappings do not preserve coordinate averages.** The fitted trapezoid's mapped center differs from the average of its corners. The *normalized DLT walkthrough* shows all the numerical stages.
6. **Count the complement when attempts repeat.** The coin quiz gives $1/8$ per attempt and $1 - (7/8)^7 \approx 60.73\%$ across seven independent attempts. See *probability*.
7. **State the sampler before quoting a RANSAC budget.** The pentagon has 25 candidate pairs and five correct pairs for one alignment. The standard independent-status approximation gives 2,876 trials for 99%; exact finite-pool rules give different counts.
8. **Symmetry is missing information.** An unlabeled regular pentagon permits ten corner alignments without orientation constraints. More trials cannot select the intended one; extra asymmetric evidence can.

3.8.2 Short quiz

These original questions test this chapter's applications.

Each question has its own complete worked answer below. Open it after trying the problem; all answers are visible in print.

Question 1

Original practice question

A centered pinhole image is 800 pixels wide with 90° horizontal FOV. What is f_x? Does that alone determine f_y?

Worked solution

Step 1 — Use the half-angle triangle

Why: A centered 90-degree horizontal field gives a 45-degree half angle.

$$f_x = \frac{800/2}{\tan 45^\circ} = \frac{400}{1} = 400 \text{ pixels.}$$

Step 2 — Identify the missing parameter

Why: A horizontal angular measurement constrains horizontal focal length only.

It does not determine f_y without a pixel-shape or other calibration assumption.

Question 2

Original practice question

If $f_x = 400$ pixels and pixel pitch is $0.004\,\text{mm/pixel}$, where is the principal point on the positive virtual image plane in camera-frame millimeters?

Worked solution

Step 1 — Convert pixel focal length to millimeters

Why: Pixel pitch supplies the missing physical length per pixel.

$$f_{phys} = (400\,\text{px})(0.004\,\text{mm/px}) = 1.6\,\text{mm.}$$

Step 2 — Locate the principal point

Why: It is on the camera optical axis at the virtual image plane.

Its camera-frame position is $(0, 0, 1.6)^T$ mm. This is different from its image-array pixel coordinate.

Question 3

Original practice question

For a scene vector $(3, 2, 30)^T$ and $f = 3$, what is its image-plane 3D intersection? What is the cross product of the two vectors?

Worked solution

Step 1 — Scale onto the image plane

Why: The ray's third coordinate must become the focal depth.

$$a = \frac{f}{Z}\begin{pmatrix}3\\2\\30\end{pmatrix} = \frac{3}{30}\begin{pmatrix}3\\2\\30\end{pmatrix} = \begin{pmatrix}0.3\\0.2\\3\end{pmatrix}.$$

Step 2 — Expand the cross product

Why: Parallel 3D ray vectors must have zero cross product.

$$\begin{pmatrix}3\\2\\30\end{pmatrix} \times \begin{pmatrix}.3\\.2\\3\end{pmatrix} = \begin{pmatrix}2(3) - 30(.2)\\30(.3) - 3(3)\\3(.2) - 2(.3)\end{pmatrix} = \begin{pmatrix}0\\0\\0\end{pmatrix}.$$

Question 4

Original practice question

A ray is $P = Z(0.5, 0, 1)^T$ and the known plane is $X + Z = 12$. Find Z and P.

Worked solution

Step 1 — Substitute the ray into the plane

Why: At the intersection both descriptions must hold.

$$X + Z = 0.5Z + Z = 12 \quad \Rightarrow \quad 1.5Z = 12 \quad \Rightarrow \quad Z = 8.$$

Step 2 — Recover and check the point

Why: The recovered depth scales the complete ray direction.

$$P = 8\begin{pmatrix} .5 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 0 \\ 8 \end{pmatrix}.$$

It has positive depth and gives $4 + 8 = 12$ in the plane equation.

Question 5

Original practice question

What happens if a ray has $\gamma = 0$ for a plane with $D \neq 0$?

Worked solution

Step 1 — Write the reduced intersection equation

Why: Substitution always produces the scalar equation gamma times depth plus D equals zero.

$$\gamma Z + D = 0.$$

With $\gamma = 0$ this reduces to $D = 0$.

Step 2 — Check the stated nonzero offset

Why: The supplied D contradicts that required equality.

Since $D \neq 0$, there is no finite solution. The viewing ray is parallel to a plane that does not contain the camera center.

Question 6

Original practice question

For Example 3.5's matrix, where does source point $(0.5, 0.5)$ go? Is it the destination vertex average?

Worked solution

Step 1 — Multiply the supplied example matrix

Why: Projective coordinates must be transformed before averaging or division.

$$\begin{pmatrix} 4 & 0 & 0 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \end{pmatrix}\begin{pmatrix} .5 \\ .5 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 1.5 \end{pmatrix}.$$

Step 2 — Normalize and compare

Why: A projective map does not generally preserve ordinary coordinate averages.

$$(u, v) = \left(\frac{2}{1.5}, \frac{1}{1.5}\right) = \left(\frac{4}{3}, \frac{2}{3}\right).$$

This differs from the destination vertex average $(1, 3/4)$.

Question 7

Original practice question

If one independent attempt succeeds with probability $1/4$, what is the chance of at least one success in three attempts?

Worked solution

Step 1 — Count three failures

Why: Independent failures are easier to count than all success patterns.

$$P(\text{no success}) = (1 - 1/4)^3 = (3/4)^3 = 27/64.$$

Step 2 — Take the complement

Why: At least one success is the complement of no successes.

$$P(\text{at least one}) = 1 - 27/64 = 37/64 = 0.578125 \approx 57.81\%.$$

Question 8

Original practice question

A regular pentagon gives several equally supported alignments. Will increasing RANSAC's trial count identify the intended alignment? Name evidence that could.

Worked solution

Step 1 — Identify the ambiguity

Why: Several models have the same corner evidence.

Increasing the number of samples does not change that evidence, so it cannot identify the intended alignment.

Step 2 — Supply asymmetric evidence

Why: The competing alignments must make different predictions about something observable.

A distinctive visible mark, labeled corner or other asymmetric scene cue can distinguish them. More identical unlabeled corners do not remove this symmetry.

CHAPTER

FOUR

LECTURE 4: EPIPOLAR GEOMETRY AND STEREO DEPTH

Original quiz questions + fully worked solutions — complete problem statements, numbered red steps with a plain-English reason, and blue mathematics showing intermediate operations.

A one-pixel mistake can move a reconstructed point by centimeters—or much more. Why? Two views turn image matches into 3D geometry, but the result depends on finding the right match and measuring its position accurately.

Start here: *Epipolar geometry, explained visually with worked math*. Learn why a pixel predicts a line, then calculate the line, test matches, reverse the mapping, find epipoles, and solve the assigned quizzes step by step.

Looking for stereo errors? Go to *depth sensitivity and signed errors*, including both quiz sets, exact versus approximate answers, and why distant points are harder to measure. The *3D stereo explorer* connects the equations to actual camera rays.

4.1 Find the worked math quickly

I want to...	Go straight to...
Understand what an epipolar line means	*Ray → plane → line*
Multiply F by a point, one row at a time	*Guided calculation A*
Test several matches and measure a miss	*Guided calculation B*
Use the transpose or solve for epipoles	*Guided calculation C*
Expand the symbolic Quiz 4 constraint	*Worked example 4.1*
Intersect third-view epipolar lines	*Worked example 4.2*
Correct the supplied Quiz 2 matrix answer	*Worked example 4.3*
Calculate ±1-pixel depth errors	*Fall quiz 4.6* and *Spring quiz 4.7*
Set a disparity search range	*Worked example 4.8*

4.2 Reading route

1. *Understand epipolar geometry and solve its quizzes*: visual intuition, fully expanded linear algebra, and checked answers.
2. *Estimate F from noisy matches*: enforce rank two and measure error in pixels.
3. *Recover camera pose*: use calibration, decompose the essential matrix, and test which solution faces the scene.
4. *Stereo depth and its errors*: derive the depth equation, solve both quiz sets, and explore the geometry in 3D.
5. *Find matches across an image*: compare patches, aggregate evidence, and handle depth boundaries.
6. *Review and source checklist*, then the final *TLDR and answered quiz*.

Source: `wk4_cs532s26_TwoViewGeometry.pdf`, 77 PDF pages. Printed numbers match PDF pages 1–34; from PDF page 35 onward, visible printed numbers are four greater. PDF page 54 has no visible number. References below use one-based **PDF pages**. The coverage register records repeated material and its original teaching location.

4.3 Epipolar geometry: visual guide and worked quiz examples

You know a point in the first photo. Where should you look in the second? Epipolar geometry narrows the search from a whole image to **one line**. It does not choose the correct match on that line; appearance or another view must finish the job.

Jump to a calculation: *point → line*, *test candidate matches*, *reverse direction and epipoles*, *Quiz 4: symbolic expansion*, *Quiz 2: third-view intersection*, *Quiz 2: supplied-matrix correction*. Then continue to *stereo depth, sensitivity and signed errors*.

The *Lecture 2 formula reference* remains the canonical introduction. Here we work through the geometry and arithmetic slowly enough to use those formulas in an exam.

Sources: lecture PDF pages 18–21; `cs532f15_quiz4.pdf` and `cs532s19_quiz4.pdf`, P1–P2, PDF page 1, both printed Spring 2019 Quiz 4; Fall 2025 Quiz 2, `cs532f25_quiz_2_solution.pdf`, Q1–Q2, PDF page 1.

4.3.1 Why a point turns into a line

Imagine a lamp seen by two cameras. In image A you click its tip.

1. **One pixel gives a ray, not a depth.** The tip could be anywhere along the ray from camera A through that pixel.
2. **Add camera B.** The ray and the two camera centers define an **epipolar plane** (except when the ray is the baseline itself).
3. **Slice image B with that plane.** Their intersection is the **epipolar line**. Every possible position along A's ray projects onto this line in B.

4. **Find the actual match.** Compare image content along the line. The two viewing rays can then locate the 3D point; noisy rays require triangulation with an error criterion.

The useful shortcut: a two-dimensional image search becomes a one-dimensional line search. A point on the line is *geometrically possible*, not necessarily the same physical feature.

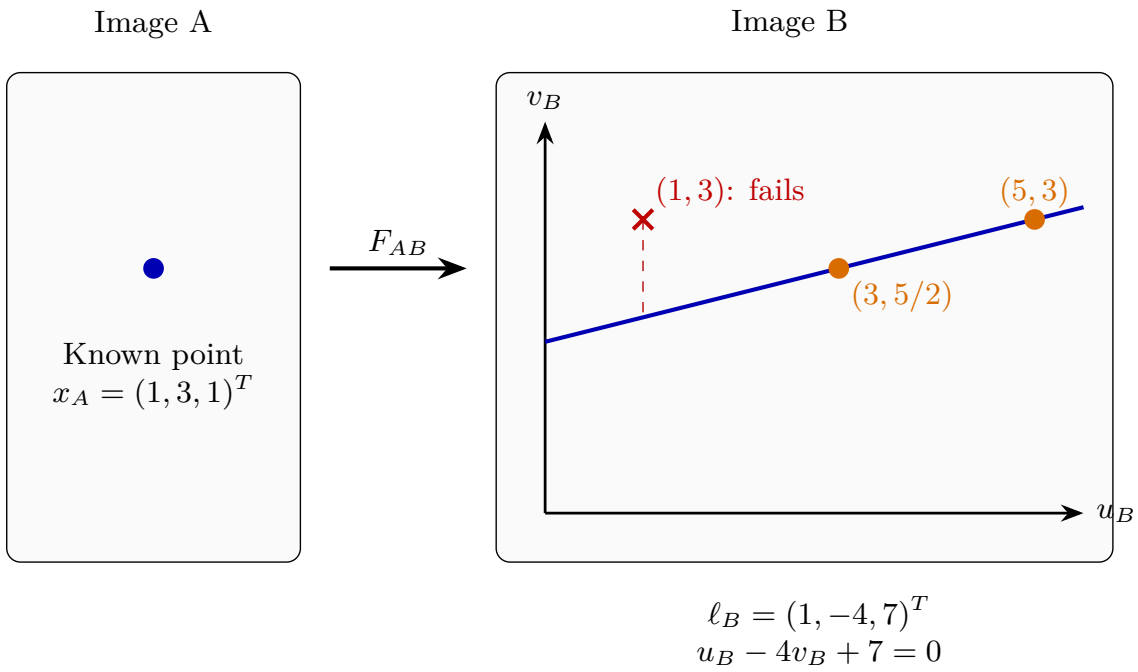


Fig. 4.1: One known pixel predicts a whole line. The numbers are worked out below; the two orange candidates pass the geometry test, so appearance must distinguish them.

Keep these words straight: the **baseline** joins the camera centers. An **epipole** is the other camera center's image; all epipolar lines in that image pass through it. For parallel rectified cameras, the epipoles are at infinity and the lines are horizontal. General epipolar lines can slope.

4.3.2 Read the symbols before multiplying

Symbol	Shape	Meaning
$x_A = (u_A, v_A, 1)^T$	3×1	Known finite image point, in homogeneous coordinates
F_{AB}	3×3	Fundamental matrix mapping a point in A to a line in B
$\ell_B = (a, b, c)^T$	3×1	Line coefficients, meaning $au_B + bv_B + c = 0$
$x_B = (u_B, v_B, 1)^T$	3×1	A candidate point in B
$x_B^T \ell_B$	1×1	A scalar: zero exactly when the candidate lies on the line

The superscript T **transposes** a column into a row. A row times a column means multiply matching entries and add. A matrix times a column repeats that operation for each row. The last coordinate 1 lets the constant term c participate in the same multiplication.

Tip

Point → line → test. First compute $\ell_B = F_{AB}x_A$. Then test $x_B^T\ell_B = 0$. The first result has three entries; the second has just one. Neither calculation gives depth by itself.

The familiar constraint $x_B^T F_{AB} x_A = 0$ simply combines those two steps. Its canonical formula and application box is (2.13). We use the convention that the subscript AB means **point in A → line in B** throughout this chapter.

4.3.3 Guided calculation A: multiply a point into its epipolar line

Use this original, valid rank-two fundamental matrix and known point:

$$F_{AB} = \begin{pmatrix} -1 & 0 & 2 \\ 0 & -1 & -1 \\ 1 & 2 & 0 \end{pmatrix}, \qquad x_A = \begin{pmatrix} 1 \\ 3 \\ 1 \end{pmatrix}.$$

Step 1 — take one row at a time.

$$\ell_B = F_{AB}x_A = \begin{pmatrix} (-1)(1) + (0)(3) + (2)(1) \\ (0)(1) + (-1)(3) + (-1)(1) \\ (1)(1) + (2)(3) + (0)(1) \end{pmatrix} = \begin{pmatrix} 1 \\ -4 \\ 7 \end{pmatrix}.$$

Step 2 — turn the coefficients into a line equation. The result means

$$u_B - 4v_B + 7 = 0 \quad \Longrightarrow \quad 4v_B = u_B + 7 \quad \Longrightarrow \quad v_B = \frac{u_B + 7}{4}.$$

For every chosen u_B, there is a corresponding v_B. **You have a line of candidates, not a unique pixel.** Do not divide the line's coefficients by its last entry and call them a point.

4.3.4 Guided calculation B: which candidates pass?

A candidate passes if its coordinates make the line equation zero. For $x_B = (5, 3, 1)^T$, show the full row-column multiplication:

$$x_B^T\ell_B = \begin{pmatrix} 5 & 3 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -4 \\ 7 \end{pmatrix} = 5(1) + 3(-4) + 1(7) = 5 - 12 + 7 = 0.$$

Candidate (u_B, v_B)	Substitute into $u_B - 4v_B + 7$	Decision
$(3, 5/2)$	$3 - 4(5/2) + 7 = 3 - 10 + 7 = 0$	On the line
$(5, 3)$	$5 - 12 + 7 = 0$	Also on the line
$(1, 3)$	$1 - 12 + 7 = -4$	Off the line

Two candidates pass. Epipolar consistency is a filter, not proof of correspondence. A repeated texture can produce several plausible matches.

With real measurements, expect a small nonzero residual. Do not interpret the raw residual -4 as a distance: multiplying F by ten multiplies that number by ten without changing the geometry. Normalize by the line's first two coefficients:

$$\text{distance} = \frac{|au_B + bv_B + c|}{\sqrt{a^2 + b^2}} = \frac{|-4|}{\sqrt{1^2 + (-4)^2}} = \frac{4}{\sqrt{17}} \approx 0.970.$$

This is a perpendicular distance in **pixels if these coordinates are pixels**. The *estimation section* gives the full symmetric-error formula and a procedure for noisy matches. Scaling the line to $(2, -8, 14)^T$ doubles both numerator and denominator, leaving this distance unchanged.

4.3.5 Guided calculation C: reverse direction, then find the epipoles

Reverse direction with the transpose. Given the candidate $x_B = (5, 3, 1)^T$, its line in A is $F_{AB}^T x_B$, not $F_{AB} x_B$:

$$\ell_A = \begin{pmatrix} -1 & 0 & 1 \\ 0 & -1 & 2 \\ 2 & -1 & 0 \end{pmatrix} \begin{pmatrix} 5 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} -5+0+1 \\ 0-3+2 \\ 10-3+0 \end{pmatrix} = \begin{pmatrix} -4 \\ -1 \\ 7 \end{pmatrix}.$$

Check the original point: $(-4)(1) + (-1)(3) + 7 = 0$. The same correspondence passes from either direction because transposing a scalar leaves it unchanged.

Find an epipole by solving a nullspace equation. The epipole in A satisfies $F_{AB} e_A = 0$. Write $e_A = (r, s, w)^T$. Then

$$-r + 2w = 0, \qquad -s - w = 0, \qquad r + 2s = 0.$$

The first two equations give $r = 2w$ and $s = -w$. The third becomes $2w + 2(-w) = 0$, so it adds no independent condition. Choose $w = 1$: $e_A = (2, -1, 1)^T$. This is the one-dimensional nullspace expected of a rank-two 3×3 matrix. The first two rows are independent and the nonzero null vector rules out rank three.

Likewise $F_{AB}^T e_B = 0$ gives

$$-r + w = 0, \qquad -s + 2w = 0, \qquad 2r - s = 0,$$

so $e_B = (1, 2, 1)^T$. Check it on our predicted line: $1 - 4(2) + 7 = 0$.

Why does every line pass through e_B? For any x_A, $e_B^T (F_{AB} x_A) = (F_{AB}^T e_B)^T x_A = 0$. That is the algebraic version of the epipolar-plane picture.

> **Warning**
>
> **Special case:** putting $x_A = e_A$ gives $F_{AB} e_A = (0, 0, 0)^T$. This is not a valid line: the baseline projects to the epipole, so it does not select a unique epipolar plane. Also remember that F and any nonzero multiple of F describe the same geometry.

4.3.6 Worked quiz examples: now use the same steps

Quiz 4: expand a symbolic matrix

Worked example 4.1 — expand the assigned constraint. *Read the original problem and every worked step.*

Quiz 2, Q1: use two views to predict a point in a third

Worked example 4.2 — two predicted lines meet. *Read the original problem and every worked step.*

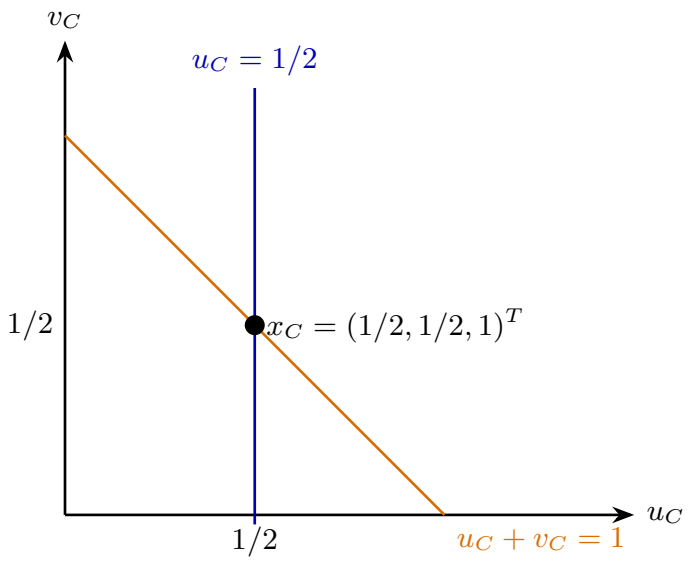


Fig. 4.2: Two independent line predictions locate the third-image point. This applies the incidence operations from Lecture 1 to a new three-view task.

Check the intersection before dividing: coincident lines give a zero cross product and no unique prediction. Distinct parallel lines meet at an ideal point, not a finite pixel. With noise, nearly parallel lines give an unstable location; incompatible matrices or an incorrect A–B match can also spoil the prediction.

Quiz 2, Q2: check the supplied matrix before trusting its name

Worked example 4.3 — correct the supplied solution. *Read the original problem and every worked step.*

4.3.7 Before moving on

You should be able to explain **why the answer is a line**, multiply a matrix row by row, test a candidate with a scalar dot product, reverse direction with F^T, and intersect two lines in the **same** image.

Next: *estimate F from noisy matches*, or go directly to *stereo depth, sensitivity and errors*. Rectification makes the lines horizontal; it makes matching simpler, but it does not remove depth uncertainty.

4.4 Estimate F from noisy matches

The *eight-point route* already supplies the linear rows and normalization order. Here we add what makes that route usable: a numerical noisy fit, an explicit rank constraint, and an error measured in pixels.

Source: lecture PDF pages 24–37; printed slides 24–34, then 39–41. The section develops the earlier Hartley–Zisserman–Pollefeys material.

4.4.1 Make the fitted matrix physically admissible

Noise usually gives a full-rank linear estimate. A true fundamental matrix must have rank two.

Formula — nearest rank-two correction

$$\widehat{F} = U\,\mathrm{diag}(\sigma_1, \sigma_2, \sigma_3)V^T, \qquad F_2 = U\,\mathrm{diag}(\sigma_1, \sigma_2, 0)V^T. \tag{4.1}$$

The 3×3 SVD has $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$. This minimizes the Frobenius distance (the square root of the sum of squared entry differences) among matrices of rank at most two. Require $\sigma_2 > 0$ for rank **exactly** two. It does not minimize image reprojection error.

Correct the normalized estimate

1. **Input:** the matrix reshaped from the normalized linear system's smallest right singular vector.
2. Compute its SVD and set only the smallest singular value to zero.
3. Undo point normalization using the existing rule $F = T'^T F_2 T$.
4. **Check:** rank two, valid epipoles, and pixel-space errors on matches you did not fit.

Normalization improves numerical conditioning; it cannot repair outliers or degenerate geometry. The source's image-size mapping to $[-1, 1]^2$ is a scaling heuristic. The worked example uses independent centroid/mean-distance normalization from Lecture 2.

4.4.2 Measure how far a point misses its predicted line

An algebraic residual ${x'}^T F x$ changes if you rescale F. Pixel distance to the line does not.

Formula — point-to-line and symmetric epipolar error

$$\begin{aligned} \operatorname{dist}(x', \ell') &= \frac{|au' + bv' + c|}{\sqrt{a^2 + b^2}}, \quad x' = (u', v', 1)^T,\ \ell' = (a, b, c)^T, \\ r &= {x'}^T F x, \quad \ell' = Fx, \quad \ell = F^T x', \\ e^2_{\text{sym}} &= r^2 \left[\frac{1}{\ell_1'^2 + \ell_2'^2} + \frac{1}{\ell_1^2 + \ell_2^2} \right]. \end{aligned} \tag{4.2}$$

Use undistorted pixel coordinates and finite lines with nonzero first-two-component norm. The symmetric error sums squared distances in both images; it is not the same as minimizing jointly corrected image positions. Near an epipole, a vanishing line makes this calculation undefined.

Score a hypothesis in image units

1. Predict a line in each image and compute its perpendicular point distance.
2. Combine the two squared distances if using the symmetric criterion.
3. Set a threshold in units matching that criterion: pixels for distance, pixels squared for squared error.
4. Inspect held-out errors and spatial coverage, not just the number of fitted points.

For line $(0, -2, 6)^T$ and point $(5, 4, 1)^T$, the distance is $|-8 + 6|/2 = 1$ pixel. Scaling the line by ten leaves the answer unchanged.

Worked example 4.4 — a complete noisy eight-point-route fit

Original numerical companion to: PDF pages 25–33. Twelve matches make this an overdetermined fit using the eight-point algorithm, not a different "twelve-point" camera model. Coordinates are pixels, rounded to two decimals **before fitting**.

u	v	u'	v'
160.14	174.24	192.03	195.37
506.19	306.72	505.53	337.77
115.57	325.55	134.85	346.85
344.14	93.47	305.48	122.09
186.44	258.74	208.55	281.31
211.28	59.43	209.20	86.37
555.25	368.93	556.98	406.35
168.42	96.09	116.58	131.45
445.56	357.02	477.41	384.87
17.61	68.56	10.62	103.15
108.56	279.43	127.33	300.86
499.77	272.68	543.72	295.87

1. Normalize both sets. Centroid/mean-distance normalization gives

$$T \approx \begin{pmatrix} .0071174 & 0 & -1.968505 \\ 0 & .0071174 & -1.578194 \\ 0 & 0 & 1 \end{pmatrix}, \quad T' \approx \begin{pmatrix} .0070558 & 0 & -1.992253 \\ 0 & .0070558 & -1.759429 \\ 0 & 0 & 1 \end{pmatrix}.$$

2. Stack one row per match. The first normalized row of (2.15) is approximately

$$(0.528172, 0.215457, -0.637327, 0.315694, 0.128781, -0.380937, -0.828729, -0.338063, 1).$$

The matrix is 12×9. Use its smallest right singular vector, reshaped row by row into $\widehat{\widetilde{F}}$.
3. Enforce rank two. This normalized matrix has singular values approximately $(0.718385, 0.695646, 0.000568)$. Set the last to zero using (4.1).
4. Undo normalization. With full-precision intermediate values, $F = T'^T F_2 T$. Rescaling only for display so $F_{33} = 1$ gives

$$F \approx \begin{pmatrix} 6.85735 \times 10^{-7} & 2.31624 \times 10^{-6} & -0.00429928 \\ 2.63343 \times 10^{-6} & 8.60518 \times 10^{-7} & -0.02609573 \\ 0.00274885 & 0.02456939 & 1 \end{pmatrix}.$$

5. Check what the fit predicts. The training root-mean-square symmetric epipolar error is about **0.49 pixels**. Four unmeasured projections from the generating cameras give about **0.33 pixels**. Those points were not used for fitting. Rounded matrix entries can alter residuals; the check uses full precision.
For reproducibility, the original synthetic scene uses 16 noncoplanar points, a 9° camera rotation, translation $(-0.8, 0.12, 0.05)^T$, and independent approximately 0.25-pixel coordinate noise on the twelve training matches. This is a controlled example, not a guarantee that real correspondences will fit this well.

4.4.3 Robust fitting: change the model and sample size

Use the existing *RANSAC explanation* and *sampling qualifications*. The new detail is that a fundamental-matrix **seven-point solver** can generate multiple candidate matrices from one sample. Its two-dimensional linear nullspace becomes $F(\lambda) = F_0 + \lambda F_1$; imposing $\det F(\lambda) = 0$ gives a cubic, generically with one or three real candidate roots. Score each candidate.

An eight-point solver needs eight matches instead. At 80% inliers and 99% clean-sample confidence, the independent-status budgets are **20** trials for seven-point samples and **26** for eight-point samples. The slide's table on PDF page 35 corresponds to about **95%**, not 99%; choose the confidence explicitly.

After scoring, refit on the inliers, search near predicted epipolar lines for additional matches, and refine geometric error. Under Gaussian image-coordinate noise, the slide's maximum-likelihood recommendation minimizes squared displacements to jointly corrected corresponding image points; this is stronger than finding the nearest matrix in entry space. PDF page 36's ±1.5-pixel band is an example tolerance, not a universal constant. Planar scenes and pure rotation remain *degenerate cases*; a plausible low residual alone does not prove a recoverable camera baseline.

4.5 Recover camera pose with calibration

A fundamental matrix works with pixels. Calibration removes each camera's internal coordinate scaling, leaving a relation between **viewing directions**. That relation can reveal rotation and translation direction.

Source: lecture PDF pages 22–23. The explicit SVD construction below is a computational companion to the slides' essential-matrix and four-solution statements. OpenCV's official decomposition and pose-recovery documentation[1] provides an implementation cross-check: four pose candidates, a translation direction, and a positive-depth test.

4.5.1 Remove intrinsics before asking about motion

Use the first camera as the reference: $P = [I \mid 0]$, $P' = [R \mid t]$ in calibrated coordinates. Here $X' = RX + t$; t **is a coordinate-transform translation**, and the second camera center in the first frame is $-R^T t$.

Formula — essential matrix and calibrated constraint

$$\begin{aligned} \hat{x} &= K^{-1}x, \quad \hat{x}' = K'^{-1}x', \\ E &= K'^{T}FK = [t]_{\times}R, \qquad \hat{x}'^{T}E\hat{x} = 0, \\ [t]_{\times} &= \begin{pmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{pmatrix}. \end{aligned} \tag{4.3}$$

K, K', F, E, R are 3×3; $t, \hat{x}, \hat{x}'$ are three-entry vectors. Use calibrated, undistorted coordinates and a nonzero baseline. E has five DOF: three for rotation and two for translation **direction**. A nonzero essential matrix has singular values $(a, a, 0)$ with $a > 0$.

Why does the constraint work? A 3D correspondence satisfies $\lambda'\hat{x}' = R\lambda\hat{x} + t$. The vector $[t]_{\times}R\hat{x}$ is perpendicular to both t and $R\hat{x}$, hence also to their combination $\hat{x}'$. Dotting them gives zero. Conversely, substituting calibrated coordinates into $x'^{T}Fx = 0$ gives $E = K'^{T}FK$.

Separate calibration from coordinate normalization

1. **Input:** valid F and both intrinsic matrices. Form $E = K'^{T}FK$.
2. Check two equal nonzero singular values and a third zero, allowing measurement noise.
3. With noisy $E = U\,\mathrm{diag}(s_1, s_2, s_3)V^T$, replace its singular values by $(a, a, 0)$, where $a = (s_1 + s_2)/2$; require $a > 0$. This is the nearest essential-form correction in Frobenius norm.
4. Continue to pose decomposition and test positive depth.

K^{-1} removes camera intrinsics. The earlier T normalization only improves numerical conditioning. They serve different purposes. Unlike a general rank-two F, an E also requires its first two singular values to agree.

[1] https://docs.opencv.org/4.12.0/d9/d0c/group__calib3d.html

4.5.2 Four algebraic poses, usually one visible scene

Formula — essential-matrix pose candidates

$$E \sim U\,\mathrm{diag}(1,1,0)V^T, \qquad W = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \tag{4.4}$$
$$R_1 = UWV^T, \quad R_2 = UW^TV^T, \qquad (R,t) \in \{(R_1, \pm u_3), (R_2, \pm u_3)\}.$$

u_3 is the third column of U. Choose SVD signs so $\det U = \det V = +1$; flipping either third column leaves the zero-singular-value product unchanged. Then both R_i are proper rotations. The unit translation fixes an arbitrary scale, not a known physical baseline.

Choose a pose by cheirality

1. Generate the four (R,t) pairs and check $R^TR = I$, $\det R = 1$.
2. For each pair, solve $\lambda'\hat{x}' = R\lambda\hat{x} + t$ for several reliable correspondences.
3. Keep the pose with positive depth in **both** cameras. This is **cheirality**: reconstructed points must lie in front of their observing cameras.
4. Check reprojection errors. With noise, use many matches and inspect an ambiguous result rather than trusting one marginal point.

Pure rotation, negligible parallax, or degenerate data can defeat this selection. General linear triangulation is developed in Lecture 6; the next example solves the two ray depths directly.

Worked example 4.5 — test all four poses

Original numerical companion to: PDF pages 22–23.

Take $E = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix}$, $\hat{x} = (0.2, 0.1, 1)^T$, and $\hat{x}' = (0, 0.1, 1)^T$. The singular values are $(1,1,0)$. Its candidates can be written as $R = I$ or $R_x(180^\circ) = \mathrm{diag}(1,-1,-1)$, each with $t = \pm(1,0,0)^T$.
Solve $\lambda'\hat{x}' = R\lambda\hat{x} + t$. Its first coordinate fixes $0 = 0.2\lambda + t_x$; the other coordinates determine λ'.

R	t_x	First depth λ	Second depth λ'	Visible in both?
I	-1	5	5	Yes
I	$+1$	-5	-5	No
$R_x(180^\circ)$	-1	5	-5	No
$R_x(180^\circ)$	$+1$	-5	5	No

The visible solution is $R = I$, $t = (-1,0,0)^T$. Its second camera center is $(1,0,0)^T$. The reconstructed point is $5\hat{x} = (1, 0.5, 5)^T$; in camera two it is $(0, 0.5, 5)^T$, which reprojects correctly.
All lengths here use the chosen unit baseline. Doubling the baseline and all point coordinates leaves the two images unchanged. Metric scale needs extra information, such as a measured baseline.

Two different singular-value rules

For singular values $(3, 2, 0.1)$, F correction uses $(3, 2, 0)$; E correction uses $(2.5, 2.5, 0)$.

4.6 Stereo depth, sensitivity and worked error examples

Hold a finger in front of your face and close one eye at a time. Its apparent horizontal shift is larger when it is close. **Disparity** measures that shift in a rectified camera pair.

Quick route: *derive depth*, *understand sensitivity*, *Fall quiz: ±1 pixel*, *Spring quiz: larger errors*, *3D explorer*, *depth range → search range*.

Sources: lecture PDF pages 58–64; Fall 2025 Quiz 3, `cs532f25_quiz_3_solution.pdf`, Q1–Q2, page 1; Spring 2019 Quiz 5, `cs532s19_quiz5-2.pdf`, P1's three bullets, page 1; Spring 2020 Quiz 5, `cs532s20_quiz5.pdf`, same questions under an overprinted 2019/2020 header.

4.6.1 Turn horizontal shift into axial depth

Assume aligned rectified cameras with a horizontal baseline $b > 0$, shared horizontal focal length f in pixels, and matched principal-point columns. Put the left center at $(0, 0, 0)$ and the right center at $(b, 0, 0)$.

For a scene point (X, Y, Z), the left camera sees horizontal position X, while the right camera sees $X - b$ relative to its own center. Perspective divides each by the same depth:

$$u_L - c_x = \frac{fX}{Z}, \qquad u_R - c_x = \frac{f(X-b)}{Z}.$$

Subtract the right observation from the left. The principal point cancels, then the unknown X cancels:

$$\begin{aligned} d &= (u_L - c_x) - (u_R - c_x) \\ &= \frac{fX}{Z} - \frac{f(X-b)}{Z} = \frac{fX - fX + fb}{Z} = \frac{fb}{Z}. \end{aligned}$$

Multiply both sides by Z, giving $dZ = fb$. Divide by positive d: $Z = fb/d$. This is why the horizontal shift alone can recover depth once the cameras are calibrated.

Formula — rectified stereo depth

$$d = u_L - u_R, \qquad Z = \frac{fb}{d}. \tag{4.5}$$

f and d use pixels; Z uses the same length unit as b. Z is axial depth. Require $d > 0$ under this camera order and forward-view convention. If principal-point columns differ, use corrected disparity $d = (u_L - c_{xL}) - (u_R - c_{xR})$ instead. A zero disparity is the ideal infinite-depth limit, not a finite reconstruction.

Reconstruct with a known baseline

1. **Input:** rectified correspondence, calibrated pixel focal length, and baseline length.
2. Check equal image rows and subtract horizontal coordinates in the stated order.
3. Compute $Z = fb/d$. Then use *calibrated backprojection* if you also need X, Y.
4. **Check:** doubling disparity halves depth; doubling baseline at fixed disparity doubles depth. Keep centimeters and meters consistent.

The source's BAD/GOOD rectification examples (PDF pages 59–60) illustrate that stereo needs compatible row alignment, not independently straightened pictures.

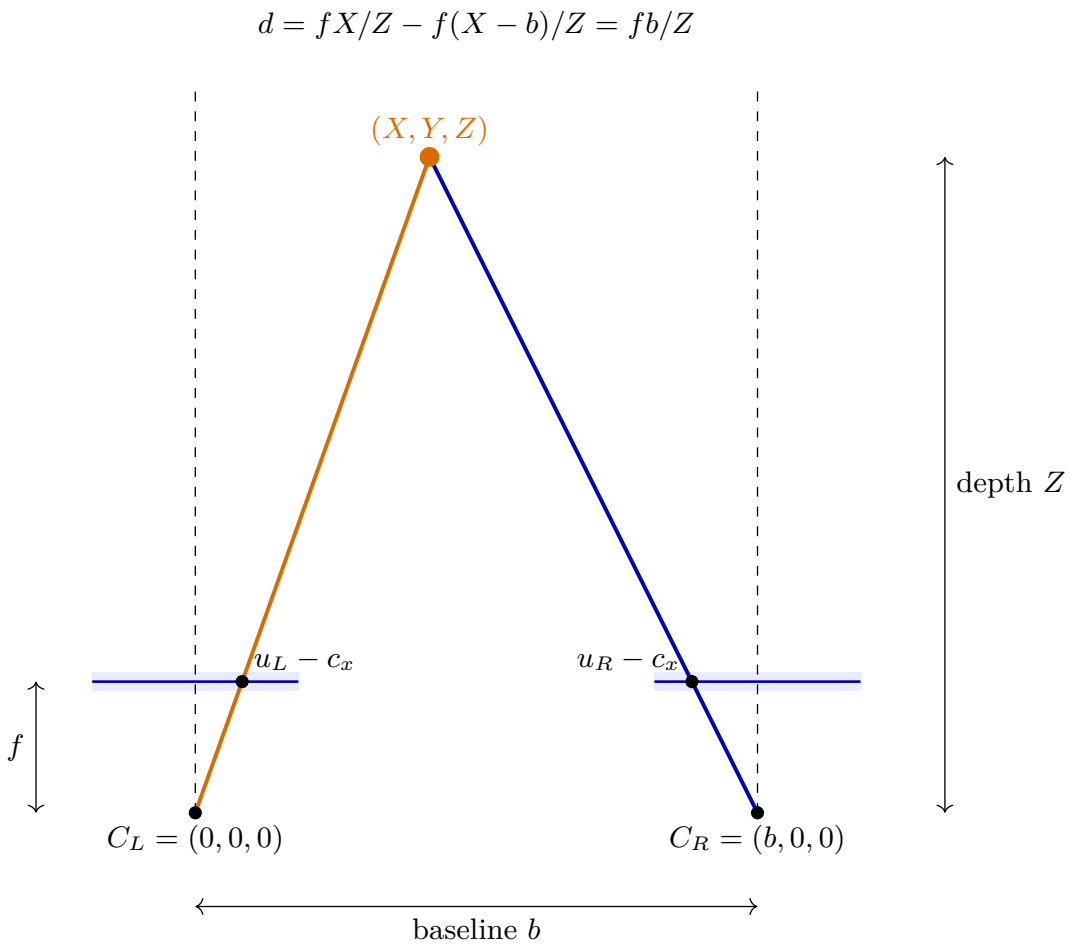


Fig. 4.3: A side view along the horizontal baseline: the difference between two projections fixes the ray intersection's depth. Virtual image planes are drawn in front of the cameras.

4.6.2 Stereo depth sensitivity: why one pixel matters

Faraway points barely move between the two images. Their disparity is small, so one pixel is a large fraction of the signal. Nearby points have a larger shift and tolerate the same pixel error better.

Let the true disparity be d and the measured disparity $d + \delta$. Here δ is a **disparity error**, not automatically the error in one particular image coordinate. Since $d = u_L - u_R$, measurement errors ϵ_L, ϵ_R give $\delta = \epsilon_L - \epsilon_R$. If the left coordinate is held fixed, a right-coordinate error has the opposite sign to δ.

Define **signed depth error** as estimated depth minus true depth. Start with the two depths, give their fractions a common denominator, and simplify:

$$\begin{aligned}\Delta Z = \widehat{Z} - Z &= \frac{fb}{d+\delta} - \frac{fb}{d} \\ &= \frac{fb\,d - fb(d+\delta)}{d(d+\delta)} \\ &= \frac{fb\,d - fb\,d - fb\,\delta}{d(d+\delta)} = -\frac{fb\,\delta}{d(d+\delta)}.\end{aligned}$$

Read the sign: $\delta > 0$ means too much disparity and an estimate too near ($\Delta Z < 0$). $\delta < 0$ means too little disparity and an estimate too far ($\Delta Z > 0$), provided measured disparity stays positive.

Formula — exact and first-order depth error

$$\begin{aligned}&\Delta Z = \frac{fb}{d+\delta} - \frac{fb}{d} = -\frac{fb\,\delta}{d(d+\delta)}, \\ &\Delta Z \approx -\frac{fb}{d^2}\delta \quad (|\delta| \ll d), \qquad \frac{|\Delta Z|}{Z} \approx \frac{|\delta|}{d}, \\ &\left|\frac{dZ}{dd}\right| = \frac{fb}{d^2} = \frac{Z^2}{fb}.\end{aligned} \tag{4.6}$$

Require $d > 0$ and $d + \delta > 0$ for two finite forward-depth estimates. The approximation is the derivative of fb/d times δ. Its magnitude is often quoted without the minus sign; retain the sign when explaining whether the point moves nearer or farther.

Report uncertainty honestly

1. **Input:** f, b, d and a signed disparity error δ.
2. Compute the true and estimated depths; subtract for an exact error.
3. Use the derivative only when $|\delta|/d$ is small; label it an approximation.
4. **Check:** overestimating positive disparity makes depth too small. Underestimating it makes depth too large, by a larger magnitude for equal-size errors.

For bounded error $|\delta| \le \epsilon < d$, the possible depths lie between $fb/(d+\epsilon)$ and $fb/(d-\epsilon)$. If the interval reaches zero disparity, there is no finite upper depth bound under this model.

Understand the approximation, not just the formula

Write $Z = fb\,d^{-1}$. Differentiating with respect to disparity gives

$$\frac{dZ}{dd} = fb(-d^{-2}) = -\frac{fb}{d^2}.$$

A derivative is the **local slope**: approximately how much depth changes for a small change in disparity. Multiply this slope by δ to get $\Delta Z \approx -fb\delta/d^2$. Its units are length per pixel times pixels, giving length.

You can also see the approximation without calculus. If $|\delta|$ is much smaller than d, replace the exact denominator $d(d+\delta)$ by d^2. Dividing the resulting error magnitude by $Z = fb/d$ gives

$$\frac{|\Delta Z|}{Z} \approx \frac{fb|\delta|}{d^2}\frac{d}{fb} = \frac{|\delta|}{d}.$$

A 1-pixel error at 25 pixels is roughly a 4% depth error. At 5 pixels it is roughly 20%. Close to zero disparity, this approximation becomes unreliable; use the exact depths instead.

Why distance makes sensitivity grow so quickly

Substitute $d = fb/Z$ into the slope magnitude:

$$\left|\frac{dZ}{dd}\right| = \frac{fb}{(fb/Z)^2} = \frac{Z^2}{fb}.$$

At fixed focal length and baseline, **double the true depth → about four times the absolute error** for the same small disparity error. At fixed true depth, a larger baseline or longer pixel focal length reduces this local sensitivity. A larger baseline also makes occlusions and matching more difficult; geometry alone does not guarantee a usable correspondence.

For $f = 50$ pixels and $b = 10\,\text{cm}$, so $fb = 500$:

True depth	True disparity	Depth if disparity is 1 pixel too small	Depth if disparity is 1 pixel too large	Local sensitivity
20 cm	25 px	20.8333 cm	19.2308 cm	0.8 cm/px
40 cm	12.5 px	43.4783 cm	37.0370 cm	3.2 cm/px
100 cm	5 px	125 cm	83.3333 cm	20 cm/px

For example, the last row uses $d = 500/100 = 5$, then $500/(5-1) = 125$ and $500/(5+1) = 83.3333$. The exact errors are $+25$ **and** $-16.6667\,\text{cm}$, despite equal one-pixel disparity errors.

A bounded error gives a depth interval

Suppose $d = 5$ pixels and the disparity error is at most 0.5 pixel. With $fb = 500$, the permitted measured disparities are $[4.5, 5.5]$. Because larger disparity means smaller depth, reverse the endpoints:

$$\widehat{Z} \in \left[\frac{500}{5.5}, \frac{500}{4.5}\right] = [90.9091, 111.1111]\,\text{cm}.$$

Relative to the true depth $100\,\text{cm}$, the errors lie in $[-9.0909, +11.1111]\,\text{cm}$. The derivative estimate would give approximately $\pm 10\,\text{cm}$. A bound is an interval, not a probability statement. If permitted disparity reaches zero, finite upper depth is no longer bounded.

Worked example 4.6 — the one-pixel Fall 2025 question. *Read the original problem and every worked step.*

Worked example 4.7 — why ±1 pixel is not symmetric. *Read the original problem and every worked step.*

4.6.3 Explore the real 3D geometry

Move the true point in depth, change the camera baseline or focal length, and perturb the measured disparity. The orange point is the truth; the purple point is the estimate from the perturbed right-image match. Orbiting the scene changes only your inspection view.

Explore stereo depth in 3D — print edition

Print/no-JavaScript experiment: use the two-camera ray diagram above with $b = 10\,\text{cm}$, $f = 50$ pixels and true $Z = 20\,\text{cm}$. The point is centered between the cameras at $X = 5\,\text{cm}$. For a principal column $c_x = 125$, its projections are $u_L = 137.5$ and $u_R = 112.5$, giving $d = 25$ pixels.

Keep u_L fixed. A disparity error δ means an observed right column $\hat{u}_R = u_R - \delta$. Intersect the left ray with the ray through that perturbed right observation:

δ	$\hat{u}_R$	Estimated depth	What happens
-1 pixel	113.5	$20.8333\,\text{cm}$	Rays meet farther away
0	112.5	$20\,\text{cm}$	Estimate meets the truth
$+1$ pixel	111.5	$19.2308\,\text{cm}$	Rays meet nearer

At true depth $100\,\text{cm}$, disparity is only 5 pixels. The same ±1-pixel error gives depths 125 and $83.3333\,\text{cm}$. Small disparity makes depth fragile. Zero or negative measured disparity does not give a finite forward point in this convention.

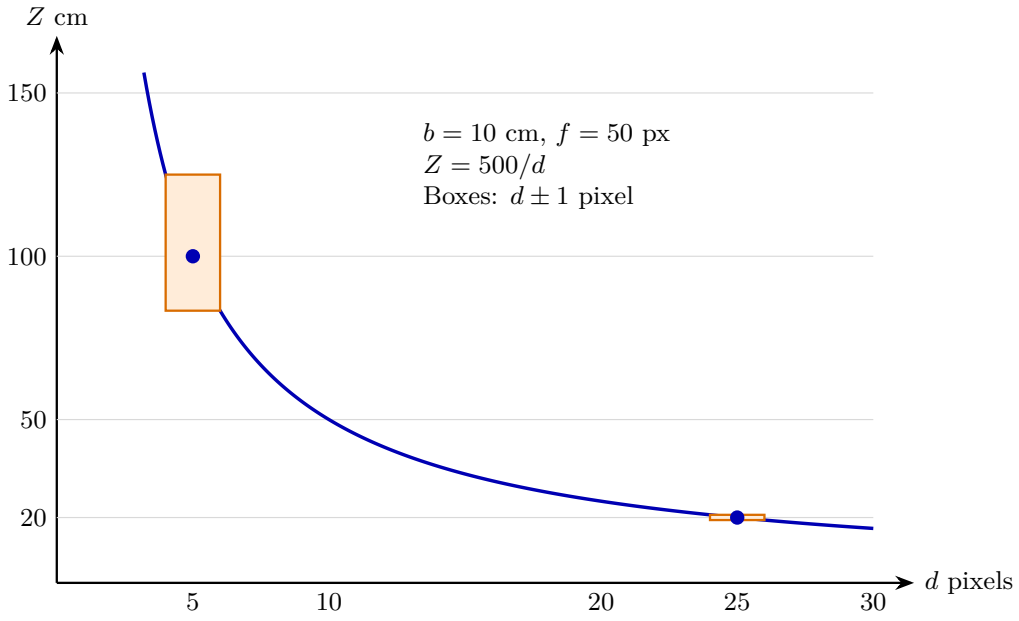


Fig. 4.4: Equal-width disparity errors produce unequal depth errors, especially near zero disparity. This static curve carries the interactive's main result into print.

4.6.4 Convert a permitted depth range into search limits

Formula — disparity search from a depth interval

$$0 < Z_{\min} \le Z \le Z_{\max} \quad \Rightarrow \quad \frac{fb}{Z_{\max}} \le d \le \frac{fb}{Z_{\min}}. \tag{4.7}$$

The inequalities reverse because depth and positive disparity are inversely related. Use consistent length units and the corrected disparity convention when principal points differ.

Limit the stereo search

1. Convert $b, Z_{\min}, Z_{\max}$ to the same length unit.
2. Compute the continuous disparity endpoints.
3. For integer hypotheses inside that interval, use the ceiling of the lower endpoint and floor of the upper endpoint. If subpixel or calibration uncertainty matters, include an appropriate margin before discretizing.
4. Check that the larger disparity belongs to the nearer depth.

Worked example 4.8 — restrict the Fall 2025 search. *Read the original problem and every worked step.*

4.7 Find matches across an image

Geometry narrows the search to a row. **Appearance chooses a position along it.** That is still difficult: repeated textures can look identical, flat regions have little evidence, and some pixels are visible in only one camera.

Source: lecture PDF pages 42–77, printed slides 46–81 except unnumbered PDF page 54. Stereo contributors named on PDF page 42: Rick Szeliski, Pascal Fua, P. Mordohai, and E. Dunn.

4.7.1 Generate, evaluate, select

The slide sequence on pages 45–53 follows one practical pipeline:

1. **Generate hypotheses:** sample possible disparities or depths. Each projects the reference pixel to a candidate in another view.
2. **Evaluate:** compare image appearance at the candidate, then combine neighboring evidence.
3. **Select:** choose a supported hypothesis, or reject the pixel when evidence is unreliable.

A **cost volume** stores one score for each image position and hypothesis: dimensions width × height × number of disparities. A depth map selects one hypothesis per reference pixel; it is view-dependent and need not contain occluded surfaces. The *earlier reconstruction pipeline* explains how depth can feed other 3D representations.

Formula — aggregate costs and select disparity

$$\overline{C}(p,d) = \frac{1}{|\Omega_p|} \sum_{q \in \Omega_p} C(q,d), \qquad \hat{d}(p) \in \arg\min_{d \in D_p} \overline{C}(p,d). \tag{4.8}$$

p, q are pixel positions, D_p the allowed disparity set, and Ω_p a support window. $C(q,d)$ is a dissimilarity between left location q and right location $q-(d,0)$. Require valid samples and nonempty support. An argmin can contain several disparities: a tie is evidence of ambiguity, not a unique measurement.

Build a simple dense stereo estimate

1. **Input:** rectified images and the permitted disparity range.
2. Evaluate a cost at every valid candidate. Stack these candidate costs into the volume.
3. Aggregate support and pick the smallest cost; retain a confidence measure such as separation from competing minima.
4. Reject inconsistent left–right matches and occlusions. Convert reliable disparities to depth.

This is **winner-take-all** selection. The slides also name feature/edge matching followed by interpolation and joint optimization using iterative updates, dynamic programming, energy minimization, or graph methods. Those alternatives add spatial reasoning; they do not make missing image evidence appear.

Reference image



Matching image



Find the same visible surface, not just the same color.
Depth changes its location between views.

Fig. 4.5: The actual reference/matching photographs from PDF page 46, printed slide 50. Look at the toy and leaf boundaries: viewpoints change which surfaces are visible. Extracted photographs with original project labels; individual photography credits and reuse terms are not identified on the slide.

4.7.2 Compare patches, not isolated colors

Photo-consistency means that projections of the same scene point have compatible image appearance under the chosen model. For a patch, write its paired samples as $L_i, R_i(d), i = 1, \dots, m$.

Formula — SAD, SSD, and zero-mean NCC

$$\begin{aligned} \mathrm{SAD}(d) &= \sum_i |L_i - R_i(d)|, \qquad \mathrm{SSD}(d) = \sum_i (L_i - R_i(d))^2, \\ \mathrm{NCC}(d) &= \frac{\sum_i (L_i - \overline{L})(R_i(d) - \overline{R}(d))}{\sqrt{\sum_i (L_i - \overline{L})^2 \sum_i (R_i(d) - \overline{R}(d))^2}}. \end{aligned} \tag{4.9}$$

$\overline{L}, \overline{R}$ are patch means. Minimize SAD/SSD; **maximize** NCC, or minimize a cost such as $1 - \mathrm{NCC}$. NCC lies in $[-1, 1]$ for nonconstant patches and is undefined if either centered patch has zero norm. SAD uses intensity units, SSD squared intensity units, NCC is dimensionless.

Choose and check a patch score

1. Use the same valid support and disparity convention for each candidate.
2. For SAD/SSD, compare aligned intensities directly. SSD penalizes large differences more strongly.

3. For NCC, subtract each patch's own mean and divide by its centered norm. Reject near-zero-variance supports.
4. Inspect the full score curve: multiple similar peaks or minima indicate uncertainty.

Positive affine brightness changes preserve ideal NCC, but clipping, reflections, shadows, and nonuniform exposure can break that assumption. The source's formula image leaves normalization factors dependent on its standard-deviation convention; the explicit sum-of-squares denominator above avoids that ambiguity.

Worked example 4.9 — when raw intensity picks the wrong patch

Original companion to: PDF pages 65–70.

Reference $L = (10, 20, 30)$ has two candidate patches: true but brighter $R_A = (30, 40, 50)$, and wrong $R_B = (10, 20, 10)$.

Candidate	SAD	SSD	NCC
A: brighter match	$20 + 20 + 20 = 60$	$3 \times 400 = 1200$	1
B: wrong texture	$0 + 0 + 20 = 20$	400	0

Raw difference scores favor B. For NCC, A's centered values equal the reference's $(-10, 0, 10)$, giving correlation one. B's centered values are $(-10/3, 20/3, -10/3)$, perpendicular to the reference vector, giving zero. NCC favors A under the uniform brightness-offset model.

A flat patch such as $(20, 20, 20)$ instead has zero centered norm. It cannot supply a defined NCC score or a distinctive texture match.

4.7.3 Window size trades detail for support

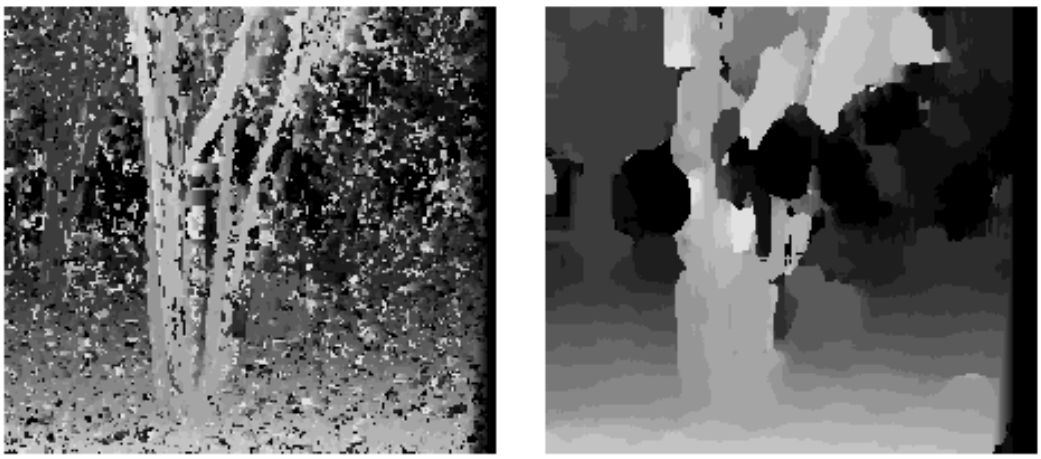


Fig. 4.6: The source's $w = 3$ versus $w = 20$ results, left to right (PDF page 66, printed slide 70). Larger support suppresses isolated mistakes but can erase detail. Extracted source illustration; its individual creator and reuse terms are not identified on that slide.

A constant-disparity window assumes **fronto-parallel support**: one axial depth. Slanted surfaces violate this assumption; windows crossing foreground and background mix different depths.

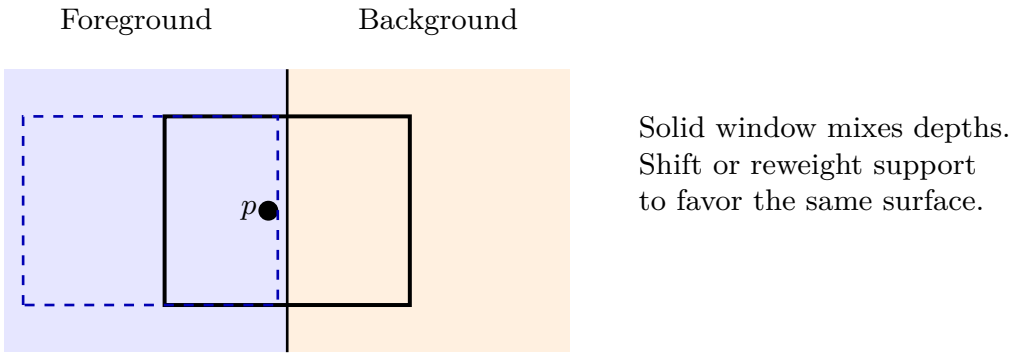


Fig. 4.7: A larger window can vote for two different depths. Adapting its position or weights changes which surface contributes evidence.

The slides offer several ways to adapt support:

- **Shiftable windows:** choose among windows that contain the reference pixel; favor support that stays on one surface.
- **Rod-shaped supports:** test elongated directions that may follow an approximately constant-disparity contour.
- **Adaptive weights:** downweight neighbors far away in position or color.
- **Oriented-plane hypotheses:** allow disparity to vary across a slanted plane instead of assuming one constant value.

All add assumptions. Similar colors can still occur at different depths, and an image edge need not be a depth edge.

4.7.4 Rank and Census use relative intensity order

Formula — rank, Census, and Hamming distance

$$
\begin{aligned}
r(p) &= \sum_{q \in \mathcal{N}_p} \mathbf{1}[I(q) < I(p)], \\
\mathcal{C}(p) &= (\mathbf{1}[I(q_1) < I(p)], \dots, \mathbf{1}[I(q_m) < I(p)]), \\
H(\mathcal{C}_L, \mathcal{C}_R) &= \sum_{i=1}^{m} \mathbf{1}[\mathcal{C}_{L,i} \neq \mathcal{C}_{R,i}].
\end{aligned}
\tag{4.10}
$$

$\mathcal{N}_p$ contains m ordered neighbors excluding the center; $\mathbf{1}[\cdot]$ is one when its condition is true and zero otherwise. Rank ranges from 0 to m; Census keeps the complete ordered bit string. Use the same neighbor order and tie convention in both images. Both preserve strictly increasing intensity remappings within the compared support.

Match relative-intensity patterns

1. Compare each neighbor with the center using the fixed strict inequality.
2. Sum the comparisons for rank, or retain their order for Census.
3. Compare rank-transformed images with a difference cost; compare Census strings by Hamming distance.
4. Check for repeated patterns and ties. Relative order is robust to some brightness changes, not to all lighting or geometry changes.

Worked example 4.10 — equal ranks can hide different patterns

Original companion to: PDF page 74, printed slide 78.
Let the center intensity be 20. Ordered neighbors $(10, 30, 15, 25)$ give Census **1010** and rank 2. A second patch with center 20 and neighbors $(5, 10, 30, 40)$ gives **1100**, also rank 2. Their Census Hamming distance is **2**: bits 2 and 3 disagree.
The rank compresses spatial evidence into one count. Census retains which neighbors were darker. Adding the same brightness offset to every pixel of either patch preserves its comparisons, provided clipping does not change them.

4.7.5 Give nearby, similar neighbors more influence

Formula — locally adaptive support

$$\begin{aligned} \Delta_c(p,q) &= \|\,\mathrm{Lab}(p) - \mathrm{Lab}(q)\|_2, \quad \Delta_s(p,q) = \|p-q\|_2, \\ w(p,q) &= \exp\left[-\frac{\Delta_c(p,q)}{\gamma_c} - \frac{\Delta_s(p,q)}{\gamma_s}\right], \\ \overline{C}_w(p,d) &= \frac{\sum_{q\in\Omega_p} w(p,q)C(q,d)}{\sum_{q\in\Omega_p} w(p,q)}. \end{aligned} \tag{4.11}$$

$\mathrm{Lab}(p) = (L_p, a_p, b_p)^T$ is a three-channel color vector. $\gamma_c > 0$ uses color-distance units; $\gamma_s > 0$ uses pixels, making both exponents dimensionless. These are reference-image weights with a nonzero sum. Other methods also include shifted right-image weights; specify the convention. A common weight multiplier cancels.

Aggregate without crossing every edge equally

1. Compute color distance in CIE Lab and spatial distance within the support.
2. Calculate positive weights with the chosen scales.
3. Form a weighted average cost for each disparity and compare candidates.
4. Inspect whether the support follows actual surfaces. Color similarity helps; it does not prove shared depth.

PDF pages 52/76 credit Yoon and Kweon (PAMI 2006); page 53 credits Lu et al. (CVPR 2013) for an oriented-plane extension. These descriptions explain their roles; they do not reproduce the full methods.

Worked example 4.11 — a two-sample depth boundary

Original companion to: PDF pages 75–76.

Two support samples are each one pixel away. One has the same color as the reference; the other differs by 40 Lab units. With $\gamma_c = 10$, $\gamma_s = 2$, their weights are $e^{-0.5} \approx 0.60653$ and $e^{-4.5} \approx 0.01111$.

At the true disparity, their costs are $(0, 10)$; at a wrong disparity, $(10, 0)$. Uniform averaging ties both at 5. Adaptive averaging gives about **0.180** for the true disparity and **9.820** for the wrong one. The same-color sample now contributes most of the evidence.

If that color boundary did not correspond to a depth boundary, this weighting preference could be misleading. The source's result images compare such support choices, not a universal guarantee of correctness.

The slide list also names **mutual information** (statistical dependence between intensities) and **learning-based methods** (learned appearance features or costs). They change how hypotheses are scored. The same geometry, visibility limits, and need to recognize uncertain matches remain.

4.8 Original quiz questions and worked solutions

Read the problem in black. Follow the numbered red steps. Work through the blue mathematics. Every step explains why it is needed. Matrix calculations show the inputs and intermediate results; written-reasoning questions explain the decision instead of inventing numerical data.

Source: the supplied lecture-organized quiz booklet. Full problems follow its wording and givens, with typography normalized; the historical PDF filenames appear below. Source-key corrections and added numerical companions are marked.

Retests have their own questions and full solutions. Problems labeled **Original practice** are the booklet's seven supplemental examples, separate from the 18 historical quizzes. Colored steps remain numbered and readable in grayscale.

4.8.1 Jump to a quiz

- *Older Quiz 4 — Epipolar line and constraint*
- *Spring 2019 Quiz 4 — Duplicate epipolar retest*
- *Fall 2025 Quiz 2 — Three-view epipolar geometry*
- *Fall 2025 Quiz 3 — Stereo depth sensitivity*
- *Spring 2019 Quiz 5 — Exact stereo errors*
- *Spring 2020 Quiz 5 — Repeat disparity test*
- *A4. Essential matrix and cheirality (L4, slides 22–41)*

4.8.2 Older Quiz 4 — Epipolar line and constraint

Historical source: `3dcvMid/olderquizzes/cs532f15_quiz4.pdf`.

Header note: this filename contains "2015"; the PDF is printed Spring 2019.

Problem 1

Original problem · quiz booklet

Let $x_A = (x, y, w)^T$ be a point in image A and $F_{AB} = [f_{ij}]$ be the fundamental matrix from image A to B. Write the epipolar-line equation in B, using both matrix entries and the row vectors $\mathbf{f}_1^T, \mathbf{f}_2^T, \mathbf{f}_3^T$. Make an illustrative sketch.

Given / relevant facts:

$$F_{AB} = \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix}$$

A homogeneous line is $\ell_B = (a, b, c)^T$.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Choose the point-to-line rule

Why: F maps the known point to possible matches in the other image.

Start from the fundamental-matrix point-to-line relation: $\ell_B = F_{AB} x_A$. This is a line, not the matched pixel itself.

Step 2 — Multiply all three rows

Why: The resulting column contains line coefficients.

Multiply each row of the full matrix by the input column:

$$\begin{aligned} \ell_B &= \begin{pmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{pmatrix} \begin{pmatrix} x \\ y \\ w \end{pmatrix} \\ &= \begin{pmatrix} f_{11}x + f_{12}y + f_{13}w \\ f_{21}x + f_{22}y + f_{23}w \\ f_{31}x + f_{32}y + f_{33}w \end{pmatrix} = \begin{pmatrix} \mathbf{f}_1^T x_A \\ \mathbf{f}_2^T x_A \\ \mathbf{f}_3^T x_A \end{pmatrix}. \end{aligned}$$

The output entries are line coefficients $(a, b, c)^T$, not point coordinates.

Step 3 — Enforce point-line incidence

Why: A true candidate must give a zero scalar dot product.

A candidate $x_B = (u_B, v_B, w_B)^T$ lies on that line when its dot product is zero:

$$\begin{aligned} 0 = x_B^T \ell_B = (f_{11}x + f_{12}y + f_{13}w)u_B \\ + (f_{21}x + f_{22}y + f_{23}w)v_B \\ + (f_{31}x + f_{32}y + f_{33}w)w_B. \end{aligned}$$

For a finite point set $w_B = 1$, giving $au_B + bv_B + c = 0$. If $b \neq 0$, rewrite it as $v_B = -(a/b)u_B - c/b$ for plotting; $b = 0$ instead gives a vertical line when $a \neq 0$.

Step 4 — Draw the predicted line

Why: One point generally predicts a line rather than a unique match.

If the point is finite, set $w_B = 1$; plot $au_B + bv_B + c = 0$. For $b \neq 0$, $v_B = -(a/b)u_B - c/b$. Draw a line in image B through the unknown match; its exact location depends on the supplied F_{AB} and x_A.

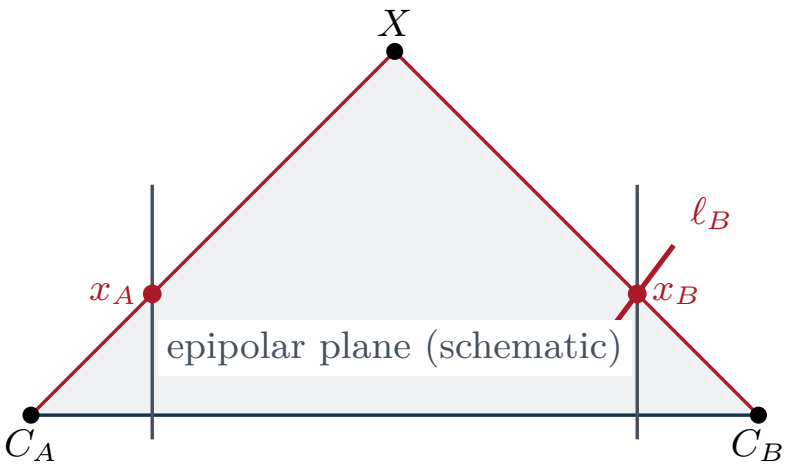


Fig. 4.8: The supplied booklet's schematic shows the two cameras, corresponding rays and epipolar plane. The line's slant is illustrative; in 3D it is the intersection of that plane with image plane B.

Problem 2

Original problem · quiz booklet

Write the explicit epipolar constraint $x_B^T F_{AB} x_A = 0$ for $x_A = (1, 0, 1)^T$ and $x_B = (0, 1, 1)^T$. Rewrite it using row-vector dot products.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Multiply F by the assigned point

Why: The zero middle coordinate removes the second column's contribution.

Multiply the actual point into the full symbolic matrix:

$$F_{AB}x_A = \begin{pmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$$
$$= \begin{pmatrix} f_{11}(1) + f_{12}(0) + f_{13}(1) \\ f_{21}(1) + f_{22}(0) + f_{23}(1) \\ f_{31}(1) + f_{32}(0) + f_{33}(1) \end{pmatrix} = \begin{pmatrix} f_{11} + f_{13} \\ f_{21} + f_{23} \\ f_{31} + f_{33} \end{pmatrix}.$$

Step 2 — Dot with the second point

Why: Its row selects the second and third line coefficients.

Transpose the other point and multiply the row by that column:

$$x_B^T F_{AB} x_A = \begin{pmatrix} 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} f_{11} + f_{13} \\ f_{21} + f_{23} \\ f_{31} + f_{33} \end{pmatrix}$$
$$= 0(f_{11} + f_{13}) + 1(f_{21} + f_{23}) + 1(f_{31} + f_{33}).$$

The result is a scalar: $(1 \times 3)(3 \times 1) = 1 \times 1$.

Step 3 — Set the scalar to zero

Why: This is the required epipolar constraint on the unknown entries.

Enforce incidence:

$$\boxed{f_{21} + f_{23} + f_{31} + f_{33} = 0}.$$

Step 4 — Rewrite with row-vector products

Why: The compact row notation must express the same scalar equation.

Rewrite using the supplied row vectors: $x_B^T F x_A = (\mathbf{f}_2^T + \mathbf{f}_3^T)x_A = \mathbf{f}_2 \cdot x_A + \mathbf{f}_3 \cdot x_A = 0$.

Source check / assumptions: The quiz provides symbolic F, so no individual numeric f_{ij} values can be solved; the requested result is a constraint on the entries.

4.8.3 Spring 2019 Quiz 4 — Duplicate epipolar retest

Historical source: `3dcvMid/olderquizzes/cs532s19_quiz4.pdf`.

Problem 1

Original problem · quiz booklet

Given $x_A = (x, y, w)^T$ and the rows $\mathbf{f}_1^T, \mathbf{f}_2^T, \mathbf{f}_3^T$ of F_{AB}, express its epipolar line in image B and make a sketch.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Check mapping direction

Why: A point in A must generate a line in B.

Choose the correct direction: F_{AB} maps a *point in A* to a *line in B*; compute $\ell_B = F_{AB} x_A$.

Step 2 — Expand the matrix multiplication

Why: Each row supplies one coefficient of that line.

Multiply each row of the full matrix by the input column:

$$\begin{aligned} \ell_B &= \begin{pmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{pmatrix} \begin{pmatrix} x \\ y \\ w \end{pmatrix} \\ &= \begin{pmatrix} f_{11}x + f_{12}y + f_{13}w \\ f_{21}x + f_{22}y + f_{23}w \\ f_{31}x + f_{32}y + f_{33}w \end{pmatrix} = \begin{pmatrix} \mathbf{f}_1^T x_A \\ \mathbf{f}_2^T x_A \\ \mathbf{f}_3^T x_A \end{pmatrix}. \end{aligned}$$

The output entries are line coefficients $(a, b, c)^T$, not point coordinates.

Step 3 — Write the complete line equation

Why: Incidence converts the coefficient vector into a condition on a candidate point.

A candidate $x_B = (u_B, v_B, w_B)^T$ lies on that line when its dot product is zero:

$$\begin{aligned} 0 = x_B^T \ell_B = (f_{11}x + f_{12}y + f_{13}w)u_B \\ + (f_{21}x + f_{22}y + f_{23}w)v_B \\ + (f_{31}x + f_{32}y + f_{33}w)w_B. \end{aligned}$$

For a finite point set $w_B = 1$, giving $au_B + bv_B + c = 0$. If $b \neq 0$, rewrite it as $v_B = -(a/b)u_B - c/b$ for plotting; $b = 0$ instead gives a vertical line when $a \neq 0$.

Step 4 — Make the requested sketch

Why: The camera rays and image line explain the algebra geometrically.

Sketch two camera centers joined by a baseline, their image planes, a common 3D point and both rays; label the line in image B that contains the possible match. The same diagram as on the preceding page applies.

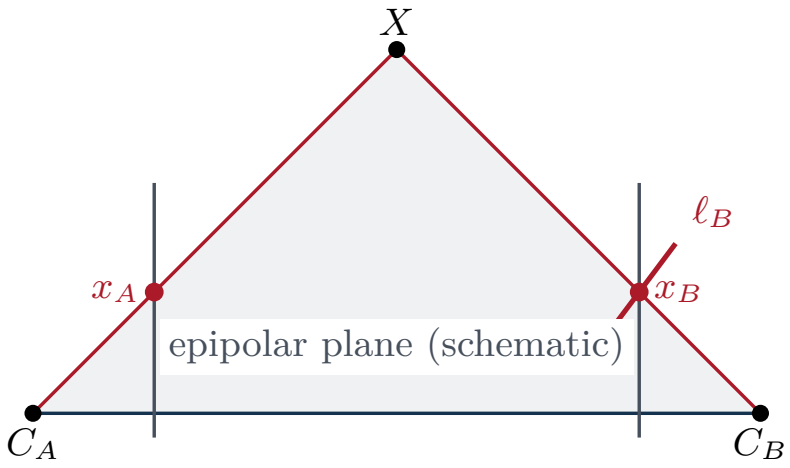


Fig. 4.9: The same schematic is reproduced beside this separate retest. The camera rays and baseline lie in one epipolar plane.

Problem 2

Original problem · quiz booklet

For $x_A = (1, 0, 1)^T$ and $x_B = (0, 1, 1)^T$, expand $x_B^T F_{AB} x_A = 0$ and express it in terms of row vectors.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Multiply the known point into F

Why: This selects and adds columns one and three.

Multiply the actual point into the full symbolic matrix:

$$F_{AB}x_A = \begin{pmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$$
$$= \begin{pmatrix} f_{11}(1) + f_{12}(0) + f_{13}(1) \\ f_{21}(1) + f_{22}(0) + f_{23}(1) \\ f_{31}(1) + f_{32}(0) + f_{33}(1) \end{pmatrix} = \begin{pmatrix} f_{11} + f_{13} \\ f_{21} + f_{23} \\ f_{31} + f_{33} \end{pmatrix}.$$

Step 2 — Perform the final row-column product

Why: Only two output coefficients contribute.

Transpose the other point and multiply the row by that column:

$$x_B^T F_{AB} x_A = \begin{pmatrix} 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} f_{11} + f_{13} \\ f_{21} + f_{23} \\ f_{31} + f_{33} \end{pmatrix}$$
$$= 0(f_{11} + f_{13}) + 1(f_{21} + f_{23}) + 1(f_{31} + f_{33}).$$

The result is a scalar: $(1 \times 3)(3 \times 1) = 1 \times 1$.

Step 3 — Enforce the zero constraint

Why: Correspondence requires incidence, not equality between a point and a line.

Set the scalar to zero: $\boxed{f_{21} + f_{23} + f_{31} + f_{33} = 0}$.

Step 4 — Express the row-vector form

Why: This checks the equivalent notation requested by the quiz.

In row notation, $\boxed{(\mathbf{f}_2^T + \mathbf{f}_3^T)x_A = 0}$; this is one scalar epipolar incidence constraint.

4.8.4 Fall 2025 Quiz 2 — Three-view epipolar geometry

Historical source: `3dcvMid/quizzesf25sol/cs532f25_quiz_2_solution.pdf`.

Question 1

Original problem · quiz booklet

Three images A, B, C observe a point. The pair $x_A \leftrightarrow x_B$ is known, but its image location x_C is unknown. Given fundamental matrices F_{AC} and F_{BC} (which map A/B points to lines in C), determine x_C.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Predict the line from image A

Why: The unknown point in C must satisfy the A-to-C geometry.

Map x_A to a line in C: $\boxed{\ell_C^{(A)} = F_{AC}x_A}$. Any correct x_C must satisfy $x_C^T\ell_C^{(A)} = 0$.

Step 2 — Predict the line from image B

Why: A second independent view supplies another constraint in the same image C.

Map x_B independently to a second line in C: $\boxed{\ell_C^{(B)} = F_{BC}x_B}$. The same x_C must satisfy $x_C^T\ell_C^{(B)} = 0$.

Step 3 — Intersect the two lines

Why: Their cross product gives the homogeneous intersection.

In the projective plane, the intersection of two lines is their cross product:

$$\boxed{x_C \sim \ell_C^{(A)} \times \ell_C^{(B)}}.$$

Step 4 — Normalize and check the intersection

Why: Homogeneous coordinates must be converted before reading a finite pixel.

Normalize $x_C = (q_1, q_2, q_3)^T$ to $(q_1/q_3, q_2/q_3)$ if $q_3 \neq 0$. Check both incidence products equal zero. If the two lines coincide, they do not determine a unique point; with noisy lines use an appropriate least-squares estimate.

Step 5 — Define a numerical companion

Why: The original symbolic question has no numerical matrices to multiply.

Original numerical companion: let $x_A = (1/2, 3/4, 1)^T$, $x_B = (1/4, 3/4, 1)^T$, and

$$F_{AC} = \begin{pmatrix} 0 & 0 & -1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}, \quad F_{BC} = \begin{pmatrix} 0 & 0 & -1 \\ 0 & 0 & -1 \\ 1 & 1 & 0 \end{pmatrix}.$$

These are compatible with identity-oriented normalized cameras centered at $(0,0,0)$, $(1,0,0)$, $(0,1,0)$ observing world point $(2,3,4)$.

Step 6 — Multiply both numerical matrices

Why: Full row products show exactly where both predicted lines come from.

Multiply both matrices, retaining every row operation:

$$\ell_C^{(A)} = \begin{pmatrix} 0 & 0 & -1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1/2 \\ 3/4 \\ 1 \end{pmatrix} = \begin{pmatrix} 0(1/2) + 0(3/4) - 1 \\ 0 \\ 1(1/2) + 0 + 0 \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \\ 1/2 \end{pmatrix},$$

$$\ell_C^{(B)} = \begin{pmatrix} 0 & 0 & -1 \\ 0 & 0 & -1 \\ 1 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1/4 \\ 3/4 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ 1/4 + 3/4 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix}.$$

Step 7 — Expand the cross product

Why: Its three components determine the common image point.

Intersect the two lines by expanding the cross product:

$$x_C \sim \begin{pmatrix} -1 \\ 0 \\ 1/2 \end{pmatrix} \times \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0(1) - (1/2)(-1) \\ (1/2)(-1) - (-1)(1) \\ (-1)(-1) - 0(-1) \end{pmatrix} = \begin{pmatrix} 1/2 \\ 1/2 \\ 1 \end{pmatrix}.$$

Divide by the last coordinate to get $(u_C, v_C) = (1/2, 1/2)$. A scaled result $(4, 4, 8)^T$ gives the same ratios.

Step 8 — Substitute into both lines

Why: Both independent constraints must pass at the answer.

Check both original line equations at the answer:

$$(\ell_C^{(A)})^T x_C = (-1)(1/2) + 0(1/2) + (1/2)(1) = 0,$$
$$(\ell_C^{(B)})^T x_C = (-1)(1/2) + (-1)(1/2) + (1)(1) = 0.$$

Coincident lines give a zero cross product and no unique point. Distinct parallel lines meet at infinity; nearly parallel lines make a finite intersection sensitive to noise. Image appearance must still confirm visibility and identity.

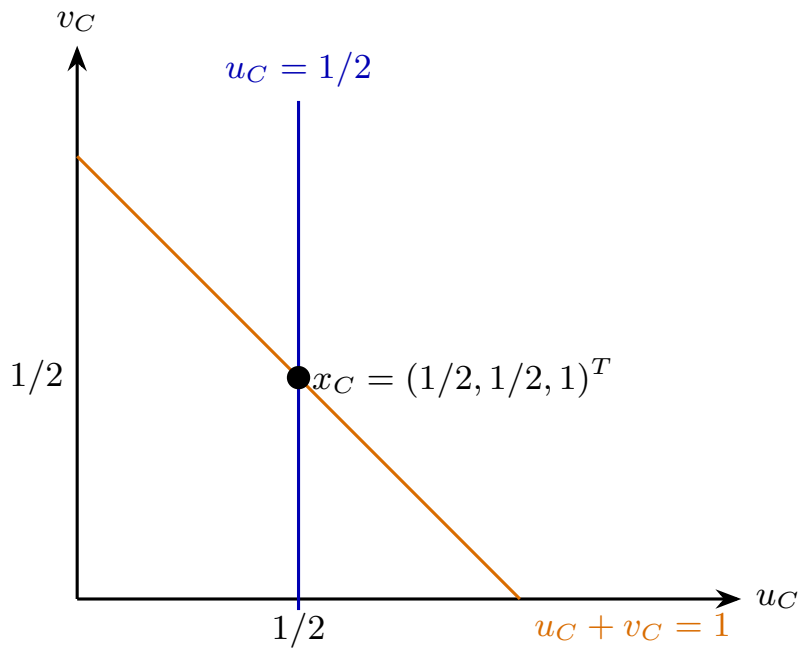


Fig. 4.10: The two numerical companion lines intersect at the predicted third-image point.

Question 2

Original problem · quiz booklet

Given $x' = (1, 2, 1)^T$ and

$$F = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 2 \\ 1 & 0 & 1 \end{bmatrix}$$

, propose an epipolar-consistent point x.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Choose the unprimed-image line

Why: The transpose maps the primed point back to a line in the unprimed image.

Start from $\boxed{x'^T F x = 0}$. To obtain the possible locations of x, form the line $\ell = F^T x'$ in the *unprimed* image.

Step 2 — Transpose and multiply explicitly

Why: The line coefficients come from row-column products.

Transpose explicitly, then multiply row by row:

$$F^T = \begin{pmatrix} 1 & 0 & 1 \\ 2 & 1 & 0 \\ 1 & 2 & 1 \end{pmatrix}, \qquad F^T x' = \begin{pmatrix} 1(1)+0(2)+1(1) \\ 2(1)+1(2)+0(1) \\ 1(1)+2(2)+1(1) \end{pmatrix} = \begin{pmatrix} 2 \\ 4 \\ 6 \end{pmatrix}.$$

Step 3 — Write the incidence equation

Why: A candidate must make the dot product zero.

Write line incidence for $x = (u, v, 1)^T$: $2u + 4v + 6 = 0$, or $\boxed{u + 2v + 3 = 0}$.

Step 4 — Choose a convenient finite point

Why: One free coordinate remains because the answer is a whole line.

Choose a convenient solution, e.g. $v = 0 \Rightarrow u = -3$. Thus $\boxed{x = (-3, 0, 1)^T}$. Alternatively $u = -1 \Rightarrow v = -1$.

Step 5 — Check the answer and the matrix rank

Why: Solving the given equation does not make an invalid matrix physically valid.

Verify the point by multiplying it into the original supplied matrix:

$$Fx = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 2 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} -3 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} -3+0+1 \\ 0+0+2 \\ -3+0+1 \end{pmatrix} = \begin{pmatrix} -2 \\ 2 \\ -2 \end{pmatrix},$$

$$x'^T Fx = \begin{pmatrix} 1 & 2 & 1 \end{pmatrix} \begin{pmatrix} -2 \\ 2 \\ -2 \end{pmatrix} = -2 + 4 - 2 = 0.$$

But $\det F = 1(1) - 2(-2) + 1(-1) = 4$, so its rank is three. The point solves the supplied equation; this matrix is not a valid fundamental matrix for pinhole cameras.

Source check / assumptions: The uploaded solution writes $x'^T F = x$; that is dimensionally/conceptually inaccurate. $x'^T F$ is a *line row-vector*, not a specific corresponding image point. There are infinitely many epipolar-consistent points on this line. Note that a true fundamental matrix must have rank 2; the numerical matrix given in the quiz is nonsingular, but its scalar incidence equation remains solvable as requested.

4.8.5 Fall 2025 Quiz 3 — Stereo depth sensitivity

Historical source: `3dcvMid/quizzesf25sol/cs532f25_quiz_3_solution.pdf`.

Question 1

Original problem · quiz booklet

Rectified stereo cameras have baseline $b = 10$ cm, common focal length $f = 50$ pixels, and true disparity $d^* = 25$ pixels. Determine the depth error caused by a 1-pixel disparity mistake.

Given / relevant facts:

Use $Z = bf/d$. The problem does not say whether disparity is over- or underestimated, so compute both; the supplied key uses a first-order approximate magnitude.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Compute the true depth

Why: We need a reference before defining an error.

Keep physical units consistent: $bf = (10\,\text{cm})(50\,\text{px}) = 500\,\text{cm px}$. True depth:

$$Z^* = 500/25 = \boxed{20\,\text{cm}}.$$

Step 2 — Calculate the underestimated-disparity case

Why: A smaller positive disparity should reconstruct a farther point.

For $\delta = -1$, measured disparity is 24 pixels:

$$\widehat{Z}_- = \frac{500}{24} = \frac{125}{6}\,\text{cm}, \qquad \Delta Z_- = \frac{125}{6} - 20 = \frac{125 - 120}{6} = \frac{5}{6}\,\text{cm} \approx +0.83333\,\text{cm}.$$

Step 3 — Calculate the overestimated-disparity case

Why: A larger disparity should reconstruct a nearer point.

For $\delta = +1$, measured disparity is 26 pixels:

$$\widehat{Z}_+ = \frac{500}{26} = \frac{250}{13}\,\text{cm}, \qquad \Delta Z_+ = \frac{250}{13} - 20 = \frac{250 - 260}{13} = -\frac{10}{13}\,\text{cm} \approx -0.76923\,\text{cm}.$$

Step 4 — Compare the first-order approximation

Why: The derivative estimates a small change rather than both exact nonlinear errors.

To obtain the first-order approximation, differentiate $Z(d) = bf/d$:

$$\frac{dZ}{dd} = -\frac{bf}{d^2}, \quad \Delta Z \approx -\frac{500}{25^2}\Delta d = -0.8\Delta d \text{ cm}.$$

Thus the *approximate magnitude* is $\boxed{0.8\,\text{cm}}$ for $|\Delta d| = 1$.

Source check / assumptions: The exact errors are asymmetric ($+0.8333$ and -0.7692 cm), whereas the linearized approximation gives ± 0.8 cm. The source key reports the approximation, not both exact cases.

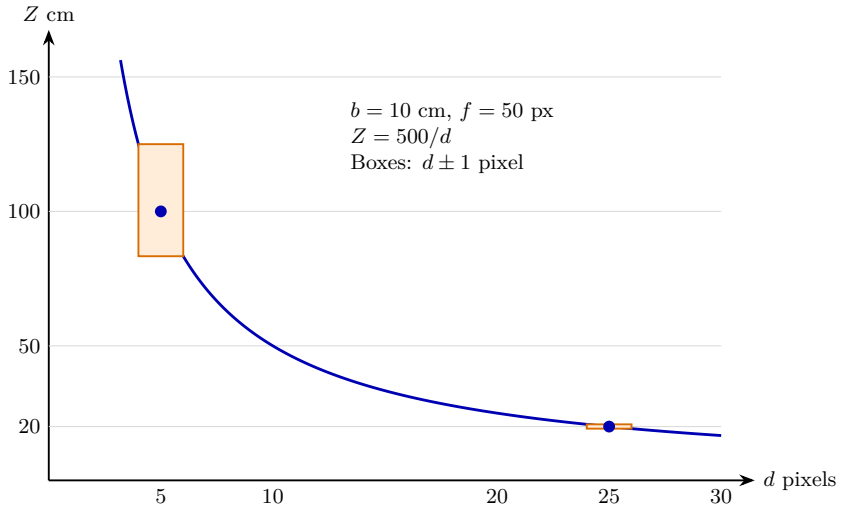


Fig. 4.11: The same pixel error produces much larger depth errors at smaller disparity.

Question 2

Original problem · quiz booklet

For the same setup, restrict the searched 3D depth range to between 1 m and 5 m. What disparity interval corresponds to this?

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Convert the baseline to meters

Why: Depth and baseline must use the same physical unit.

Convert baseline: $b = 10\,\text{cm} = 0.1\,\text{m}$. Then $bf = 0.1(50) = 5\,\text{m px}$.

Step 2 — Invert the depth formula

Why: Search limits are requested in disparity rather than depth.

Invert the depth equation: $d = bf/Z$. Larger depth implies smaller disparity.

Step 3 — Evaluate the far endpoint

Why: The farthest permitted point gives the smallest positive disparity.

At farthest depth $Z_{\text{max}} = 5$ m: $d_{\text{min}} = 5/5 = \boxed{1\text{ px}}$.

Step 4 — Evaluate the near endpoint

Why: The nearest permitted point gives the largest disparity.

At nearest depth $Z_{\text{min}} = 1$ m: $d_{\text{max}} = 5/1 = \boxed{5\text{ px}}$. Search $\boxed{d \in [1, 5]\text{ px}}$.

4.8.6 Spring 2019 Quiz 5 — Exact stereo errors

Historical source: `3dcvMid/olderquizzes/cs532s19_quiz5-2.pdf`.

Problem 1(a–c)

Original problem · quiz booklet

Rectified cameras have baseline $b = 10$ cm and focal length $f = 5$ pixels. True disparity is $d^* = 3$ pixels. (a) Find the depth error when disparity is underestimated by one pixel; (b) when overestimated by one pixel; (c) explain whether these errors remain constant at larger disparity.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Insert focal length and baseline

Why: Their product determines the depth scale.

Write the depth-disparity law and insert the givens: $Z(d) = bf/d = (10)(5)/d = 50/d$ cm.

Step 2 — Compute the true depth

Why: Signed errors are measured relative to this value.

True depth is $Z^* = 50/3 = \boxed{16\tfrac{2}{3}\text{ cm}} \approx 16.6667$ cm.

Step 3 — Underestimate disparity by one

Why: Subtract exact fractions before rounding the error.

Underestimate disparity by one: $\hat{d} = 3 - 1 = 2$. Then

$$\widehat{Z} = \frac{50}{2} = 25\,\text{cm}, \qquad \Delta Z = 25 - \frac{50}{3} = \frac{75 - 50}{3} = \frac{25}{3}\,\text{cm} \approx +8.3333\,\text{cm}.$$

A positive signed error means the estimated point is too far away.

Step 4 — Overestimate disparity by one

Why: The nonlinear inverse relation produces a different error magnitude.

Overestimate disparity by one: $\hat{d} = 3 + 1 = 4$. Then

$$\widehat{Z} = \frac{50}{4} = \frac{25}{2}\,\text{cm}, \qquad \Delta Z = \frac{25}{2} - \frac{50}{3} = \frac{75 - 100}{6} = -\frac{25}{6}\,\text{cm} \approx -4.1667\,\text{cm}.$$

A negative signed error means the estimate is too near. The two magnitudes are unequal.

Step 5 — Generalize the two exact errors

Why: Their denominators show how the magnitudes change with disparity.

No: $Z(d) = bf/d$ is nonlinear. The exact error for +1 px is $Z(d+1) - Z(d) = -bf/[d(d+1)]$; for -1 px it is $Z(d-1) - Z(d) = +bf/[d(d-1)]$ for $d > 1$. Both magnitudes diminish at larger d.

Step 6 — Interpret the local derivative

Why: It explains the trend for small perturbations.

The derivative $dZ/dd = -bf/d^2$ gives the local approximation $|\Delta Z| \approx bf|\Delta d|/d^2$. Because $1/d^2$ decreases as d grows, disparity errors cause smaller depth errors for nearer points.

4.8.7 Spring 2020 Quiz 5 — Repeat disparity test

Historical source: `3dcvMid/olderquizzes/cs532s20_quiz5.pdf`.

Header note: the supplied PDF contains mixed/overprinted year labels. The filename and separate retest entry are retained.

Problem 1(a–c)

Original problem · quiz booklet

Rectified stereo cameras: $b = 10$ cm, $f = 5$ pixels, true $d^* = 3$ pixels. Determine (a) error for $\hat{d} = 2$, (b) error for $\hat{d} = 4$, and (c) how errors behave as disparity increases.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Set up the stereo depth law

Why: The same physical and pixel units must be used consistently.

Write $Z(d) = bf/d$, using $bf = (10\,\text{cm})(5\,\text{px}) = 50\,\text{cm px}$.

Step 2 — Find the true depth

Why: This is the reference for both signed errors.

True depth: $Z^* = 50/3 = 16.6667$ cm.

Step 3 — Solve the smaller-disparity case

Why: A common denominator makes the positive error explicit.

Underestimate disparity by one: $\hat{d} = 3 - 1 = 2$. Then

$$\widehat{Z} = \frac{50}{2} = 25\,\text{cm}, \qquad \Delta Z = 25 - \frac{50}{3} = \frac{75-50}{3} = \frac{25}{3}\,\text{cm} \approx +8.3333\,\text{cm}.$$

A positive signed error means the estimated point is too far away.

Step 4 — Solve the larger-disparity case

Why: The exact subtraction exposes the unequal negative error.

Overestimate disparity by one: $\hat{d} = 3 + 1 = 4$. Then

$$\widehat{Z} = \frac{50}{4} = \frac{25}{2}\,\text{cm}, \qquad \Delta Z = \frac{25}{2} - \frac{50}{3} = \frac{75-100}{6} = -\frac{25}{6}\,\text{cm} \approx -4.1667\,\text{cm}.$$

A negative signed error means the estimate is too near. The two magnitudes are unequal.

Step 5 — Explain the asymmetry

Why: Equal disparity changes do not give equal changes in inverse disparity.

Explain why the two numbers differ: Z is proportional to $1/d$, not linear in d; changes of +1 and -1 around the same d need not have equal effects.

Step 6 — State the large-disparity trend

Why: The derivative and exact formula both predict decreasing sensitivity.

Generalize: $\frac{dZ}{dd} = -bf/d^2$, so a fixed disparity error typically matters less at larger disparities. For exact +1 px, $\Delta Z = -bf/[d(d+1)]$.

4.8.8 A4. Essential matrix and cheirality (L4, slides 22–41)

Original practice from the supplied booklet; not a historical quiz question.

Illustrative problem

Original problem · quiz booklet

Two calibrated cameras have relative rotation $R = I$ and translation $t = (1, 0, 0)^T$ in compatible normalized units. Form E, identify its rank and singular values, and explain why pose decomposition is not unique without a positive-depth test.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Write the essential-matrix construction

Why: Translation and rotation determine the calibrated two-view relation.

The essential matrix is $\boxed{E = [t]_\times R}$, with cross-product matrix

$$[t]_\times = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}$$

.

Step 2 — Insert the translation and rotation

Why: The cross-product matrix fixes every entry.

Insert $t = (1, 0, 0)^T$ and $R = I$:

$$\boxed{E = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}}.$$

Step 3 — Multiply E transpose by E

Why: Its eigenvalues determine E's singular values.

Compute the singular-value check from the complete product:

$$\begin{aligned} E^T E &= \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}. \end{aligned}$$

Its eigenvalues are $1, 1, 0$, so the singular values of E are their nonnegative square roots: $1, 1, 0$. Two are nonzero, hence rank two.

Step 4 — Connect calibrated and pixel coordinates

Why: Intrinsic matrices convert between the essential and fundamental relations.

For calibrated normalized image coordinates $\hat{x}, \hat{x}'$, corresponding points satisfy $\hat{x}'^T E \hat{x} = 0$. If pixel intrinsics are known, $E = K_R^T F K_L$.

Step 5 — List the pose ambiguity

Why: The SVD gives two rotations and two translation signs.

Decomposing a general essential matrix via SVD gives two rotation candidates R_1, R_2 and a translation *direction* with two signs, i.e. four candidates $(R_1, +t)$, $(R_1, -t)$, $(R_2, +t)$, $(R_2, -t)$.

Step 6 — Apply the positive-depth test

Why: The visible scene must lie in front of both observing cameras.

Triangulate one or more matched 3D points under each candidate. Keep the one for which the points have positive camera-frame Z in *both* views (cheirality). The magnitude of t remains unobservable without external scale.

4.9 Review and source checklist

Start with the *epipolar visual guide and calculations A–C*: point-to-line multiplication, candidate checks, transpose, and epipole nullspaces. Then work through the expanded quiz examples below. The *stereo sensitivity walkthrough* explains exact errors, the derivative, distance-squared sensitivity, and uncertainty intervals.

4.9.1 Formula and application map

Task	Formula box and procedure
Correct a noisy fitted F to rank two	(4.1), *estimation*
Measure pixel distance from epipolar predictions	(4.2)
Use calibration to isolate camera motion	(4.3), *pose recovery*
Generate and test four pose candidates	(4.4)
Compute rectified stereo depth	(4.5), *stereo*
Distinguish exact from first-order error	(4.6)
Restrict disparity using a depth interval	(4.7)
Aggregate a cost volume and select a hypothesis	(4.8), *matching*
Compare patch appearance	(4.9)
Encode relative brightness order	(4.10)
Adapt support at boundaries	(4.11)

The canonical fundamental constraint, linear eight-point rows, radial correction, and rectification formulas remain in *Lecture 2*. Normalized point-coordinate transforms remain in *Lecture 2 estimation*.

4.9.2 Worked examples and assigned quizzes

Example	Source or purpose
4.1	Both Spring 2019 Quiz 4 files: P1 line, P2 explicit constraint; duplicate retained for retest
4.2	Fall 2025 Quiz 2 Q1: third-view intersection and degeneracies
4.3	Same quiz Q2: consistent points, line/point correction, invalid rank-three F
4.4	Original noisy normalized fit and held-out numerical checks
4.5	Original essential-matrix/four-pose calculation
4.6	Fall 2025 Quiz 3 Q1: both exact signs versus the supplied linearization
4.7	Spring 2019 Quiz 5, all three bullets; Spring 2020 duplicate retest
4.8	Fall 2025 Quiz 3 Q2: depth range to disparity bounds
4.9	Original SAD/SSD/NCC calculation
4.10	Original rank/Census distinction
4.11	Original boundary-weighting calculation

4.9.3 Slide coverage without repeated teaching

PDF pages	Disposition
1	Title and source metadata
2–21	Distortion, epipolar planes/epipoles/F and linear rows already taught in L2; guided geometry recap and expanded quiz applications here
22–23	Essential-matrix constraints, four poses and cheirality
24–37	Noisy normalized F fitting, rank projection, residuals, seven-point robust sampling and model degeneracy
38–41	Search reduction and rectification already taught in L2; linked as prerequisites
42–57	Dense depth hypotheses, cost volumes, support choices, matching criteria and representation/algorithm overview
58–64	Rectification linked to L2; new stereo derivation and depth applications
65–77	Window size, cost curves, texture/occlusion ambiguity, rank/Census, shiftable/rod/adaptive support

What to check in an exam answer: image order, coordinate units, matrix rank, transform direction, positive depths in both cameras, signed versus absolute errors, and whether a score must be minimized or maximized.

Finish with the *TLDR and short quiz*.

4.10 TLDR

Lecture 4 in one sentence: estimate valid two-view geometry, choose a camera pose that sees the scene, and turn reliable stereo matches into depths with honest error estimates.

4.10.1 The important things

1. **Keep points, lines, and scalars distinct.** A pixel defines a ray; that ray and the baseline define a plane whose intersection with the other image is a line. Compute that line with Fx, then test a candidate using $x'^T Fx = 0$. The assigned quizzes practice symbolic expansion and third-view prediction. Two lines in a third image can locate a point, unless their intersection is degenerate. See *quiz practice*.
2. **Check rank before trusting "F."** Fall 2025 Quiz 2 supplies a rank-three matrix. Its bilinear equation can be solved, but it is not valid pinhole-camera geometry. A noisy estimate requires a rank-two correction and pixel-space residual checks. See *estimation*.
3. **Calibration supplies an essential matrix.** $E = K'^T F K = [t]_\times R$ has equal nonzero singular values and one zero. It recovers translation direction, not metric scale. Numerical conditioning transforms and camera calibration are different operations. See *essential geometry*.
4. **Test all four poses.** Two rotations and two translation signs produce four candidates. Positive reconstructed depths in both cameras select the visible configuration for suitable nondegenerate data. Translation t is not the second camera center.
5. **Disparity is inverse depth.** In the specified rectified convention, $Z = fb/d$. Use focal length and disparity in pixels, baseline and depth in one length unit. Account for unequal principal-point columns. See *stereo*.
6. **A one-pixel depth error is not symmetric.** Underestimating disparity pushes the point farther by more than an equal overestimate pulls it nearer. Use the exact error when the disparity error is a substantial fraction of disparity. Small disparities are especially fragile: local error magnitude is approximately $Z^2|\delta|/(fb)$. At fixed cameras and small disparity error, doubling depth roughly quadruples absolute error.
7. **Appearance chooses among geometric candidates.** Dense stereo generates hypotheses, evaluates costs, aggregates support, and selects or rejects. Minimize SAD/SSD; maximize NCC. Flat patches, repetitions, and occlusions can defeat a low-cost match. See *matching*.
8. **Support has assumptions.** A constant-disparity window favors fronto-parallel surfaces; large windows can mix depths. Shifted, elongated, weighted, or oriented-plane support can help. Rank retains a count; Census retains the ordered relative-intensity pattern. None removes ambiguity automatically.

4.10.2 Short quiz

These original questions test this chapter's additions.

Each question has its own complete worked answer below. Open it after trying the problem; all answers are visible in print.

Question 1

Original practice question

For $x_A = (0, 1, 1)^T$, $x_B = (1, 0, 1)^T$, expand $x_B^T F x_A$ in matrix entries.

Worked solution

Step 1 — Multiply the point into F

Why: The zero first coordinate selects columns two and three.

$$\begin{aligned} Fx_A &= \begin{pmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} f_{12} + f_{13} \\ f_{22} + f_{23} \\ f_{32} + f_{33} \end{pmatrix}. \end{aligned}$$

Step 2 — Dot with the second point

Why: Its transpose selects the first and third output entries.

$$\begin{pmatrix} 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} f_{12} + f_{13} \\ f_{22} + f_{23} \\ f_{32} + f_{33} \end{pmatrix} = f_{12} + f_{13} + f_{32} + f_{33} = 0.$$

Question 2

Original practice question

Two predicted lines in image C are proportional. Does their cross product give a unique finite correspondence?

Worked solution

Step 1 — Write proportional line vectors

Why: Nonzero scaling represents the same homogeneous line.

Let $\ell_2 = k\ell_1$ with $k \neq 0$. Then $\ell_1 \times \ell_2 = k(\ell_1 \times \ell_1) = 0$.

Step 2 — Interpret the zero result

Why: The zero homogeneous vector cannot represent a unique image point.

No unique finite correspondence is determined: the two constraints describe the same line.

Question 3

Original practice question

A noisy matrix has singular values $(4, 2, 0.2)$. What singular values do rank-two F correction and essential-form correction use?

Worked solution

Step 1 — Apply the rank-two F rule

Why: Only the smallest singular value must be zero.

$$\Sigma_F = \begin{pmatrix} 4 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Step 2 — Apply the essential-matrix rule

Why: The first two singular values must also agree.

$$a = (4 + 2)/2 = 3, \qquad \Sigma_E = \begin{pmatrix} 3 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Thus F uses $(4, 2, 0)$ and E uses $(3, 3, 0)$.

Question 4

Original practice question

A candidate pose reconstructs a point with depths $+5$ and -5 in the two cameras. Is that point a valid visible correspondence under that pose?

Worked solution

Step 1 — Check the first camera

Why: A visible point must have positive camera-frame depth.

The first depth $+5 > 0$ passes the first camera's test.

Step 2 — Check the second camera

Why: Cheirality requires positive depth in both observing cameras.

The second depth $-5 < 0$ fails. Reject this point/pose combination as a valid visible correspondence.

Question 5

Original practice question

Rectified cameras have $b = 0.2\,\text{m}$, $f = 600$ pixels, $d = 30$ pixels. What is the depth? If measured disparity becomes 31, is the point reconstructed nearer or farther?

Worked solution

Step 1 — Compute the true depth

Why: Baseline and depth both use meters; focal length and disparity both use pixels.

$$fb = 600(0.2) = 120\,\text{m px}, \qquad Z = 120/30 = 4\,\text{m}.$$

Step 2 — Compute the perturbed depth and sign

Why: A larger disparity should place the estimate nearer.

$$\widehat{Z} = \frac{120}{31}\,\text{m}, \qquad \Delta Z = \frac{120}{31} - 4 = \frac{120 - 124}{31} = -\frac{4}{31}\,\text{m} \approx -0.1290\,\text{m}.$$

The estimate is nearer.

Question 6

Original practice question

For that setup, which disparity interval corresponds to depths from 2 to 6 meters?

Worked solution

Step 1 — Evaluate the far limit

Why: Inverse depth makes the far endpoint the smaller disparity.

$$d_{min} = \frac{fb}{Z_{max}} = \frac{120}{6} = 20\,\text{pixels}.$$

Step 2 — Evaluate the near limit

Why: The closer endpoint has the larger disparity.

$$d_{max} = \frac{120}{2} = 60\,\text{pixels}, \qquad d \in [20, 60].$$

Question 7

Original practice question

Which direction do you optimize SAD and NCC? Can a constant-intensity patch supply a defined zero-mean NCC score?

Worked solution

Step 1 — Identify what each score rewards

Why: Difference costs reward small disagreement; correlation rewards similar centered patterns.

Minimize SAD. Maximize zero-mean NCC, or equivalently minimize a consistently defined negative correlation cost.

Step 2 — Check the NCC denominator

Why: A flat patch has no centered contrast to normalize.

For constant $I_i = c$, every $I_i - \bar{I} = 0$, so $\sqrt{\sum_i (I_i - \bar{I})^2} = 0$. The NCC denominator vanishes; the normalized score is undefined.

Question 8

Original practice question

With center intensity 10 and ordered neighbors $(8, 12, 7, 11)$, what are the Census string and rank under the strict darker-than-center convention?

Worked solution

Step 1 — Compare each neighbor with the center

Why: The strict darker-than-center convention assigns one to values below 10.

$$\begin{pmatrix} 8 < 10 \\ 12 < 10 \\ 7 < 10 \\ 11 < 10 \end{pmatrix} \quad \Rightarrow \quad \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}.$$

Step 2 — Read the string and count its ones

Why: Census retains order; rank retains only the count.

The Census string is 1010. The rank is $1 + 0 + 1 + 0 = 2$.

CHAPTER

FIVE

LECTURE 5: FIND POINTS YOU CAN TRACK

Which part of a picture can you recognize after the camera moves? A plain wall gives no location cue. An edge gives only one direction. A corner can constrain both. This chapter turns that observation into a detector, a tracker, and a matching procedure.

Original quiz questions and fully worked solutions include Spring 2019 Quiz 6, Fall 2025 Quiz 4, and the numerical Harris/Lucas–Kanade booklet problems. Every original question has red numbered steps, a short reason, and blue mathematical workings.

5.1 Reading route

1. *Find distinctive features*: the mountain panorama and Mars Rover matches.
2. *Understand Harris corners*: shifted windows → gradients → a 2×2 matrix → eigenvalues → selected corners. Includes the 3D error-surface explorer.
3. *Track with Lucas–Kanade*: build and solve the patch equations, then handle large motion, drift, and exposure changes.
4. *Describe and match features*: BRIEF, ORB, BRISK, Hamming distance, ratio tests, and measured matching quality.
5. *Full quiz workbook*, *formula/source checklist*, and final *TLDR with answered quiz*.

Source: `wk5_cs532_Corners.pdf`, 91 PDF pages. References use one-based PDF pages; visible page labels match them where printed. Early unnumbered slides are identified by their PDF positions. Source illustrations retain their credits in captions and the figure register. The chapter develops new feature-selection/tracking mathematics; Lecture 4 remains the reference for patch costs and stereo geometry.

5.2 Find a small set of recognizable points

To stitch two mountain photos, first find the same places in both. Aligning all pixels blindly is expensive and easily confused by sky or repeated snow. A smaller set of distinctive neighborhoods gives the alignment process useful anchors.

Source: Lecture 5 PDF pages 3–10.

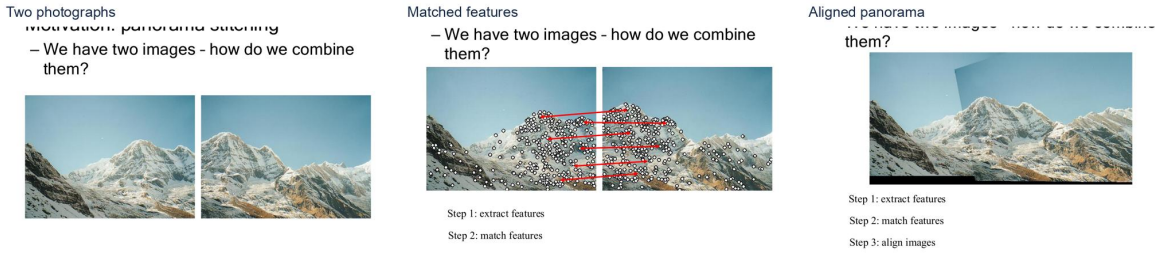


Fig. 5.1: The lecture's panorama sequence: detect points, find correspondences, then align. Alignment uses the homography methods already taught in Lectures 2–3; here the question is how to obtain their inputs. Source PDF pages 4–6; individual photographer/reuse terms unidentified.

A **feature detector** finds an interesting location. A **descriptor** summarizes its surrounding appearance. A **matcher** proposes which descriptor in another image belongs to the same physical feature. These are three separate decisions: a strong corner can still get a wrong match.

Useful features combine four properties:

- **Repeatability:** the same physical location can be detected again after viewpoint or illumination changes.
- **Distinctiveness:** its patch differs from plausible alternatives.
- **Locality:** the description uses a limited neighborhood, helping with clutter and partial occlusion.
- **Compactness:** there are far fewer features than pixels, and comparing their descriptions is efficient.

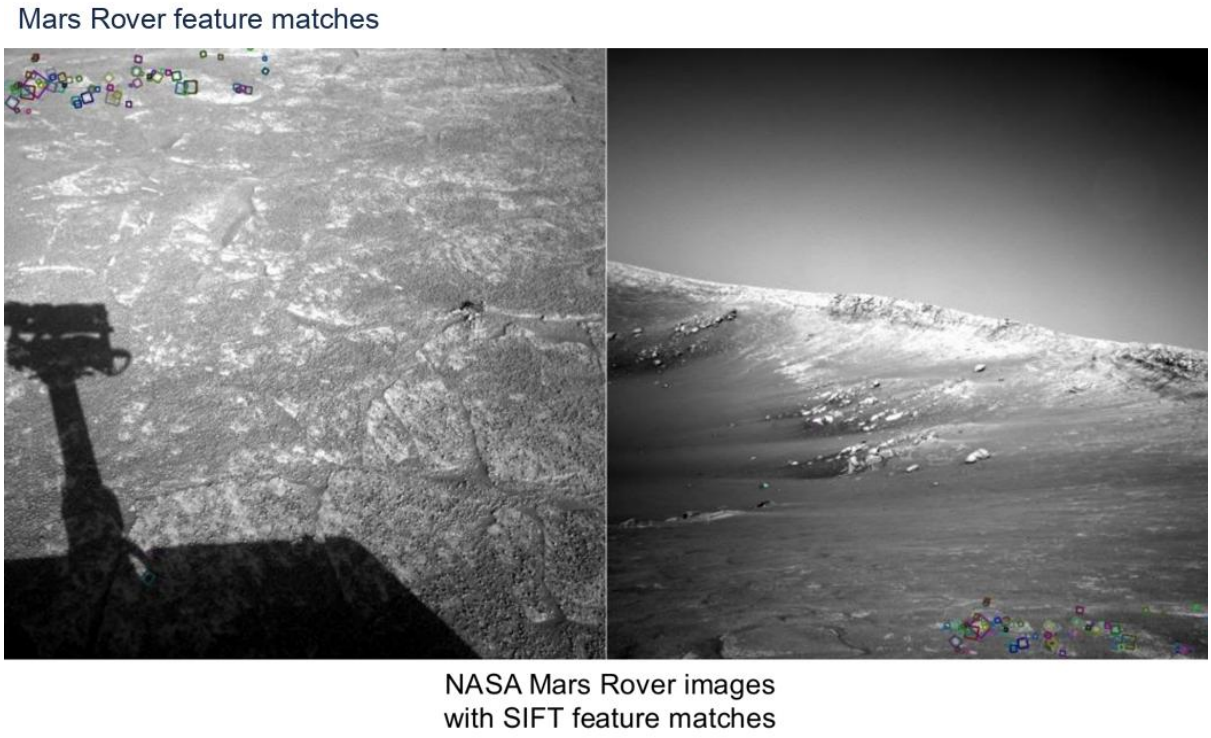


Fig. 5.2: The lecture's difficult Mars Rover pair, with SIFT matches revealed. NASA imagery; match figure credited to Noah Snavely in PDF page 10. Look at the tiny corresponding marks rather than expecting the two entire views to look alike. SIFT motivates the problem; this deck's detailed descriptor material is BRIEF, ORB, and BRISK.

What makes this hard? A distinctive rock may change size or orientation. Repeated texture produces look-alikes. A feature may disappear behind another surface. Features help image alignment, reconstruction, tracking, navigation, retrieval, and recognition, but none of those applications makes a proposed correspondence automatically correct.

Next, *test whether a patch fixes a location in both directions*.

5.3 Why corners are easier to locate

Slide a small image window. Does its appearance change? If it barely changes, you cannot tell where it belongs. Harris corner detection measures that change in every direction.

Source: PDF pages 11–33. Numerical booklet companion: *Harris gradients and response*. Tracking interpretation: *Spring 2019 Quiz 6*.

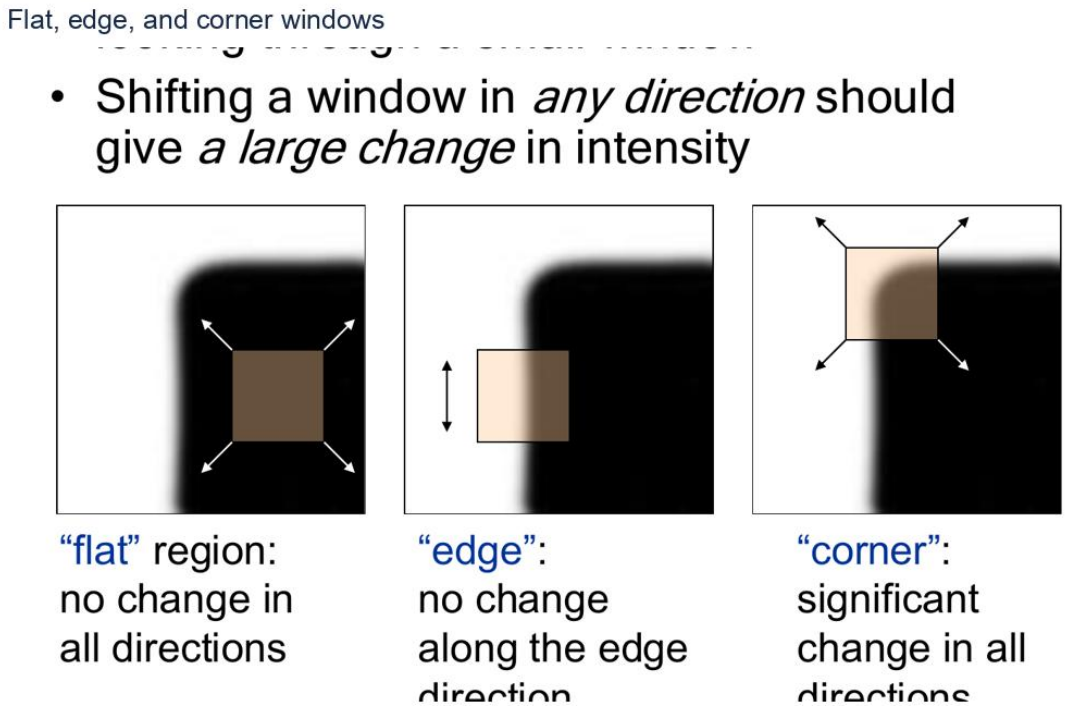


Fig. 5.3: Source PDF page 11. Flat: little change in any direction. Edge: little change along the edge. Corner: appreciable change in every direction. "Corner" describes a locally well-constrained patch, not just a sharp-looking center pixel.

5.3.1 From a shifted patch to a 2×2 matrix

Let $I(x, y)$ be intensity and $d = (u, v)^T$ a small shift in pixels. Let $w_p \geq 0$ weight sample p in the patch; a Gaussian emphasizes samples near its center. The source starts with unit weights and later introduces Gaussian weights.

Formula — patch change and its local approximation

$$
\begin{aligned}
E(d) &= \sum_{p \in W} w_p [I(p+d) - I(p)]^2, \\
I(p+d) &\approx I(p) + I_x(p)u + I_y(p)v, \\
E(d) &\approx d^T M d, \qquad M = \sum_{p \in W} w_p \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}.
\end{aligned}
\tag{5.1}
$$

M is the **structure tensor**, also called the second-moment matrix. I_x, I_y are intensity change per pixel. M is 2×2, d is 2×1, and E is a scalar squared-intensity cost. The first line is the chosen exact patch objective; the quadratic form is a first-order small-shift approximation. Assume a differentiable/interpolated image and consistent sample weights.

Why does the matrix appear? Substitute the Taylor approximation into the intensity difference. The unchanged

$I(p)$ cancels, leaving $I_x u + I_y v$. Squaring gives

$$I_x^2 u^2 + 2 I_x I_y uv + I_y^2 v^2.$$

Summing those three coefficients gives the diagonal and off-diagonal entries of M. Multiplying $[u\ v] M (u, v)^T$ expands back to the same expression. A gradient column $g_p = (I_x, I_y)^T$ contributes the outer product $w_p g_p g_p^T$, not the scalar dot product $g_p^T g_p$.

Build and use the structure tensor

1. Compute both spatial derivatives at each patch sample.
2. Form its full outer product; multiply by the sample weight.
3. Add the matrices entry by entry.
4. Evaluate a trial shift with $d^T M d$, or inspect eigenvalues to find weak directions.

Weights and intensity scaling affect response magnitudes, so thresholds must match that convention. See the full products in *the numerical Harris problem*.

5.3.2 Eigenvalues tell you the weak direction

An **eigenvector** is a direction the matrix scales without turning. Its **eigenvalue** is the scale. For this symmetric nonnegative tensor, choose perpendicular unit eigenvectors. Writing a shift in those directions makes the cost especially simple.

Formula — tensor eigenvalues and directional change

$$\begin{aligned} &M = Q\,\mathrm{diag}(\lambda_1, \lambda_2) Q^T, \quad Q^T Q = I, \quad \lambda_i \geq 0, \\ &d = Qz \quad \Longrightarrow \quad E(d) \approx \lambda_1 z_1^2 + \lambda_2 z_2^2, \\ &M = \begin{pmatrix} a & c \\ c & b \end{pmatrix} \quad \Longrightarrow \quad \lambda_\pm = \frac{a + b \pm \sqrt{(a-b)^2 + 4c^2}}{2}. \end{aligned} \tag{5.2}$$

The eigenvalue formula comes from $\det(M - \lambda I) = (a - \lambda)(b - \lambda) - c^2 = 0$. For a positive cost level k, the ellipse semiaxes are $\sqrt{k/\lambda_i}$ when both eigenvalues are positive. A small eigenvalue makes a long, weakly constrained direction. Eigenvalues need not be equal for a useful corner.

Classify the patch

1. Compute both eigenvalues and sort them as $\lambda_{max} \geq \lambda_{min}$.
2. Both small: flat patch. One small: edge-like patch. Both sufficiently large: useful two-directional structure.
3. Also check conditioning $\lambda_{max}/\lambda_{min}$ when $\lambda_{min} > 0$; a very large ratio means much weaker information in one direction.
4. Interpret "large" relative to noise, intensity scale, weights, and patch size.

For $M = \mathrm{diag}(4, 9)$, unit horizontal motion costs 4, unit vertical motion costs 9. The constant-cost contour $4u^2 + 9v^2 = 1$ has semiaxes $1/2$ and $1/3$. Both directions are constrained, although the vertical one changes faster.

For a diagonal edge with two gradients $(2, 2)^T$, summation gives

$$M = \begin{pmatrix} 8 & 8 \\ 8 & 8 \end{pmatrix}, \qquad M \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

Both I_x and I_y can be large while motion along $(1, -1)^T$ remains invisible. This answers the slanted-edge part of *the original trackability quiz*.

5.3.3 Turn the score into selected corners

Formula — Harris and Shi–Tomasi responses

$$R_H = \det M - k(\operatorname{tr} M)^2 = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2, \qquad R_{ST} = \lambda_{min}(M). \tag{5.3}$$

k is a dimensionless parameter, typically 0.04–0.06 in these slides. A large positive Harris response suggests a corner; an edge usually gives a negative response, and a weak flat patch gives a small response. A positive response alone does not set a practical detection threshold. Shi–Tomasi directly measures the least-constrained direction.

Run Harris detection

1. Differentiate the image; smooth the derivative products with the chosen Gaussian window.
2. Compute M and R_H at every candidate pixel.
3. Keep responses above a chosen threshold.
4. Apply **non-maximum suppression (NMS)**: retain a local peak rather than every nearby high response. Define a tie-breaking rule and minimum separation.
5. Reject image-border candidates without sufficient valid support.

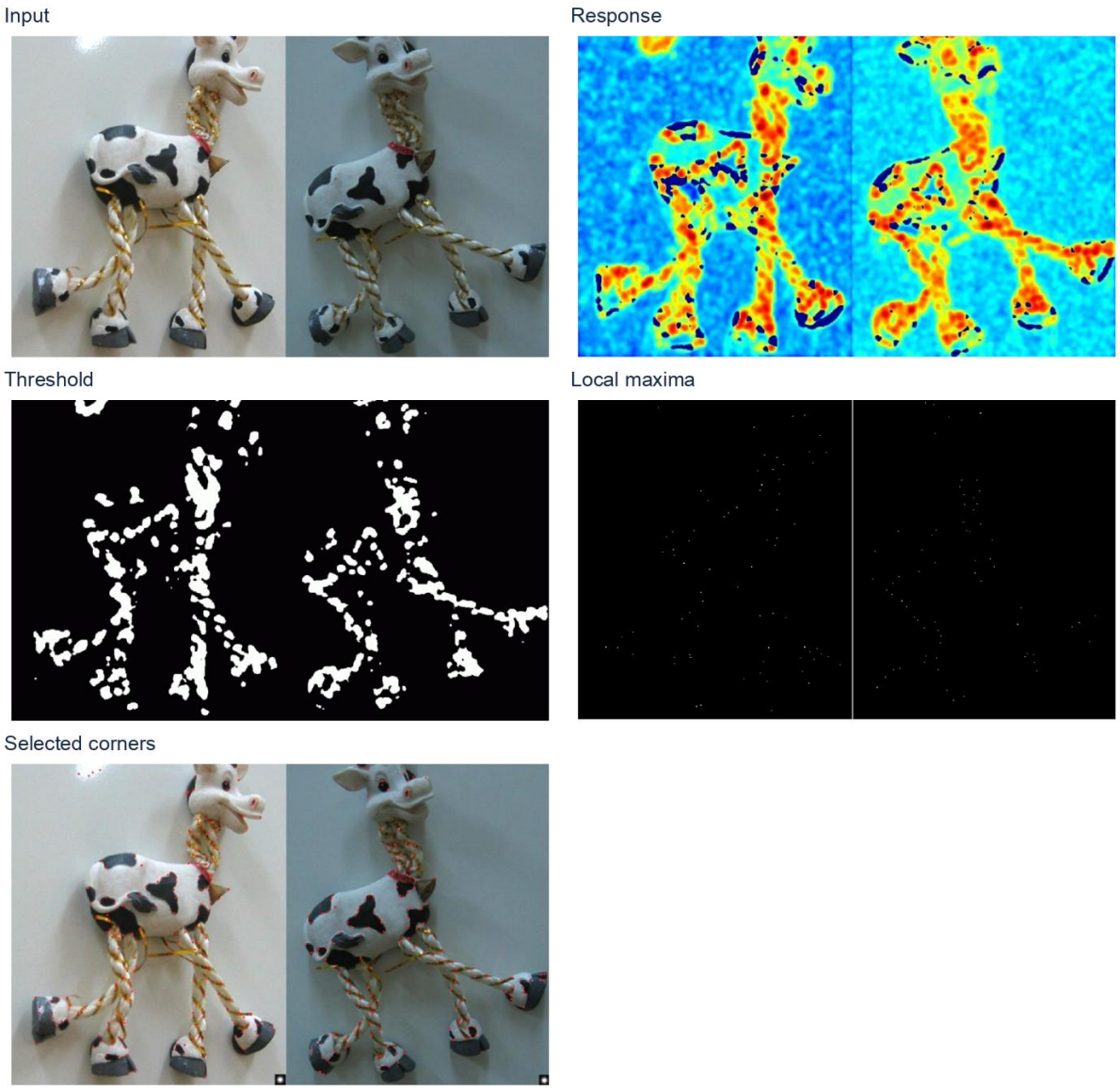


Fig. 5.4: The source toy-animal sequence, PDF pages 28–33. Thresholding produces clusters; NMS selects representative peaks. The source local-maxima marks are tiny: compare that sparse panel with the threshold mask and final overlay. Source image creator/reuse terms unidentified.

Worked example 5.1 — a threshold is not NMS

Original practice question

Original companion to: PDF pages 30–32. Suppose the local response grid is

$$R = \begin{pmatrix} 1 & 2 & 1 & 0 & 0 \\ 2 & 9 & 8 & 0 & 1 \\ 1 & 7 & 3 & 1 & 2 \\ 0 & 0 & 1 & 6 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

Step 1 — Apply the threshold

Why: Weak responses should not become candidates.
With the strict threshold $R > 5$, the surviving values are $9, 8, 7, 6$. Use one-based row/column locations: $(2, 2), (2, 3), (3, 2), (4, 4)$.

Step 2 — Compare each candidate with its neighbors

Why: Adjacent high responses can describe the same local feature.
In a 3×3 neighborhood, the 9 suppresses the neighboring 8 and 7. The 6 at $(4, 4)$ is larger than its own neighbors, so it survives. The two retained centers are $(2, 2)$ and $(4, 4)$.

Step 3 — Check what the procedure guarantees

Why: A local maximum is a selection rule, not proof of a correct correspondence.
Both surviving responses exceed the threshold and are local peaks. A later matching step must still identify the corresponding physical points.

5.3.4 Explore the surface in 3D

Explore feature trackability in 3D — print edition

Switch among a flat patch, an edge, and a corner. The image panel is schematic; the **two listed gradient samples** define the exact model. Change the trial shift and rotate the error surface to inspect the weak directions. Orbiting only changes the inspection view.

Static experiment: use unit weights and these two-sample models:

Patch	Gradient samples	Tensor M	Eigenvalues	Trial cost at $(u, v) = (1, 0)$
Flat	$(0, 0), (0, 0)$	zero matrix	$0, 0$	0
Edge	$(2, 2), (2, 2)$	$\left(\begin{smallmatrix} 8 & 8 \\ 8 & 8 \end{smallmatrix}\right)$	$16, 0$	8
Corner	$(2, 0), (0, 3)$	$\mathrm{diag}(4, 9)$	$9, 4$	4

The exact displayed quadratic model is $E_{lin} = d^T M d$. Its relationship to the nonlinear image-shift cost is approximate. For the edge, shifts $(1, -1)$ cost zero; for the corner that shift costs $4 + 9 = 13$. This is the difference between an ambiguous valley and a bowl with a unique minimum.

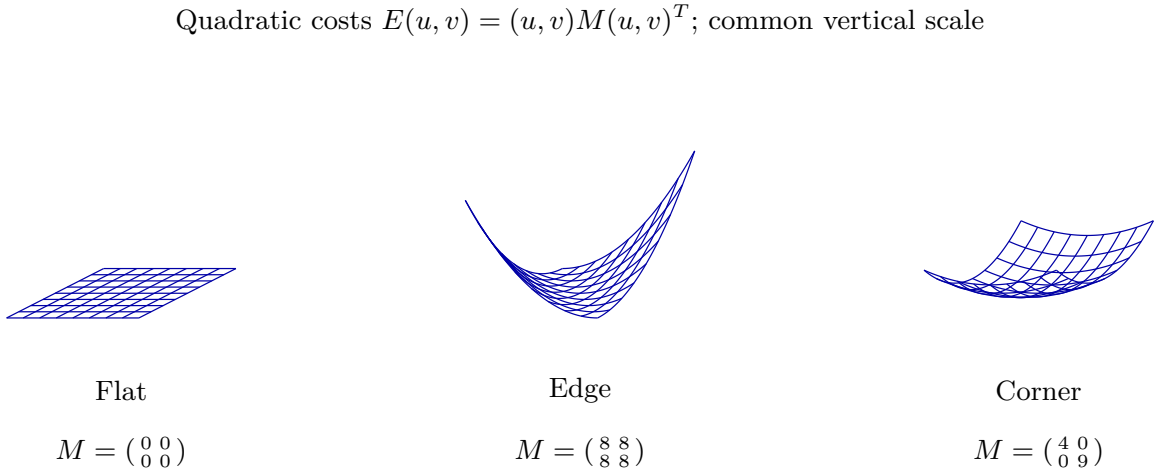


Fig. 5.5: Original surfaces for the three stated tensors. Every printed result uses the same matrices as the interactive.

5.4 Track a feature by solving for its motion

Which small shift makes the next patch resemble this one? Instead of trying every pixel displacement, Lucas–Kanade uses local intensity derivatives to predict an improving shift.

Source: PDF pages 34–60. Full assigned problems: *exposure changes*, *trackability*, and *the numerical displacement calculation*.

Camera motion



36

Object motion



37

Fig. 5.6: Source PDF pages 36–37. Either camera motion or object motion changes image locations. The tracker estimates local image displacement; that displacement alone does not identify its 3D cause.

5.4.1 One pixel gives one equation, not two

Let I be the first frame and J the next. **Brightness constancy** assumes the same visible surface has the same intensity after motion. **Spatial coherence** assumes a sufficiently small patch shares one translation. **Small motion** permits a local Taylor approximation.

Formula — brightness constancy and patch least squares

$$\begin{aligned} J(p+d) &\approx I(p), \qquad J(p+d) \approx J(p) + J_x u + J_y v, \\ J_x u + J_y v &\approx -I_t, \qquad I_t = J(p) - I(p), \\ A &= \begin{pmatrix} J_x(p_1) & J_y(p_1) \\ \vdots & \vdots \\ J_x(p_n) & J_y(p_n) \end{pmatrix}, \quad b = -\begin{pmatrix} I_t(p_1) \\ \vdots \\ I_t(p_n) \end{pmatrix}, \\ d^* &= \arg\min_d \|Ad - b\|^2, \qquad A^T A d = A^T b. \end{aligned} \tag{5.4}$$

A is $n \times 2$, b is $n \times 1$, d is 2×1. Derivatives and displacement use consistent intensity/pixel units. This is the forward-additive local linearization using gradients of J at the current warp. Reference-gradient/inverse-compositional variants change the update derivation; do not mix their signs or formulas. A single pixel supplies one scalar equation for two motion components.

Why least squares? Different patch pixels provide different gradient directions. Stack their equations and choose the displacement that minimizes their total squared residual. Differentiating $(Ad - b)^T (Ad - b)$ gives $2A^T(Ad - b)$; set this to zero to obtain the normal equations. In numerical code use QR/SVD or a stable solve, rather than explicitly forming an inverse.

The normal matrix is precisely the unweighted structure tensor:

$$A^T A = \begin{pmatrix} \sum J_x^2 & \sum J_x J_y \\ \sum J_x J_y & \sum J_y^2 \end{pmatrix} = M.$$

That is the connection between **finding** a good feature and **tracking** it. Harris tests whether the same matrix will give a useful motion estimate. A weighted tracker uses $A^T W A$ and $A^T W b$, with $W = \mathrm{diag}(w_p)$.

Compute a Lucas–Kanade displacement

1. Select a textured patch; compute the current temporal differences and consistently chosen spatial derivatives.
2. Stack every row of A and entry of b; keep the minus sign in $b = -I_t$.
3. Check the smallest eigenvalue and conditioning of $A^T A$. Reject ambiguous patches.
4. Solve the least-squares system. Interpret the result as the next-frame location minus the first-frame location under this convention.
5. Warp the next patch, recompute residuals, and verify that the match improved.

The *booklet numerical problem* shows the full matrices, normal equations and displacement $(2, 1)^T$.

5.4.2 Why an edge can fool you

The **aperture problem** means a local edge reveals motion across the edge, but not motion along it. Adding more pixels with parallel gradients repeats the same directional information; it does not create a second independent equation.

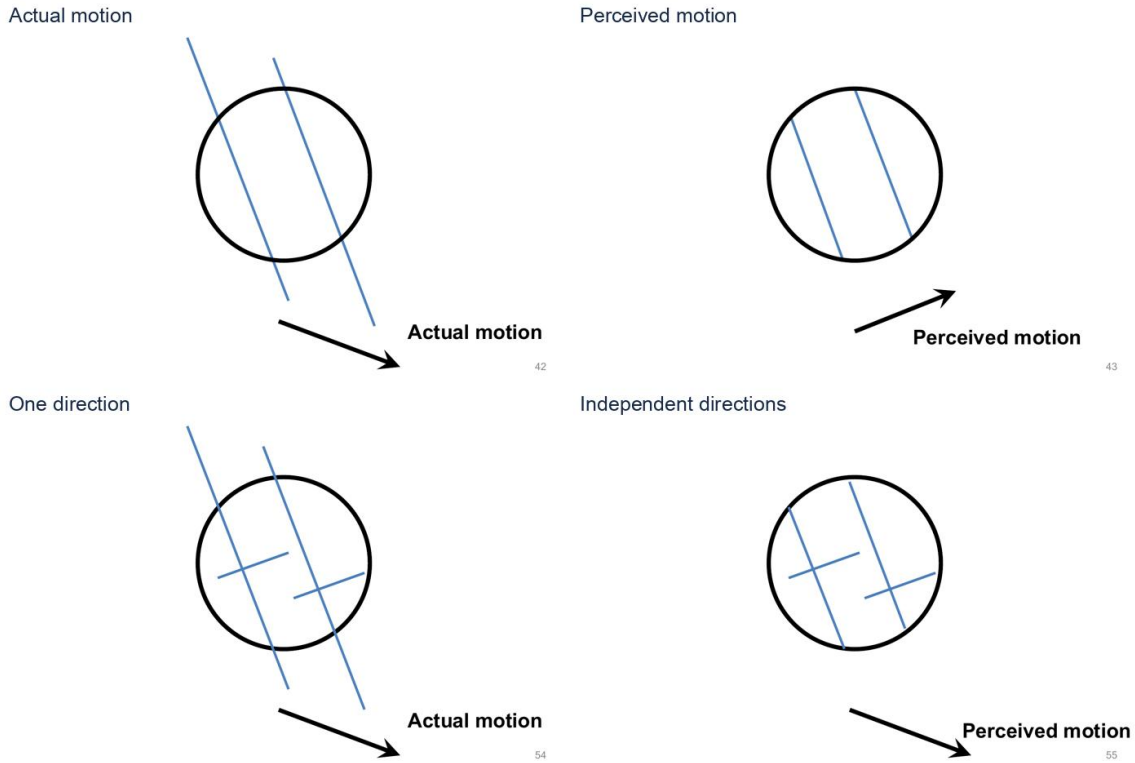


Fig. 5.7: Source PDF pages 42–43 and 54–55. A second independent edge direction resolves the ambiguity. The barber-pole example in pages 44–45 illustrates the same effect perceptually; the moving stripes' local appearance need not reveal their full physical motion.

The original *junction/edge/isolated-dot quiz* makes an important distinction: two nonzero derivative components do not ensure two independent gradient directions. Feature quality depends on the **whole tracking patch**, not whether its center pixel is black or white. The source flat/edge/high-texture tree examples (pages 51–53) apply the same two-eigenvalue test.

5.4.3 Iterate, then use a pyramid when necessary

Formula — iterative tracking and coarse-to-fine transfer

$$
\begin{aligned}
& r_p(d_k) = J(p + d_k) - I(p), \\
& A_k \Delta d \approx -r(d_k), \qquad d_{k+1} = d_k + \Delta d, \\
& d_{\ell-1}^{init} = 2 d_\ell \quad \text{for adjacent pyramid levels differing by factor two.}
\end{aligned}
\tag{5.5}
$$

A_k contains the warped next-image gradients for this forward-additive version. Interpolate fractional coordinates. Displacements are measured in the pixels of their current pyramid level. Recompute gradients if the chosen formulation requires it; stop on a small update, small residual improvement, or an iteration limit. The local translation/appearance model can still fail.

Track beyond one small step

1. Smooth and downsample both frames to create compatible Gaussian pyramids.
2. Estimate motion at a coarse level, where large original motion occupies fewer pixels.
3. Double the displacement when moving to the next finer level; warp and refine there.
4. Reject failed tracks using residuals, conditioning, visibility, and forward/backward or reference-patch consistency.

For example, a displacement of 12 full-resolution pixels becomes 3 pixels after two factor-two reductions. A coarse estimate of $(3, 1)$ initializes $(6, 2)$ at the next level and $(12, 4)$ at full resolution. A pyramid helps but does not guarantee convergence.

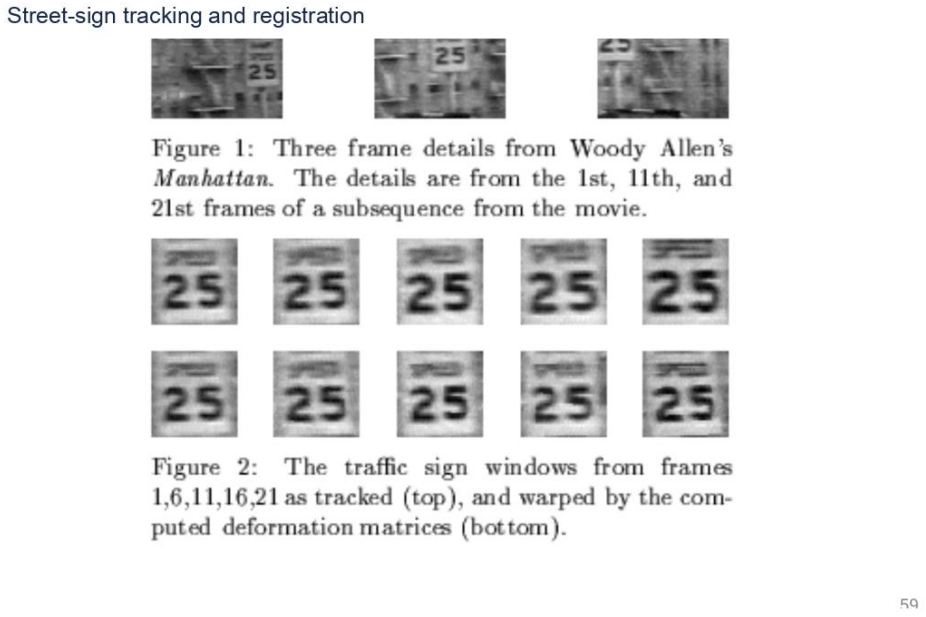


Fig. 5.8: The lecture's "25" sign sequence, PDF page 59, attributed there to frames from Woody Allen's Manhattan and the tracking illustration. Compare the changing observed windows with the registered patches. Their agreement tests a track; incremental motion updates alone do not prevent drift. Source reuse terms unidentified.

Shi–Tomasi tracking (page 58) starts with features whose smallest tensor eigenvalue is large enough, tracks with a local translation model, then checks registration against the **first** observed patch. An affine consistency check can absorb moderate local shape change. Repeatedly replacing the template without checking the original can accumulate **drift**. Add newly visible points and remove occluded, out-of-image, weak or inconsistent ones.

5.4.4 What if the camera changes exposure?

A correct match can violate raw brightness constancy when $J \approx aI + b$. The original *Fall 2025 exposure quiz* derives mean/standard-deviation patch normalization and checks an actual numerical pair.

Formula — exposure-normalized patch

$$\mu = \frac{1}{n}\sum_{p} I(p), \quad \sigma^2 = \frac{1}{n}\sum_{p}[I(p)-\mu]^2, \quad \tilde{I}(p) = \frac{I(p)-\mu}{\sigma+\varepsilon}. \tag{5.6}$$

For positive gain a and nonzero variance, setting $\varepsilon = 0$ exactly cancels an affine brightness change. Positive ε makes that cancellation approximate. It prevents division by zero but does not make a flat patch informative. Clipping, cast shadows, specularities and spatially varying exposure can defeat this model.

Adapt the tracker to exposure changes

1. Reject nearly flat/saturated patches before normalizing.
2. Compute the normalization and track normalized appearance with consistent derivatives.
3. Specify whether patch statistics are frozen within a local solve or differentiated as part of the full residual; update them consistently after warping.
4. Alternatively estimate gain and offset alongside motion, then check residuals and conditioning.

Normalization is also the core of the NCC comparison already covered in Lecture 4; the new task here is using it consistently in the tracking linearization.

5.5 Describe a patch, then judge its matches

A detector says "look here." A descriptor says "this is what it looks like." Two distant views may need descriptor matching even when frame-to-frame tracking is impossible.

Source: PDF pages 61–91. The deck's BRIEF/ORB/BRISK material credits J. Heinly and the original method authors. Distortion and viewpoint changes can alter a descriptor even when its intended invariance helps.

5.5.1 Binary descriptors make comparisons cheap

Formula — binary tests, smoothing and Hamming distance

$$\begin{aligned} \bar{I}_\sigma(p) &= \sum_q G_\sigma(q) I(p-q), \quad \sum_q G_\sigma(q) = 1, \\ \tau_i &= \mathbf{1}[\bar{I}(p_i) < \bar{I}(q_i)], \qquad B = (\tau_1, \dots, \tau_K), \\ D_H(B,C) &= \sum_{i=1}^{K} \mathbf{1}[B_i \neq C_i]. \end{aligned} \tag{5.7}$$

Each ordered sample pair produces one bit; equality gives zero under the stated strict comparison. G_σ is

a Gaussian smoothing kernel, K is the number of tests, and Hamming distance counts differing positions. This is related to Lecture 4's Census bit comparisons, but these methods use selected point pairs rather than necessarily comparing every neighbor to the center. Normalize a truncated Gaussian's weights.

Build and compare a binary description

1. Smooth the patch to reduce sensitivity to pixel noise.
2. Apply the same ordered sampling pattern in both patches, with the method's orientation/scale handling.
3. Store one bit per intensity comparison. A 256-bit description occupies 32 bytes.
4. Count XOR bits to compare descriptors. Reject ambiguous candidates rather than assuming the smallest distance is correct.

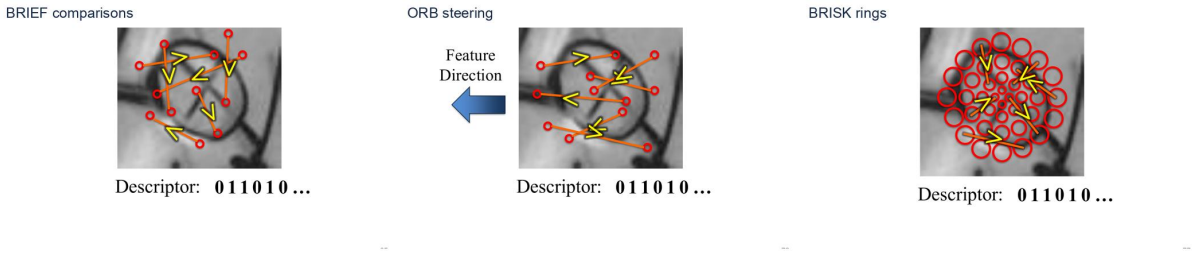


Fig. 5.9: BRIEF, ORB and BRISK on the lecture eye patch: PDF pages 65, 72 and 77. Credits: J. Heinly slides, with Calonder et al., Rublee et al., and Leutenegger et al. named in their source sections. The sampling geometry changes; the output remains a compact bit string.

BRIEF uses fixed intensity-pair tests, with Gaussian-distributed sample locations in the source illustration. It is fast, but an unrotated fixed pattern does not itself compensate for image rotation or scale change. The slides list 128, 256 or 512 bits.

ORB combines FAST corner detection with orientation and rotated BRIEF tests. FAST looks for a sufficiently long contiguous bright or dark arc around a center relative to a threshold; the lecture illustrates a 12-pixel arc on a 16-pixel circle and rapid rejection tests. Other FAST variants use other arc lengths. Harris ranking/NMS selects useful candidates.

Formula — orientation from intensity moments

$$m_{pq} = \sum_{x,y} x^p y^q I(x,y), \quad C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right), \quad \theta = \text{atan2}(m_{01}, m_{10}), \quad p_i^{steered} = R(\theta) p_i. \tag{5.8}$$

Patch coordinates are centered on the keypoint; use a consistent intensity convention and circular support. C is the intensity centroid. A zero/near-zero centroid direction gives unstable orientation; $m_{00} = 0$ makes the centroid undefined. Steering rotates the **sampling pattern**, not the physical object.

Align the description before comparing it

1. Estimate the patch's orientation from the intensity moments.
2. Rotate all test locations consistently with the selected reference orientation.
3. Interpolate/smooth samples and compute bits in the same test order.
4. Use comparisons with good variance and low mutual correlation; redundant bits contribute less independent information.

For example, $m_{10} = 0$, $m_{01} > 0$ yields $\theta = 90°$. Rotating pair endpoint $(1, 0)^T$ gives $(0, 1)^T$. The geometry is the rotation rule from Lecture 1 applied to descriptor sampling.

Scale qualification: the lecture summary lists no scale invariance for ORB. The full ORB pipeline detects features over an image pyramid, providing multiscale handling; a single steered descriptor at a fixed scale does not supply that by itself. See the official ORB description[2] and ORB class documentation[3]. Distinguish orientation compensation, scale selection, and robustness instead of promising perfect invariance.

BRISK estimates location and scale, uses a concentric sampling pattern with Gaussian-smoothed intensities, estimates orientation from long-distance pairs, and encodes short-distance comparisons. The source's "avoid short-distance comparisons" bullet on page 81 is misleading: long pairs estimate orientation; short pairs build the descriptor. The 512-bit descriptor uses 64 bytes. Its detection/description pipeline targets rotation and scale changes, while smoothing improves noise robustness. The authors' implementation[4] and original paper[5] support this distinction. The scale diagram is conceptual; do not infer that BRISK is simply SIFT's DoG detector.

5.5.2 A small distance is not enough

With m and n descriptors of K bits, exhaustive comparison takes $O(mnK)$ bit-level work (or fewer machine-word XOR/popcount operations). The cross-image distance matrix has shape $m \times n$. It need not be square, symmetric, or have a zero diagonal. Those properties apply to distances within the same ordered set, as qualified from source page 85.

Formula — nearest-neighbor distance ratio

$$j_1 = \arg\min_j D(A, B_j), \qquad \rho = \frac{d_1}{d_2}, \qquad \text{accept if } d_1 < \tau d_2, \quad 0 < \tau < 1. \tag{5.9}$$

$d_1 \leq d_2$ are the first and second candidate distances for the **same query**, with $d_2 > 0$. A small ratio indicates a distinctive nearest candidate. If $d_2 = 0$, there are tied exact candidates; reject or resolve the ambiguity explicitly. A threshold is a chosen policy, not a universal constant. A mutual-nearest test checks agreement in both matching directions.

Filter proposed correspondences

1. Compute a metric appropriate to the descriptor; use Hamming for the binary convention above.

[2] https://docs.opencv.org/4.13.0/d1/d89/tutorial_py_orb.html
[3] https://docs.opencv.org/3.4.7/db/d95/classcv_1_1ORB.html
[4] https://github.com/ethz-mrl/brisk
[5] https://www.iro.umontreal.ca/~mignotte/IFT6150/Articles/BriskDetector.pdf

2. Compare an absolute threshold, nearest-neighbor choice, and the ratio test for the intended task.
3. Require mutual matching or spatial/geometric verification when appropriate.
4. Pass surviving pairs to the earlier RANSAC/epipolar/homography procedures; descriptor similarity is appearance evidence, not a geometry proof.

Worked example 5.2 — a distinctive binary match

Original practice question

Original companion to: pages 68 and 89–90. Query $A = 10110010$ has candidates $B = 10110011$ and $C = 01001100$. Apply a ratio threshold of 0.8 and state the next verification step.

Step 1 — Count differing positions

Why: Binary descriptors are compared by Hamming distance.
$A \oplus B = 00000001$ has one set bit, so $D_H(A, B) = 1$. $A \oplus C = 11111110$ has seven, so $D_H(A, C) = 7$.

Step 2 — Apply an explicit acceptance rule

Why: The second-nearest candidate reveals how distinctive the best one is.
With $\tau = 0.8$, $\rho = 1/7 \approx 0.143 < 0.8$, so accept B provisionally. If the two distances were 7 and 8, the ratio 0.875 would fail even though one candidate is closest.

Step 3 — Check geometry

Why: A unique appearance match can still pair different physical surfaces.
Verify the candidate with the relevant geometric model before using it for reconstruction.

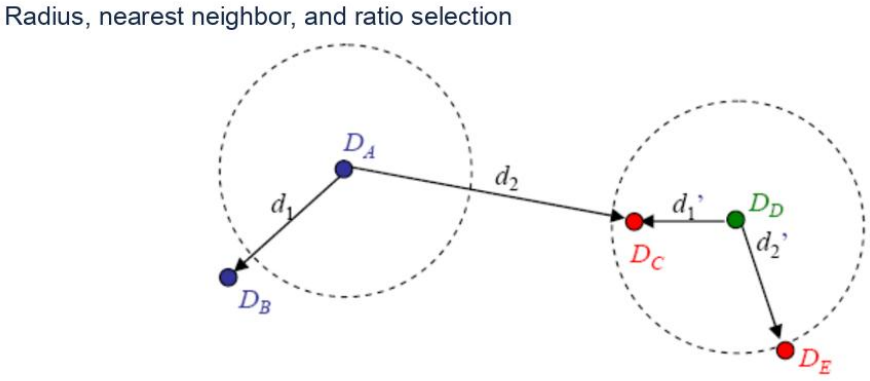


Figure 4.24 Fixed threshold, nearest neighbor, and nearest neighbor distance ratio matching. At a fixed distance threshold (dashed circles), descriptor D_A fails to match D_B and D_D incorrectly matches D_C and D_E. If we pick the nearest neighbor, D_A correctly matches D_B but D_D incorrectly matches D_C. Using nearest neighbor distance ratio (NNDR) matching, the small NNDR d_1/d_2 correctly matches D_A with D_B, and the large NNDR d_1'/d_2' correctly rejects matches for D_D.

Fig. 5.10: Source PDF page 90. A distance threshold uses a fixed radius; nearest-neighbor matching always selects the closest candidate; the ratio test asks whether that choice is sufficiently better than the next one.

5.5.3 Measure the quality of the decisions

Formula — matching evaluation

$$\begin{aligned} \text{precision} &= \frac{TP}{TP+FP}, \quad \text{recall} = TPR = \frac{TP}{TP+FN}, \\ FPR &= \frac{FP}{FP+TN}, \qquad \text{accuracy} = \frac{TP+TN}{TP+FP+FN+TN}. \end{aligned} \tag{5.10}$$

TP/FP/FN/TN are true positives, false positives, false negatives and true negatives against known match labels. Require nonzero denominators for each reported ratio. The ROC curve plots TPR against FPR as the threshold changes; its ideal operating region has high TPR and low FPR. Accuracy can look high merely because nonmatches dominate.

Evaluate a matching rule

1. Label the evaluation pairs and count all four outcomes.
2. State the threshold and compute the relevant ratios.
3. Vary the threshold to inspect the false-match/missed-match tradeoff.
4. Keep the evaluation dataset and candidate-pair definition fixed when comparing methods.

Worked example 5.3 — the slide's 100-pair table

Source slide example

Source: PDF page 87 gives $TP = 18$, $FP = 4$, $FN = 2$, $TN = 76$. Compute precision, recall, false-positive rate and accuracy.

Step 1 — Check the counts

Why: The four cells must account for the same evaluation population.
There are $18 + 4 + 2 + 76 = 100$ pairs, $18 + 2 = 20$ real matches and $4 + 76 = 80$ real nonmatches.

Step 2 — Compute precision and recall

Why: Accepted-match reliability and recovered-match coverage answer different questions.

$$\text{precision} = \frac{18}{18+4} = \frac{18}{22} \approx 0.8182, \qquad \text{recall} = \frac{18}{18+2} = 0.9.$$

Step 3 — Compute false-positive rate and accuracy

Why: These use different denominators from precision.

$$FPR = \frac{4}{4+76} = 0.05, \qquad \text{accuracy} = \frac{18+76}{100} = 0.94.$$

This reproduces the source values. A false-positive rate of 5% does not mean 95% of accepted matches are correct; their precision is about 82%.

For large descriptor databases, multidimensional trees or hashing can reduce search work. Their speed and approximation quality depend on dimensionality, distance metric and data distribution; exact exhaustive search remains the comparison baseline. These are the indexing alternatives on PDF page 91, not a guarantee of logarithmic query time.

5.6 Original quiz questions and worked solutions

Read the problem in black. Follow the numbered red steps. Work through the blue mathematics. Every step explains why it is needed. Matrix calculations show the inputs and intermediate results; written-reasoning questions explain the decision instead of inventing numerical data.

Source: the supplied lecture-organized quiz booklet. Full problems follow its wording and givens, with typography normalized; the historical PDF filenames appear below. Source-key corrections and added numerical companions are marked.

Retests have their own questions and full solutions. Problems labeled **Original practice** are the booklet's seven supplemental examples, separate from the 18 historical quizzes. Colored steps remain numbered and readable in grayscale.

5.6.1 Jump to a quiz

- *Fall 2025 Quiz 4 — KLT with auto-exposure*
- *Spring 2019 Quiz 6 — Feature trackability*
- *A2. Harris response from gradients (L5, slides 11–30)*
- *A3. Lucas–Kanade least-squares displacement (L5, slides 39–50)*

5.6.2 Fall 2025 Quiz 4 — KLT with auto-exposure

Historical source: `3dcvMid/quizzesf25sol/cs532f25_quiz_4_solution.pdf`.

Question 1

Original problem · quiz booklet

You track features over a long monocular video with automatic exposure that changes brightness from frame to frame. Propose modifications to the KLT brightness-constancy formulation, explaining why they improve robustness without invalidating the local linearization.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — State brightness constancy

Why: Ordinary KLT explains image changes using motion alone.

State original KLT assumption: $I_2(x+\mathbf{d}) \approx I_1(x)$, with $\mathbf{d} = (u,v)^T$. Linearizing gives $I_x u + I_y v + I_t \approx 0$.

Step 2 — Model the exposure change

Why: Gain and offset can change intensities even at the correct match.

Model auto-exposure as a frame-dependent affine brightness change: $I_2(x+\mathbf{d}) \approx aI_1(x) + b$, where $a > 0$ rescales contrast and b changes brightness. This violates raw brightness constancy even if motion is correct.

Step 3 — Normalize each patch

Why: Subtracting the mean removes offset and standardizing removes positive gain.

For each local patch W, calculate its mean $\mu = (1/|W|)\sum_W I$ and standard deviation $\sigma = \sqrt{(1/|W|)\sum_W (I-\mu)^2}$. Define normalized intensity $\boxed{\tilde{I}(x) = (I(x)-\mu)/(\sigma+\varepsilon)}$, with a small ε preventing division by zero.

Step 4 — Prove the cancellation

Why: The affine exposure terms cancel under nonzero patch variance.

Show why the correction works when $\varepsilon \approx 0$: if $I_2 = aI_1 + b$ and $a > 0$, then $\mu_2 = a\mu_1 + b$ and $\sigma_2 = a\sigma_1$. Therefore

$$\tilde{I}_2 = \frac{aI_1 + b - (a\mu_1 + b)}{a\sigma_1} = \frac{I_1 - \mu_1}{\sigma_1} = \tilde{I}_1.$$

Step 5 — Linearize the normalized images

Why: Tracking still needs derivatives of the residual that is actually optimized.

Now apply the usual local Taylor expansion to *normalized* patches: $\tilde{I}_2(x+\mathbf{d}) \approx \tilde{I}_2(x) + \nabla\tilde{I}_2(x)^T\mathbf{d}$. The tracking equations retain the Lucas–Kanade/KLT least-squares form but use normalized derivatives.

Step 6 — Consider joint exposure estimation

Why: Gain and offset can instead be treated as extra unknowns.

Optional alternative: estimate exposure nuisance parameters jointly by minimizing $\sum_W [I_2(x+\mathbf{d}) - aI_1(x) - b]^2$; linearize the residual with respect to small increments in u, v, a, b.

Step 7 — Verify a numerical patch pair

Why: Matching normalized columns demonstrate the correction and its assumptions.

Original numerical companion: patches $I_1 = (10,20,30)^T$ and $I_2 = (30,50,70)^T = 2I_1 + 10$ have means 20 and 50. Their centered columns are $(-10,0,10)^T$ and $(-20,0,20)^T$; population standard deviations are

$\sqrt{200/3}$ and $2\sqrt{200/3}$. With $\varepsilon = 0$ they normalize to the same column:

$$\tilde{I}_1 = \tilde{I}_2 = \begin{pmatrix} -\sqrt{3/2} \\ 0 \\ \sqrt{3/2} \end{pmatrix}.$$

With a small positive ε, invariance is approximate. Freeze the selected patch statistics while forming each local linear system, and recompute after updating the warp; if differentiating the full normalized residual instead, include derivatives of those statistics. Reject flat or saturated patches rather than trusting the denominator safeguard to create texture.

Source check / assumptions: The uploaded key specifically proposes patch mean/std normalization. This method assumes patches have nonzero texture and a roughly affine photometric change. Saturation, shadows and spatially nonuniform lighting can still break the model.

5.6.3 Spring 2019 Quiz 6 — Feature trackability

Historical source: `3dcvMid/olderquizzes/cs532s19_quiz6-2.pdf`.

Problem 1(a–d)

Original problem · quiz booklet

A binary image contains three black lines meeting at one vertex v: horizontal ℓ_h, vertical ℓ_v, and a 45° line ℓ_s. There is also one isolated black pixel $p = (x, y, 1)$ surrounded by white pixels. (a) Express intersection vertex coordinates in terms of the horizontal/vertical lines. (b) Is the common vertex good to track? (c) What about an arbitrary point along the slanted edge with both I_x and I_y large? (d) Is the isolated black pixel better than a neighboring white pixel?

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Find the junction geometrically

Why: Intersecting the two known lines gives its position.

Represent a homogeneous line as $\ell = (a, b, c)^T$, with line equation $ax + by + cw = 0$. Two noncoincident lines intersect at their cross product, giving

$$\boxed{v \sim \ell_h \times \ell_v}.$$

Verify that the diagonal line also passes through the vertex: $\ell_s^T v = 0$.

Step 2 — Explain why the junction is trackable

Why: Multiple gradient directions constrain both motion components.

Use the Harris/KLT structure tensor $M = \sum_W w \nabla I \nabla I^T$. A corner-like intersection provides intensity gradients in multiple directions, making *both* eigenvalues sufficiently large. So yes, the vertex is generally good to track if patch size/contrast resolve the junction.

Step 3 — Test the diagonal-edge claim

Why: Two nonzero gradient components do not imply two independent directions.

For a straight diagonal edge, gradients may have *both coordinate components* $I_x \neq 0$, $I_y \neq 0$, but every local gradient is approximately parallel to the same edge normal g. Then $M \approx (\sum w\alpha_i^2)gg^T$, a rank-1 matrix with one near-zero eigenvalue. Thus no, not reliably trackable along the edge: the aperture problem remains.

Step 4 — Compare the dot and neighboring pixel

Why: Tracking quality belongs to a window, not to a pixel's color.

For an isolated dot, local intensity changes in both x and y directions; the dot as a *patch* is distinctive. However a nearby white pixel whose tracking *window includes the same dot* sees essentially the same local gradients and is not necessarily worse. So feature quality depends on the patch, not the center pixel color.

Step 5 — Check what the window contains

Why: A flat neighboring window has no useful motion information.

If the tracking window around the neighboring white pixel *misses* the dot completely, then it is flat and poor. The correct answer requires specifying the window and its gradients.

Step 6 — Work out the line and rank calculations

Why: A full matrix shows precisely why the diagonal edge remains ambiguous.

Original algebraic companion to parts (a–c): a horizontal line at $v = v_0$ and vertical line at $u = u_0$ have

$$\ell_h = \begin{pmatrix} 0 \\ 1 \\ -v_0 \end{pmatrix}, \quad \ell_v = \begin{pmatrix} 1 \\ 0 \\ -u_0 \end{pmatrix}, \quad \ell_h \times \ell_v = \begin{pmatrix} -u_0 \\ -v_0 \\ -1 \end{pmatrix} \sim \begin{pmatrix} u_0 \\ v_0 \\ 1 \end{pmatrix}.$$

For N identical diagonal-edge gradients $g = (1, 1)^T$, the tracking matrix is

$$M = N \begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \end{pmatrix} = N \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

Its characteristic equation is $(N - \lambda)^2 - N^2 = \lambda(\lambda - 2N) = 0$. One eigenvalue is zero even though both gradient components are nonzero. Motion along the edge cannot be recovered from that patch alone.

5.6.4 A2. Harris response from gradients (L5, slides 11–30)

Original practice from the supplied booklet; not a historical quiz question.

Illustrative problem

Original problem · quiz booklet

A small patch contains two gradient vectors $(2,0)^T$ and $(0,3)^T$, equally weighted. Compute its second-moment matrix, eigenvalues, and Harris score for $k = 0.04$. Classify it.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Set up the structure tensor

Why: Gradient outer products measure changes in every direction.

Construct the unweighted second-moment matrix as the sum of gradient outer products: $M = \sum_i g_i g_i^T$.

Step 2 — Multiply and add the full matrices

Why: The off-diagonal and diagonal entries come from explicit products.

Multiply each 2×1 gradient by its 1×2 transpose, then add:

$$g_1 g_1^T = \begin{pmatrix} 2 \\ 0 \end{pmatrix} \begin{pmatrix} 2 & 0 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 0 \end{pmatrix},$$

$$g_2 g_2^T = \begin{pmatrix} 0 \\ 3 \end{pmatrix} \begin{pmatrix} 0 & 3 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 9 \end{pmatrix},$$

$$M = \begin{pmatrix} 4 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 9 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix}.$$

Here equal weights mean both weights are one; averaging instead would scale the eigenvalues and response.

Step 3 — Read the eigenvalues

Why: A diagonal matrix makes the independent gradient strengths visible.

Because M is diagonal, its eigenvalues are $\lambda_1 = 4$ and $\lambda_2 = 9$; intensity changes in two independent directions.

Step 4 — Compute determinant and trace

Why: Harris combines these two scalar summaries.

Compute determinant and trace: $\det M = 4(9) = 36$, $\operatorname{tr} M = 4 + 9 = 13$.

Step 5 — Evaluate the Harris response

Why: Substitute the supplied k after computing the summaries.

Harris response: $R = \det M - k(\operatorname{tr} M)^2 = 36 - 0.04(13)^2 = 36 - 6.76 = \boxed{29.24}$.

Step 6 — Classify and select the feature

Why: A corner-like score still needs a threshold and nonmaximum suppression.

This positive score is *corner-like*; actual detector selection also requires an image-dependent threshold and spatial nonmaximum suppression. If instead $M = \operatorname{diag}(20, 0)$, the response would be $-0.04(20)^2 = -16$ (edge-like and singular).

5.6.5 A3. Lucas–Kanade least-squares displacement (L5, slides 39–50)

Original practice from the supplied booklet; not a historical quiz question.

Illustrative problem

Original problem · quiz booklet

Two pixels in one small tracked patch have spatial gradients $(I_x, I_y) = (2, 0)$ and $(0, 3)$, with temporal derivatives $I_t = -4$ and -3, respectively. Find the shared displacement (u, v).

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Write brightness constancy's linearization

Why: Small motion turns each pixel into one linear equation.

Start with linearized brightness constancy at each pixel: $I_x u + I_y v + I_t = 0$.

Step 2 — Substitute both pixels

Why: Different gradient directions constrain different displacement components.

Substitute pixel 1: $2u + 0v - 4 = 0 \Rightarrow 2u = 4$. Pixel 2: $0u + 3v - 3 = 0 \Rightarrow 3v = 3$.

Step 3 — Stack the linear system

Why: The patch shares one displacement vector.

Write matrix form: $Ad = b$, with

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

, $d = (u, v)^T$, $b = (4, 3)^T$.

Step 4 — Multiply out the normal equations

Why: This exposes the structure tensor and right-hand side completely.

Multiply the full normal-equation matrices:

$$A^T A = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix},$$

$$A^T b = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 4 \\ 3 \end{pmatrix} = \begin{pmatrix} 8 \\ 9 \end{pmatrix}.$$

Thus the two least-squares equations are

$$\begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} 8 \\ 9 \end{pmatrix}.$$

Step 5 — Solve and verify the displacement

Why: A zero residual checks the proposed motion in the stated linear model.

For this diagonal example, show the inverse explicitly:

$$\begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} 1/4 & 0 \\ 0 & 1/9 \end{pmatrix} \begin{pmatrix} 8 \\ 9 \end{pmatrix} = \begin{pmatrix} 8/4 \\ 9/9 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \text{ pixels.}$$

Check $A(2,1)^T - b = (0,0)^T$. In implementations, solve the system rather than explicitly inverting it; use warping/pyramids when the local small-motion approximation is inadequate.

Step 6 — Connect solvability to corner quality

Why: An edge makes the same normal matrix nearly singular.

Notice $A^T A = M$, the same second-moment matrix as in Harris. When both eigenvalues are positive and not too small, the optical-flow estimate is better determined; on an edge one eigenvalue tends toward zero and the aperture problem returns.

5.7 Formula and source checklist

Use this as a map, then work the *original questions and full solutions*. This checklist adds no second version of the lecture.

5.7.1 Find a formula and its procedure

Formula	Where to learn and apply it
(5.1) — patch change and its local approximation	*Corners*; the adjacent application box gives the steps and assumptions
(5.2) — tensor eigenvalues and directional change	*Corners*; the adjacent application box gives the steps and assumptions
(5.3) — Harris and Shi–Tomasi responses	*Corners*; the adjacent application box gives the steps and assumptions
(5.7) — binary tests, smoothing and Hamming distance	*Descriptors*; the adjacent application box gives the steps and assumptions
(5.8) — orientation from intensity moments	*Descriptors*; the adjacent application box gives the steps and assumptions
(5.9) — nearest-neighbor distance ratio	*Descriptors*; the adjacent application box gives the steps and assumptions
(5.10) — matching evaluation	*Descriptors*; the adjacent application box gives the steps and assumptions
(5.4) — brightness constancy and patch least squares	*Tracking*; the adjacent application box gives the steps and assumptions
(5.5) — iterative tracking and coarse-to-fine transfer	*Tracking*; the adjacent application box gives the steps and assumptions
(5.6) — exposure-normalized patch	*Tracking*; the adjacent application box gives the steps and assumptions

5.7.2 Trace the lecture slides

PDF pages	Teaching location	Covered material
3–10	*Features*	Panorama, repeatable/salient/local features, Mars Rover matches
11–33	*Corners*	Shifted SSD, Taylor expansion, weighted tensor, eigenvalues/ellipses, Harris and NMS
34–60	*Tracking*	KLT assumptions, aperture problem, least squares, iterative/pyramid tracking, drift and exposure
61–83	*Descriptors*	BRIEF, FAST/ORB, BRISK; orientation, scale and smoothing qualifications
84–91	*Descriptors*	Distance matrices, thresholds, nearest neighbors/ratio, confusion table/ROC, search

Pages 1–2 supply title/outline metadata. Repeated pictures and derivation builds are consolidated into the listed explanations. The final TLDR tests this lecture's additions; the original workbook remains the source for complete historical questions.

5.7.3 Quiz route

- *Trackable patches and the slanted-edge trap*.
- *Exposure changes and normalization*.
- *Harris numerical tensor/eigenvalue calculation*.
- *Lucas–Kanade full matrix solve*.

Source qualifications: ORB's full pyramid pipeline handles multiple scales; fixed-scale steering alone does not. BRISK uses long pairs for orientation and short pairs for descriptor bits. A cross-image distance matrix need not be symmetric. See the cited primary references in *descriptors*.

Detailed page-level dispositions and editable formula/figure inventories are in the repository's `authoring/coverage.csv`, `lecture05-audit.md`, and `lecture05-formulas.csv`.

5.8 TLDR

Find it → track it → describe it → verify it. These are different jobs, even when one pipeline does all four.

- A good feature is repeatable, distinctive, compact, and local. Flat patches constrain neither motion direction; edges constrain one; useful corners constrain both.
- The structure tensor collects gradient outer products. Its smallest eigenvalue exposes the weak direction. Two large gradient components at one edge do not imply two independent constraints.
- Harris scores candidates; thresholding removes weak responses; NMS removes nearby duplicates. Shi–Tomasi tests the smallest eigenvalue directly.
- Lucas–Kanade linearizes brightness constancy, stacks patch equations, solves for displacement, warps, and repeats. Pyramids reduce the apparent motion; consistency checks detect drift and failures.

- Exposure normalization can remove positive affine brightness changes, but a flat or clipped patch remains problematic.
- Binary descriptions store intensity comparisons. Hamming distance counts different bits. ORB steers comparisons; BRISK uses long pairs for orientation and short pairs for bits.
- A ratio test checks distinctiveness; geometry checks whether a match is physically plausible. Precision and recall use different denominators.

5.8.1 Short quiz

These are **original practice questions**, separate from the historical/booklet *quiz workbook*. Try each before opening its answer. All answers are printed in the PDF.

Question 1

Original practice question

A patch has two gradient samples $(2, 2)^T$, each with weight one. Is it a stable two-directional feature?

Worked solution

Step 1 — Form the outer products

Why: Different pixels help only if they supply independent directions.

$$M = 2\begin{pmatrix} 4 & 4 \\ 4 & 4 \end{pmatrix} = \begin{pmatrix} 8 & 8 \\ 8 & 8 \end{pmatrix}.$$

Step 2 — Test its weak direction

Why: A zero eigenvalue means an unconstrained shift.

$$M(1, -1)^T = (0, 0)^T, \qquad \lambda = 16, 0.$$

It is edge-like, not a stable two-directional feature.

Question 2

Original practice question

For $M = \mathrm{diag}(4, 9)$ and $k = 0.04$, compute the Harris response and Shi–Tomasi score.

Worked solution

Step 1 — Compute determinant and trace

Why: Harris uses both directional responses.

$$\det M = 36, \quad \operatorname{tr} M = 13, \quad R_H = 36 - 0.04(169) = 29.24.$$

Step 2 — Read the weaker eigenvalue

Why: Shi–Tomasi measures the least-constrained direction.

$$R_{ST} = \min(4, 9) = 4.$$

Compare scores against thresholds defined for these weights and intensity units.

Question 3

Original practice question

Solve $Ad = b$ for $A = \left(\begin{smallmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ and $b = (2, 1, 3)^T$.

Worked solution

Step 1 — Write the normal equations

Why: All patch rows must contribute to the displacement estimate.

$$A^T A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \quad A^T b = \begin{pmatrix} 5 \\ 4 \end{pmatrix}.$$

Step 2 — Solve and substitute

Why: Substitution checks signs and arithmetic.

$$2u + v = 5, \quad u + 2v = 4 \Rightarrow d = (2, 1)^T,$$

$$Ad = (2, 1, 3)^T = b.$$

Question 4

Original practice question

A tracker estimates $(3, 1)$ pixels at a pyramid level two reductions below full resolution. What full-resolution initialization follows?

Worked solution

Step 1 — Transfer one level

Why: Each finer image has twice as many pixels per displacement.

$$d_{next} = 2(3,1)^T = (6,2)^T.$$

Step 2 — Transfer the next level

Why: Repeat the same conversion; refine after each transfer in an actual tracker.

$$d_{full} = 2(6,2)^T = (12,4)^T.$$

Question 5

Original practice question

The two nearest descriptor distances are 7 and 8. Does the match pass a strict ratio threshold of 0.8?

Worked solution

Step 1 — Compute the ratio

Why: The second-best candidate measures ambiguity.

$$\rho = 7/8 = 0.875.$$

Step 2 — Apply the stated rule

Why: Being closest does not mean being distinctive.

$$0.875 \not< 0.8.$$

Reject this proposed match under the given policy.

Question 6

Original practice question

Among accepted matches, 18 are correct and 4 incorrect. Two true matches were missed. What are precision and recall?

Worked solution

Step 1 — Compute accepted-match reliability

Why: Precision counts accepted decisions in its denominator.

$$P = 18/(18 + 4) = 9/11 \approx 81.82\%.$$

Step 2 — Compute true-match coverage

Why: Recall counts all actual matches in its denominator.

$$R = 18/(18 + 2) = 90\%.$$

CHAPTER

SIX

LECTURE 6: RECONSTRUCT A SCENE—AND IMPROVE IT

The goal: turn tracks across photographs into camera poses and 3D points. Then adjust both until their projections agree with the measurements.

Read *what reconstruction cannot determine*, *affine factorization*, *triangulation*, *the multi-view pipeline*, and *bundle adjustment*. Finish with the *complete quiz workbook*, *formula and source map*, and *TLDR + short quiz*.

Source: all 72 PDF pages of `wk6_cs532s24_SFM1.pdf`, Enrique Dunn. The deck credits Szeliski, Seitz, Snavely, Lazebnik, Hebert, and Choudhary. Page references below are one-based PDF pages. Repeated titles and repeated optimization slides are consolidated.

We use Lecture 4's camera and epipolar geometry rather than teach it again. This chapter adds reconstruction ambiguity, many-view estimation, factorization, and optimization. Exactly one interactive appears at the point where a numerical update becomes a visible 3D improvement.

6.1 What can the photographs actually determine?

A good-looking reconstruction can still have the wrong scale. Photographs measure directions to points. Without a known length, a miniature scene close to the cameras can look identical to a large scene farther away.

Source: PDF pages 4–19. Worked exam companions: *reconstruction upgrades, Fall 2025 Quiz 5* and *its second scenario*.

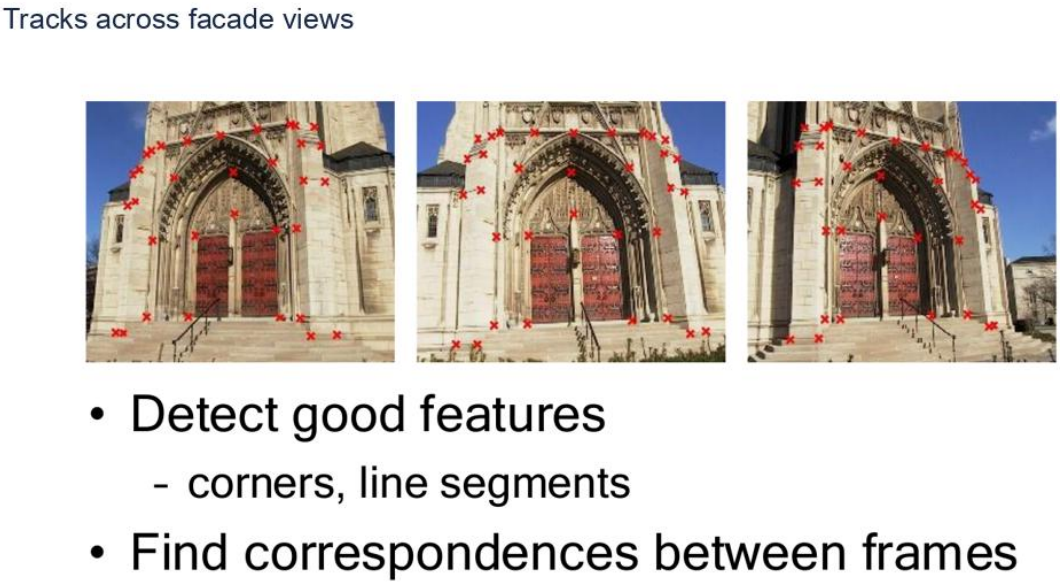


Fig. 6.1: The source facade tracks, PDF page 5. Each track should represent one physical 3D point observed in several images. Attribution follows the lecture deck; original image creator is not individually identified.

6.1.1 Count measurements, then count unknowns

Each observed image point supplies two coordinates. A calibrated camera has six pose parameters; each 3D point has three. But an arbitrary common translation, rotation, and scale changes no image. Those seven free choices are the **similarity gauge**.

Formula — necessary measurement counts

$$\begin{aligned} \text{Calibrated:} \quad & 2mn \geq 6m + 3n - 7, \\ \text{Projective:} \quad & 2mn \geq 11m + 3n - 15, \\ \text{Affine:} \quad & 2mn \geq 8m + 3n - 12. \end{aligned} \tag{6.1}$$

m is the number of views and n the number of points, assuming every point is visible in every view and generic geometry. A projective camera has 11 parameters up to scale; an affine camera has 8. The gauge freedoms have dimensions 15 and 12 respectively. These inequalities are **necessary counts, not sufficient guarantees of a unique or stable solution**. Missing observations reduce the left side to twice the number actually observed.

Use a count without overclaiming

1. Identify the camera model and count actual observations.
2. Subtract the appropriate common gauge from the unknowns.
3. Check the inequality. Failure means insufficient data; success means only that the count permits a solution.

4. Check configuration, ambiguity, outliers, and numerical conditioning separately.

For two calibrated views, $4n \geq 12 + 3n - 7$ gives $n \geq 5$. For two projective views, $4n \geq 22 + 3n - 15$ gives $n \geq 7$. These explain the minimal counts behind five-point and seven-point methods; they do not remove their multiple candidate solutions.

6.1.2 Why a different 3D shape can produce the same images

Formula — projective reconstruction ambiguity

$$x_{ij} \sim P_i X_j = (P_i H^{-1})(H X_j), \qquad \det H \neq 0. \tag{6.2}$$

X_j is a homogeneous 4×1 point, P_i a 3×4 camera, and H one common invertible 4×4 projective transformation. Transforming all points and compensating every camera preserves their image projections. Arbitrary H need not preserve lengths, angles, parallelism, or calibration.

Check a claimed reconstruction upgrade

1. State what was recovered: projective, affine, or metric geometry.
2. State the extra information: parallel directions, orthogonality, calibration, or known lengths.
3. Use it to constrain the allowable common transformation.
4. Report the ambiguity that remains. A metric reconstruction usually still has arbitrary common rotation, translation, and scale.

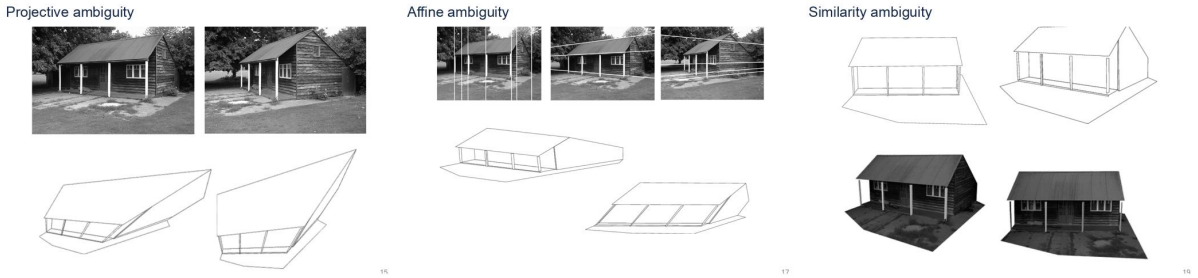


Fig. 6.2: Selected house reconstructions from PDF pages 15, 17, and 19. Different reconstruction classes preserve different geometric properties. These are source examples, not three stages computed by this book.

A concrete scale example: replace every Euclidean point X and camera center C_i by $2X$ and $2C_i$, leaving rotations unchanged. Camera coordinates become $R_i(2X - 2C_i) = 2R_i(X - C_i)$. Dividing by depth cancels the factor two. Merely multiplying a homogeneous point vector by two describes the *same* point and is not this physical change of scale.

6.1.3 What the upgrades add

A projective reconstruction has not located the **plane at infinity** correctly. Establishing that plane gives an affine reconstruction, where parallel lines remain parallel. Recovering the metric additionally constrains angles and relative lengths, commonly through calibration or sufficient orthogonality constraints.

A known length fixes global scale after a metric reconstruction. One right angle alone generally does not determine an entire metric upgrade. Use the independent constraints and assumptions in the specific question; the two Fall 2025 scenarios are worked separately in the workbook.

6.2 Factorization: split one matrix into cameras and shape

Put the tracks in a table. A low-rank split separates motion from shape. This works particularly cleanly for an affine camera, which has no division by depth.

Source: PDF pages 20–38. Full numerical booklet problem: *rank-three factorization*.

6.2.1 Center first

Formula — affine measurements and centering

$$\begin{aligned} x_{ij} &= A_i X_j + b_i, \\ \bar{x}_i &= \frac{1}{n}\sum_j x_{ij} = A_i \bar{X} + b_i, \\ x_{ij} - \bar{x}_i &= A_i (X_j - \bar{X}). \end{aligned} \tag{6.3}$$

x_{ij}, b_i are 2×1, A_i is 2×3, and X_j is Euclidean 3×1. Here the same complete set of n points is visible in every view. Orthographic and weak-perspective cameras are special affine models. Centering removes image translation and chooses the shape centroid as the origin.

Build the centered measurement matrix

1. For each image, compute the mean horizontal and vertical coordinates of the same tracked points.
2. Subtract these means from that image's measurements.
3. Stack two rows per image into $D \in \mathbb{R}^{2m \times n}$.
4. Do not enter zero for a missing observation; zero is a valid centered coordinate. Basic SVD here assumes complete tracks.

Formula — rank-three affine factorization

$$
\begin{aligned}
D &= MS, \\
M &= \begin{pmatrix} A_1 \\ \vdots \\ A_m \end{pmatrix} \in \mathbb{R}^{2m\times 3}, \\
S &= \begin{pmatrix} X_1 - \bar{X} & \cdots & X_n - \bar{X} \end{pmatrix} \in \mathbb{R}^{3\times n}, \\
&\operatorname{rank} D \le 3, \\
D &= U\Sigma V^T \approx U_3\Sigma_3 V_3^T, \\
\widetilde{M} &= U_3\Sigma_3^{1/2}, \qquad \widetilde{S} = \Sigma_3^{1/2} V_3^T.
\end{aligned}
\tag{6.4}
$$

With noise, retaining the three largest singular values gives the best rank-at-most-three approximation in Frobenius norm. A generic nondegenerate scene can have rank three; a planar/degenerate configuration can have lower rank. Do not force a nonexistent third dimension from noise.

Recover affine motion and shape

1. Center the complete tracks and compute their SVD.
2. Inspect singular values: do three dimensions plausibly explain the data?
3. Form the balanced factors above. Their product reproduces the truncated measurements.
4. Restore each image mean when predicting original coordinates.
5. Treat the factors as affine estimates until additional metric constraints are enforced.

For any invertible 3×3 matrix C, $(\widetilde{M}C)(C^{-1}\widetilde{S})$ gives the same D. This is the remaining affine ambiguity after centering.

6.2.2 Turn an orthographic factorization into metric shape

For calibrated, unit-scale orthographic rows a_i, b_i, require unit lengths and perpendicularity. If weak perspective has a different unknown scale in each image, the unit-length constraints must be changed; do not apply this model blindly.

Formula — orthographic metric upgrade

$$
\begin{aligned}
M = \widetilde{M}C, \quad S = C^{-1}\widetilde{S}, \quad L = CC^T, \\
\tilde{a}_i L \tilde{a}_i^T = 1, \quad \tilde{b}_i L \tilde{b}_i^T = 1, \quad \tilde{a}_i L \tilde{b}_i^T = 0.
\end{aligned}
\tag{6.5}
$$

L is symmetric with six unknown entries. Each row pair gives three equations linear in those entries. A valid invertible upgrade requires a positive definite L and sufficient independent constraints. Factor it as CC^T, for example with Cholesky. Noisy estimates may need constrained fitting; an indefinite result is not a usable Euclidean metric.

Solve the metric upgrade

1. Write $L = \begin{pmatrix} l_{11} & l_{12} & l_{13} \\ l_{12} & l_{22} & l_{23} \\ l_{13} & l_{23} & l_{33} \end{pmatrix}$.
2. Expand the three constraints per view; solve for its six distinct entries.
3. Check definiteness and constraint residuals.
4. Recover C and update **both** factors. Verify $MS = \widetilde{M}\widetilde{S}$.

Worked example 6.1 — remove unequal axis scaling

Original practice question

Three orthographic views give the estimated motion rows below. Find a diagonal metric upgrade and display the corrected motion matrix.

$$\widetilde{M} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1/2 \\ 2 & 0 & 0 \\ 0 & 0 & 1/2 \end{pmatrix}.$$

Step 1 — Expand all independent constraints

Why: Unit rows set axis scales; perpendicular rows set the cross terms.

$$4l_{11} = 1, \quad l_{22} = 1, \quad \tfrac{1}{4}l_{33} = 1,$$

$$2l_{12} = 0, \quad \tfrac{1}{2}l_{23} = 0, \quad l_{13} = 0.$$

The first camera supplies the xy orthogonality constraint, the second yz, and the third xz. Thus

$$L = \begin{pmatrix} 1/4 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 4 \end{pmatrix}.$$

Step 2 — Factor the metric

Why: Multiplying motion by this factor must make its rows unit length.

$$C = \begin{pmatrix} 1/2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \qquad CC^T = L.$$

Step 3 — Correct motion and compensate shape

Why: Correcting only motion would change the predicted measurements.

$$M = \widetilde{M}C = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix},$$

$$S = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1/2 \end{pmatrix} \widetilde{S}, \qquad MS = \widetilde{M}CC^{-1}\widetilde{S} = D.$$

Each camera's two rows are perpendicular unit vectors. A common rotation/reflection of the recovered coordinate system remains possible.

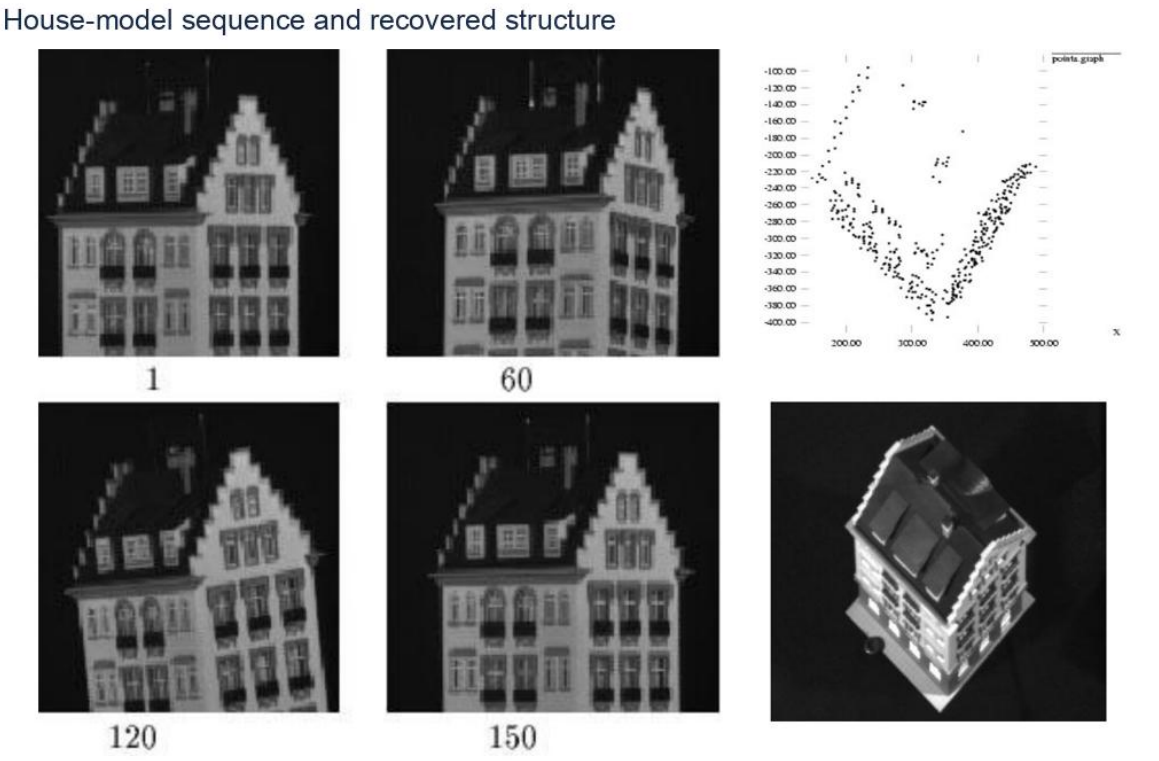


Fig. 6.3: The house-model sequence and result, PDF page 38, credited in the deck to Tomasi and Kanade. The matrix method reconstructs structure from tracks; the pictures alone are not the measurement matrix.

6.3 Triangulation: find the 3D point behind two pixels

Each pixel gives a ray. Triangulation finds a point consistent with several rays. Real rays usually miss because image measurements contain error.

Source: PDF pages 48–52. For full camera matrices, row construction, and substitution checks, work through *the booklet triangulation example*.

6.3.1 Turn projection into linear equations

Write the rows of camera P_i as $p_{i1}^T, p_{i2}^T, p_{i3}^T$. For an observed homogeneous image point $(u_i, v_i, 1)^T$,

$$u_i = \frac{p_{i1}^T X}{p_{i3}^T X}, \qquad v_i = \frac{p_{i2}^T X}{p_{i3}^T X}.$$

Multiply by the denominator and move the numerator to the left. Each image supplies two independent rows.

Formula — linear homogeneous triangulation

$$A_i = \begin{pmatrix} u_i p_{i3}^T - p_{i1}^T \\ v_i p_{i3}^T - p_{i2}^T \end{pmatrix}, \quad A = \begin{pmatrix} A_1 \\ \vdots \\ A_m \end{pmatrix}, \quad \widehat{X} = \underset{\|X\|=1}{\arg\min} \|AX\|^2. \tag{6.6}$$

A is $2m \times 4$. Its right singular vector with smallest singular value is $\widehat{X}$. If $\widehat{X}_4 \neq 0$, divide by that entry to obtain Euclidean coordinates. A zero last entry describes a point at infinity, not a finite coordinate you can obtain by dividing by zero.

Triangulate and check

1. Use image coordinates consistent with the cameras. If normalizing pixels by T_i, transform the camera to $T_i P_i$ too.
2. Build every row explicitly and take the last right singular vector.
3. Dehomogenize only if safe; reproject into every camera.
4. Check positive physical depth (**cheirality**) in the calibrated camera frames.
5. Reject large residuals and very small ray angles. Nearly parallel rays make depth unstable.

The linear method minimizes an **algebraic** residual. Rescaling rows or changing coordinates can change its noisy answer; it is not automatically invariant under arbitrary projective coordinate changes. Use it as a good initializer, then minimize the image error you care about.

6.3.2 Refine the point in image coordinates

Formula — geometric triangulation objectives

$$
\begin{aligned}
X^* &= \arg\min_X \sum_i \|\pi(P_i X) - x_i\|^2, \\
(\bar{x}, \bar{x}')^* &= \arg\min_{\bar{x},\bar{x}'} (\|\bar{x} - x\|^2 + \|\bar{x}' - x'\|^2), \\
&\text{subject to } \bar{x}'^T F \bar{x} = 0.
\end{aligned}
\tag{6.7}
$$

$\pi(a, b, c)^T = (a/c, b/c)^T$; norms in the second formulation compare the two finite image coordinates, with a homogeneous 1 appended only for the epipolar constraint. The first optimizes a 3D point with **fixed cameras**. The second finds the closest consistent pair of image points for a valid two-view F. Both assume the chosen unweighted pixel-error model; unequal uncertainty calls for suitable weights.

Choose the refinement

1. Obtain a linear estimate and inspect validity and residuals.
2. For fixed known cameras, refine the 3D point against reprojection error.
3. For the source's optimal two-view correction, parameterize corresponding epipolar lines and optimize the two corrected points before triangulating.
4. Compare all valid candidates and include boundary/infinite-parameter cases where required by that algorithm.

What is the sixth-degree polynomial on slide 52? Parameterizing the pair of epipolar lines by one scalar t turns the correction cost into a rational function. Setting its derivative to zero yields a polynomial of degree up to six. Its real roots are candidate stationary solutions; evaluating the original cost chooses the best valid candidate. The degree does not mean six iterations or six cameras. The slide presents the reduction, not the full polynomial coefficients.

6.4 Grow a reconstruction one view at a time

Structure from motion (SfM) estimates both the scene and the cameras. Lecture 5 supplies tracks; Lecture 4 supplies two-view geometry. This section joins those pieces into a reconstruction.

Source: PDF pages 39–47 and 53–55. Exam companion: *weak overlap and hierarchical reconstruction*.

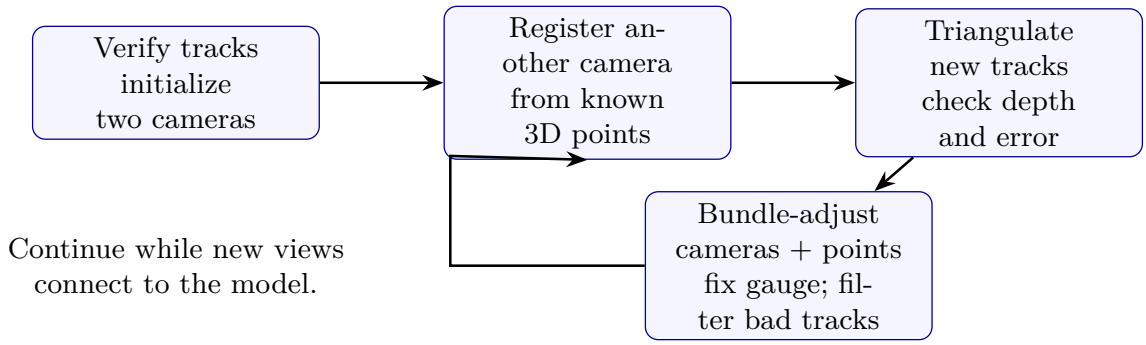


Fig. 6.4: Original reconstruction loop based on PDF pages 43–46. New cameras need links to existing structure; bundle adjustment revisits earlier estimates.

6.4.1 Start with a reliable pair

Use robust matches, a useful baseline, and enough shared nondegenerate structure. A nearly pure rotation can support image alignment while providing little or no depth information. A large baseline can improve triangulation but make matching harder.

For calibrated cameras, estimate an essential matrix and recover a relative rotation and translation direction, testing cheirality as in Lecture 4. Without calibration, a fundamental matrix initializes a **projective** reconstruction.

Formula — one canonical camera pair from a fundamental matrix

$$F^T e' = 0, \quad P = [I \mid 0], \quad P' = [-[e']_\times F \mid e']. \tag{6.8}$$

F has rank two, $e' \neq 0$ is the second-image epipole, and $[e']_\times$ is its cross-product matrix. This produces cameras consistent with F up to scale and projective gauge. The left 3×3 block of P' is generally **not a physical rotation**. This is a projective construction, not calibrated pose extraction.

Initialize an uncalibrated pair

1. Robustly estimate a rank-two F with a consistent image-order convention.
2. Find a nonzero vector in the nullspace of F^T.
3. Build the two camera matrices above and triangulate shared tracks.
4. Keep the result labeled projective until justified upgrade constraints are available.

Why is this consistent? The pair induces $[e']_\times(-[e']_\times F)$. Since $[e']_\times^2 = e'e'^T - \|e'\|^2 I$ and $e'^T F = 0$, the product is $\|e'\|^2 F$, the same fundamental matrix up to scale.

6.4.2 Add cameras, then add points

1. **Register a new camera** using already reconstructed 3D points and their 2D observations (**PnP** in the calibrated case).
2. **Triangulate new tracks** seen in sufficiently separated registered cameras.
3. **Check** reprojection error, cheirality, and track consistency. Remove outliers; do not keep a point merely because a solver returned it.
4. **Bundle-adjust** the current cameras and points, then continue. Revisit drift rather than assuming each early estimate is final.

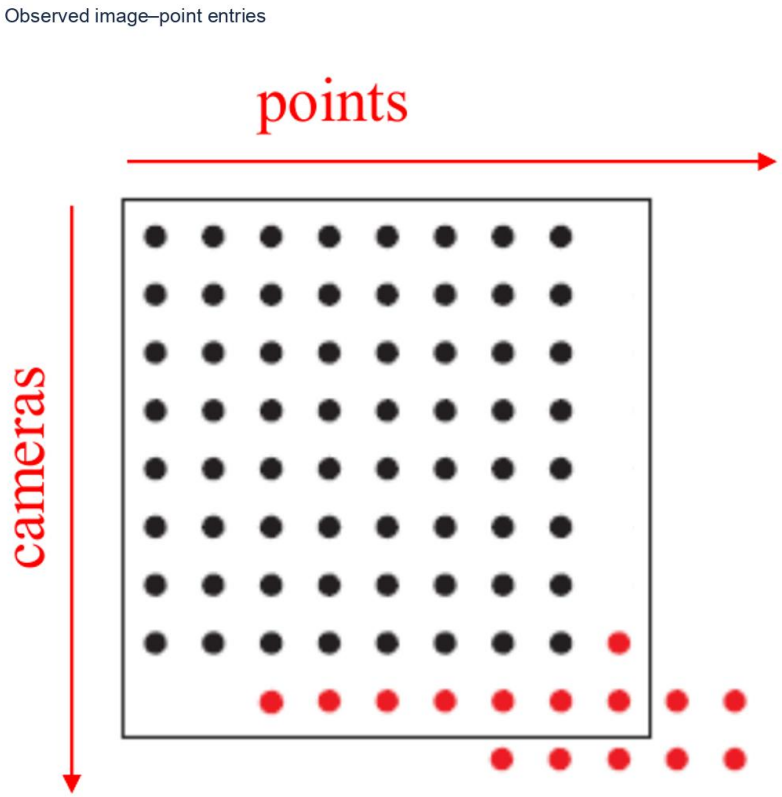


Fig. 6.5: Source PDF page 45. Each marked observation connects one camera and one point. Empty entries are missing data, not zero-valued pixel measurements.

6.4.3 Do not confuse the minimal pose problems

Problem	Known data	Unknown	Minimal sample and ambiguity
Calibrated relative pose	Matched pixels in two calibrated views	Relative rotation and translation direction	Five correspondences; up to ten algebraic candidates in the general five-point solver
P3P	Three known 3D points and their calibrated image rays	One camera pose	Three correspondences; up to four real pose candidates
Practical robust registration	Many noisy matches, possibly with outliers	Reliable pose	RANSAC plus extra observations, cheirality, and reprojection scoring

A sample can generate several hypotheses. More observations select and refine a valid one. These are different problems: two unknown views versus one camera looking at an existing 3D map.

6.4.4 What if calibration is unknown?

Self-calibration uses constraints such as constant intrinsics across views, zero skew, known principal-point assumptions, or geometric information such as vanishing points. Sufficient independent constraints can upgrade a projective reconstruction toward metric geometry. Constant intrinsics alone is not a universal guarantee: critical camera motions and degenerate scenes can leave ambiguities.

For long sequences, local reconstructions or keyframes can be joined through shared tracks and refined together. Weak overlap makes merging fragile; a global solver cannot invent missing connectivity. The workbook explains the quiz's hierarchical strategy, including when it helps and what still needs overlap.

6.5 Bundle adjustment: move the model until the pixels agree

A reconstructed point should project back to the pixel where you measured it. Bundle adjustment changes cameras and points together to reduce those discrepancies. "Bundle" refers to the bundle of viewing rays.

Source: PDF pages 46 and 56–72. Fully worked numerical companions: *a Gauss–Newton reprojection update* and *long-sequence reconstruction*.

6.5.1 Define exactly what is being minimized

Formula — weighted reprojection least squares

$$\begin{aligned} r_{ij}(\theta) &= \pi(K_i[R_i \mid t_i]X_j) - x_{ij}, \\ E(\theta) &= \tfrac{1}{2} \sum_{(i,j)\in\mathcal{O}} r_{ij}^T W_{ij} r_{ij}. \end{aligned} \tag{6.9}$$

θ collects the camera parameters and finite 3D point coordinates; intrinsics may be fixed or estimated as

appropriate. $\mathcal{O}$ contains actual observations only. Each residual is 2×1, in pixels, and W_{ij} is a symmetric positive definite 2×2 information matrix (often inverse covariance). The source sometimes uses observed-minus-predicted residuals; we consistently use predicted-minus-observed. Changing both residual and Jacobian signs leaves the least-squares update unchanged.

Prepare a meaningful objective

1. Choose which parameters are unknown and which are fixed.
2. Fix the gauge: for calibrated monocular SfM, fixing the first pose and one nonzero baseline/length removes the usual seven freedoms.
3. Stack only valid observed residuals with consistent units and uncertainty weights.
4. Remove gross outliers or use an appropriate robust loss. Ordinary squared error gives large residuals strong influence.
5. Compare actual objective values after every proposed update.

6.5.2 The Jacobian says how a parameter moves a pixel

For zero skew, let camera coordinates be $q = (x_c, y_c, z_c)^T$ and let

$$u = f_x x_c / z_c + c_x, \qquad v = f_y y_c / z_c + c_y.$$

Differentiate one coordinate at a time. For example, $\partial u / \partial x_c = f_x / z_c$ and $\partial u / \partial z_c = -f_x x_c / z_c^2$.

Formula — projection and pose Jacobian blocks

$$\begin{aligned} B &= \begin{pmatrix} f_x/z_c & 0 & -f_x x_c / z_c^2 \\ 0 & f_y/z_c & -f_y y_c / z_c^2 \end{pmatrix}, \\ J_{point} &= BR_i, \qquad J_{pose} = B \begin{pmatrix} -[q]_\times & I \end{pmatrix}. \end{aligned} \tag{6.10}$$

B is 2×3, J_{point} is 2×3, and J_{pose} is 2×6. Here the local pose increment is ordered $(\delta\omega, \delta t)$ and acts on camera coordinates as $q' \approx q + \delta\omega \times q + \delta t$ (a left local rigid-motion increment). A different pose convention changes the block; do not copy it into a different parameterization without checking. Depth must be nonzero. Intrinsics derivatives require extra columns if intrinsics are optimized.

Build a Jacobian row pair

1. Transform the point into the camera frame and compute B.
2. Multiply by R_i for world-point derivatives.
3. Use the stated local pose convention for camera derivatives.
4. Insert these blocks into the columns for that camera and that point; unrelated columns are zero.
5. Compare with small finite differences before trusting a new implementation.

6.5.3 Four update rules, one local picture

Linearize $r(\theta+\delta) \approx r + J\delta$. The derivative of $\frac{1}{2}r^T W r$ is $g = J^T W r$. Setting the derivative of the quadratic approximation to zero gives the Gauss–Newton equation.

Formula — gradient, Newton, Gauss–Newton, and damping

$$
\begin{aligned}
g = J^T W r, \qquad & H_{exact} = J^T W J + \sum_k (Wr)_k \nabla^2 r_k, \\
\delta_{GD} = -\eta g, \qquad & H_{exact}\delta_N = -g, \\
(J^T W J)\delta_{GN} &= -g, \\
(J^T W J + \lambda D)\delta_{LM} &= -g.
\end{aligned}
\tag{6.11}
$$

W is held constant in this expression; D is a positive diagonal scaling matrix and $\lambda \geq 0$. Gradient descent needs a suitable step size η. Newton uses the full Hessian. Gauss–Newton drops the residual second-derivative term, often useful near a good fit. Levenberg–Marquardt damps the solve; damping alone does not guarantee an accepted step. Large λ gives a scaled gradient-like direction. Parameter scaling matters because meters and radians have different meanings.

Run an iteration

1. Evaluate residuals, weights, Jacobian, and current cost.
2. Solve the damped normal system; do not explicitly invert it.
3. Apply the camera increment with its proper rotation parameterization.
4. Recompute the **actual** cost and verify valid depths.
5. Accept an improving step; otherwise increase damping or reduce the step. Stop using cost, gradient, and step-size criteria.

Worked example 6.2 — a full two-variable point update

Original practice question

Fixed rectified cameras have $f = 800$ pixels and baseline $b = 0.2$ m. Their measured horizontal coordinates are $u_L = 260$ and $u_R = 219$. Refine the horizontal point block (X, Z) from $(1.3, 4)$ m using one Gauss–Newton step. This block example holds the cameras fixed; it is not a complete joint camera-and-point solve.

Step 1 — Evaluate the residual and Jacobian

Why: The update must describe the present prediction error and its local sensitivity.

$$
r = \begin{pmatrix} 800X/Z - 260 \\ 800(X-0.2)/Z - 219 \end{pmatrix}_{(1.3,4)} = \begin{pmatrix} 0 \\ 1 \end{pmatrix},
$$

$$
J = \begin{pmatrix} 800/Z & -800X/Z^2 \\ 800/Z & -800(X-0.2)/Z^2 \end{pmatrix} = \begin{pmatrix} 200 & -65 \\ 200 & -55 \end{pmatrix}.
$$

Step 2 — Write and solve the complete normal system

Why: Both parameters affect both observations; solve their coupled equations together.

$$J^T J = \begin{pmatrix} 80000 & -24000 \\ -24000 & 7250 \end{pmatrix}, \quad J^T r = \begin{pmatrix} 200 \\ -55 \end{pmatrix},$$

$$\begin{pmatrix} 80000 & -24000 \\ -24000 & 7250 \end{pmatrix} \begin{pmatrix} \delta_X \\ \delta_Z \end{pmatrix} = \begin{pmatrix} -200 \\ 55 \end{pmatrix}.$$

Equivalently, $200\delta_X - 65\delta_Z = 0$ and $200\delta_X - 55\delta_Z = -1$. Subtracting gives $10\delta_Z = -1$, hence

$$\delta = \begin{pmatrix} -0.0325 \\ -0.1 \end{pmatrix}, \qquad \begin{pmatrix} X_{new} \\ Z_{new} \end{pmatrix} = \begin{pmatrix} 1.2675 \\ 3.9 \end{pmatrix}.$$

Step 3 — Reproject and compare actual costs

Why: A linearized improvement must be verified in the original nonlinear model.

$$u_L^{new} = \frac{800(1.2675)}{3.9} = 260, \qquad u_R^{new} = \frac{800(1.2675 - 0.2)}{3.9} \approx 218.974359.$$

$$E_{old} = \tfrac{1}{2}(0^2 + 1^2) = 0.5, \qquad E_{new} \approx \tfrac{1}{2}(0^2 + (-0.025641)^2) = 0.00032873.$$

Both depths are positive and the cost falls. The exact horizontal solution is $Z = 160/(260 - 219) \approx 3.902439$ m and $X = 260Z/800 \approx 1.268293$ m. One nonlinear iteration gets close; it need not finish exactly.

6.5.4 See a point update in real 3D

Explore reprojection refinement in 3D — print edition

The cameras stay fixed. The point is constrained to the left measured ray, with $X = 260Z/800$ and $Y = 120Z/800$. Rotate the scene to see that moving along this ray changes the right projection. Adjust the initial depth or right measurement, then take a Gauss–Newton/LM step.

Static equivalent: $u_R(Z) = 260 - 160/Z$, $r(Z) = u_R(Z) - 219$, and $J = 160/Z^2$. At $Z = 4$, $u_R = 220$, $r = 1$, and $J = 10$. An undamped step gives $\delta_Z = -r/J = -0.1$ m. At $Z = 3.9$, $r \approx -0.025641$; the next update moves back toward 3.902439 m. The left projection remains $(260, 120)$ throughout.

For this one-parameter subset, damping solves $(J^2 + \lambda)\delta_Z = -Jr$. The interactive checks positive depth and actual cost, reducing the step if necessary. The measured right pixel defines another ray; the moving point's projection approaches it. This is a visual model of a point refinement, while full bundle adjustment also updates camera parameters.

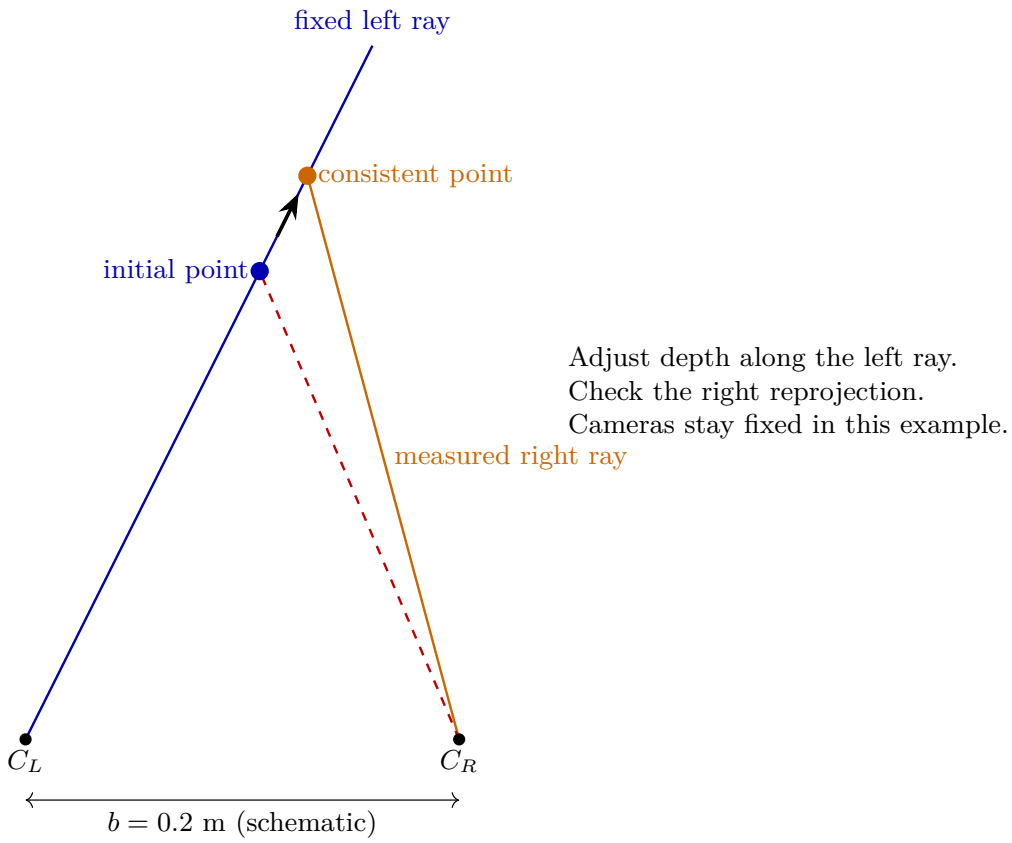


Fig. 6.6: Original diagram. Camera centers and the left ray stay fixed; the point's right ray changes as depth is refined.

6.5.5 Why large reconstructions can still be solved

Each image residual depends on **one camera and one point**, not every unknown. That creates sparse Jacobian blocks.

Formula — eliminate point updates with the Schur complement

$$\begin{pmatrix} U & V \\ V^T & W_p \end{pmatrix} \begin{pmatrix} \delta_c \\ \delta_p \end{pmatrix} = - \begin{pmatrix} g_c \\ g_p \end{pmatrix}, \tag{6.12}$$

$$(U - VW_p^{-1}V^T)\delta_c = -g_c + VW_p^{-1}g_p,$$
$$W_p\delta_p = -g_p - V^T\delta_c.$$

U is the camera block, W_p the point block, and V their coupling in the (possibly damped) normal system. W_p is block diagonal for independent point parameters. Inverse notation expresses elimination; implementations solve the small blocks. Gauge fixing and sufficient observations are needed for nonsingular systems. Here W_p is not the observation-weight matrix W above.

Solve a sparse bundle system

1. Assemble blocks only where observations connect a camera and a point.
2. Eliminate each independent point block to form the reduced camera system.
3. Solve for camera updates, then back-substitute for point updates.
4. Use appropriate ordering, scaling, and preconditioning; inspect connectivity and degeneracy.

The source ends with software examples. The important transferable ideas are the residual definition, a consistent Jacobian, gauge fixing, sparse structure, and checking actual improvement. A large parameter count alone does not tell you whether a reconstruction is well constrained.

6.6 Original quiz questions and worked solutions

Read the problem in black. Follow the numbered red steps. Work through the blue mathematics. Every step explains why it is needed. Matrix calculations show the inputs and intermediate results; written-reasoning questions explain the decision instead of inventing numerical data.

Source: the supplied lecture-organized quiz booklet. Full problems follow its wording and givens, with typography normalized; the historical PDF filenames appear below. Source-key corrections and added numerical companions are marked.

Retests have their own questions and full solutions. Problems labeled **Original practice** are the booklet's seven supplemental examples, separate from the 18 historical quizzes. Colored steps remain numbered and readable in grayscale.

6.6.1 Jump to a quiz

- *Fall 2025 Quiz 5 — SfM reconstruction upgrades*
- *Fall 2025 Quiz 6 — Large-scale Structure from Motion*
- *A5. Linear triangulation with two projection matrices (L6, slides 48–52)*
- *A6. Gauss–Newton and reprojection error (L6, slides 46 and 62–70)*
- *A7. Affine structure-from-motion factorization (L6, slides 25–37)*

6.6.2 Fall 2025 Quiz 5 — SfM reconstruction upgrades

Historical source: `3dcvMid/quizzesf25sol/cs532f25_quiz_5_solution-1.pdf`.

Question 1, Scenario A

Original problem · quiz booklet

You photograph an office facade with parallel horizontal bands and parallel pedestrian crossing lines on the road. Is this geometry sufficient to upgrade an entire-scene projective reconstruction? To what type? What extra information is needed?

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — State what each reconstruction preserves

Why: The target upgrade determines which geometric constraints are needed.

Recall reconstruction hierarchy: projective transformations preserve incidence; affine preserves parallelism; similarity preserves angles and length ratios; Euclidean/metric reconstruction preserves true lengths when scale is fixed.

Step 2 — Identify the supplied ideal points

Why: Each parallel-line family contributes one direction at infinity.

Each known family of 3D parallel lines gives a vanishing/ideal point in projective reconstruction. Horizontal facade bands provide one direction; road crossing lines provide a second.

Step 3 — Count the plane-at-infinity constraints

Why: Two points leave a family of possible planes.

To identify the plane at infinity π_∞ (a 3D plane in projective space), three independent noncollinear ideal points are generally needed. Two ideal points alone do not specify this plane.

Step 4 — Decide whether an affine upgrade is determined

Why: A third independent ideal point is generally needed.

Therefore the two supplied families are insufficient for a unique entire-scene affine upgrade under the simple count used by the quiz. A third independent family, such as vertical facade edges, may provide the missing ideal point; with suitable calibrated correspondences it constrains the affine plane.

Step 5 — Separate metric constraints and scale

Why: Parallelism alone does not recover angles or absolute lengths.

To reach metric/similarity reconstruction, also require enough Euclidean information, such as camera calibration, orthogonality/angular or absolute-conic constraints with adequate geometry. One known physical length then fixes absolute scale.

Question 1, Scenario B

Original problem · quiz booklet

You film a planar kitchen floor tiled with regular square tiles. Can this geometry upgrade the reconstruction, and what else is required for metric scale?

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Identify the square-grid constraints

Why: Parallel directions, a right angle and equal lengths constrain the observed plane.

A square tile grid contains two line directions with known 3D parallelism, and their intersection angle is 90°. Equal-length squares also supply metric relations *within the floor plane*.

Step 2 — State the identifiable upgrade

Why: Plane rectification is weaker than upgrading arbitrary off-plane structure.

The square grid directly permits a *metric rectification of the floor plane up to scale*. An *entire-scene similarity reconstruction (metric up to scale)* additionally requires sufficient camera-calibration and nondegenerate 3D constraints; diverse views of a single plane alone are not a general guarantee.

Step 3 — Recover scale when a length is known

Why: The ratio of real to reconstructed tile size fixes units.

If tile side length L is given (e.g. 30 cm), compare reconstructed tile dimension $\hat{L}$ with L and scale all translations/points by $s = L/\hat{L}$ to fix absolute units.

Step 4 — State the full-scene limitation

Why: A planar grid alone does not determine general 3D metric geometry.

Scope qualification: the known square grid alone, viewed only as a single plane without adequate camera/model constraints, does not in general uniquely determine full 3D scene metric geometry. State the extra assumption of sufficient viewpoints/calibration constraints when claiming an *entire-scene* upgrade.

Question 2

Original problem · quiz booklet

An image sequence has weak overlap: A overlaps B, B overlaps C, but A and C never share points, and so on. Is a single global factorization or a sequential SfM approach better?

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Inspect the visibility pattern

Why: Neighboring-only tracks leave many entries missing.

Form the tracking/measurement matrix mentally: many 3D features occur in only neighboring frames, leaving missing entries outside a narrow image-track visibility band.

Step 2 — Check complete-data factorization's assumptions

Why: Ordinary dense SVD expects a suitable filled measurement matrix.

Classical complete-data factorization $D \approx MS$ presumes adequate common visibility. With widespread missing observations, naive dense SVD cannot be applied directly and estimates can become poorly conditioned.

Step 3 — Choose incremental reconstruction

Why: It can grow using successive overlapping views.

Choose sequential / incremental SfM: start with a strongly overlapping initial pair; estimate two-view geometry and poses, triangulate their tracks, add the next overlapping image via pose estimation, then triangulate new points.

Step 4 — Refine and verify the result

Why: Bundle adjustment uses the visible measurements across the growing model.

Periodically refine all visible cameras and points with bundle adjustment; robust matching handles outliers. This respects the chain-like visibility structure.

Source check / assumptions: The original answer key describes a similarity upgrade. The rigorous distinction is between rectifying the observed tile plane and uniquely upgrading arbitrary off-plane 3D structure; the latter needs extra camera/scene constraints.

6.6.3 Fall 2025 Quiz 6 — Large-scale Structure from Motion

Historical source: `3dcvMid/quizzesf25sol/cs532f25_quiz_6_solution.pdf`.

Question 1

Original problem · quiz booklet

Given about 500,000 weakly overlapping photographs of Manhattan, choose incremental, global, hierarchical or hybrid SfM and defend your choice.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Read the dataset constraints

Why: Scale, weak overlap and repetition determine the practical bottlenecks.

Read the constraints: huge dataset, weak overlap, large repetitive urban textures (windows/facades), and significant computational burden from repeatedly refining poses and points.

Step 2 — Compare the candidate pipelines

Why: Each approach handles computation and bad pairwise matches differently.

Incremental SfM grows a reconstruction camera by camera; it gives good local control but updating an enormous model can become expensive. Global SfM estimates camera motions jointly but bad pairwise matches in repetitive facades can contaminate the solution.

Step 3 — Choose a hierarchical strategy

Why: Connected clusters make a huge reconstruction more manageable.

Choose hierarchical SfM (a hierarchical–hybrid strategy is also defensible). Build an image-overlap graph and partition it into moderately sized, sufficiently connected image clusters.

Step 4 — Reconstruct and refine locally

Why: Verified tracks are needed before neighboring pieces can be merged.

Run robust local two-view matching/geometric verification, initialize and triangulate points within each cluster, and perform local bundle adjustment.

Step 5 — Merge overlapping reconstructions

Why: Shared cameras or points provide alignment constraints.

Merge neighboring cluster reconstructions using common tracks/cameras and an appropriate similarity/alignment transformation; reject inconsistent alignments. Perform regional/global adjustment once the merged model is stable.

Step 6 — Defend the tradeoff

Why: The choice follows from the dataset rather than a universal complexity claim.

State why this choice matches the question: computation is localized; false repetitive-feature matches are easier to reject; partial overlaps need not be tracked in a single fully dense global factorization.

Source check / assumptions: The uploaded key uses an $O(N^3)$ assertion about pure incremental BA. That is not a universal complexity for sparse bundle adjustment, so the defensible argument is high computational cost that depends on graph structure, implementation and solver.

Question 2

Original problem · quiz booklet

The images are crowd-sourced and contaminated by "WTF" watermarks/text/frames that may produce false geometric matches. Suggest one simple modification to the RANSAC geometric verification step.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Inspect the consensus beyond its size

Why: Repeated overlays can create a large false match set.

Run RANSAC for each candidate image pair, estimating a valid geometric model and collecting its consensus inliers. A high inlier count *alone* may be deceptive when static overlays match each other.

Step 2 — Measure border concentration

Why: Frames and watermarks often produce suspicious edge-localized inliers.

Measure the spatial distribution of inliers: compute the proportion near thin image-border strips, as watermark/frames often occur along borders.

Step 3 — Check displacement consistency

Why: Overlay matches may behave like one common image translation.

For those border inliers, calculate pixel-coordinate displacements $\delta_i = x_i' - x_i$ after dehomogenization (and accounting for image size/crop if needed). Check whether most are approximately the same 2D translation with little parallax or 3D structure.

Step 4 — Reject or downweight suspicious models

Why: The combined evidence can identify a false consensus.

If both conditions hold—large border concentration and near-translation consistency—*reject or downweight the model/image pair*, despite its inlier count. Optionally mask border strips and rerun robust estimation.

Step 5 — State the heuristic's limitation

Why: Genuine scene geometry can also produce nearly translational matches.

To avoid false rejection of genuine pure translations, apply this rule as a *heuristic* together with spatial-coverage and scene-consistency checks, not as an infallible theorem.

6.6.4 A5. Linear triangulation with two projection matrices (L6, slides 48–52)

Original practice from the supplied booklet; not a historical quiz question.

Illustrative problem

Original problem · quiz booklet

Rectified cameras have $f = 800$ px and baseline $b = 0.2$ m. Let

$$P_L = \begin{bmatrix} 800 & 0 & 0 & 0 \\ 0 & 800 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

and

$$P_R = \begin{bmatrix} 800 & 0 & 0 & -160 \\ 0 & 800 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

. Observed pixels are $(260, 120)$ and $(220, 120)$. Reconstruct $X = (X, Y, Z)$.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Write two constraints per camera

Why: The projected point must lie on each observed viewing ray.

For each point (u, v) in a camera with matrix rows p_1^T, p_2^T, p_3^T, the two homogeneous equations are $(up_3^T - p_1^T)\widetilde{X} = 0$ and $(vp_3^T - p_2^T)\widetilde{X} = 0$, with $\widetilde{X} = (X, Y, Z, 1)^T$.

Step 2 — Stack the complete homogeneous system

Why: Both image observations constrain the same four-component point.

Stack two equations from each actual camera:

$$\begin{pmatrix} -800 & 0 & 260 & 0 \\ 0 & -800 & 120 & 0 \\ -800 & 0 & 220 & 160 \\ 0 & -800 & 120 & 0 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

The matrix is 4×4 and the point is 4×1. For a finite point choose $W = 1$ after finding a nonzero homogeneous solution.

Step 3 — Read the first camera's equations

Why: They express lateral coordinates in terms of depth.

Row 1 gives $X = (260/800)Z$ and row 2 gives $Y = (120/800)Z$. Row 3 gives $-800X + 220Z + 160 = 0$.

Step 4 — Substitute into the second camera

Why: The baseline resolves the remaining depth ambiguity.

Substitute X: $-260Z + 220Z + 160 = 0 \Rightarrow Z = 4$ m. Then $X = 260(4)/800 = 1.3$ m, $Y = 120(4)/800 = 0.6$ m.

Step 5 — Reproject the answer

Why: Both original observations should be recovered.

Answer $\boxed{(X, Y, Z) = (1.3, 0.6, 4) \text{ m}}$. Check by projecting into each camera: $(800X/Z, 800Y/Z) = (260, 120)$; $(800(X - 0.2)/Z, 800Y/Z) = (220, 120)$.

Step 6 — Explain the noisy SVD solution

Why: Noisy constraints require a best-fit homogeneous null vector.

With noisy measurements the four rays/equations are inconsistent. Use SVD $A = U\Sigma V^T$; take the last right singular vector as homogeneous $\widetilde{X}$, then divide by its fourth coordinate. Refine if needed by minimizing *reprojection* error rather than algebraic residual.

Step 7 — Verify the full nullspace product

Why: Every row should vanish for the exact supplied measurements.

Explicit nullspace check for the answer:

$$A\begin{pmatrix}1.3\\0.6\\4\\1\end{pmatrix} = \begin{pmatrix}-800(1.3)+260(4)\\-800(0.6)+120(4)\\-800(1.3)+220(4)+160\\-800(0.6)+120(4)\end{pmatrix} = \begin{pmatrix}0\\0\\0\\0\end{pmatrix}.$$

An SVD may return this column divided by $\sqrt{19.05}$, with either sign. Divide its first three entries by its fourth to recover $(1.3, 0.6, 4)$; do not interpret a unit homogeneous vector as ordinary coordinates.

6.6.5 A6. Gauss–Newton and reprojection error (L6, slides 46 and 62–70)

Original practice from the supplied booklet; not a historical quiz question.

Illustrative problem

Original problem · quiz booklet

For calibrated rectified stereo with $u_L = 260$, $bf = 160$ pixel-meters, and an observed right pixel $u_R^{obs} = 219$, refine an initial depth $Z_0 = 4$ m by one Gauss–Newton iteration.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Define prediction and residual

Why: Optimization needs an explicit error whose sign is fixed.

Predicted right pixel from stereo geometry is $\hat{u}_R(Z) = 260 - 160/Z$. Define signed residual $r(Z) = \hat{u}_R(Z) - 219 = 41 - 160/Z$. Minimize $E(Z) = r(Z)^2$.

Step 2 — Evaluate the initial error

Why: This gives the starting point for improvement.

Evaluate initial prediction: $\hat{u}_R(4) = 260 - 160/4 = 220$, hence $r(4) = 1$ pixel and $E(4) = 1$ squared pixel.

Step 3 — Differentiate the residual

Why: The Jacobian predicts how the error responds to a depth change.

Differentiate: $J(Z) = dr/dZ = 160/Z^2$, so $J(4) = 160/16 = 10$ pixels/meter.

Step 4 — Solve the normal equation

Why: Gauss–Newton chooses an increment that cancels the linearized residual.

Gauss–Newton normal equation is $(J^T J)\Delta Z = -J^T r$. In this one-variable case $10^2 \Delta Z = -(10)(1)$, giving $\Delta Z = -0.1$ m.

Step 5 — Update and re-evaluate

Why: The nonlinear residual must actually decrease.

Update $\boxed{Z_1 = Z_0 + \Delta Z = 3.9 \text{ m}}$. New pixel prediction is $260 - 160/3.9 \approx 218.97436$, residual -0.02564, error $E(Z_1) \approx 0.000657$.

Step 6 — Compare with the exact solution

Why: The small example clarifies the role of damping and multivariable extension.

The exact zero-residual solution is $Z^* = 160/41 \approx 3.902439$ m. Full bundle adjustment extends this to many camera pose and 3D-point variables: $\min_\theta \sum \|x^{obs} - \pi(P(\theta)X(\theta))\|^2$. LM adds damping: $(J^T J + \lambda I)\Delta\theta = -J^T r$.

Source check / assumptions: This is a one-variable reprojection refinement, not full joint bundle adjustment. The purpose is to demonstrate a complete numerical optimization update before scaling to the sparse multivariable system.

6.6.6 A7. Affine structure-from-motion factorization (L6, slides 25–37)

Original practice from the supplied booklet; not a historical quiz question.

Illustrative problem

Original problem · quiz booklet

Four 3D points are $(0,0,0)$, $(2,0,0)$, $(0,2,0)$ and $(0,0,2)$. The first orthographic camera records (X, Y) and the second records (Y, Z). Build the centered observation matrix and explain its rank and SVD factorization.

Fully worked solution — red labels explain the step; blue mathematics shows the calculation.

Step 1 — Center the first view

Why: Subtracting the image centroid removes its translation.

In view 1, image x coordinates are $(0,2,0,0)$ with mean 0.5, and y coordinates are $(0,0,2,0)$ with mean 0.5. Centered rows: $(-.5, 1.5, -.5, -.5)$ and $(-.5, -.5, 1.5, -.5)$.

Step 2 — Center the second view

Why: Every view must use its own centroid.

In view 2, image coordinates are (Y, Z). Their centered rows are $(-.5, -.5, 1.5, -.5)$ and $(-.5, -.5, -.5, 1.5)$.

Step 3 — Stack all measured rows

Why: The centered matrix contains two rows per image.

Stack views to obtain the $2m \times n = 4 \times 4$ matrix:

$$\boxed{D = \begin{bmatrix} -.5 & 1.5 & -.5 & -.5 \\ -.5 & -.5 & 1.5 & -.5 \\ -.5 & -.5 & 1.5 & -.5 \\ -.5 & -.5 & -.5 & 1.5 \end{bmatrix}}.$$

Step 4 — Check rank three

Why: Only three independent shape-coordinate rows are needed.

Rows 2 and 3 are identical, so rank ≤ 3. Rows 1,2,4 are linearly independent; therefore $\boxed{\text{rank}(D) = 3}$. This illustrates the centered affine/orthographic SfM rank-three condition.

Step 5 — Split the SVD factors

Why: Three singular components produce motion and centered structure factors.

Perform SVD $D = U\Sigma V^T$, retain the top three singular terms, and form $\widetilde{M} = U_3\Sigma_3^{1/2}$, $\widetilde{S} = \Sigma_3^{1/2}V_3^T$. Then $D = \widetilde{M}\widetilde{S}$ in this noise-free example.

Step 6 — Explain the affine ambiguity

Why: An invertible middle transformation leaves the measurements unchanged.

The factorization is not unique: for any invertible $Q \in \mathbb{R}^{3\times3}$, $(\widetilde{M}Q)(Q^{-1}\widetilde{S}) = D$. Metric constraints and camera-model assumptions are needed to upgrade from arbitrary affine coordinates. With missing feature observations, naive dense SVD also needs a missing-data strategy.

Step 7 — Multiply an exact geometric factorization

Why: Full matrices show one concrete motion-and-shape pair.

An exact geometric factorization makes the matrix multiplication visible:

$$M = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad S = \begin{pmatrix} -.5 & 1.5 & -.5 & -.5 \\ -.5 & -.5 & 1.5 & -.5 \\ -.5 & -.5 & -.5 & 1.5 \end{pmatrix}.$$

Multiplying MS selects row 1, row 2, row 2 again, then row 3 of S, reproducing every row of D. These are one valid camera/centered-shape pair; the balanced SVD factors generally look different but reproduce the same measurements.

6.7 Formula and source checklist

Use this as a map, then work the *original questions and full solutions*. This checklist adds no second version of the lecture.

6.7.1 Find a formula and its procedure

Formula	Where to learn and apply it
(6.1) — necessary measurement counts	*Ambiguity*; the adjacent application box gives the steps and assumptions
(6.2) — projective reconstruction ambiguity	*Ambiguity*; the adjacent application box gives the steps and assumptions
(6.3) — affine measurements and centering	*Factorization*; the adjacent application box gives the steps and assumptions
(6.4) — rank-three affine factorization	*Factorization*; the adjacent application box gives the steps and assumptions
(6.5) — orthographic metric upgrade	*Factorization*; the adjacent application box gives the steps and assumptions
(6.9) — weighted reprojection least squares	*Optimization*; the adjacent application box gives the steps and assumptions
(6.10) — projection and pose Jacobian blocks	*Optimization*; the adjacent application box gives the steps and assumptions
(6.11) — gradient, Newton, Gauss–Newton, and damping	*Optimization*; the adjacent application box gives the steps and assumptions
(6.12) — eliminate point updates with the Schur complement	*Optimization*; the adjacent application box gives the steps and assumptions
(6.8) — one canonical camera pair from a fundamental matrix	*Pipeline*; the adjacent application box gives the steps and assumptions
(6.6) — linear homogeneous triangulation	*Triangulation*; the adjacent application box gives the steps and assumptions
(6.7) — geometric triangulation objectives	*Triangulation*; the adjacent application box gives the steps and assumptions

6.7.2 Trace the lecture slides

PDF pages	Teaching location	Covered material
4–19	*Ambiguity*	Unknowns, necessary counts, gauge, projective/affine/similarity ambiguity and house examples
20–38	*Factorization*	Affine/orthographic cameras, centering, SVD, rank at most three, metric upgrade and house sequence
39–47	*Pipeline*	Perspective count, canonical cameras from F, sequential SfM, visibility, self-calibration
48–52	*Triangulation*	Linear system/SVD, noisy rays, geometric objectives and sixth-degree candidate equation
53–55	*Pipeline*	Calibrated five-point versus P3P, multiple solutions and verification
56–72	*Optimization*	Reprojection least squares, gradient/Newton/GN/LM, Jacobians, gauge and sparse elimination

Pages 1–3 supply title/credits; repeated section titles and the duplicated BA definition on pages 57/59 are consolidated. Page 72's software examples are acknowledged in the optimization discussion rather than reproduced as installation instructions.

6.7.3 Quiz route

- *Facade and floor reconstruction constraints*, *second scenario*, and *pipeline choice*.
- *Weak overlaps* and *contaminated matches*.
- *Linear triangulation with full matrices*.
- *Numerical Gauss–Newton reprojection update*.
- *Centered measurement matrix and factorization*.

Source qualifications: rank is at most three, with equality requiring nondegeneracy. Parameter counts alone do not establish uniqueness. A projective camera block is not a rotation. Metric upgrade requires enough independent constraints and a valid positive metric. Linear triangulation optimizes algebraic error; the geometric objective is different.

Detailed page-level dispositions and editable formula/figure inventories are in the repository's `authoring/coverage.csv`, `lecture06-audit.md`, and `lecture06-formulas.csv`.

6.8 TLDR

Tracks give measurements. Camera models turn those measurements into equations. Optimization makes their predictions agree.

- SfM estimates camera motion and 3D structure together. Without external metric information, common gauge freedoms remain.
- A projective reconstruction preserves the images but may distort 3D angles and parallelism. Additional independent constraints justify affine/metric upgrades; a known length can fix metric scale.
- Under affine cameras, center complete tracks, form the measurement matrix, and retain its leading SVD components. Its ideal rank is at most three. Missing entries are not zeros.
- Triangulation stacks two linear rows per view, takes a right singular vector, and checks dehomogenization, cheirality, reprojection, and parallax. Refine geometric image error when needed.
- Sequential SfM initializes a pair, registers cameras against known 3D points, triangulates new tracks, filters, and bundle-adjusts. Five-point relative pose and P3P solve different problems.
- Bundle adjustment minimizes reprojection error over observed camera–point pairs. Define residual signs, parameter conventions, weights, and gauge before constructing the Jacobian.
- Gauss–Newton solves a local least-squares model; LM adds damping. Always check the actual nonlinear cost. Sparsity and point elimination make large solves practical.

6.8.1 Short quiz

These are **original practice questions**, separate from the historical/booklet *quiz workbook*. Try each before opening its answer. All answers are printed in the PDF.

Question 1

Original practice question

If all world points and camera centers double in Euclidean coordinates, with rotations fixed, do image projections change?

Worked solution

Step 1 — Find camera coordinates

Why: Both the scene and camera translation must scale consistently.

$$q^{new} = R(2X - 2C) = 2R(X - C) = 2q.$$

Step 2 — Divide by depth

Why: Perspective removes a common nonzero scale.

$$u^{new} = f_x \frac{2q_x}{2q_z} + c_x = u.$$

The same holds for the vertical coordinate. Metric scale needs extra information.

Question 2

Original practice question

A complete centered affine measurement matrix has shape $8imes10$. What factor shapes and ideal rank bound apply?

Worked solution

Step 1 — Identify cameras and points

Why: There are two rows per camera and one column per point.

$$m = 4, \quad n = 10, \quad M \in \mathbb{R}^{8\times3}, \quad S \in \mathbb{R}^{3\times10}.$$

Step 2 — Apply the product rank bound

Why: A product cannot have rank greater than either factor.

$$D = MS \quad \Longrightarrow \quad \text{rank}(D) \leq 3.$$

Rank equals three only for suitable nondegenerate motion and shape.

Question 3

Original practice question

Given $u = 2, v = 3$ and $P = [I \mid 0]$, display the two triangulation rows.

Worked solution

Step 1 — Write the camera rows

Why: Multiplication by the projection denominator removes the unknown image scale.

$$p_1^T = (1,0,0,0),\ p_2^T = (0,1,0,0),\ p_3^T = (0,0,1,0).$$

Step 2 — Assemble the row pair

Why: Each observed image point supplies two constraints.

$$A_i = \begin{pmatrix} -1 & 0 & 2 & 0 \\ 0 & -1 & 3 & 0 \end{pmatrix}.$$

One image still leaves depth undetermined; add another camera's constraints.

Question 4

Original practice question

A centered factorization changes motion to MC for invertible C. How must shape change to preserve the image measurements?

Worked solution

Step 1 — Cancel the inserted transformation

Why: The inner factors must multiply to the identity.

$$S^{new} = C^{-1}S.$$

Step 2 — Verify the product

Why: Preserving the product preserves centered image predictions.

$$M^{new}S^{new} = (MC)(C^{-1}S) = MS = D.$$

Question 5

Original practice question

For $r(Z) = 260 - 160/Z - 219$ at $Z = 4$, compute one undamped Gauss–Newton depth update.

Worked solution

Step 1 — Evaluate the local model

Why: The residual and derivative set the scalar least-squares problem.

$$r = 1, \qquad J = 160/Z^2 = 10.$$

Step 2 — Solve and check

Why: The correct sign must reduce the actual nonlinear error.

$$\delta = -\frac{Jr}{J^2} = -0.1, \quad Z_{new} = 3.9,$$

$$r(3.9) \approx -0.025641, \quad E_{new} \approx 0.00032873 < 0.5.$$

Question 6

Original practice question

Why are a first-pose constraint and a nonzero baseline constraint useful in calibrated monocular bundle adjustment?

Worked solution

Step 1 — Identify the freedoms

Why: A common rigid transform and scale do not change image predictions.

$$3 \text{ translation} + 3 \text{ rotation} + 1 \text{ scale} = 7 \text{ gauge freedoms.}$$

Step 2 — Anchor the coordinate system

Why: A solver should not chase directions that leave the objective unchanged.

Fixing the first pose removes six freedoms. Fixing a nonzero baseline or known length fixes the common scale. Other degeneracies can still remain.

Study and reference

Every entry points to a named formula box, its assumptions, derivation, and application procedure. All six lectures are populated.

Foundation	Formula	Explanation and application
Homogeneous scale	(1)	*Notation and coordinate conventions*

Lecture 1

See the *Lecture 1 formula and source checklist* for the full inventory. Each entry links to its formula box, assumptions, and companion application procedure.

Lecture 2

See the *Lecture 2 formula and source checklist* for quaternion conversions, directional scaling, homographies, image sampling, DLT/normalization, RANSAC, radial correction, and epipolar constraints. Repeated Lecture 1 formulas retain their original canonical boxes.

Lecture 3

See the *Lecture 3 formula and source checklist* for focal length from field of view and known-plane depth. The new projective DLT and RANSAC quiz applications link to their original Lecture 2 formula boxes.

Lecture 4

See the *Lecture 4 formula and application map* for noisy rank-two estimation, epipolar pixel errors, essential matrices/poses, stereo depth and exact uncertainty, disparity bounds, and dense matching costs/support. Existing fundamental and rectification formulas keep their Lecture 2 teaching location.

Lecture 5

See the *Lecture 5 formula and source checklist* for the structure tensor, eigenvalues, Harris/Shi–Tomasi, Lucas–Kanade, pyramids, normalization, binary descriptors, orientation, ratio tests and evaluation.

Lecture 6

See the *Lecture 6 formula and source checklist* for gauge/counts, affine centering/SVD/metric upgrade, triangulation, canonical cameras, reprojection objectives, Jacobians, update rules and sparse elimination.

Every assigned quiz now has its complete problem statements and worked solutions. Open a lecture workbook below. Questions use the supplied booklet's wording, with typography normalized; historical PDF filenames identify the original sources. Each numbered red step gives a reason, and its blue mathematics shows the calculation. Duplicate retests have separate complete solutions.

The 18 historical quizzes contain **39 problem groups** (including every grouped subpart); the seven clearly labeled original supplemental problems bring the total to **46**. Main lecture explanations for Lectures 5–6 remain unfinished; their quiz workbooks are available now.

Lecture 1

***Open the Lecture 1 quiz workbook*.**

Fall 2025 Quiz 1 — Probability and rigid camera poses

Source: `cs532f25_quiz_1_solution.pdf`.

- *Question 1 — original problem and 3 worked steps*.
- *Question 2 — original problem and 4 worked steps*.
- *Question 3 — original problem and 6 worked steps*.
- *Question 4 — original problem and 6 worked steps*.

Fall 2017 Quiz 9 — Depth maps and tetrahedron normals

Source: `cs532_f17_quiz9.pdf`.

- *Problem 1 — original problem and 6 worked steps*.
- *Problem 2(a) — original problem and 6 worked steps*.
- *Problem 2(b) — original problem and 5 worked steps*.

Lecture 2

***Open the Lecture 2 quiz workbook*.**

Spring 2020 Quiz 1 — Projective points and camera projection

Source: `cs532s20_quiz1.pdf`.

- *Problem 1 — original problem and 4 worked steps*.
- *Problem 2(a–b) — original problem and 3 worked steps*.
- *Problem 3(a–b) — original problem and 5 worked steps*.

Lecture 3

***Open the Lecture 3 quiz workbook*.**

Spring 2019 Quiz 3 — Pentagon, homography, RANSAC

Source: `cs532s19-quiz3.pdf`.

- *Problem 1(a–d) — original problem and 6 worked steps*.

Spring 2019 Quiz 2 — FOV, calibration, homogeneous scaling

Source: `cs532s19_quiz2-2.pdf`.

- *Problem 1 — original problem and 3 worked steps*.
- *Problem 2 — original problem and 3 worked steps*.
- *Problem 3(a–b) — original problem and 3 worked steps*.
- *Problem 4 — original problem and 3 worked steps*.

Spring 2020 Quiz 3 — Duplicate camera model

Source: `cs532s20_quiz3.pdf`.

- *Problem 1 — original problem and 3 worked steps*.
- *Problem 2 — original problem and 3 worked steps*.
- *Problem 3(a–b) — original problem and 3 worked steps*.
- *Problem 4 — original problem and 3 worked steps*.

Spring 2020 Quiz 2 — Pinhole ray and cross product

Source: `cs532s20_quiz2.pdf`.

- *Problem 1 — original problem and 4 worked steps*.
- *Problem 2 — original problem and 4 worked steps*.

Older Quiz 9 — Pinhole ray intersects a 3D plane

Source: `cs532f15_quiz9.pdf`.

- *Problem 1 — original problem and 7 worked steps*.

A1. Homography, DLT and SVD (L3, slides 34–45)

Source: `CS532_18_Quizzes_By_Lecture.tex`. **Original practice from the booklet.**

- *Illustrative problem — original problem and 8 worked steps*.

Lecture 4

***Open the Lecture 4 quiz workbook*.**

Older Quiz 4 — Epipolar line and constraint

Source: `cs532f15_quiz4.pdf`.

- *Problem 1 — original problem and 4 worked steps*.
- *Problem 2 — original problem and 4 worked steps*.

Spring 2019 Quiz 4 — Duplicate epipolar retest

Source: `cs532s19_quiz4.pdf`.

- *Problem 1 — original problem and 4 worked steps*.
- *Problem 2 — original problem and 4 worked steps*.

Fall 2025 Quiz 2 — Three-view epipolar geometry

Source: `cs532f25_quiz_2_solution.pdf`.

- *Question 1 — original problem and 8 worked steps*.
- *Question 2 — original problem and 5 worked steps*.

Fall 2025 Quiz 3 — Stereo depth sensitivity

Source: `cs532f25_quiz_3_solution.pdf`.

- *Question 1 — original problem and 4 worked steps*.
- *Question 2 — original problem and 4 worked steps*.

Spring 2019 Quiz 5 — Exact stereo errors

Source: `cs532s19_quiz5-2.pdf`.

- *Problem 1(a–c) — original problem and 6 worked steps*.

Spring 2020 Quiz 5 — Repeat disparity test

Source: `cs532s20_quiz5.pdf`.

- *Problem 1(a–c) — original problem and 6 worked steps*.

A4. Essential matrix and cheirality (L4, slides 22–41)

Source: `CS532_18_Quizzes_By_Lecture.tex`. **Original practice from the booklet.**

- *Illustrative problem — original problem and 6 worked steps*.

Lecture 5

***Open the Lecture 5 quiz workbook*.**

Fall 2025 Quiz 4 — KLT with auto-exposure

Source: `cs532f25_quiz_4_solution.pdf`.

- *Question 1 — original problem and 7 worked steps*.

Spring 2019 Quiz 6 — Feature trackability

Source: `cs532s19_quiz6-2.pdf`.

- *Problem 1(a–d) — original problem and 6 worked steps*.

A2. Harris response from gradients (L5, slides 11–30)

Source: `CS532_18_Quizzes_By_Lecture.tex`. **Original practice from the booklet.**

- *Illustrative problem — original problem and 6 worked steps*.

A3. Lucas–Kanade least-squares displacement (L5, slides 39–50)

Source: `CS532_18_Quizzes_By_Lecture.tex`. **Original practice from the booklet.**

- *Illustrative problem — original problem and 6 worked steps*.

Lecture 6

***Open the Lecture 6 quiz workbook*.**

Fall 2025 Quiz 5 — SfM reconstruction upgrades

Source: `cs532f25_quiz_5_solution-1.pdf`.

- *Question 1, Scenario A — original problem and 5 worked steps*.
- *Question 1, Scenario B — original problem and 4 worked steps*.
- *Question 2 — original problem and 4 worked steps*.

Fall 2025 Quiz 6 — Large-scale Structure from Motion

Source: `cs532f25_quiz_6_solution.pdf`.

- *Question 1 — original problem and 6 worked steps*.
- *Question 2 — original problem and 5 worked steps*.

A5. Linear triangulation with two projection matrices (L6, slides 48–52)

Source: `CS532_18_Quizzes_By_Lecture.tex`. **Original practice from the booklet.**

- *Illustrative problem — original problem and 7 worked steps*.

A6. Gauss–Newton and reprojection error (L6, slides 46 and 62–70)

Source: `CS532_18_Quizzes_By_Lecture.tex`. **Original practice from the booklet.**

- *Illustrative problem — original problem and 6 worked steps*.

A7. Affine structure-from-motion factorization (L6, slides 25–37)

Source: `CS532_18_Quizzes_By_Lecture.tex`. **Original practice from the booklet.**

- *Illustrative problem — original problem and 7 worked steps*.

Outside the core scope

Fall 2025 Quizzes 7 (ICP) and 8 (mesh topology) remain excluded from these six lectures. They are not among the 18 historical quizzes in the supplied booklet.

GLOSSARY

Terms are added as their explanations are written. First-use definitions, glossary terms, and PDF index entries share stable references.

Homogeneous coordinates
: Coordinates representing a projective point by a nonzero vector up to nonzero scale. See *Notation and coordinate conventions* and (1) for the definition and comparison procedure.

Coordinate frame
: A reference origin and axes used to express coordinates. Transform directions are specified in *Notation and coordinate conventions*; detailed geometry is developed in *Reference frames and two worked quiz problems*.

Computer vision
: Inferring scene properties from images. See *Why reconstruct a world from photographs?* for the inverse-problem viewpoint.

Structure from motion
: Joint recovery of camera motion and scene structure from corresponding image measurements. See *Why reconstruct a world from photographs?*; full estimation is in *Grow a reconstruction one view at a time*.

Depth map
: An image-indexed array of depth measurements or estimates. Its depth convention and camera model must be specified. See *Image formation: where does a 3D point appear?*.

Point cloud
: A collection of 3D point positions, often expressed in one common frame. See *Example 1.16*.

Projective plane
: Nonzero 3-vectors identified under nonzero scale, representing finite and ideal 2D points. See *Points, lines, and the meaning of infinity*.

Incidence
: A point lying on a line or plane, tested through a zero dot product of homogeneous coordinates and coefficients. See (1.1).

Ideal point
: A projective point with last homogeneous coordinate zero; also called a point at infinity. See (1.3).

Line at infinity
: The projective line containing all ideal points of a plane. See *Points, lines, and the meaning of infinity*.

Vanishing point
: An image location corresponding to a scene direction, often where projected parallel scene lines intersect. It need not be an ideal point of the image plane. See *Points, lines, and the meaning of infinity*.

Horizon
: The image of a scene plane's line at infinity. See *Points, lines, and the meaning of infinity*.

Degrees of freedom
: The number of independent continuous parameters. Matrix entries can be constrained or equivalent under scale. See (1.6) and *Rotations: several descriptions of the same motion*.

Nullspace
The vectors mapped to zero by a matrix. Homogeneous geometry seeks a nonzero solution, often unique up to scale. See (1.10).

Plücker coordinates
A homogeneous representation of a spatial line; for a finite line the direction/moment pair is $(d, p \times d)$ with $d^T m = 0$. See (1.12).

Reciprocal product
A scalar formed from two valid direction/moment line representations; it tests their coplanarity and supports a shortest-distance formula. See (1.13).

Surface normal
A direction perpendicular to local surface tangent directions. An oriented face normal requires a winding or outward convention. See *Example 1.12*.

Rotation matrix
A real orthogonal matrix with determinant $+1$ representing a proper rotation. See (1.15).

Axis–angle
A rotation representation consisting of a unit axis and an angle. Extraction needs special handling near zero and a half-turn. See (1.17).

Quaternion
A scalar–vector algebraic object; unit quaternions represent rotations under a declared multiplication and active/passive convention. See (1.19).

Intrinsic rotation
A rotation about an axis attached to the moving body. It is unrelated to camera intrinsic calibration despite the shared word. See (1.16).

Extrinsic rotation
A rotation about a fixed reference axis. See (1.16).

Gimbal lock
A singular three-angle description in which independent parameter effects coincide. The physical orientation still exists. See *Rotations: several descriptions of the same motion*.

Rigid transformation
A proper rotation and translation preserving distances and angles. See (1.20).

Circular points
Two complex ideal points shared by all Euclidean circles in a plane, encoding metric structure projectively. These are a vocabulary preview in *Move points without losing the geometry*.

Absolute conic
A complex conic on the plane at infinity encoding the 3D Euclidean metric. Its role in metric upgrades is deferred to Lecture 6; see *Reference frames and two worked quiz problems* for the hierarchy context.

Pinhole camera
An ideal central-projection model in which rays pass through one camera center. See (1.23).

Principal point
The pixel location where the optical axis intersects the ideal image plane. It is not automatically the pixel origin. See (1.24); physical versus normalized representations are clarified in *From field of view to a camera matrix*.

Intrinsic parameters
Camera calibration parameters that map normalized camera coordinates to pixels, including focal entries, skew, and principal point. See (1.24).

Extrinsic parameters
Rotation and translation linking world and camera frames, with a declared transform direction. See *Image formation: where does a 3D point appear?*.

Backprojection
Mapping a pixel to a camera ray, then using specified depth to recover a 3D point. See (1.25).

Axial depth
The point's coordinate along the camera's forward axis; it differs from Euclidean range along the ray. See (1.25).

Homography
An invertible projective mapping between planes, represented by a nonsingular 3×3 matrix up to scale. See (2.4) and *Homographies: straighten a photographed plane*.

Cross-ratio
An ordered ratio of four collinear positions preserved by projective mappings. See (2.6).

Invariant
A property unchanged by a specified transformation family. See *Homographies: straighten a photographed plane*.

SLERP
Spherical linear interpolation of unit quaternions, giving an orientation path with constant angular speed when parameterized uniformly. See (2.3).

Inverse warping
Filling each output pixel by looking up a source coordinate through the inverse geometric mapping. See (2.7).

Bilinear interpolation
Blending four neighboring grid samples using horizontal and vertical fractional-coordinate weights. See (2.8).

Direct linear transformation
DLT: estimating a projective mapping from a homogeneous linear system. See (2.9); coordinate normalization is in (2.10).

Inlier
A measurement consistent with a model within a specified residual threshold. An outlier fails that test. See *Estimate a reliable mapping*.

RANSAC
Random sample consensus: fit small random subsets, score their support, and refit a selected consensus. See (2.11) for sampling assumptions.

Radial distortion
Lens-induced coordinate displacement along directions from a distortion center. See (2.12), including mapping-direction conventions.

Epipolar line
The image of an epipolar plane; a known point in one view restricts its counterpart to this line in the other.

See (2.13).

Epipole
The image of the other camera's center, shared by the epipolar lines of a view. See *Distortion and two-view geometry*.

Fundamental matrix
A rank-two 3×3 matrix mapping image points to epipolar lines in another view, defined up to scale. See (2.13).

Rectification
Warping a pair of images so corresponding epipolar lines lie on matching horizontal rows. See (2.16).

Field of view
Angle between boundary rays of an image. Centered horizontal FOV relates width to pixel focal length through (3.1).

Pixel pitch
Physical size of a sensor pixel along an axis; multiplying pixel focal length by that pitch gives physical focal length. See *From field of view to a camera matrix*.

Ray–plane intersection
Finding the point shared by a viewing ray and a known plane; special cases include parallel rays and rays lying in the plane. See (3.2).

Symmetry ambiguity
Multiple alignments supported by identical visible evidence. Regular-pentagon corner matches illustrate why extra trials cannot recover missing identity information. See *Probability and the pentagon puzzle*.

Essential matrix
Calibrated two-view relation $E = [t]_\times R$, with two equal nonzero singular values and one zero. See (4.3).

Cheirality
The requirement that reconstructed visible points have positive depth in both observing cameras. Used to choose pose candidates; see *Recover camera pose with calibration*.

Disparity
Corrected horizontal coordinate difference in a rectified stereo pair; inversely proportional to depth under the stated convention. See (4.5).

Cost volume
Scores for every reference pixel and disparity/depth hypothesis, before selecting a depth map. See (4.8).

Photo-consistency
Agreement of projected image appearance under a specified comparison model. See *Find matches across an image*.

Census transform
Ordered bit string comparing each neighbor's intensity with its center; compare strings by Hamming distance. See (4.10).

Rank transform
Count of neighbors darker than the center under a fixed comparison convention. See (4.10).

Adaptive support
Neighborhood contributions weighted by similarity and proximity rather than uniformly. See (4.11).

Structure tensor
: Gradient outer-product sum describing local directional intensity change. See (5.1) and *Why corners are easier to locate*.

Harris corner detector
: Detector using determinant and squared trace of the patch tensor, followed by thresholding and local peak selection. See (5.3).

Non-maximum suppression
: Selection of local score peaks, with a specified neighborhood and tie policy. See *the NMS example*.

Lucas–Kanade tracking
: Local iterative motion estimation by linearized brightness-constancy least squares. See (5.4) and *Track a feature by solving for its motion*.

Aperture problem
: A single local edge does not constrain motion along that edge. See *Track a feature by solving for its motion*.

BRIEF
: Binary patch descriptor formed from ordered intensity-pair comparisons. See *Describe a patch, then judge its matches*.

ORB
: Oriented FAST and rotated BRIEF pipeline with multiscale detection. See *Describe a patch, then judge its matches* for the scale qualification.

BRISK
: Binary feature pipeline using long pairs for orientation and short pairs for descriptor bits. See *Describe a patch, then judge its matches*.

Hamming distance
: Number of differing positions in equal-length binary descriptors. See (5.7).

Nearest-neighbor distance ratio
: Best-to-second-best candidate distance ratio for one query, used to reject ambiguous matches. See (5.9).

Gauge freedom
: Common coordinate changes that leave the observations unchanged. See *What can the photographs actually determine?* and (6.2).

Affine factorization
: Splitting a centered affine measurement matrix into motion and shape factors. See (6.4).

Metric upgrade
: Constraining a reconstruction to recover Euclidean geometric properties, usually up to a similarity. See (6.5) and *What can the photographs actually determine?*.

Bundle adjustment
: Joint camera-and-point refinement against observed image reprojection error. See (6.9).

Gauss–Newton
: Least-squares iteration using the linearized residual Jacobian. See (6.11) and *the numerical update*.

Levenberg–Marquardt
: Damped least-squares update with an acceptance policy. See (6.11).

Schur complement
: Block elimination that reduces a bundle system to camera parameters before recovering point updates. See (6.12).

P3P
: Calibrated pose estimation from three known 3D points and corresponding image rays, potentially with several solutions. See *Grow a reconstruction one view at a time*.

Self-calibration
: Inferring intrinsic calibration through multi-view/scene constraints, subject to sufficient independent information and noncritical configurations. See *Grow a reconstruction one view at a time*.

The planned text follows the six supplied CS 532 lecture PDFs and the quiz-to-lecture mapping in `quizByLec.md`. The lecture decks identify Enrique Dunn and credit additional contributors on individual sections. Preserve those specific credits when adapting their material; do not infer authorship or reuse permission from filenames alone.

Lecture 1 gives verified source ranges and identifies original versus adapted examples. The geometry section credits M. Pollefeys and D. Cappelleri (PDF page 62); image formation credits John Oliensis (PDF page 104). The notation and matrix-dimension examples in this foundation edition are original explanatory examples.

The pixel-axis diagram in the notation guide is an original project illustration with an editable TikZ source and paired SVG/PDF assets.

Lecture 1 includes three embedded image assets extracted without retouching from the supplied deck: the railway photograph (PDF page 68), fountain input (page 50), and fountain depth output (page 51). Their original individual creators are not identified on those slides; this source uncertainty is retained in the captions and figure register. The source deck is preserved. Twelve additional geometric/workflow diagrams are original project drawings, with paired SVG/PDF and editable TikZ sources. A repository's text license does not automatically establish the reuse permissions of every embedded source figure.

Reading list named in the lecture

The deck's PDF pages 9–10 name Richard Szeliski's *Computer Vision: Algorithms and Applications* (2010), David M. Mount's CMSC 754 computational-geometry lecture notes (2012), and Xiang Gao and Tao Zhang's *Introduction to Visual SLAM: From Theory to Practice*. These are source-listed further reading; the present chapter was developed from the supplied slides and quizzes rather than claiming those books were consulted in full.

Lecture 2

`cs532_f25_Week2.pdf` credits M. Pollefeys and D. Cappelleri for geometry (PDF page 2), John Oliensis for image formation (51), Steve Seitz for warping (72), and R. Hartley, A. Zisserman, and M. Pollefeys for estimation, distortion, and two-view geometry (82, 95, 105, 111).

The basketball-court photograph is extracted from PDF page 71, printed slide 72. Its individual photographer and reuse terms are not identified there. The annotated source/rectangular-warp comparison is computed locally from four disclosed approximate corner coordinates. The chosen output aspect ratio is illustrative. Four additional original vector diagrams explain quiz intersections, warp sampling, consensus, and rectified epipolar lines. Editable sources and paired print/web assets are retained in the figure register.

The interactive camera-motion and perspective-warp diagrams are original local SVG visualizations of the chapter models. The perspective explorer also has a paired static SVG/PDF figure for print and no-JavaScript readers, with an editable TikZ source. Its four numbered collinear points have cross-ratio 4/3 before and after the warp.

Lecture 1's rotatable 3D projection scene uses locally bundled Three.js and OrbitControls r180 (MIT license retained with the library). The scene design, math linkage, and static projection diagram are original project work; the library provides rendering and inspection controls.

Lecture 3

`wk3_cs532s26_ImageFormation+Homographies-1.pdf` credits John Oliensis for image formation (PDF page 2), Steve Seitz for warping (23), and R. Hartley, A. Zisserman, and M. Pollefeys for estimation, robust fitting, distortion, and two-view geometry (33, 46, 56, 62). Repeated source illustrations link to their existing teaching locations rather than being copied again.

Five original vector diagrams cover FOV, ray–plane depth, the numerical projective map, pentagon symmetries, and RANSAC sampling probability. Paired SVG/PDF assets and editable TikZ sources are in `images/lecture03/`. The local SVG sampling simulator is original project code and uses explicitly stated independent match statuses; its complete static explanation and numerical results appear in print. Quiz filenames, printed-title discrepancies, and adapted versus original numerical examples are identified beside the work.

Lecture 4

`wk4_cs532s26_TwoViewGeometry.pdf` names Enrique Dunn on its title slide; the radial/two-view sections credit R. Hartley, A. Zisserman, and M. Pollefeys (PDF pages 2 and 8). The stereo section credits Rick Szeliski, Pascal Fua, P. Mordohai, and E. Dunn (PDF page 42, printed slide 46). Author metadata embedded in a PDF is not substituted for these visible credits.

The actual toy/plant photographs are extracted unchanged from PDF page 46 (printed slide 50), then paired with original project labels. The window-size result illustration is extracted unchanged from PDF page 66 (printed slide 70). Individual image creators and reuse terms are not identified on these slides; that uncertainty is retained. Original extracted files and editable panel source are preserved.

Five original paired SVG/PDF diagrams explain sloping epipolar lines and candidate checks, the third-view intersection, stereo rays, depth sensitivity, and a window crossing a depth boundary. The 3D stereo explorer uses the existing locally bundled Three.js r180/OrbitControls with the retained MIT license. Its scene, mathematical linkage, and static experiment are original project work. No new remote rendering dependency was introduced.

The essential-matrix decomposition was cross-checked against OpenCV's official camera-geometry documentation[6]. Its role here is implementation verification of four candidates, translation direction, and cheirality; the supplied lecture and independently worked examples remain the teaching sources.

[6] https://docs.opencv.org/4.12.0/d9/d0c/group__calib3d.html

Complete quiz workbooks

The 18 historical quizzes and seven original supplemental problems follow the full statements in the supplied `CS532_18_Quizzes_By_Lecture.tex` booklet. Its wording lightly edits the historical PDFs; original filenames and printed-header discrepancies remain beside each quiz. Solutions are expanded independently. The camera/object-plane and epipolar-plane sketches are reproduced from the booklet's editable TikZ source as local SVG/PDF pairs; reuse terms were not specified in that source. Existing credited project figures are reused where they illustrate the same question.

Red numbered step headings, short reasons, and blue mathematical workings are shared between HTML and LaTeX. Numbering and explicit labels carry the meaning when printed in grayscale. The source register distinguishes all duplicate retests and the seven newly authored booklet supplements.

Lecture 5–6 illustrations and method references

Selected source illustrations are cropped from the supplied Enrique Dunn lecture decks; individual credits remain in figure captions. These include NASA Mars imagery/Snavely matching illustrations, Heinly descriptor slides and the original method authors, the street-sign tracking illustration from Manhattan, and the Tomasi–Kanade house experiment. Where the deck does not identify an image creator or reuse terms, the register records that uncertainty. Extraction parameters are saved beside the assets. Original error-surface and refinement-ray diagrams have editable TikZ sources. ORB/BRISK primary documentation and papers are linked where the lecture statements require qualification.

The supplied study notes flag these source issues. Completed chapters give independently checked corrections below; later chapters retain provisional entries.

Source	Issue	Treatment or status
Fall 2025 Quiz 2	A line-coefficient vector is described as a point	Verified in *Lecture 4*: line/point equality corrected; supplied matrix has rank three
Fall 2025 Quiz 5	A planar grid supports a full-scene metric-upgrade claim	State calibration/view assumptions and distinguish plane from full-scene recovery in Lecture 6
`cs532f15_quiz4.pdf`, `cs532f15_quiz9.pdf`	Filenames suggest 2015; printed headers identify Spring 2019	Retain both names in citations
`cs532s20_quiz3.pdf`	Printed header mixes years/quiz labels	Record the filename and printed wording

Completed chapters report their independent checks; later entries remain provisional. Relevance scores describe slide relevance, not exam probabilities.

Lecture 1: verified qualifications and convention translations

- **Projective-plane shorthand (PDF page 71):** nonzero homogeneous vectors must be identified under common scale; they are not distinct projective points. See *Points, lines, and the meaning of infinity*.
- **Quaternion convention (PDF page 92):** the source sandwich is $q^{-1}Pq$. This book uses active Hamilton rotation qPq^{-1} and demonstrates the inverse relationship explicitly. Conjugation alone gives the inverse only for unit quaternions. See *Rotations: several descriptions of the same motion*.
- **Camera symbols (PDF pages 111–113):** camera-to-world pose and world-to-camera extrinsics use inverse transformations. The book translates the slide's notation and clarifies that projection maps 3D to 2D; its inverse without depth is a ray. See *Image formation: where does a 3D point appear?*.
- **Plane uniqueness (Fall 2025 Quiz 1 Q4):** camera centers and the chosen object-frame origin determine a unique plane only when noncollinear. See *Example 1.11*.
- **Nullspace wording (PDF pages 83–85):** solve the right nullspace of the stacked incidence matrix, not "the nullspace of" the unknown point/plane. See *Points, planes, and lines in 3D*.

Numerical companions are checked by `scripts/check_lecture01_math.py`. These qualifications do not imply that every flagged issue in later lectures has already been checked.

Lecture 2: verified corrections and assumptions

- **Matrix-to-quaternion formula, PDF page 49:** the scalar component needs $w = \frac{1}{2}\sqrt{1 + \operatorname{tr} R}$. The slide omits $+1$; identity, quarter-turn, and cyclic-axis checks expose the error. Near a half-turn, use a component branch instead of dividing by w. See *Quaternions: convert without changing the rotation*.
- **Quaternion convention, PDF pages 43, 48–49:** the sandwich and quaternion-to-matrix slide use the inverse/passive convention relative to the book, while the displayed skew-entry extraction uses active signs. The book uses active Hamilton formulas consistently and verifies matrix round trips.
- **Geometry shorthand, PDF pages 11, 23–25:** identify homogeneous vectors under nonzero scale; compute the right nullspace of the incidence coefficient matrix, not of the unknown vector. These repeat Lecture 1's qualifications.
- **Camera direction, PDF page 60:** the displayed projection goes from 3D to 2D despite the prose saying 2D to 3D. Without depth, inverse projection gives a ray. See *Foundation quiz practice*.
- **Radius notation, PDF page 109:** the displayed polynomial and radius definition mix squared-radius conventions. The book defines r^2 explicitly and states the correction direction and coefficient units. See (2.12).
- **Fundamental-matrix uniqueness, PDF page 124:** uniqueness is up to scale for a fixed nondegenerate camera pair; a finite match set can be ambiguous. Coincident camera centers do not yield a nonzero fundamental matrix. See *Distortion and two-view geometry*.
- **Quiz degeneracy, Spring 2020 Quiz 1 P1:** repeated points make the cross-product line undefined, although the determinant still correctly indicates linear dependence and existence of a containing line. P3 requires $d > 0$ for distinct vertical lines. See *Foundation quiz practice*.

Lecture 2's numerical examples are checked by `scripts/check_lecture02_math.py`.

Lecture 3 qualifications

- **Spring 2020 Quiz 3:** the page carries mixed Spring 2019 Quiz 2 / Spring 2020 Quiz 3 headers and near-duplicate camera questions; it is retained as a retest.
- **Camera quiz P2/P4:** horizontal FOV does not specify all of K or a physical focal length. *The solutions* state the extra assumptions, image-center convention, and pixel-pitch requirement.
- **Pentagon quiz:** "equilateral" alone does not imply regular. Ten equally supported corner alignments require the regular-polygon assumption. *The RANSAC solution* also distinguishes standard independent-status trial counts from exact finite-pool counts and explicitly chooses 99% confidence.
- **Repeated slide issues:** PDF page 11's projection-direction wording and page 60's radial-radius notation retain the corrections already explained in Lectures 1–2.

Lecture 4 verified corrections and conventions

- **Fall 2025 Quiz 2 Q2:** $x'^T F$ is a line row, not a point. The condition is $x'^T F x = 0$. The supplied matrix has determinant four, so it is rank three and cannot be a physical fundamental matrix. *Example 4.3* solves the given equation without relabeling it as valid camera geometry.
- **Fall 2025 Quiz 3 Q1:** 0.8 cm is the first-order error magnitude. The exact signed errors are +0.8333 cm for disparity 24 and −0.7692 cm for disparity 26. *Stereo solutions* define estimated-minus-true signs.
- **L4 PDF page 35, printed slide 39:** the shown seven-point RANSAC table uses approximately 95% confidence. A 99% budget at 80% inliers is 20 trials, rather than the table's 13.
- **Spring 2020 Quiz 5:** the printed header overprints 2019/2020; its problem repeats Spring 2019 Quiz 5. Both filenames and headers remain documented.
- **NCC, PDF page 68:** use a self-consistent centered-norm denominator and reject zero-variance patches. The *explicit formula* avoids ambiguity about whether standard deviations include a sample-count factor.
- **Repeated radial notation, PDF page 6:** the radius/squared-radius clarification remains at its canonical Lecture 2 location.

Lecture 5–6 slide qualifications

- **L5, page 75:** fixed-scale steering alone supplies no scale invariance; the full ORB pyramid pipeline handles multiple scales.
- **L5, page 81:** BRISK uses long pairs for orientation and short pairs for descriptor comparisons; the blanket instruction to avoid short pairs is misleading.
- **L5, page 85:** symmetry and zero diagonal apply to self-distances, not an arbitrary cross-image descriptor matrix.
- **L6, pages 28–35:** affine centered measurements have rank at most three; equality needs nondegeneracy.
- **L6, pages 7,25,41:** observation counts are necessary checks, not proofs of unique recovery.
- **L6, page 37:** metric upgrade must state its camera-scale assumptions and produce a valid positive definite metric from enough independent constraints.
- **L6, pages 49–52:** linear algebraic triangulation and geometric image-error optimization are distinct objectives.

INDEX

A

Absolute conic, **244**
Adaptive support, 146, **246**
Affine factorization, 208, **247**
Aperture problem, 180, **247**
Axial depth, **245**
axial depth, 43
Axis-angle, **244**
axis-angle, 33

B

Backprojection, **245**
backprojection, 43
Bilinear interpolation, **245**
bilinear interpolation, 74
BRIEF, 190, **247**
Brightness constancy, 185
BRISK, 190, **247**
Bundle adjustment, 218, **247**

C

Census transform, 146, **246**
Cheirality, 135, 210, **246**
Circular points, **244**
collinearity, 45
Computer vision, **243**
computer vision
  inverse problem, 11
Coordinate frame, **243**
Cost volume, 143, **246**
Cross-ratio, **245**
cross-ratio, 72

D

Degrees of freedom, **243**
Depth map, **243**
Depth uncertainty, 137
Direct linear transformation, **245**
direct linear transformation, 78
Disparity, 137, **246**
Drift, 185

E

Epipolar geometry, 128
Epipolar line, **245**
epipolar line, 83
Epipole, **246**
epipole, 83
Essential matrix, 135, **246**
Extrinsic parameters, **245**
extrinsic parameters, 43
Extrinsic rotation, **244**
extrinsic rotation, 33

F

Feature descriptor, 174
Feature detector, 174
Field of view, **246**
  Pixel pitch; Principal point, 95
Fundamental matrix, 128, **246**
fundamental matrix, 83

G

Gauge freedom, 205, **247**
Gauss-Newton, 218, **247**
geometric reconstruction, 11
Gimbal lock, **244**
gimbal lock, 33

H

Hamming distance, 190, **247**
Harris corner detector, 180, **247**
Homogeneous coordinates, **243**
homogeneous coordinates
  scale equivalence, 4
Homography, **245**
homography, 72

Horizon, **243**

I

Ideal point, **243**
Incidence, **243**
incidence, 15
Inlier, **245**
inlier, 78
Intrinsic parameters, **245**
intrinsic parameters, 43
Intrinsic rotation, **244**
intrinsic rotation, 33
Invariant, **245**
invariant, 72
Inverse warping, **245**
inverse warping, 74

L

Levenberg-Marquardt, 218, **247**
Line at infinity, **243**
line at infinity, 15
Lucas-Kanade tracking, 185, **247**

M

Metric upgrade, 208, **247**

N

Nearest-neighbor distance ratio, **247**
Nearest-neighbor distance ratio, 190
Non-maximum suppression, **247**
Non-maximum suppression, 180
normal
  outward orientation, 37
normalized DLT, 78
Nullspace, **244**
nullspace, 22

O

Occlusion, 143
ORB, 190, **247**

P

P3P, 213, **248**
perspective projection, 43
Photo-consistency, **246**
Photo-consistency, 143
Pinhole camera, **244**
pinhole camera, 43
Pixel pitch, **246**
plane
  normal, 22
Plücker coordinates, 22, **244**
point at infinity, 15
Point cloud, **243**
Principal point, **244**
principal point, 43
Projective plane, **243**
projective plane, 15
Projective reconstruction ambiguity, 205

Q

Quaternion, **244**
quaternion, 33

R

Radial distortion, **245**
radial distortion, 83
Rank transform, 146, **246**
RANSAC, 78, **245**
Ray-plane intersection, **246**
  Axial depth, 96
Reciprocal product, **244**
Rectification, **246**
rectification, 83
reference frame, 37
Rigid transformation, **244**
rigid transformation, 37
Rotation matrix, **244**
rotation matrix, 33

S

Schur complement, 218, **248**
Self-calibration, **248**
Self-calibration, 213
semantic information, 11
SLERP, 68, **245**
Stereo depth, 137
Structure from motion, 213, **243**
Structure tensor, 180, **247**
Surface normal, **244**
surface normal, 22
Symmetry ambiguity, 102, **246**

T

transform composition, 37
Translation scale, 135
Triangulation, 210

V

Vanishing point, **243**